

İST347 İstatistiksel Analiz Uygulamaları

R ile Veri Analizine Giriş — Ders Notları, 2026-2027 Güz Dönemi

Dr. Onur Toka

5 Ekim 2026

İçindekiler

Önsöz	3
Ders Bilgileri	3
Öğrenme Çıktıları	3
Haftalık Plan	3
Değerlendirme	4
Kitapta Kullanılan Veri Setleri	5
Bu Notları Derlemek	6
Kaynaklar	6
I Kısım I — R ile Çalışmak	8
1 R ve RStudio	9
1.1 R Nedir? Neden R?	9
1.2 RStudio	10
1.3 Projeler, Çalışma Dizini ve Görelî Yollar	13
1.4 Paketler	13
1.5 Yardım Almak	14
1.6 R'ı Hesap Makinesi Olarak Kullanma	15
1.7 Değişkenlere Değer Atama	18
1.8 Temel Veri Tipleri	18
1.9 İlk R Oturumu	20
Kendini Sına	21
2 Veri Yapıları	23
2.1 Vektörler	23
2.2 Matrisler ve Diziler (Array)	27
2.3 Faktörler: Nominal ve Ordinal Yapılar	30
2.4 Listeler	32
2.5 Veri Çerçevesi (Data Frame)	34
Kendini Sına	37
3 Raporlama: Quarto ve R Markdown	39
3.1 Neden Tekrarlanabilir Rapor?	39
3.2 R Markdown ve Quarto	39
3.3 Yeni Bir Belge Oluşturma	40
3.4 Bir Quarto Belgesinin Üç Parçası	40
3.5 Satır İçi (Inline) Kod	43
3.6 Tablolar ve Grafikler	43
3.7 Quarto'ya Özgü Kullanışlı Özellikler	44
3.8 Özet	44
3.9 Histogram	45
3.10 Ödev Raporu Şablonu	45
3.11 Derleme ve Sık Yapılan Hatalar	46
3.12 Quarto Kitabı	46
Kendini Sına	47

4	Veri Okuma/Yazma ve Veri Düzenleme	48
4.1	Hazır Veri Setleri	48
4.2	Dosyadan Veri Okuma	49
4.3	Dosyaya Veri Yazma	52
4.4	base R ile Seçme ve Sıralama	53
4.5	dplyr ile Veri Düzenleme	54
4.6	Uygulama: Müşteri Verisini Analize Hazırlama	61
4.7	Veriyi Yeniden Şekillendirme: Uzun ve Geniş Biçim	63
4.8	Tabloları Birleştirme	64
	Kendini Sına	65
5	Kontrol Yapıları ve Fonksiyon Yazma	66
5.1	Koşul Yapıları	66
5.2	Döngüler	68
5.3	Fonksiyon Yazma	71
5.4	Döngü Yerine: apply Ailesi	76
5.5	Veri Tablosuyla Çalışan Fonksiyonlar	77
	Kendini Sına	78
II	Kısım II — Veriyi Görmek ve Özetlemek	79
6	Görselleştirme: base R, ggplot2 ve plotly	80
6.1	Neden Önce Görselleştirme?	80
6.2	Hangi Soruya Hangi Grafik?	81
6.3	base R Grafikleri	82
6.4	ggplot2: Grafiklerin Dilbilgisi	85
6.5	Grafik Çizen Fonksiyonlar Yazmak	95
6.6	plotly: Etkileşimli Grafikler	97
	Kendini Sına	99
7	Betimsel İstatistikler	101
7.1	Konum Ölçüleri	101
7.2	Değişim Ölçüleri	105
7.3	Şekil Ölçüleri: Çarpıklık ve Basıklık	109
7.4	Standart Değerler ve Aykırı Değerler	110
7.5	Kategorik Değişkenler: Frekans ve Çapraz Tablolar	111
7.6	Gruplara Göre Özetler	113
7.7	Raporlama İlkeleri	115
	Kendini Sına	115
8	Ara Sınav: Tekrar ve Hazırlık	117
8.1	Kapsam	117
8.2	Hazırlık Listesi	117
8.3	Örnek Sorular	118
III	Kısım III — İstatistiksel Çıkarım	119
9	Olasılık Dağılımları	120
9.1	Temel Kavramlar	120
9.2	R'da Dağılım Fonksiyonları: d, p, q, r	120
9.3	Rastgele Sayı Üretmek ve set.seed()	121
9.4	Kesikli Dağılımlar	122

9.5	Sürekli Dağılımlar	124
9.6	Örnekleme Dağılımları: t, Ki-kare ve F	128
9.7	Veri Normal Dağılıma Uyuyor mu?	130
	Kendini Sına	132
10	Örnekleme, Merkezi Limit Teoremi ve Güven Aralıkları	134
10.1	Kitle, Örneklem, Parametre, İstatistik	134
10.2	Örnekleme Dağılımı	134
10.3	Merkezi Limit Teoremi	136
10.4	Standart Hata	138
10.5	Ortalama İçin Güven Aralığı	138
10.6	Oran İçin Güven Aralığı	142
10.7	Bootstrap	143
	Kendini Sına	145
11	Hipotez Testleri	146
11.1	Hipotez Testinin Mantığı	146
11.2	Parametrik ve Parametrik Olmayan Testler	148
11.3	Varsayımların Denetlenmesi	149
11.4	Tek Örneklem Testleri	150
11.5	İki Varyansın Karşılaştırılması: F Testi	153
11.6	İki Bağımsız Örneklem	154
11.7	Eşleştirilmiş (Bağımlı) Örneklem	158
11.8	Oranlar İçin Testler	160
11.9	Kategorik Değişkenler: Ki-kare Testleri	161
11.10	Çoklu Karşılaştırma Sorunu	165
11.11	Sonuçları Raporlamak	165
	Kendini Sına	165
12	Varyans Analizi (ANOVA)	167
12.1	Neden ANOVA?	167
12.2	Değişimin Parçalanması	167
12.3	Varsayımlar	170
12.4	Çoklu Karşılaştırmalar (Post-hoc Testler)	173
12.5	ANOVA Sonuçlarını Görselleştirmek	175
12.6	Kruskal–Wallis Testi	177
12.7	İki Yönlü ANOVA	178
12.8	Sonuçları Raporlamak	181
	Kendini Sına	181
13	Korelasyon ve Basit Doğrusal Regresyon	183
13.1	İki Sayısal Değişken Arasındaki İlişki	183
13.2	Basit Doğrusal Regresyon	187
13.3	Tahmin: Güven Aralığı ve Kestirim Aralığı	194
13.4	Model Varsayımları ve Artık Analizi	195
13.5	Eski Notlardan: Worms Verisinde Regresyon	198
13.6	Çoklu Regresyona Kısa Bir Bakış	199
13.7	Sonuçları Raporlamak	202
	Kendini Sına	202

IV Kısım IV — Sunmak	204
14 Shiny: Etkileşimli Uygulamalar	205
14.1 Shiny Nedir?	205
14.2 Bir Shiny Uygulamasının Yapısı	205
14.3 İlk Uygulama	206
14.4 Girdiler ve Çıktılar	207
14.5 Reaktiflik ve reactive()	208
14.6 Bütünleştirici Örnek: Penguen Keşif Uygulaması	208
14.7 Uygulamayı Çalıştırmak ve Paylaşmak	210
14.8 Dönem Sonu Projesi İçin Öneriler	211
Kendini Sına	211
Ekler	212
A Hangi Test? Hızlı Başvuru Rehberi	212
A.1 Karar Şeması	213
A.2 R Fonksiyonları Özeti	213
A.3 Etki Büyüklükleri	215

Önsöz

Bu notlar, Hacettepe Üniversitesi İstatistik Bölümü **İST347 İstatistiksel Analiz Uygulamaları** dersi için hazırlanmıştır. Dersin amacı, öğrencilere **R programlama dilini sıfırdan** öğretmek temel istatistiksel analizleri güvenle uygulayabilecekleri, **tekrarlanabilir** ve **raporlanabilir** bir çalışma alışkanlığı kazandırmaktır.

Kitap dört kısımdan oluşur:

- **Kısım I — R ile Çalışmak (1.–5. haftalar):** R ve RStudio, veri yapıları, Quarto ile raporlama, veri okuma ve dplyr ile veri düzenleme, koşullar, döngüler ve fonksiyon yazma.
- **Kısım II — Veriyi Görmek ve Özetlemek (6.–8. haftalar):** base R, ggplot2 ve plotly ile görselleştirme; betimsel istatistikler; ara sınav.
- **Kısım III — İstatistiksel Çıkarım (9.–13. haftalar):** olasılık dağılımları, örnekleme ve Merkezi Limit Teoremi, güven aralıkları ve bootstrap, parametrik ve **parametrik olmayan** hipotez testleri, ANOVA ve Kruskal–Wallis, korelasyon ve basit doğrusal regresyon.
- **Kısım IV — Sunmak (14. hafta):** Shiny ile analizleri etkileşimli bir uygulamaya dönüştürmek.

Ders Bilgileri

Ders	İST347 İstatistiksel Analiz Uygulamaları
Dönem	2026-2027 Güz
Ders sorumlusu	Dr. Onur Toka
E-posta	onur.toka@hacettepe.edu.tr
Web	www.onurtoka.com.tr

Öğrenme Çıktıları

Dönem sonunda öğrenci:

1. Bir istatistiksel problemi R koduna dökebilir.
2. Analiz + çıktı + yorumu tek bir Quarto belgesinde üretebilir.
3. Uygun görselleştirmeyi seçip ggplot2 (ve gerektiğinde plotly) ile hazırlayabilir.
4. Bir hipotez testi / ANOVA / regresyon sonucunu doğru yorumlayabilir, varsayımlarını denetleyebilir ve varsayımlar sağlanmadığında uygun **parametrik olmayan** yöntemi seçebilir.
5. Basit bir Shiny uygulaması ile analizlerini etkileşimli olarak sunabilir.

Haftalık Plan

Hafta	Konu	Bölüm
1	R & RStudio: arayüz, paketler, temel sözdizimi, yardım	1
2	Veri yapıları: vektör, matris, faktör, liste, data frame	2
3	Raporlama: Quarto (& R Markdown)	3
4	Veri okuma/yazma + veri düzenleme (dplyr)	4
5	Kontrol yapıları (if/else, for/while) + fonksiyon yazma	5
6	Görselleştirme: base R, ggplot2 , plotly	6
7	Betimsel istatistikler	7
8	Ara Sınav	8
9	Olasılık dağılımları (d/p/q/r, normal, t, ki-kare, F, binom, Poisson)	9
10	Örnekleme, Merkezi Limit Teoremi, güven aralıkları, bootstrap	10
11	Hipotez testleri: t-testleri, oran ve ki-kare testleri, parametrik olmayan testler	11
12	ANOVA (tek/iki yönlü, varsayımlar, post-hoc) ve Kruskal–Wallis	12
13	Korelasyon ve basit doğrusal regresyon	13
14	Shiny: etkileşimli uygulama	14

Değerlendirme

Bileşen	Ağırlık
Ara sınav	%30–40
Ödevler (Quarto ile)	%20
Final	%40–50

Devamsızlık sınırı **4 hafta / 12 ders saatidir**. Ödevler kod + çıktı + yorum içeren tek bir Quarto raporu (.html ya da .pdf) olarak teslim edilir; ekran görüntüsü değil, çalışan kod ve çıktı paylaşılır.

i Yapay zekâ kullanımı

Fikir alışverişi teşvik edilir. Yapay zekâ araçlarının **kod yazımında** kullanımı kabul edilir; ancak hangi aracın nasıl kullanıldığı rapora yazılmalıdır. Yorumların kopyalanması ve yapay

zekâ tarafından üretilmiş yorumlar kabul edilmez: **sonuçları yorumlamak istatistikçinin işidir.**

Kitapta Kullanılan Veri Setleri

Örneklerin çoğu, R'a kolayca yüklenebilen ve iyi bilinen veri setleriyle yapılmıştır. Böylece aynı veriyi farklı haftalarda farklı sorularla tekrar tekrar görür, verinin kendisini öğrenmeye zaman harcamazsınız.

Veri seti	Kaynak	İçerik	Kullanıldığı yerler
penguins	palmerpenguins paketi	Antarktika'daki 3 penguen türünden 344 penguenin gaga, yüzgeç, ağırlık ölçümleri	Hemen her bölüm
iris	base R	3 süsen türünden 150 çiçeğin taç ve çanak yaprak ölçümleri	Veri yapıları, görselleştirme, ANOVA
worms	veri/worms.txt (Crawley)	20 tarlada solucan yoğunluğu, toprak pH'ı, bitki örtüsü	Eski ders notlarındaki tüm örnekler
musteri	veri/dataset_ex2040	2014 müşteri demografik bilgileri ve harcamaları ("kirli" gerçek veri)	Veri okuma ve temizleme
faithful	base R	Yellowstone'daki Old Faithful gayzerinin püskürme süresi ve bekleme süresi	Basit doğrusal regresyon
sleep	base R	İki ilacın aynı 10 hastada uyku süresine etkisi	Eşleştirilmiş testler
mtcars	base R	32 otomobilin yakıt tüketimi ve teknik özellikleri	Alıştırmalar

Bu Notları Derlemek

Bu notlar bir **Quarto kitabıdır** (3. hafta). Kaynak dosyalardan HTML kitabı üretmek için:

1. **R** ve **RStudio** (Quarto ile birlikte gelir) kurulmalıdır.
2. R konsolunda gerekli paketler **bir kez** kurulmalıdır:

```
install.packages(c(
  "tidyverse", "palmerpenguins", "plotly", "readxl", "writexl",
  "car", "rstatix", "broom", "patchwork", "GGally", "Hmisc", "psych", "shiny",
  "rmarkdown", "knitr"
))
```

3. **QUARTO_NOTLAR** klasöründe terminalden **quarto render** (ya da RStudio'da **_quarto.yml** açırken **Build** → **Render Book**) çalıştırılır.
4. Çıktı **_book/index.html** dosyasındadır.

Klasör / dosya	İçerik
01-r-rstudio.qmd, 02-veri-yapilari.qmd, ... _common.R	Haftalık bölümler Her bölümün başında çalışan ortak ayarlar (paketler, tema)
images/ veri/	RStudio ekran görüntüleri worms.txt , dataset_ex.csv ve derste üretilen veri dosyaları
shiny_ornekleri/	14. haftanın çalıştırılmaya hazır Shiny uygulamaları

💡 R 4.5 ve sonrası için not

R 4.5 sürümüyle birlikte **penguins** verisi base R'a da eklenmiştir; ancak oradaki sütun adları farklıdır (**bill_len**, **flipper_len** gibi). Bu kitaptaki kodlar **palmerpenguins** paketindeki adları (**bill_length_mm**, **flipper_length_mm** ...) kullanır. **library(palmerpenguins)** çalıştırıldığında paketteki sürüm kullanılır, bu yüzden kodlar her iki R sürümünde de aynı şekilde çalışır.

Kaynaklar

Ana kaynaklar (ücretsiz, çevrimiçi)

1. Wickham, H., Çetinkaya-Rundel, M. & Grolemund, G. (2023). *R for Data Science* (2. baskı). <https://r4ds.hadley.nz>
2. Çetinkaya-Rundel, M. & Hardin, J. *Introduction to Modern Statistics* (2. baskı). <https://openintro-ims.netlify.app>
3. Chang, W. (2018). *R Graphics Cookbook* (2. baskı). <https://r-graphics.org>
4. Long, J. D. & Teetor, P. (2019). *R Cookbook* (2. baskı). <https://rc2e.com>
5. Wickham, H., Navarro, D. & Pedersen, T. L. *ggplot2: Elegant Graphics for Data Analysis* (3. baskı). <https://ggplot2-book.org>
6. Wickham, H. (2021). *Mastering Shiny*. <https://mastering-shiny.org>
7. Quarto resmi belgeleri: <https://quarto.org/docs/guide/>

Ek kaynaklar

8. Crawley, M. J. (2015). *Statistics: An Introduction Using R* (2. baskı). Wiley.
9. Verzani, J. (2014). *Using R for Introductory Statistics* (2. baskı). CRC Press.
10. Golemund, G. *Hands-On Programming with R*. <https://rstudio-education.github.io/hopr>
11. Sievert, C. (2020). *Interactive Web-Based Data Visualization with R, plotly, and shiny*. <https://plotly-r.com>
12. Horst, A. M., Hill, A. P. & Gorman, K. B. (2020). *palmerpenguins: Palmer Archipelago (Antarctica) penguin data*. <https://allisonhorst.github.io/palmerpenguins>

Kısım I

Kısım I — R ile Çalışmak

1 R ve RStudio

i Bu haftanın hedefleri

Bu bölümün sonunda şunları yapabiliyor olmalısınız:

- R ile RStudio arasındaki farkı açıklamak ve ikisini kurmak,
- RStudio'daki pencereleri (Editor, Console, Environment, Plots, Packages, Help) tanımak,
- Bir **RStudio projesi** oluşturmak ve dosyaları **görelî yol** ile çağırmak,
- Paket **kurmak** ve **yüklemek** arasındaki farkı bilmek,
- R'ı hesap makinesi gibi kullanmak, değişken tanımlamak, temel veri tiplerini tanımak,
- Yardım sayfalarını okumak ve hata mesajlarından ne anlaşılması gerektiğini bilmek.

1.1 R Nedir? Neden R?

R, istatistiksel hesaplama ve grafik için geliştirilmiş **ücretsiz ve açık kaynak** bir programlama dilidir. Ross Ihaka ve Robert Gentleman tarafından 1993 yılında Yeni Zelanda'daki Auckland Üniversitesi'nde geliştirilmeye başlanmış; bugün **R Core Team** tarafından sürdürülmektedir. R, Bell Laboratuvarları'nda geliştirilen **S** dilinin açık kaynak bir uyarlaması olarak düşünülebilir.

R'ın özelliği	Ne anlama gelir?
Ücretsiz ve açık kaynak İstatistik için tasarlanmış	Lisans ücreti yoktur, kodu herkes inceleyebilir t-testinden karma modellere kadar her yöntem hazırdır
Vektörel çalışır	Bir işlem tek tek elemanlara değil, bütün veriye aynı anda uygulanır
Güçlü grafik	Yayın kalitesinde grafikler (ggplot2) ve etkileşimli grafikler (plotly)
Paket ekosistemi	CRAN (Comprehensive R Archive Network) üzerinde 20.000'den fazla paket
Tekrarlanabilir	Kod + çıktı + yorum Quarto ile tek belgede (Bölüm 3)

1.1.1 R Programını İndirme

<https://cran.r-project.org/> adresinden işletim sisteminize (Windows / macOS / Linux) uygun R sürümünü indirebilirsiniz. **Önce R, sonra RStudio** kurulmalıdır; RStudio kendi başına R içermez, bilgisayarındaki R'ı kullanır.

1.2 RStudio

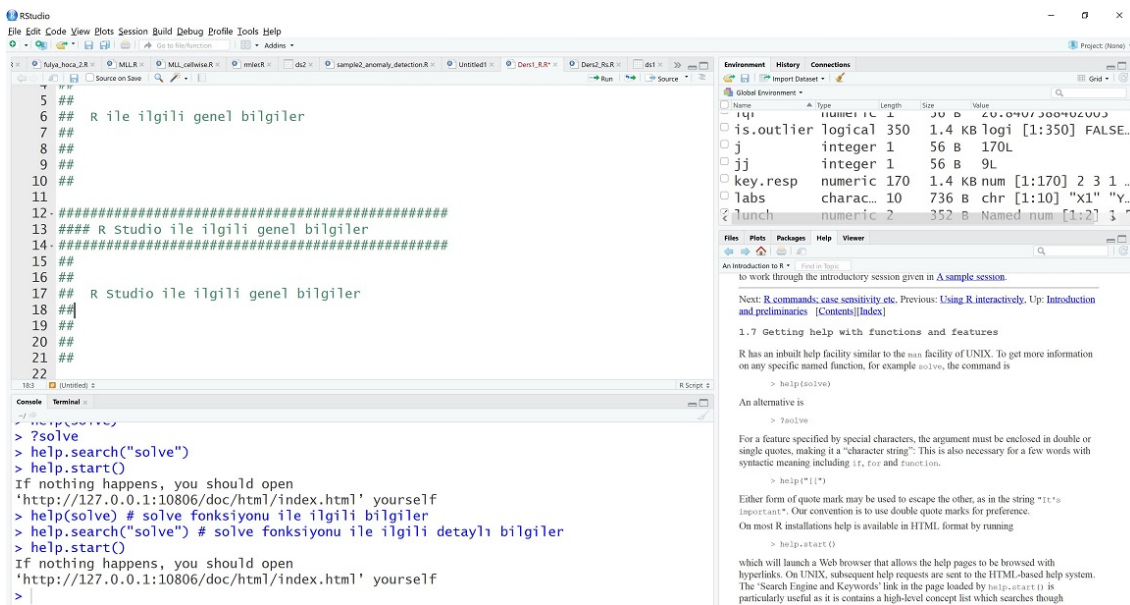
RStudio, R ile çalışmayı kolaylaştıran bir **bütünleşik geliştirme ortamıdır** (IDE, *Integrated Development Environment*). Kısaca: **R motordur, RStudio ise gösterge paneli**. Kodları R çalıştırır; RStudio bize editör, grafik penceresi, dosya gezgini, yardım gibi araçları tek ekranda sunar.

- RStudio, Joseph J. Allaire'in kurduğu şirket tarafından geliştirilmiş, ilk sürümü Şubat 2011'de çıkmıştır.
- Şirketin adı 2022'de **Posit** olarak değişmiştir; program hâlâ "RStudio" adıyla dağıtılmaktadır.
- Aynı ekibin geliştirdiği önemli araçlar: **tidyverse** (dplyr, ggplot2, readr...), **shiny**, **rmarkdown**, **knitr** ve **Quarto**.
- <https://posit.co/download/rstudio-desktop/> adresinden ücretsiz "RStudio Desktop" sürümünü indirebilirsiniz. Aynı ekibin yeni nesil editörü **Positron** da R ile kullanılabilir.

1.2.1 RStudio Ekran Görünümü

RStudio açıldığında ekran genellikle **dört bölme**ye ayrılır:

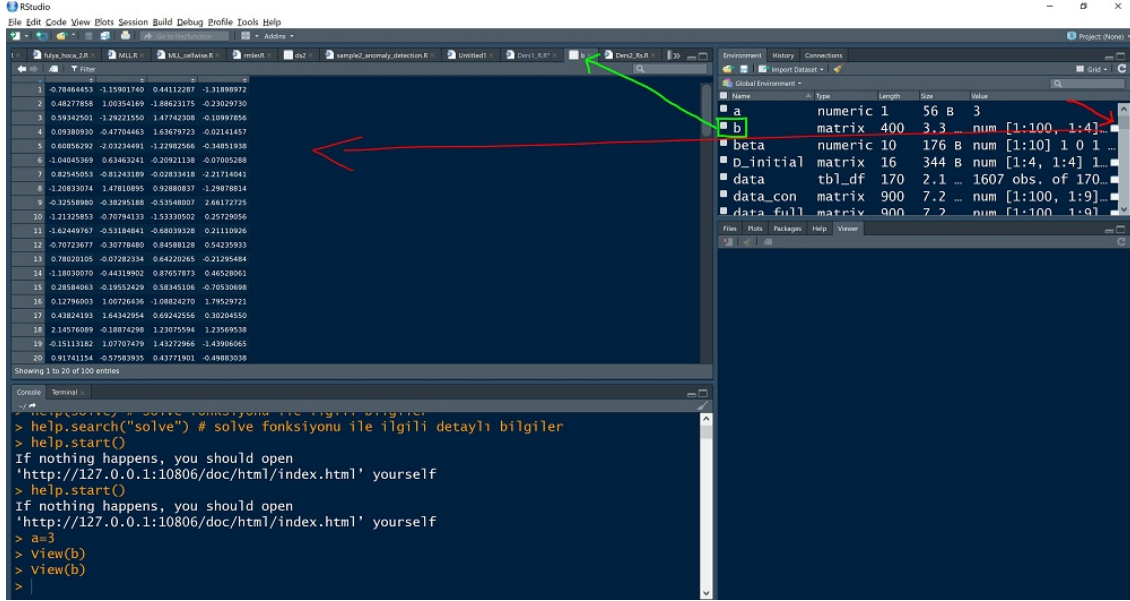
Bölme	Konum (varsayılan)	Ne işe yarar?
Editor (Source)	Sol üst	Kod dosyalarını (.R, .qmd, .Rmd) yazıp kaydettiğimiz yer
Console	Sol alt	Kodların gerçekten çalıştığı yer; sonuçlar burada görünür
Environment / History	Sağ üst	Oluşturulan nesneler ve daha önce çalıştırılan kodlar
Files / Plots / Packages / Help / Viewer	Sağ alt	Dosyalar, grafikler, paketler, yardım sayfaları



Şekil 1.1: RStudio ekran görünümü (ekran görüntüleri RStudio'nun eski bir sürümünden alınmıştır; menülerin yeri güncel sürümde küçük farklılıklar gösterebilir).

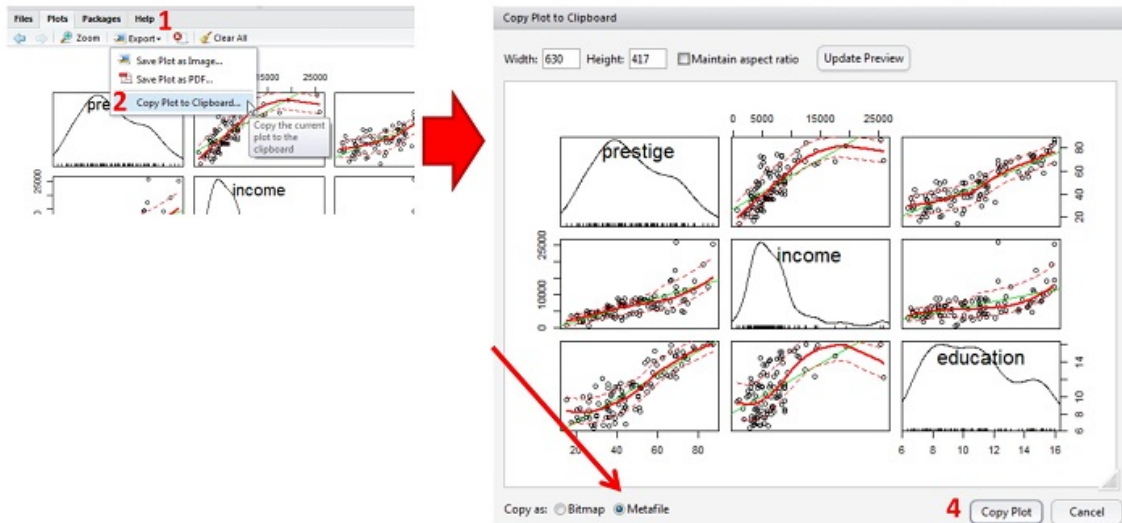
1.2.2 RStudio Program Penceresi

- **Console:** Kod yazıp *Enter* tuşuna bastığınız anda kod çalışır. > işareti R'ın komut beklediğini, + işareti ise komutun **tamamlanmadığını** (genellikle bir parantez ya da tırnak açık kaldığını) gösterir. Takıldığınızda **Esc** tuşuna basın.
- **Editor:** Kodları bir dosyaya yazıp kaydedebilirsiniz. Yazılan kod, çalıştırılmadıkça (*Run* düğmesi ya da **Ctrl + Enter** / Mac'te **Cmd + Enter**) çalışmaz. **Kalıcı olması gereken her kod editörde yazılmalıdır.**
- **Environment:** Çalıştırılan kodlarla üretilmiş tüm nesneleri (sayı, vektör, matris, veri tablosu, fonksiyon...) gösterir. Bir veri tablosunun yanındaki tablo simgesine tıklanarak veri incelenebilir (*View()* fonksiyonu ile aynı işi görür).



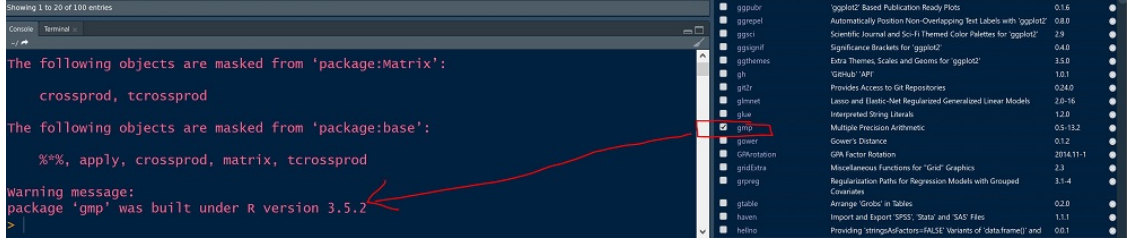
Şekil 1.2: Environment penceresi ve View() ile veri inceleme

- **History:** Daha önce çalıştırılmış kodları gösterir.
- **Plots:** Çizdirilen grafikler burada görünür. **Export** menüsü ile grafikler resim ya da PDF olarak kaydedilebilir veya panoya kopyalanabilir.



Şekil 1.3: Plots penceresinden grafiği dışı aktarma

- **Packages:** Bilgisayarda kurulu paketleri listeler. Yanındaki kutucuk işaretliyse paket o oturumda **yüklenmiş** demektir.

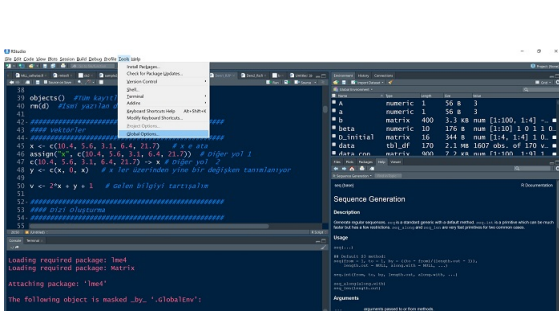


Şekil 1.4: Packages penceresi

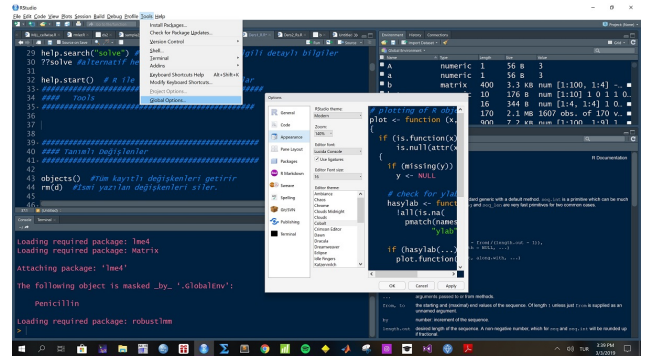
- **Help:** Fonksiyonların açıklamaları, argümanları ve örnek kodları burada görünür.
- **Viewer:** Etkileşimli grafikler (plotly) ve Quarto çıktıları burada önizlenir.
- **Connections / Git / Tutorial:** Veri tabanı bağlantıları, sürüm kontrolü ve learnr ile hazırlanmış etkileşimli dersler içindir.

1.2.3 Görünümü Değiştirme

Tools → Global Options menüsünden yazı tipi, tema (açık/koyu), bölmelerin yerleşimi gibi ayarlar değiştirilebilir.



Şekil 1.5: Tools menüsü



Şekil 1.6: Global Options → Appearance

🌟 Önerilen iki ayar

Tools → Global Options → General altında “*Restore .RData into workspace at startup*” kutusunun işaretini kaldırın ve “*Save workspace to .RData on exit*” seçeneğini **Never** yapın. Böylece her oturuma temiz bir sayfayla başlarsınız ve analizinizin yalnızca kodunuza dayandığından emin olursunuz.

1.2.4 Sık Kullanılan Kısayollar

İşlem	Windows / Linux	macOS
Satırı / seçimi çalıştır	Ctrl + Enter	Cmd + Enter
Tüm dosyayı çalıştır	Ctrl + Shift + Enter	Cmd + Shift + Enter
Atama oku <-	Alt + -	Option + -
Pipe >	Ctrl + Shift + M	Cmd + Shift + M

İşlem	Windows / Linux	macOS
Kod parçası ekle (Quarto)	Ctrl + Alt + I	Cmd + Option + I
Satırı yorum yap / kaldır	Ctrl + Shift + C	Cmd + Shift + C
Konsolu temizle	Ctrl + L	Ctrl + L
Otomatik tamamlama	Tab	Tab

1.3 Projeler, Çalışma Dizini ve Göreli Yollar

R bir dosyayı okurken ya da kaydederken **çalışma dizinine** (*working directory*) bakar.

```
getwd() # Şu anki çalışma dizinini gösterir
setwd("C:/Kullanici/Belgeler/ist347") # Çalışma dizinini değiştirir (ÖNERİLMEZ)
list.files() # Çalışma dizinindeki dosyaları listeler
```

⚠ Mutlak yol yerine göreli yol kullanın

"C:/Users/onurt/Desktop/worms.txt" gibi **mutlak yollar** yalnızca o bilgisayarda çalışır. Dosya başka bir bilgisayara, başka bir klasöre ya da Mac'e taşındığında kod hata verir.

1. **File** → **New Project** ile bir **RStudio projesi** (.Rproj) oluşturun ve çalışmaya her zaman bu dosyayı açarak başlayın. Proje açıldığında çalışma dizini otomatik olarak proje klasörü olur.
2. Dosyaları proje klasörüne göre **göreli yol** ile çağırın: "veri/worms.txt", "images/r1.jpg" gibi. (.. "bir üst klasör" demektir.)
3. Windows'ta yol yazarken ters bölü \ yerine düz bölü / kullanın.
4. Quarto belgesi derlenirken çalışma dizini **belgenin bulunduğu klasördür**.

Bu kitabın klasör yapısı da bu mantıkla kurulmuştur:

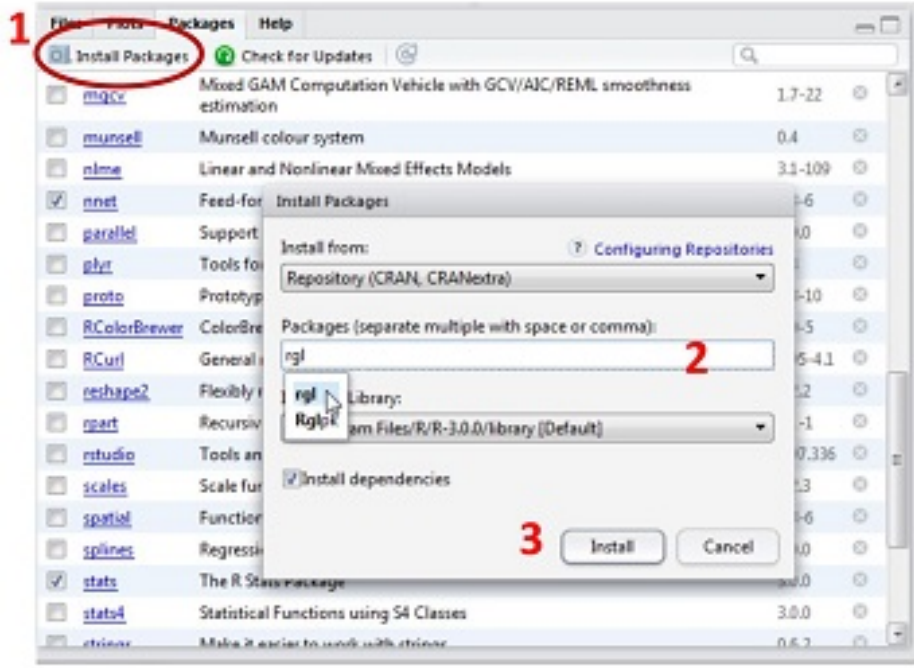
```
QUARTO_NOTLAR/
├─ _quarto.yml      <- kitabın ayarları
├─ index.qmd        <- önsöz
├─ 01-r-rstudio.qmd <- bölümler
├─ ...
├─ images/          <- ekran görüntüleri
└─ veri/            <- worms.txt, dataset_ex.csv
```

1.4 Paketler

R'ın temel kurulumunda gelen fonksiyonlara **base R** denir. Ek özellikler **paketler** ile gelir. Paketlerle çalışırken iki adımı ayırt etmek gerekir:

Adım	Ne zaman?	Kod
Kurma (install)	Bilgisayara bir kez	<code>install.packages("dplyr")</code>
Yükleme (load)	Her yeni oturumda	<code>library(dplyr)</code>

Benzetme: `install.packages()` kitabı satın alıp rafa koymak, `library()` ise kitabı raftan alıp masaya açmaktır.



Şekil 1.7: Packages → Install ile paket kurma

```
install.packages(c("tidyverse", "palmerpenguins", "plotly")) # bir kez
library(palmerpenguins) # her oturumda
dplyr::filter          # Paketi yüklemeyen tek bir fonksiyonu kullanma:
↳ paket::fonksiyon
```

i Not

Farklı paketlerde aynı isimli fonksiyonlar olabilir. Bir paket yüklenirken konsolda “*The following objects are masked from ...*” uyarısı görürseniz, son yüklenen paketin fonksiyonu öncekini “örtmüş” demektir. Bu durumda `stats::filter()` ya da `dplyr::filter()` şeklinde hangi paketin fonksiyonunu istediğinizi açıkça yazabilirsiniz.

tidyverse tek bir paket değil, aynı tasarım felsefesini paylaşan paketlerin bir koleksiyonudur: `library(tidyverse)` yazıldığında `ggplot2`, `dplyr`, `tidyr`, `readr`, `purrr`, `tibble`, `stringr`, `forcats` ve `lubridate` birlikte yüklenir. Bu derste **hibrit** bir yol izliyoruz: temel mantığı base R ile kurup günlük işlerde `dplyr` ve `ggplot2` kullanacağız.

1.5 Yardım Almak

- `help(mean)` veya `?mean`: `mean` fonksiyonunun yardım sayfasını açar.
- `help.search("solve")` veya `??solve`: İçinde “solve” geçen tüm yardım sayfalarında arama yapar.
- `example(mean)`: Yardım sayfasındaki örnekleri çalıştırır.
- `vignette("dplyr")`: Bir paketin uzun anlatımlı kullanım kılavuzunu açar.
- `help.start()`: R’in genel kılavuz sayfalarını açar.

Yardım sayfalarında en çok işe yarayan kısımlar: **Usage** (nasıl çağrılır), **Arguments** (argümanlar), **Value** (ne döndürür) ve **Examples** (örnekler).

1.5.1 Hata Mesajlarını Okumak

R'da üç tür mesaj görürsünüz. Kırmızı yazıların hepsi hata **değildir**:

Tür	Başlangıç	Anlamı
Error	Error in ...	Kod durdu , sonuç üretilmedi. Düzeltmek zorundasınız.
Warning	Warning message:	Kod çalıştı ama R bir şeyden şüpheleniyor. Okuyun!
Message	(düz metin)	Bilgilendirme, örneğin paket yüklenirken.

```
log(-1) # Uyarı: NaN üretildi
```

```
#> Warning in log(-1): NaNs produced
```

```
#> [1] NaN
```

```
sqrt("dört") # Hata: sayı olmayan bir argüman
```

```
#> Error in sqrt("dört"): non-numeric argument to mathematical function
```

```
ortalama # Hata: böyle bir nesne yok
```

```
#> Error in eval(expr, envir, enclos): object 'ortalama' not found
```

En sık karşılaşılan hatalar: nesne adının yanlış yazılması (R büyük/küçük harfe duyarlıdır), paketin yüklenmemesi (could not find function), kapanmayan parantez ya da tırnak ve dosya yolunun yanlış olması (cannot open file).

1.6 R'ı Hesap Makinesi Olarak Kullanma

işaretinden sonra yazılanlar **yorumdur**, çalıştırılmaz.

```
3 + 5 # Toplama
```

```
#> [1] 8
```

```
9 - 2 # Çıkarma
```

```
#> [1] 7
```

```
2 * 3      # Çarpma
```

```
#> [1] 6
```

```
9 / 4      # Bölme
```

```
#> [1] 2.25
```

```
2^10       # Üs alma
```

```
#> [1] 1024
```

```
17 %/% 5   # Tam bölme
```

```
#> [1] 3
```

```
17 %% 5    # Mod (kalan)
```

```
#> [1] 2
```

```
log(exp(5)) # Doğal logaritma
```

```
#> [1] 5
```

```
log10(100)
```

```
#> [1] 2
```

```
sqrt(100)
```

```
#> [1] 10
```

```
round(pi, 3)
```

```
#> [1] 3.142
```

Yuvarlama fonksiyonları sık karıştırılır:

```
x <- c(-2.7, -2.5, 2.5, 2.7)
round(x)      # Matematiksel yuvarlama (.5 çift sayıya yuvarlanır!)
```

```
#> [1] -3 -2  2  3
```

```
ceiling(x)    # Bir üst tam sayıya
```

```
#> [1] -2 -2  3  3
```

```
floor(x)      # Bir alt tam sayıya
```

```
#> [1] -3 -3  2  2
```

```
trunc(x)      # Sıfıra doğru (ondalığı atar)
```

```
#> [1] -2 -2  2  2
```

1.6.1 Mantıksal İşlemler

```
# Karşılaştırma:  <, <=, >, >=, == (eşit mi?), != (eşit değil mi?)  
# Birleştirme:    c1 & c2 (ve),   c1 | c2 (veya),   !c1 (değil)  
3 > 13
```

```
#> [1] FALSE
```

```
!2 > 13
```

```
#> [1] TRUE
```

```
!(5 < 2) & (8 != 4)
```

```
#> [1] TRUE
```

```
(1 > 5) | (4 == 5)
```

```
#> [1] FALSE
```

! Önemli

= atama, == karşılaştırma içindir. `x == 3` “x 3’e eşit mi?” diye sorar; `x = 3` ise x’e 3 değerini verir.

Ondalık sayılar bilgisayarda tam olarak saklanamaz. Bu yüzden ondalıklı sayıları == ile karşılaştırmak sürprizlere yol açabilir:

```
0.1 + 0.2 == 0.3
```

```
#> [1] FALSE
```

```
isTRUE(all.equal(0.1 + 0.2, 0.3)) # Güvenli karşılaştırma
```

```
#> [1] TRUE
```

1.7 Değişkenlere Değer Atama

R'da atama için en yaygın kullanılan işaret `<-`'dir (RStudio kısayolu **Alt + -**). `=` de çalışır.

```
x <- c(10.4, 5.6, 3.1, 6.4, 21.7) # x'e ata
assign("x1", c(10.4, 5.6, 3.1, 6.4, 21.7)) # Diğer yol 1
c(10.4, 5.6, 3.1, 6.4, 21.7) -> x2 # Diğer yol 2
x3 = c(10.4, 5.6, 3.1, 6.4, 21.7) # Diğer yol 3
y <- c(x, x) # x'ler üzerinden yeni bir değişken
v <- 2*x + y + 1 # Gelen bilgiyi tartışalım: x 5, y 10 elemanlı!
v
```

```
#> [1] 32.2 17.8 10.3 20.2 66.1 32.2 17.8 10.3 20.2 66.1
```

i Geri dönüşüm (recycling) kuralı

Yukarıda x 5, y 10 elemanlıdır. R, kısa olan vektörü uzun olanın boyuna ulaşana kadar **tekrar eder**. Uzunluklar birbirinin katı değilse işlem yine yapılır ama **uyarı** verilir:

```
y1 <- c(x, 0, x) # 11 elemanlı
v1 <- 2*x + y1 + 1 # 11, 5'in katı değil -> uyarı
```

```
#> Warning in 2 * x + y1: longer object length is not a multiple of shorter object
#> length
```

Değişken isimlendirme kuralları: Harf ya da nokta ile başlar; rakam ile başlayamaz; boşluk içeremez; büyük/küçük harfe duyarlıdır (**Veri veri**). Okunabilirlik için **ogrenci_notu** gibi **alt çizgili** (*snake_case*) isimler önerilir. Türkçe karakterler çalışır ama başka bilgisayarlarda kodlama sorunu çıkarabileceği için nesne adlarında kullanmamak daha güvenlidir.

1.8 Temel Veri Tipleri

Tip	Örnek	class() sonucu
Mantıksal	TRUE, FALSE	"logical"
Sayısal (ondalıklı)	3, 4.8	"numeric"
Tam sayı	5L	"integer"
Karakter	"Karakter"	"character"
Eksik değer	NA	(tipine göre)

```
a <- TRUE; b <- 3; c <- 4.8; d <- 5L; e <- "Karakter"
class(a); class(b); class(c); class(d); class(e)
```

```
#> [1] "logical"
```

```
#> [1] "numeric"
```

```
#> [1] "numeric"
```

```
#> [1] "integer"
```

```
#> [1] "character"
```

```
str(e)
```

```
#> chr "Karakter"
```

Tip dönüşümleri `as.` ile başlayan fonksiyonlarla, tip sorgulama `is.` ile başlayan fonksiyonlarla yapılır:

```
as.numeric("3.14")
```

```
#> [1] 3.14
```

```
as.character(42)
```

```
#> [1] "42"
```

```
as.integer(4.9)      # Yuvarlamaz, ondalığı atar!
```

```
#> [1] 4
```

```
as.numeric(TRUE)     # TRUE = 1, FALSE = 0
```

```
#> [1] 1
```

```
as.numeric("elma")   # Sayıya çevrilemez -> NA ve uyarı
```

```
#> Warning: NAs introduced by coercion
```

```
#> [1] NA
```

`TRUE`'nın 1, `FALSE`'un 0 sayılması çok kullanışlıdır: mantıksal bir vektörün toplamı “kaç tane doğru var?”, ortalaması “doğruların oranı nedir?” sorusunu cevaplar.

```
notlar <- c(45, 82, 67, 91, 30)
sum(notlar >= 60)      # Kaç kişi geçti?
```

```
#> [1] 3
```

```
mean(notlar >= 60)     # Geçenlerin oranı
```

```
#> [1] 0.6
```

1.8.1 Eksik Değerler

Eksik değerler **NA** (*Not Available*) ile gösterilir ve **bulaşıcıdır**: içinde **NA** olan bir hesaplamanın sonucu da **NA** olur.

```
f <- c(4, NA, 6)
is.na(f)
```

```
#> [1] FALSE TRUE FALSE
```

```
mean(f)                # NA
```

```
#> [1] NA
```

```
mean(f, na.rm = TRUE)  # NA'ları atarak hesapla
```

```
#> [1] 5
```

```
sum(is.na(f))          # Kaç eksik değer var?
```

```
#> [1] 1
```

NA ile **NaN** (*Not a Number*, $0/0$ gibi tanımsız işlemler), **Inf** ($1/0$) ve **NULL** (boş nesne) farklı şeylerdir.

1.9 İlk R Oturumu

Bu bölümde öğrendiklerimizi küçük bir örnekle birleştirelim. Bir sınıftaki öğrencilerin notlarını özetleyelim:

```
notlar <- c(70, 85, 90, 65, 78, 55, 92, 60, 74, 81)
length(notlar)      # gözlem sayısı
```

```
#> [1] 10
```

```
mean(notlar)      # ortalama
```

```
#> [1] 75
```

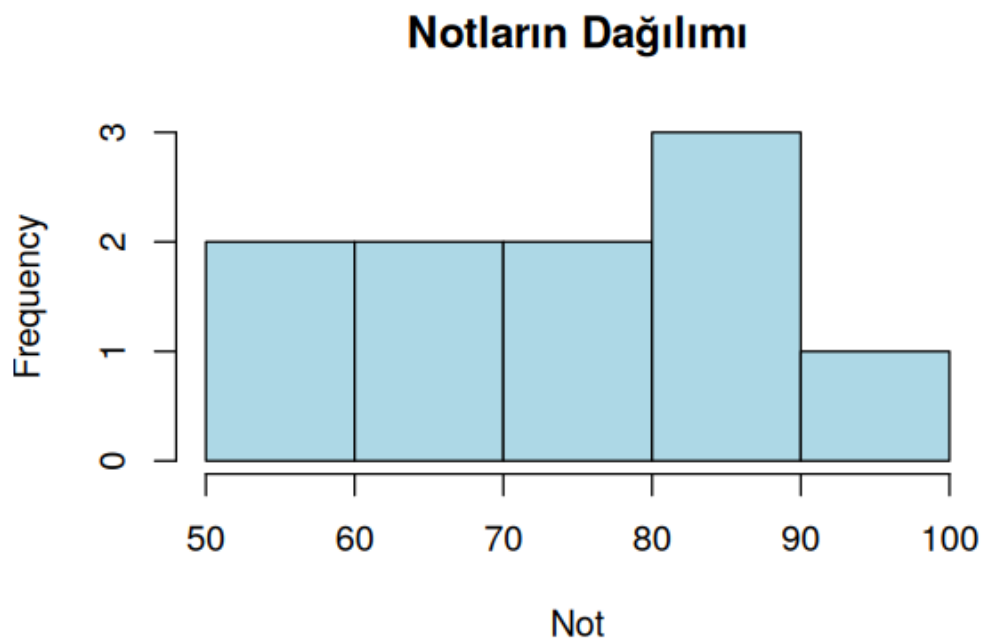
```
sd(notlar)        # standart sapma
```

```
#> [1] 12.52
```

```
summary(notlar)   # beş sayı özeti + ortalama
```

```
#>   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>   55.0   66.2   76.0   75.0   84.0   92.0
```

```
hist(notlar, main = "Notların Dağılımı", xlab = "Not", col = "lightblue")
```



Kendini Sına

- 💡 1. Yarıçapı 7 cm olan bir dairenin alanını hesaplayıp `alan` adlı bir değişkene atayın.

```
alan <- pi * 7^2
alan   # 153.938
```

💡 2. `install.packages("ggplot2")` çalıştırdınız ama `ggplot()` fonksiyonu “could not find function” hatası veriyor. Neden?

Paket **kurulmuş** ama bu oturumda **yüklenmemiştir**. `library(ggplot2)` çalıştırılmalıdır. Kurulum bir kez, yükleme her oturumda yapılır.

💡 3. `as.integer("12a")` ne döndürür? Neden?

NA döndürür ve uyarı verir, çünkü “12a” bir sayı olarak yorumlanamaz. R hata vermez; bu yüzden veri okurken sayısal olması gereken bir sütunun karakter olarak gelmesi sessiz bir sorun yaratabilir.

💡 4. `?round` yardım sayfasını açın. `round()` ile `signif()` arasındaki fark nedir?

`round(x, 2)` virgülden sonra 2 **basamağa**, `signif(x, 2)` ise toplam 2 **anlamalı basamağa** yuvarlar. Örneğin `round(123.456, 1)` 123.5, `signif(123.456, 2)` ise 120 verir.

💡 5. `c(1, "a", TRUE)` vektörünün tipi nedir? Neden?

"character". Bir vektörün bütün elemanları aynı tipte olmalıdır; farklı tipler bir araya geldiğinde R hepsini en “genel” tipe çevirir: mantıksal → tam sayı → sayısal → karakter.

2 Veri Yapıları

i Bu haftanın hedefleri

- R'daki temel veri yapılarını (**vektör**, **matris/array**, **faktör**, **liste**, **data frame**) tanımak ve oluşturmak,
- Bu yapıların içinden eleman seçmek (**indeksleme**),
- Vektörel işlem mantığını kavramak,
- Kategorik değişkenleri **faktör** olarak saklamak ve düzeylerini yönetmek,
- Bir veri setinin yapısını **str()**, **head()**, **summary()** ile incelemek.

İstatistikte bir değişken ölçme düzeyine göre farklı şekilde saklanır ve analiz edilir. R'daki veri yapıları bu farklılıklara karşılık gelir:

Tablo 2.1: R'daki temel veri yapıları

Yapı	Boyut	Aynı tip mi olmalı?	Örnek kullanım
Vektör	1	Evet	Bir değişkenin gözlemleri
Matris	2	Evet	Korelasyon matrisi, tasarım matrisi
Dizi (array)	2+	Evet	Çok boyutlu tablolar
Faktör	1	Kategoriler	Tür, cinsiyet, eğitim düzeyi
Liste	1	Hayır	Bir analizin tüm çıktıları
Data frame	2	Sütun içinde evet	Veri seti (sıra = gözlem, sütun = değişken)

2.1 Vektörler

Vektör, **aynı tipteki** değerlerin sıralı topluluğudur ve **c()** (*combine*) fonksiyonu ile oluşturulur. R'da tek bir sayı bile aslında 1 elemanlı bir vektördür.

2.1.1 Dizi Oluşturma

```
1:10 # 1'den 10'a kadar
```

```
#> [1] 1 2 3 4 5 6 7 8 9 10
```

```
2*1:5 # 2'den 10'a kadar çiftler çiftler
```

```
#> [1] 2 4 6 8 10
```

```
10:1 # 10'dan 1'e kadar
```

```
#> [1] 10 9 8 7 6 5 4 3 2 1
```

```
seq(2, 10)
```

```
#> [1] 2 3 4 5 6 7 8 9 10
```

```
seq(-1, 1, by = .25)
```

```
#> [1] -1.00 -0.75 -0.50 -0.25 0.00 0.25 0.50 0.75 1.00
```

```
seq(0, 1, length.out = 5) # Eşit aralıklı 5 nokta
```

```
#> [1] 0.00 0.25 0.50 0.75 1.00
```

```
x <- c(10.4, 5.6, 3.1, 6.4, 21.7)
rep(x, times = 2) # Vektörün tamamını 2 kez tekrarla
```

```
#> [1] 10.4 5.6 3.1 6.4 21.7 10.4 5.6 3.1 6.4 21.7
```

```
rep(c("A", "B"), each = 3) # Her elemanı 3'er kez tekrarla
```

```
#> [1] "A" "A" "A" "B" "B" "B"
```

Minik bir yorumlama: İşlem önceliğine dikkat!

```
n <- 10
1:n-1 # Önce 1:n oluşturulur, sonra her elemandan 1 çıkarılır
```

```
#> [1] 0 1 2 3 4 5 6 7 8 9
```

```
1:(n-1)
```

```
#> [1] 1 2 3 4 5 6 7 8 9
```

2.1.2 Vektör Üzerinde İşlemler

İşlemler vektörün **tüm elemanlarına aynı anda** uygulanır (vektörel işlem). Döngü yazmaya gerek yoktur.

```
notlar <- c(65, 80, 45, 90, 72)
notlar + 5          # Herkese 5 puan
```

```
#> [1] 70 85 50 95 77
```

```
notlar > 60          # Kim geçti?
```

```
#> [1] TRUE TRUE FALSE TRUE TRUE
```

```
length(notlar); sum(notlar); mean(notlar); max(notlar)
```

```
#> [1] 5
```

```
#> [1] 352
```

```
#> [1] 70.4
```

```
#> [1] 90
```

```
sort(notlar, decreasing = TRUE)
```

```
#> [1] 90 80 72 65 45
```

```
order(notlar)        # Sıralama için hangi sıradaki elemanların alınacağı
```

```
#> [1] 3 1 5 2 4
```

```
rev(notlar)          # Ters çevirme
```

```
#> [1] 72 90 45 80 65
```

```
cumsum(notlar)       # Kümülatif toplam
```

```
#> [1] 65 145 190 280 352
```

i Not

`sort()` değerleri sıralar; `order()` ise **sıralamayı sağlayacak indeksleri** verir. `order()` özellikle bir veri tablosunu bir sütuna göre sıralarken kullanılır (Bölüm 4).

2.1.3 İndeksleme (Eleman Seçme)

Köşeli parantez [] ile eleman seçilir. R'da indeksleme 1'den başlar.

Yazım	Anlamı
x[2]	2. eleman
x[c(1, 3)]	1. ve 3. elemanlar
x[-1]	1. eleman hariç hepsi
x[x > 5]	Koşulu sağlayanlar
x["isim"]	İsimli vektörlerde isimle seçim

```
x[!is.na(x)] # NA olmayanlar
```

```
#> [1] 10.4 5.6 3.1 6.4 21.7
```

```
(x + 1)[(!is.na(x)) & x > 1]
```

```
#> [1] 11.4 6.6 4.1 7.4 22.7
```

```
x[1:7] # x 5 elemanlı; olmayan elemanlar NA gelir
```

```
#> [1] 10.4 5.6 3.1 6.4 21.7 NA NA
```

```
x[-(1:5)] # İlk 5 eleman hariç -> boş vektör
```

```
#> numeric(0)
```

```
c("x", "y")[rep(c(1, 2, 2, 1), times = 2)]
```

```
#> [1] "x" "y" "y" "x" "x" "y" "y" "x"
```

```
fruit <- c(5, 10, 1, 20)
names(fruit) <- c("orange", "banana", "apple", "peach")
fruit
```

```
#> orange banana apple peach
#>      5      10      1      20
```

```
fruit[c("apple", "orange")]
```

```
#> apple orange
#>      1      5
```

```
w <- c(-3, 4, -1, 7)
w[w < 0] <- -w[w < 0]      # Negatifleri pozitive çevir
w
```

```
#> [1] 3 4 1 7
```

```
which(w > 3)                # Koşulu sağlayan elemanların sırası
```

```
#> [1] 2 4
```

2.2 Matrisler ve Diziler (Array)

- Matris **iki boyutlu**, array ise **iki ya da daha fazla boyutlu** yapıdır. `matrix()` fonksiyonu `nrow / ncol`, `array()` fonksiyonu `dim` argümanı ile kullanılır.
- Matrislerde eleman seçimi [**satır**, **sütun**] şeklindedir. Birini boş bırakmak “hepsi” anlamına gelir.

```
g <- 1:10
dim(g) <- c(5, 2)          # dim() ile vektöre boyut verme
g
```

```
#>      [,1] [,2]
#> [1,]    1    6
#> [2,]    2    7
#> [3,]    3    8
#> [4,]    4    9
#> [5,]    5   10
```

```
g[5, 1]
```

```
#> [1] 5
```

```
x <- array(1:20, dim = c(4, 5))  # 4x5 boyutlu matris
i <- array(c(1:3, 3:1), dim = c(3, 2))  # 3x2 boyutlu indeks matrisi
x[i]                             # i'deki (satır, sütun) gözlerini x'ten çek
```

```
#> [1] 9 6 3
```

```
x[i] <- 0
x
```

```
#>      [,1] [,2] [,3] [,4] [,5]
#> [1,]    1    5    0   13   17
#> [2,]    2    0   10   14   18
#> [3,]    0    7   11   15   19
#> [4,]    4    8   12   16   20
```

```
Z <- matrix(1:12, nrow = 3) # Sütun sütun doldurulur
Z_satir <- matrix(1:12, nrow = 3, byrow = TRUE) # Satır satır doldurma
Z; Z_satir
```

```
#>      [,1] [,2] [,3] [,4]
#> [1,]    1    4    7   10
#> [2,]    2    5    8   11
#> [3,]    3    6    9   12
```

```
#>      [,1] [,2] [,3] [,4]
#> [1,]    1    2    3    4
#> [2,]    5    6    7    8
#> [3,]    9   10   11   12
```

```
Z[2, ] # 2. satır
```

```
#> [1]  2  5  8 11
```

```
Z[, 3] # 3. sütun
```

```
#> [1] 7 8 9
```

Matrisler istatistiksel işlemlerde çok önemli yer tutar (regresyon, kovaryans, özdeğer...):

```
A <- matrix(1:9, nrow = 3)
B <- t(A) # Devrik (transpose)
dim(A); nrow(A); ncol(A)
```

```
#> [1] 3 3
```

```
#> [1] 3
```

```
#> [1] 3
```

```
A * B # Eleman eleman çarpım
```

```
#>      [,1] [,2] [,3]
#> [1,]    1    8   21
#> [2,]    8   25   48
#> [3,]   21   48   81
```

```
A %*% B # Matris çarpımı
```

```
#>      [,1] [,2] [,3]
#> [1,]   66   78   90
#> [2,]   78   93  108
#> [3,]   90  108  126
```

```
diag(A)      # Köşegen elemanlar
```

```
#> [1] 1 5 9
```

```
# Denklem sistemi:  $x + y + z = 3$ ;  $x + 2y + 3z = 0$ ;  $x + 3y + 4z = -2$   
A <- matrix(c(1, 1, 1, 1, 2, 3, 1, 3, 4), nrow = 3); b <- c(3, 0, -2)  
solve(A, b)
```

```
#> [1] 5 -1 -1
```

```
solve(A) %*% b      # A'nın tersi ile b'nin çarpımı -> aynı sonuç
```

```
#>      [,1]  
#> [1,] 5  
#> [2,] -1  
#> [3,] -1
```

```
# Özdeğerler  
eigen(A %*% t(A))$values
```

```
#> [1] 42.45885 0.49341 0.04773
```

Satır ve sütun birleştirme `rbind()` (*row bind*) ve `cbind()` (*column bind*) ile, satır/sütun bazında özet ise `rowSums()`, `colMeans()` gibi fonksiyonlarla yapılır:

```
x1 <- rep(0, 3); x2 <- rep(1, 3)  
cbind(1, x1, x2)
```

```
#>      x1 x2  
#> [1,] 1 0 1  
#> [2,] 1 0 1  
#> [3,] 1 0 1
```

```
rbind(x1, x2)
```

```
#>      [,1] [,2] [,3]  
#> x1      0      0      0  
#> x2      1      1      1
```

```
olcumler <- as.matrix(iris[1:5, 1:4])  
colMeans(olcumler)
```

```
#> Sepal.Length Sepal.Width Petal.Length Petal.Width  
#>          4.86          3.28          1.40          0.20
```

```
rowSums(olcumler)
```

```
#>      1      2      3      4      5  
#> 10.2   9.5   9.4   9.4  10.2
```

2.3 Faktörler: Nominal ve Ordinal Yapılar

İstatistiksel analizde yapılacak analizler, değişkenin **ölçme düzeyine** göre değişir. Kategorik değişkenler R’da **faktör** olarak saklanır:

- **Nominal** (sınıflama): Kategoriler arasında sıralama yoktur (tür, ada, kan grubu).
- **Ordinal** (sıralama): Kategoriler arasında doğal bir sıra vardır (eğitim düzeyi, unvan, memnuniyet).

Faktörün alabileceği kategorilere **düzey** (*level*) denir.

```
cinsiyet <- c("Kadın", "Kadın", "Erkek", "Kadın", "Erkek", "Erkek", "Kadın", "Erkek")  
cinsiyet_faktor <- factor(cinsiyet)  
cinsiyet_faktor
```

```
#> [1] Kadın Kadın Erkek Kadın Erkek Erkek Kadın Erkek  
#> Levels: Erkek Kadın
```

```
levels(cinsiyet_faktor) # Varsayılan sıralama alfabetiktir
```

```
#> [1] "Erkek" "Kadın"
```

```
table(cinsiyet_faktor)
```

```
#> cinsiyet_faktor  
#> Erkek Kadın  
#>      4      4
```

```
as.integer(cinsiyet_faktor) # Arka planda her düzey bir tam sayı ile saklanır
```

```
#> [1] 2 2 1 2 1 1 2 1
```

```
# Ordinal faktör: sırayı levels ile biz veriyoruz  
gorev_unvan <- c("Genel Müdür", "Uzman", "Uzman", "Uzman Yardımcısı",  
                "Danışman", "Danışman", "Uzman", "Uzman Yardımcısı")  
gorev_faktor <- factor(gorev_unvan, ordered = TRUE,  
                      levels = c("Uzman Yardımcısı", "Uzman", "Danışman", "Genel  
                                ↪ Müdür"))  
gorev_faktor
```

```
#> [1] Genel Müdür      Uzman                Uzman                Uzman Yardımcısı
#> [5] Danışman           Danışman             Uzman                Uzman Yardımcısı
#> Levels: Uzman Yardımcısı < Uzman < Danışman < Genel Müdür
```

```
gorev_faktor[1] > gorev_faktor[2] # Ordinal faktörde karşılaştırma yapılabilir
```

```
#> [1] TRUE
```

Penguen verisindeki `species`, `island` ve `sex` değişkenleri birer faktördür:

```
str(penguins$species)
```

```
#> Factor w/ 3 levels "Adelie","Chinstrap",...: 1 1 1 1 1 1 1 1 1 1 ...
```

```
levels(penguins$species)
```

```
#> [1] "Adelie"      "Chinstrap"   "Gentoo"
```

```
table(penguins$species, penguins$island)
```

```
#>
#>           Biscoe Dream Torgersen
#> Adelie      44      56          52
#> Chinstrap    0      68           0
#> Gentoo     124      0           0
```

Faktörlerle çalışırken en çok ihtiyaç duyulan işlemler:

```
tur <- penguins$species
# Düzeylerin sırasını değiştirme (grafiklerde ve modellerde referans düzeyi belirler)
levels(relevel(tur, ref = "Gentoo"))
```

```
#> [1] "Gentoo"      "Adelie"      "Chinstrap"
```

```
# Düzeyleri yeniden adlandırma
levels(factor(tur, labels = c("Adelie", "Çene Kayıslı", "Gentoo")))
```

```
#> [1] "Adelie"      "Çene Kayıslı" "Gentoo"
```

2.3.1 Sürekli Değişkeni Sınıflara Ayırma

`cut()` fonksiyonu sayısal bir değişkeni aralıklara bölerek faktöre çevirir. Örnek veri: Avustralya eyaletlerinden 30 vergi mükellefinin eyalet ve gelir bilgisi.

```

state <- c("tas", "sa", "qld", "nsw", "nsw", "nt", "wa", "wa",
          "qld", "vic", "nsw", "vic", "qld", "qld", "sa", "tas",
          "sa", "nt", "wa", "vic", "qld", "nsw", "nsw", "wa",
          "sa", "act", "nsw", "vic", "vic", "act")
statef <- factor(state)
incomes <- c(60, 49, 40, 61, 64, 60, 59, 54, 62, 69, 70, 42, 56,
            61, 61, 61, 58, 51, 48, 65, 49, 49, 41, 48, 52, 46,
            59, 46, 58, 43)
incomef <- cut(incomes, breaks = 35 + 10*(0:4))
table(incomef)

```

```

#> incomef
#> (35,45] (45,55] (55,65] (65,75]
#>      4      10      14      2

```

```

table(incomef, statef)      # Çapraz tablo

```

```

#>      statef
#> incomef  act nsw nt qld sa tas vic wa
#> (35,45]   1  1  0  1  0  0  1  0
#> (45,55]   1  1  1  1  2  0  1  3
#> (55,65]   0  3  1  3  2  2  2  1
#> (65,75]   0  1  0  0  0  0  1  0

```

```

tapply(incomes, statef, mean)  # Eyaletlere göre ortalama gelir

```

```

#>   act   nsw   nt   qld   sa   tas   vic   wa
#> 44.50 57.33 55.50 53.60 55.00 60.50 56.00 52.25

```

tapply(değişken, grup, fonksiyon) yapısı, ileride dplyr'ın group_by() + summarise() ikilisiyle yapacağımız işin base R karşılığıdır.

2.4 Listeler

Liste; sayı, vektör, matris, veri tablosu, hatta başka listeler gibi **farklı tipteki** nesneleri bir arada tutabilen yapıdır.

```

Lst <- list(isim = "Ahmet", es = "Ayşe", çocuk_sayisi = 3,
            çocuk_yaslari = c(4, 7, 9))
Lst

```

```

#> $isim
#> [1] "Ahmet"
#>
#> $es
#> [1] "Ayşe"
#>

```

```
#> $cocuk_sayisi
#> [1] 3
#>
#> $cocuk_yaslari
#> [1] 4 7 9
```

```
Lst[[4]]
```

```
#> [1] 4 7 9
```

```
Lst[[4]][1]
```

```
#> [1] 4
```

```
Lst$isim
```

```
#> [1] "Ahmet"
```

```
length(Lst)
```

```
#> [1] 4
```

```
Lst$meslek <- "Mühendis"      # Yeni eleman ekleme
str(Lst)
```

```
#> List of 5
#> $ isim      : chr "Ahmet"
#> $ es        : chr "Ayşe"
#> $ cocuk_sayisi : num 3
#> $ cocuk_yaslari: num [1:3] 4 7 9
#> $ meslek      : chr "Mühendis"
```

i [] ile [[]] farkı

`Lst[1]` sonucu yine bir **listedir** (“kutunun içindeki kutu”). `Lst[[1]]` ise o elemanın **kendisini** verir (“kutunun içindeki eşya”). `Lst$isim` ifadesi `Lst[["isim"]]` ile aynıdır.

Listeler neden önemli? Çünkü R’daki hemen her analiz fonksiyonu sonucunu **liste** olarak döndürür. Örneğin bir t-testi (Bölüm 11):

```
test <- t.test(penguenler$body_mass_g, mu = 4000)
class(test)
```

```
#> [1] "htest"
```

```
names(test)
```

```
#> [1] "statistic" "parameter" "p.value" "conf.int" "estimate"  
#> [6] "null.value" "stderr" "alternative" "method" "data.name"
```

```
test$p.value
```

```
#> [1] 3.952e-06
```

```
test$conf.int
```

```
#> [1] 4120 4294  
#> attr(,"conf.level")  
#> [1] 0.95
```

2.5 Veri Çerçevesi (Data Frame)

Data frame, istatistikte en çok kullanılan veri yapısıdır: her **satır bir gözlem**, her **sütun bir değişkendir**. Farklı sütunlar farklı tipte olabilir ama tüm sütunlar aynı uzunluktadır. Teknik olarak data frame, eşit uzunluklu vektörlerden oluşan bir listedir.

```
ogrenci <- data.frame(  
  ad = c("Ali", "Ayşe", "Can", "Deniz", "Ece"),  
  bolum = factor(c("İst", "Akt", "İst", "İst", "Akt")),  
  vize = c(55, 80, 40, 90, 72),  
  final = c(60, 85, 50, 88, 65)  
)  
ogrenci
```

```
#>      ad bolum vize final  
#> 1  Ali   İst  55    60  
#> 2 Ayşe  Akt  80    85  
#> 3  Can   İst  40    50  
#> 4 Deniz İst  90    88  
#> 5  Ece  Akt  72    65
```

```
str(ogrenci)
```

```
#> 'data.frame': 5 obs. of 4 variables:  
#> $ ad : chr "Ali" "Ayşe" "Can" "Deniz" ...  
#> $ bolum: Factor w/ 2 levels "Akt","İst": 2 1 2 2 1  
#> $ vize : num 55 80 40 90 72  
#> $ final: num 60 85 50 88 65
```

Hazır bir veri setini incelerken ilk bakılacak fonksiyonlar:

```
dim(iris)           # Satır ve sütun sayısı
```

```
#> [1] 150    5
```

```
names(iris)         # Sütun adları
```

```
#> [1] "Sepal.Length" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"
```

```
head(iris, 3)       # İlk 3 satır
```

```
#>   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
#> 1         5.1         3.5         1.4         0.2   setosa
#> 2         4.9         3.0         1.4         0.2   setosa
#> 3         4.7         3.2         1.3         0.2   setosa
```

```
str(iris)           # Yapı: sütun adları ve tipleri
```

```
#> 'data.frame':   150 obs. of  5 variables:
#> $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
#> $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
#> $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
#> $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
#> $ Species     : Factor w/ 3 levels "setosa","versicolor",...: 1 1 1 1 1 1 1 1 1 1 ...
```

```
summary(iris)       # Her sütunun özeti
```

```
#>   Sepal.Length   Sepal.Width   Petal.Length   Petal.Width   Species
#> Min.   :4.30   Min.   :2.00   Min.   :1.00   Min.   :0.1   setosa   :50
#> 1st Qu.:5.10   1st Qu.:2.80   1st Qu.:1.60   1st Qu.:0.3   versicolor:50
#> Median :5.80   Median :3.00   Median :4.35   Median :1.3   virginica :50
#> Mean   :5.84   Mean   :3.06   Mean   :3.76   Mean   :1.2
#> 3rd Qu.:6.40   3rd Qu.:3.30   3rd Qu.:5.10   3rd Qu.:1.8
#> Max.   :7.90   Max.   :4.40   Max.   :6.90   Max.   :2.5
```

Sütun ve satır seçimi, matrislerdeki gibi [satır, sütun] ya da \$ ile yapılır:

```
ogrenci$vize        # Bir sütun (vektör olarak)
```

```
#> [1] 55 80 40 90 72
```

```
ogrenci[, "final"]  # Aynı işlem
```

```
#> [1] 60 85 50 88 65
```

```
ogrenci[2, ] # 2. satır
```

```
#>      ad bolum vize final  
#> 2 Ayşe Akt 80 85
```

```
ogrenci[ogrenci$vize > 60, ] # Vizesi 60'tan büyük olanlar
```

```
#>      ad bolum vize final  
#> 2 Ayşe Akt 80 85  
#> 4 Deniz İst 90 88  
#> 5 Ece Akt 72 65
```

```
ogrenci[ogrenci$bolum == "İst", c("ad", "vize")]
```

```
#>      ad vize  
#> 1 Ali 55  
#> 3 Can 40  
#> 4 Deniz 90
```

```
ogrenci$ortalama <- 0.4*ogrenci$vize + 0.6*ogrenci$final # Yeni sütun  
ogrenci
```

```
#>      ad bolum vize final ortalama  
#> 1 Ali İst 55 60 58.0  
#> 2 Ayşe Akt 80 85 83.0  
#> 3 Can İst 40 50 46.0  
#> 4 Deniz İst 90 88 88.8  
#> 5 Ece Akt 72 65 67.8
```

2.5.1 Tibble

`readr` ile okunan ya da `tidyverse` fonksiyonlarıyla üretilen veriler **tibble** olarak gelir. Tibble modern bir data frame'dir: konsolda yalnızca ilk satırları ve ekrana sığan sütunları gösterir, her sütunun tipini yazar. `penguins` da bir tibble'dır:

```
penguins
```

```
#> # A tibble: 344 x 8  
#>   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g  
#>   <fct>   <fct>         <dbl>         <dbl>             <int>      <int>  
#> 1 Adelie Torgersen     39.1           18.7             181       3750  
#> 2 Adelie Torgersen     39.5           17.4             186       3800  
#> 3 Adelie Torgersen     40.3            18             195       3250  
#> 4 Adelie Torgersen     NA              NA              NA         NA  
#> 5 Adelie Torgersen     36.7           19.3             193       3450  
#> 6 Adelie Torgersen     39.3           20.6             190       3650  
#> # i 338 more rows  
#> # i 2 more variables: sex <fct>, year <int>
```

Data frame ile yapılan her işlem tibble ile de yapılabilir; `as.data.frame()` ve `tibble::as_tibble()` ile birbirine çevrilebilirler.

2.5.2 `attach()`, `detach()` ve `with()`

`attach()` bir data frame'in sütunlarını \$ yazmadan, doğrudan isimleriyle çağırmanı sağlar; `detach()` bu kullanımı kaldırır. Eski ders notlarında sık kullanılır:

```
attach(iris)
mean(Sepal.Length)
```

```
#> [1] 5.843
```

```
detach(iris)
```

Uyarı

`attach()` pratik görünse de aynı isimli nesneler birbirini örttüğünde karışıklığa yol açar; `attach` edildikten **sonra** veriye eklenen sütunlar görünmez ve `detach()` unutulunca hatalar çıkar. Bunun yerine **`with()`** fonksiyonu ya da Bölüm 4'de göreceğimiz **`dplyr`** fonksiyonları önerilir.

```
with(iris, mean(Sepal.Length))
```

```
#> [1] 5.843
```

```
with(iris, tapply(Petal.Length, Species, mean))
```

```
#>      setosa versicolor virginica
#>      1.462      4.260      5.552
```

Kendini Sına

 1. 10'dan 100'e kadar 5'er artan bir vektör oluşturun. Kaç elemanı vardır? 50'den büyük olanlar kaç tanedir?

```
v <- seq(10, 100, by = 5)
length(v)      # 19
sum(v > 50)    # 10
```

 2. `iris` verisinde taç yaprak uzunluğu (`Petal.Length`) 5'ten büyük olan çiçeklerin türlerinin frekans tablosunu çıkarın.

```
table(iris$Species[iris$Petal.Length > 5])
```

Hiçbir *setosa* çiçeğinin taç yaprağı 5 cm'den uzun değildir.

💡 3. “düşük”, “orta”, “yüksek” düzeylerine sahip ordinal bir faktör oluşturup `table()` ile frekanslarını bulun.

```
f <- factor(c("orta", "düşük", "yüksek", "orta", "orta"),  
            levels = c("düşük", "orta", "yüksek"), ordered = TRUE)  
table(f)
```

`levels` verilmeseydi düzeyler alfabetik sıralanırdı (“düşük”, “orta”, “yüksek” burada tesadüfen doğru sıradadır; “az”, “çok”, “orta” olsaydı yanlış olurdu).

💡 4. `penguins` verisinde kaç gözlem ve kaç değişken var? Hangi değişkenler faktördür? Kaç eksik değer var?

```
dim(penguins)           # 344 satır, 8 sütun  
sapply(penguins, is.factor) # species, island, sex  
sum(is.na(penguins))    # 19  
colSums(is.na(penguins)) # En çok eksik: sex (11)
```

💡 5. `matrix(1:6, nrow = 2)` ile `matrix(1:6, nrow = 2, byrow = TRUE)` arasındaki fark nedir?

İlki değerleri **sütun sütun** (1,2 / 3,4 / 5,6), ikincisi **satır satır** (1,2,3 / 4,5,6) yerleştirir.

3 Raporlama: Quarto ve R Markdown

i Bu haftanın hedefleri

- Tekrarlanabilir raporlamanın neden gerekli olduğunu açıklamak,
- Bir Quarto belgesinin üç parçasını (YAML, Markdown, kod parçası) tanımak,
- Kod parçası seçenekleriyle neyin görünüp neyin gizleneceğini kontrol etmek,
- Satır içi kod, uyarı kutuları, çapraz referans ve sekmeleri kullanmak,
- Bir ödev raporunu HTML, Word ya da PDF olarak derlemek.

Şu an okuduğunuz bu kitap da bir **Quarto kitabıdır**.

3.1 Neden Tekrarlanabilir Rapor?

Klasik yöntemde analiz R’da yapılır, sonuçlar ve grafikler kopyalanıp Word’e yapıştırılır. Veri değiştiğinde **her şeyi baştan kopyalamak** gerekir ve hata yapmak çok kolaydır.

Quarto / R Markdown ile **kod + sonuç + yorum** aynı dosyada bulunur. Veri değişirse belgeyi yeniden derlemek (*render*) yeterlidir; tüm tablolar, grafikler ve metin içindeki sayılar kendiliğinden güncellenir. Buna **tekrarlanabilir araştırma** (*reproducible research*) denir. Altı ay sonra “Bu sayıyı nasıl bulmuştum?” sorusunun cevabı her zaman belgenin içindedir.

3.2 R Markdown ve Quarto

- **R Markdown** (.Rmd): 2014’ten beri kullanılan, rmarkdown paketi tabanlı raporlama sistemi.
- **Quarto** (.qmd): Aynı ekibin (Posit) geliştirdiği **yeni nesil** sistem. R’ın yanında Python ve Julia’yı da destekler; ayrı bir program olarak kurulur ama **RStudio’nun güncel sürümleriyle birlikte gelir**.

İkisinin arka planda çalışma mantığı aynıdır:

```
.qmd / .Rmd --(knitr: kodları çalıştırır)--> .md --(Pandoc: dönüştürür)--> HTML /  
↪ PDF / Word / sunum / kitap
```

Tablo 3.1: R Markdown ve Quarto karşılaştırması

Özellik	R Markdown (.Rmd)	Quarto (.qmd)
Dosya uzantısı	.Rmd	.qmd
Çıktı türü (YAML)	output: html_document	format: html
Parça seçenekleri	{r, echo=FALSE} başlıkta	# echo: false parçanın içinde
Derleme	Knit düğmesi	Render düğmesi

Özellik	R Markdown (.Rmd)	Quarto (.qmd)
Diller	Esas olarak R	R, Python, Julia, Observable
Uyarı kutuları, çapraz referans	Ek paket gerekir	Yerleşik
Kitap, web sitesi, sunum	bookdown, blogdown, xaringan	Tek sistemde yerleşik

Not

Eski .Rmd dosyalarınız Quarto ile de derlenebilir (`quarto render dosya.Rmd`). Bu nedenle R Markdown'ı öğrenen biri Quarto'ya çok kolay geçer. Derste R Markdown notları verildiğinde Quarto'nun farklı bir şey olduğunu düşünmeye gerek yok. Bu derste **Quarto** kullanacağız.

3.3 Yeni Bir Belge Oluşturma

1. **File** → **New File** → **Quarto Document...** (R Markdown için: *R Markdown...*)
2. Başlık ve yazar adını girin, çıktı türü olarak **HTML** seçin.
3. Dosyayı proje klasörüne kaydedin (ör. `hafta3.qmd`).
4. **Render** düğmesine (ya da **Ctrl/Cmd + Shift + K**) basın. Çıktı Viewer'da ya da tarayıcıda açılır.

RStudio'da **Source** (kaynak kod) ve **Visual** (Word benzeri görsel editör) olmak üzere iki düzenleme modu vardır. Visual modda tablo, resim ve başlık eklemek menülerle yapılabilir; ama arka planda yazılan hâlâ düz metindir.

3.4 Bir Quarto Belgesinin Üç Parçası

3.4.1 1) YAML Başlığı

Belgenin en üstünde, `---` çizgileri arasında yer alır ve belgenin **ayarlarını** içerir. YAML'da **girinti (boşluk) önemlidir**; sekme (Tab) karakteri kullanılmaz.

```
---
title: "Hafta 3 Raporu"
author: "Adınız Soyadınız"
date: today
lang: tr
format:
  html:
    toc: true          # İçindekiler tablosu
    number-sections: true
    code-fold: true    # Kodları katlanabilir yap
    embed-resources: true # Tek bir .html dosyası üret (e-posta ile gönderilebilir)
docx: default         # Word çıktısı
pdf: default          # PDF çıktısı (LaTeX gerekir)
execute:
  echo: true          # Kodları göster
```

```
warning: false          # Uyarıları gösterme
---
```

Aynı başlığın **R Markdown** karşılığı:

```
---
title: "Hafta 3 Raporu"
author: "Adınız Soyadınız"
date: "2026-10-05"
output:
  html_document:
    toc: true
    number_sections: true
  word_document: default
---
```

i Not

PDF çıktısı için bilgisayarda bir LaTeX dağıtımı olmalıdır. En kolay yol: terminalde bir kez `quarto install tinytex`.

3.4.2 2) Markdown ile Metin Biçimlendirme

Markdown, düz metne basit işaretler ekleyerek biçimlendirme yapan bir yazım dilidir.

Tablo 3.2: Temel Markdown yazımı

Yazdığınız	Görünen
# Başlık 1, ## Başlık 2, ### Başlık 3	Başlık düzeyleri
kalın	kalın
italik	<i>italik</i>
`kod`	kod
* madde ya da - madde	Madde işaretli liste
1. madde	Numaralı liste
[Quarto](https://quarto.org)	Quarto
![Açıklama](images/r1.jpg)	Resim ekleme
$\bar{x} = \frac{1}{n} \sum x_i$	$\bar{x} = \frac{1}{n} \sum x_i$
\$\$... \$\$	Ortalanmış ayrı satırda formül

Tablo yazımı:

```
| Değişken | Tip |
|:-----:|:-----:|
| yas      | sayısal |
| cinsiyet | faktör  |
```

İstatistik raporlarında sık kullanılan formüller:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

! Önemli

Başlık işaretinden (#) sonra **bir boşluk** bırakılmalı; başlıkların, listelerin ve kod parçalarının önünde **bir boş satır** olmalıdır. Aksi hâlde biçimlendirme çalışmaz.

3.4.3 3) Kod Parçaları (Code Chunks)

Kod parçaları ```{r}` ile başlar, ````` ile biter. RStudio'da **Ctrl + Alt + I** (Mac: **Cmd + Option + I**) kısayolu ile eklenir. Parça içindeki kod belge derlenirken çalıştırılır ve sonucu belgeye eklenir.

Quarto yazımı (seçenekler `#|` ile parçanın içine yazılır):

```
``{r}
#| label: fig-penguen
#| echo: false
#| fig-cap: "Penguenlerin yüzgeç uzunluğu ve ağırlığı"
plot(penguins$flipper_length_mm, penguins$body_mass_g)
```
```

**R Markdown yazımı** (seçenekler başlığa yazılır):

```
``{r fig-penguen, echo=FALSE, fig.cap="Penguenlerin yüzgeç uzunluğu ve ağırlığı"}
plot(penguins$flipper_length_mm, penguins$body_mass_g)
```
```

En sık kullanılan parça seçenekleri:

Tablo 3.3: Kod parçası seçenekleri

Seçenek (Quarto)	R Markdown	Etkisi
<code>echo: false</code>	<code>echo=FALSE</code>	Kodu gizle, sonucu göster
<code>eval: false</code>	<code>eval=FALSE</code>	Kodu göster ama çalıştırma
<code>include: false</code>	<code>include=FALSE</code>	Kodu çalıştır, ama ne kodu ne sonucu göster (paket yükleme için ideal)
<code>output: false</code>	<code>results="hide"</code>	Kodu göster, çıktıyı gizle
<code>warning: false</code>	<code>warning=FALSE</code>	Uyarıları gizle
<code>message: false</code>	<code>message=FALSE</code>	Mesajları gizle

Seenek (Quarto)	R Markdown	Etkisi
<code>error: true</code>	<code>error=TRUE</code>	Hata olsa bile derlemeyi durdurma (hatayı gster)
<code>fig-width: 6</code>	<code>fig.width=6</code>	Grafik geniřlięi (in)
<code>fig-cap: "..."</code> <code>label: ...</code>	<code>fig.cap="..."</code> {r etiket}	Grafik bařlıęı Paranın adı (Trke karakter ve bořluk kullanmayın)

Tm belge iin geerli varsayılanlar YAML'daki `execute:` altında (R Markdown'da `knitr::opts_chunk$set()` ile) verilir.

3.5 Satır İi (Inline) Kod

Metnin iine hesaplanmış bir deęer yazmak iin tek ters tırnak iinde `r` kullanılır:

```
Veride 333 penguen vardır ve ortalama aęırlık
4207 gramdır.
```

Derlendięinde řyle grnr: Veride 333 penguen vardır ve ortalama aęırlık 4207 gramdır. Veri deęiřirse bu sayılar da kendilięinden deęiřir; **rapora elle sayı yazmayın**.

3.6 Tablolar ve Grafikler

Bir data frame'i dzgn bir tablo olarak gstermek iin `knitr::kable()` kullanılır:

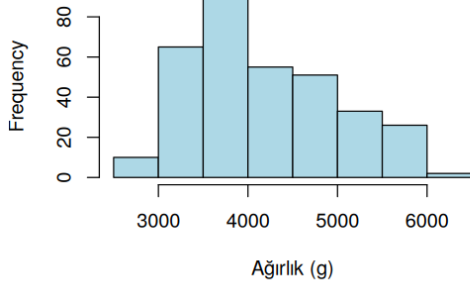
```
penguenler |>
  group_by(species) |>
  summarise(n = n(), ort_agirlik = mean(body_mass_g)) |>
  knitr::kable(digits = 1, col.names = c("Tr", "n", "Ortalama aęırlık (g)"))
```

Tablo 3.4: Trlere gre penguen sayısı ve ortalama aęırlık

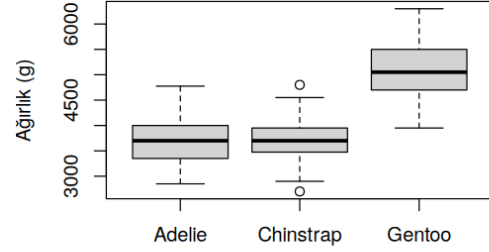
Tr	n	Ortalama aęırlık (g)
Adelie	146	3706
Chinstrap	68	3733
Gentoo	119	5092

Birden fazla grafięi yan yana koymak iin `layout-ncol` seeneęi kullanılır:

```
hist(penguenler$body_mass_g, main = "", xlab = "Ağırlık (g)", col = "lightblue")
boxplot(body_mass_g ~ species, data = penguenler, xlab = "", ylab = "Ağırlık (g)")
```



(a) Histogram



(b) Kutu grafiği

Şekil 3.1: Aynı kod parçasından iki grafik

3.7 Quarto’ya Özgü Kullanışlı Özellikler

Çapraz referans: Grafiğe `fig-`, tabloya `tbl-`, bölüme `sec-` ile başlayan bir etiket verip metinde `@fig-etiket` yazdığınızda numara otomatik atanır. Örneğin `@tbl-ozet` yazınca Tablo 3.4, `@fig-yanyana` yazınca Şekil 3.1, `@sec-veri-yapilari` yazınca Bölüm 2 görünür.

Uyarı kutuları (callout):

```
::: {.callout-tip}
## İpucu
Kodlarınızı sık sık çalıştırarak ilerleyin.
:::
```

Kutu türleri: `callout-note`, `callout-tip`, `callout-warning`, `callout-important`, `callout-caution`. `collapse="true"` eklenirse kutu kapalı başlar (bu kitaptaki “Kendini Sına” soruları böyle hazırlanmıştır).

Sekmeler (tabset):

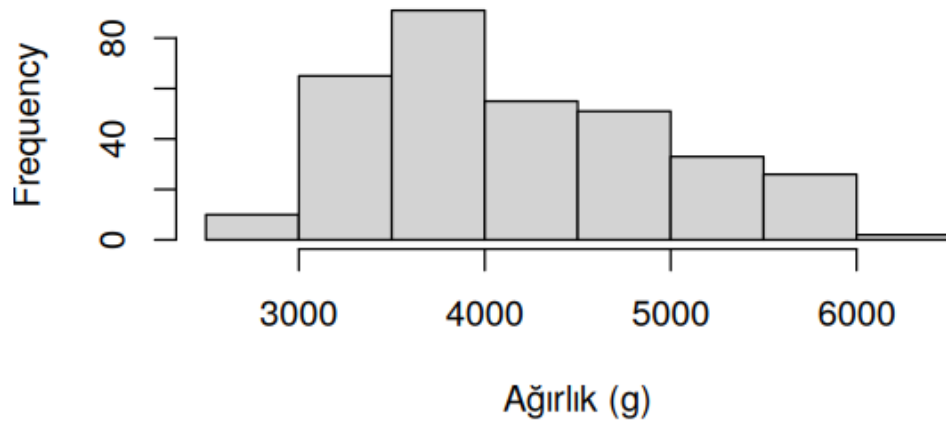
3.8 Özet

```
summary(penguenler$body_mass_g)
```

```
#>   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>  2700   3550   4050   4207   4775   6300
```

3.9 Histogram

```
hist(penguenler$body_mass_g, main = "", xlab = "Ağırlık (g)", col = "lightgray")
```



```
 ::: {.panel-tabset}
## Özet
...
## Histogram
...
:::
```

3.10 Ödev Raporu Şablonu

Bu dersteki ödevler aşağıdaki iskelete göre hazırlanabilir:

```
---
title: "İST347 Ödev 1"
author: "Ad Soyad (Öğrenci No)"
date: today
lang: tr
format:
  html:
    toc: true
    embed-resources: true
---

```${r}
#| include: false
library(tidyverse)
library(palmerpenguins)
```

```

```

## Amaç
Bu raporda ... sorusunu inceliyorum.

## Veri
Veri seti 344 gözlem içermektedir. ...

## Analiz
```{r}
kod
```

## Yorum
Sonuçlar ... göstermektedir.

## Kullanılan araçlar
Kod yazımında ... aracından yararlanılmıştır.

```

3.11 Derleme ve Sık Yapılan Hatalar

- Belge derlenirken **temiz, yeni bir R oturumu** açılır. Konsolda tanımladığınız ama belgeye yazmadığınız nesneler belgede **yoktur**. “object ‘x’ not found” hatasının en yaygın nedeni budur.
- Paketler belgenin içinde `library()` ile yüklenmelidir. `install.packages()` ise belgeye **yazıl-mamalıdır**.
- Veri dosyaları **görelî yol** ile okunmalıdır (Bölüm 1.3).
- Aynı etiket (`label`) iki parçaya verilemez.
- Etkileşimli grafikler (`plotly`, Bölüm 6) yalnızca **HTML** çıktısında çalışır.

3.12 Quarto Kitabı

Birden çok `.qmd` dosyasını bir **kitap** ya da **web sitesi** hâline getirmek için bir proje klasörüne `_quarto.yml` dosyası eklenir. Bu kitabın ayar dosyasının bir kısmı:

```

project:
  type: book
book:
  title: "İST347 İstatistiksel Analiz Uygulamaları"
  chapters:
    - index.qmd
    - 01-r-rstudio.qmd
    - 02-veri-yapilari.qmd
format:
  html:
    theme: cosmo

```

Terminalde `quarto render` komutu bütün bölümleri derler ve `_book/` klasörüne bir web sitesi üretir.

Kendini Sına

💡 1. Bir kod parçasının kodunu gizleyip yalnızca grafiğini göstermek için hangi seçenek kullanılır?

`#| echo: false`. Kodun hiç çalışmaması için `#| eval: false`, çıktının gizlenmesi için `#| output: false`, hem kodun hem çıktının gizlenip kodun yine de çalışması için `#| include: false` kullanılır.

💡 2. Konsolda `veri <- read.csv("veri.csv")` çalıştırdınız, belgede ise yalnızca `summary(veri)` var. Render ederken “object ‘veri’ not found” hatası alıyorsunuz. Neden?

Quarto belgeyi **yeni ve boş** bir R oturumunda derler; konsoldaki nesneleri görmez. Veriyi okuyan kod da belgenin içine yazılmalıdır.

💡 3. `mtcars` verisindeki arabaların ortalama yakıt tüketimini (`mpg`) metnin içine satır içi kod ile yazın.

```
Arabaların ortalama yakıt tüketimi 20.1 mil/galondur.
```

💡 4. YAML’da `format: html` altına yazdığınız `toc: true` çalışmıyor. Muhtemel neden?

YAML’da girinti hatası. `toc: true`, `html:` satırının altına **iki boşluk daha içeriden** yazılmalı ve `format:` satırı `format:` (yalnız) olmalıdır:

```
format:
  html:
    toc: true
```

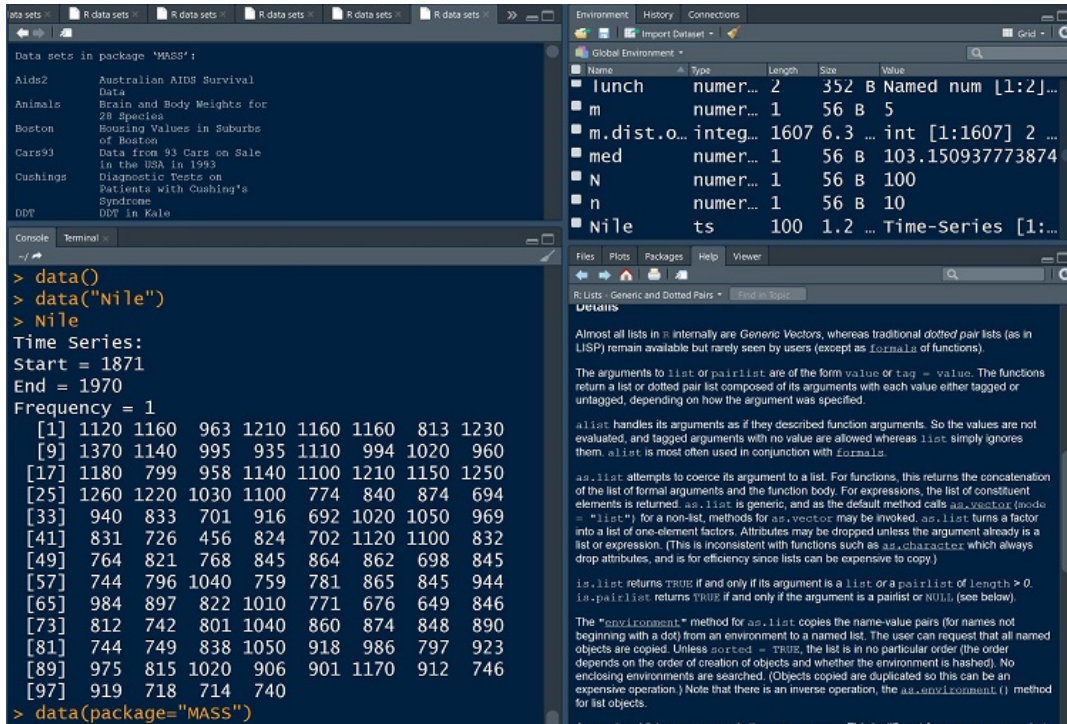
4 Veri Okuma/Yazma ve Veri Düzenleme

i Bu haftanın hedefleri

- R'daki hazır veri setlerine ulaşmak; txt, csv ve Excel dosyalarını okumak ve yazmak,
- Bir dosyayı okumadan önce **ayırıcı, başlık satırı ve ondalık işareti** kontrol etmek,
- base R ile satır/sütun seçmek ve sıralamak,
- dplyr ile **seçme, süzme, sıralama, yeni değişken türetme ve gruplara göre özetleme** yapmak,
- Pipe (`|>`) ile işlemleri okunabilir bir zincir hâline getirmek,
- Gerçek ("kirli") bir veriyi analize hazırlamak, veriyi uzun/geniş biçime çevirmek ve tabloları birleştirmek.

4.1 Hazır Veri Setleri

R ve kurulu paketlerin içindeki veri kümelerine `data()` fonksiyonu ile erişilebilir.



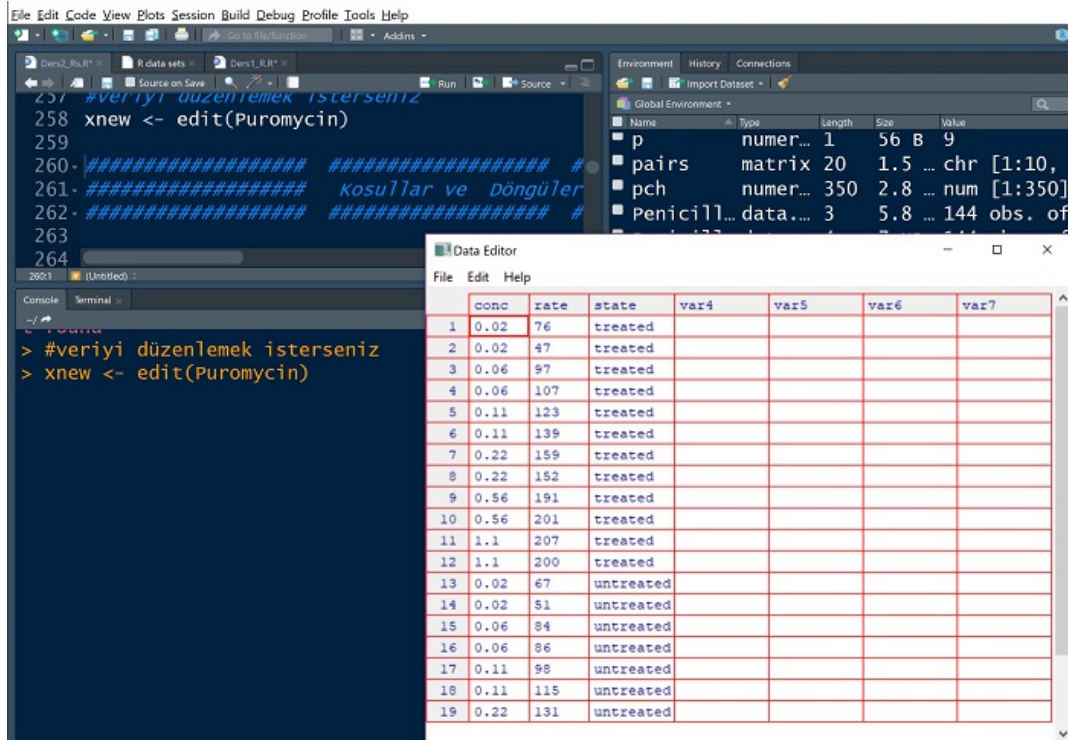
Şekil 4.1: `data()` ile hazır veri setlerini listeleme

```
data() # Tüm hazır veri setlerini listeler (ayrı pencerede)
data(package = "MASS") # Belli bir paketin içindeki verileri listeler
?penguins # Veri setinin yardım sayfası: değişkenlerin anlamı
```

```
data(Puromycin, package = "datasets")
head(Puromycin, 3)
```

```
#>   conc rate  state
#> 1 0.02   76 treated
#> 2 0.02   47 treated
#> 3 0.06   97 treated
```

Veriyi elle düzenlemek için `edit()` fonksiyonu kullanılabilir. Ancak yapılan değişiklikler kodda iz bırakmaz, yani **tekrarlanabilir değildir**; gerçek çalışmalarda düzeltmeleri kodla yapmak gerekir.



Şekil 4.2: `edit()` ile açılan veri düzenleme penceresi

4.2 Dosyadan Veri Okuma

Tablo 4.1: Dosya türlerine göre okuma fonksiyonları

| Dosya türü | Base R | readr / readxl |
|--|---|-----------------------------------|
| Sekme ile ayrılmış (.txt, .tsv) | <code>read.delim()</code> /
<code>read.table(header = TRUE)</code> | <code>read_tsv()</code> |
| Virgül ile ayrılmış (.csv) | <code>read.csv()</code> | <code>read_csv()</code> |
| Noktalı virgül ile ayrılmış (Türkçe Excel'in csv'si) | <code>read.csv2()</code> | <code>read_csv2()</code> |
| Excel (.xlsx) | — | <code>readxl::read_excel()</code> |
| R nesnesi (.rds) | <code>readRDS()</code> | <code>read_rds()</code> |

| Dosya türü | Base R | readr / readxl |
|--------------------|--------|--|
| SPSS / Stata / SAS | — | haven::read_sav(), read_dta(),
read_sas() |

Okumadan önce dosyayı bir metin editöründe açıp **ayırıcı karakterin** ne olduğuna (virgül, noktalı virgül, sekme), ilk satırda **değişken adları** olup olmadığına ve **ondalık ayırıcıya** (nokta mı virgül mü) bakmak gerekir.

4.2.1 Örnek 1: worms.txt

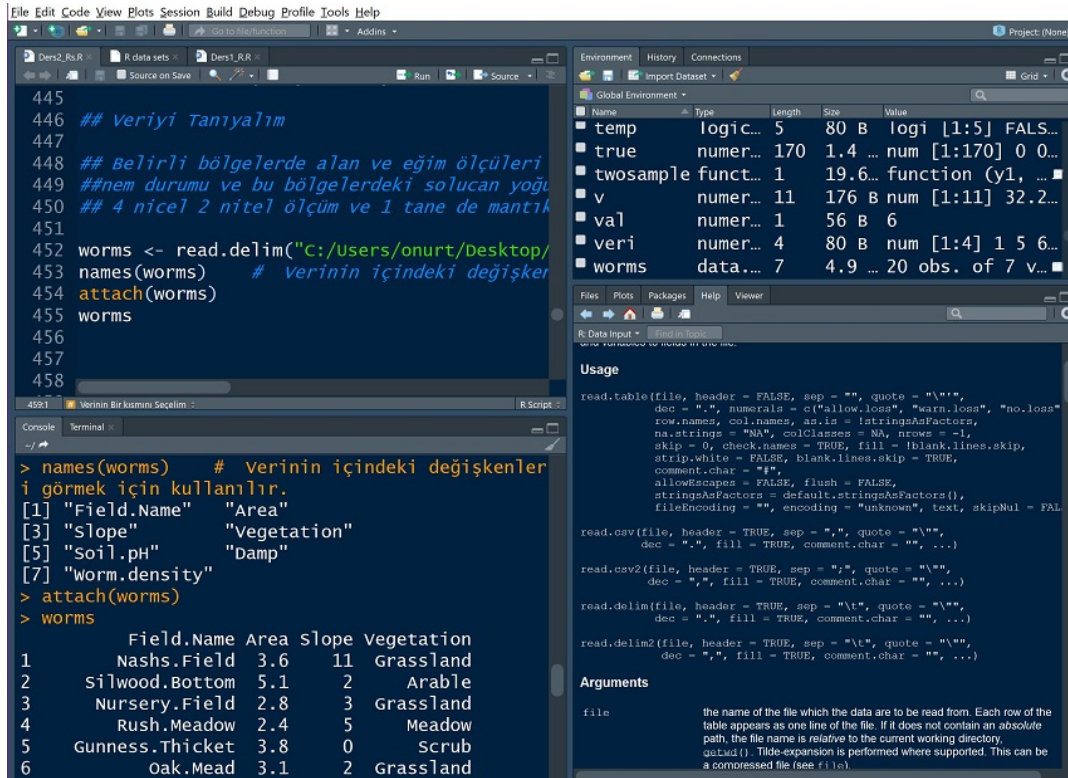
worms.txt, 20 tarladaki solucan yoğunluğunu ve tarlaların alan, eğim, bitki örtüsü, toprak pH'ı ve nemlilik bilgilerini içeren küçük bir veri setidir (Crawley, *The R Book*). Değerler **sekme** ile ayrılmıştır. Veride 4 nicel, 2 nitel ve 1 mantıksal değişken vardır.

```
worms <- read.delim("veri/worms.txt")      # Göreli yol!
# Alternatif: read.table("veri/worms.txt", header = TRUE)
str(worms)
```

```
#> 'data.frame':    20 obs. of  7 variables:
#> $ Field.Name : chr "Nashs.Field" "Silwood.Bottom" "Nursery.Field" "Rush.Meadow" ...
#> $ Area       : num  3.6 5.1 2.8 2.4 3.8 3.1 3.5 2.1 1.9 1.5 ...
#> $ Slope      : int   11 2 3 5 0 2 3 0 0 4 ...
#> $ Vegetation : chr "Grassland" "Arable" "Grassland" "Meadow" ...
#> $ Soil.pH    : num  4.1 5.2 4.3 4.9 4.2 3.9 4.2 4.8 5.7 5 ...
#> $ Damp       : logi FALSE FALSE FALSE TRUE FALSE FALSE ...
#> $ Worm.density: int   4 7 2 5 6 2 3 4 9 7 ...
```

worms

```
#>      Field.Name Area Slope Vegetation Soil.pH Damp Worm.density
#> 1    Nashs.Field  3.6    11  Grassland   4.1 FALSE         4
#> 2   Silwood.Bottom  5.1     2   Arable    5.2 FALSE         7
#> 3   Nursery.Field  2.8     3  Grassland   4.3 FALSE         2
#> 4    Rush.Meadow  2.4     5   Meadow    4.9  TRUE         5
#> 5  Gunness.Thicket  3.8     0    Scrub    4.2 FALSE         6
#> 6      Oak.Mead    3.1     2  Grassland   3.9 FALSE         2
#> 7   Church.Field  3.5     3  Grassland   4.2 FALSE         3
#> 8      Ashurst    2.1     0   Arable    4.8 FALSE         4
#> 9   The.Orchard   1.9     0   Orchard   5.7 FALSE         9
#> 10  Rookery.Slope  1.5     4  Grassland   5.0  TRUE         7
#> 11   Garden.Wood  2.9    10    Scrub    5.2 FALSE         8
#> 12  North.Gravel  3.3     1  Grassland   4.1 FALSE         1
#> 13  South.Gravel  3.7     2  Grassland   4.0 FALSE         2
#> 14 Observatory.Ridge 1.8     6  Grassland   3.8 FALSE         0
#> 15    Pond.Field  4.1     0   Meadow    5.0  TRUE         6
#> 16   Water.Meadow  3.9     0   Meadow    4.9  TRUE         8
#> 17    Cheapside   2.2     8    Scrub    4.7  TRUE         4
#> 18    Pound.Hill  4.4     2   Arable    4.5 FALSE         5
#> 19   Gravel.Pit   2.9     1  Grassland   3.5 FALSE         1
#> 20    Farm.Wood   0.8    10    Scrub    5.1  TRUE         3
```



Şekil 4.3: read.delim() ile worms verisini okuma

4.2.2 Örnek 2: dataset_ex.csv (müşteri kişilik analizi verisi)

Bu veri, bir şirketin 2240 müşterisinin demografik bilgilerini (doğum yılı, eğitim, medeni durum, gelir, evdeki çocuk sayısı) ve son iki yıldaki ürün gruplarına göre harcamalarını (Mnt... sütunları) içerir.

Uzantısı .csv olmasına rağmen dosya aslında **sekme** ile ayrılmıştır. read.csv() ile okursak tüm satır tek bir sütuna yığılır:

```
yanlis <- read.csv("veri/dataset_ex.csv")
dim(yanlis)      # 29 sütun olmalıydı, 1 sütun geldi!
```

```
#> [1] 2240      1
```

Doğru ayıracıyı belirtince sorun çözülür:

```
musteri <- read_tsv("veri/dataset_ex.csv", show_col_types = FALSE)
# Base R karşılığı: read.delim("veri/dataset_ex.csv")
dim(musteri)
```

```
#> [1] 2240     29
```

```
glimpse(musteri)      # dplyr'ın str() benzeri özet fonksiyonu
```

```
#> Rows: 2,240
#> Columns: 29
#> $ ID <dbl> 5524, 2174, 4141, 6182, 5324, 7446, 965, 6177, 485~
#> $ Year_Birth <dbl> 1957, 1954, 1965, 1984, 1981, 1967, 1971, 1985, 19~
#> $ Education <chr> "Graduation", "Graduation", "Graduation", "Graduat~
#> $ Marital_Status <chr> "Single", "Single", "Together", "Together", "Marri~
#> $ Income <dbl> 58138, 46344, 71613, 26646, 58293, 62513, 55635, 3~
#> $ Kidhome <dbl> 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1,~
#> $ Teenhome <dbl> 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1,~
#> $ Dt_Customer <chr> "04-09-2012", "08-03-2014", "21-08-2013", "10-02-2~
#> $ Recency <dbl> 58, 38, 26, 26, 94, 16, 34, 32, 19, 68, 11, 59, 82~
#> $ MntWines <dbl> 635, 11, 426, 11, 173, 520, 235, 76, 14, 28, 5, 6,~
#> $ MntFruits <dbl> 88, 1, 49, 4, 43, 42, 65, 10, 0, 0, 5, 16, 61, 2, ~
#> $ MntMeatProducts <dbl> 546, 6, 127, 20, 118, 98, 164, 56, 24, 6, 6, 11, 4~
#> $ MntFishProducts <dbl> 172, 2, 111, 10, 46, 0, 50, 3, 3, 1, 0, 11, 225, 3~
#> $ MntSweetProducts <dbl> 88, 1, 21, 3, 27, 42, 49, 1, 3, 1, 2, 1, 112, 5, 1~
#> $ MntGoldProds <dbl> 88, 6, 42, 5, 15, 14, 27, 23, 2, 13, 1, 16, 30, 14~
#> $ NumDealsPurchases <dbl> 3, 2, 1, 2, 5, 2, 4, 2, 1, 1, 1, 1, 1, 3, 1, 1, 3,~
#> $ NumWebPurchases <dbl> 8, 1, 8, 2, 5, 6, 7, 4, 3, 1, 1, 2, 3, 6, 1, 7, 3,~
#> $ NumCatalogPurchases <dbl> 10, 1, 2, 0, 3, 4, 3, 0, 0, 0, 0, 0, 4, 1, 0, 6, 0~
#> $ NumStorePurchases <dbl> 4, 2, 10, 4, 6, 10, 7, 4, 2, 0, 2, 3, 8, 5, 3, 12,~
#> $ NumWebVisitsMonth <dbl> 7, 5, 4, 6, 5, 6, 6, 8, 9, 20, 7, 8, 2, 6, 8, 3, 8~
#> $ AcceptedCmp3 <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,~
#> $ AcceptedCmp4 <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
#> $ AcceptedCmp5 <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,~
#> $ AcceptedCmp1 <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,~
#> $ AcceptedCmp2 <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
#> $ Complain <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
#> $ Z_CostContact <dbl> 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3,~
#> $ Z_Revenue <dbl> 11, 11, 11, 11, 11, 11, 11, 11, 11, 11, 11, 11, 11, 11, 11,~
#> $ Response <dbl> 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0,~
```

💡 Okuduktan sonra her zaman kontrol edin

1. `dim()`: Satır ve sütun sayısı beklediğiniz gibi mi?
2. `str()` / `glimpse()`: Sayısal olması gereken sütunlar sayısal mı?
3. `summary()`: Mantıksız değerler (negatif yaş, 999 gibi kodlar) ya da eksik değerler var mı?

4.3 Dosyaya Veri Yazma

```
write.csv(worms, "veri/worms_kopya.csv", row.names = FALSE) # csv (base R)
write_csv(worms, "veri/worms_kopya.csv") # csv (readr)
writexl::write_xlsx(worms, "veri/worms_kopya.xlsx") # Excel
saveRDS(worms, "veri/worms.rds") # R nesnesi (tipleri
  ↳ korur)
worms2 <- readRDS("veri/worms.rds")
```

4.4 base R ile Seçme ve Sıralama

Eski ders notlarımızdaki worms alıştırmalarıyla başlayalım. Köşeli parantezde **virgülün solu satırları, sağ sütunları** seçer.

```
worms1 <- worms[, 1:3]           # İlk üç değişken
worms2 <- worms[5:15, ]          # Ortadaki 11 gözlem
worms5 <- worms[worms$Area > 3 & worms$Slope < 3, ] # Alanı 3'ten büyük, eğimi
  ↪ 3'ten küçük
worms5
```

```
#>      Field.Name Area Slope Vegetation Soil.pH Damp Worm.density
#> 2  Silwood.Bottom  5.1     2   Arable    5.2 FALSE          7
#> 5  Gunness.Thicket 3.8     0   Scrub    4.2 FALSE          6
#> 6    Oak.Mead     3.1     2 Grassland  3.9 FALSE          2
#> 12 North.Gravel   3.3     1 Grassland  4.1 FALSE          1
#> 13 South.Gravel   3.7     2 Grassland  4.0 FALSE          2
#> 15   Pond.Field   4.1     0  Meadow   5.0  TRUE          6
#> 16 Water.Meadow   3.9     0  Meadow   4.9  TRUE          8
#> 18   Pound.Hill   4.4     2   Arable   4.5 FALSE          5
```

```
worms6 <- worms[order(worms$Area), ]          # Alana göre küçükten büyüğe
worms7 <- worms[order(worms$Area), c(2, 3, 5, 7)] # Sıralı ve yalnızca nicel
  ↪ değişkenler
head(worms7)
```

```
#>      Area Slope Soil.pH Worm.density
#> 20  0.8     10    5.1          3
#> 10  1.5      4    5.0          7
#> 14  1.8      6    3.8          0
#> 9   1.9      0    5.7          9
#> 8   2.1      0    4.8          4
#> 17  2.2      8    4.7          4
```

```
worms[rev(order(worms$Soil.pH)), c(5, 7)] |> head() # pH'a göre büyükten küçüğe
```

```
#>      Soil.pH Worm.density
#> 9          5.7          9
#> 11         5.2          8
#> 2          5.2          7
#> 20         5.1          3
#> 15         5.0          6
#> 10         5.0          7
```

```
subset(worms, Vegetation == "Grassland", select = c(Field.Name, Worm.density))
```

```
#>      Field.Name Worm.density
#> 1    Nashs.Field          4
```

```
#> 3      Nursery.Field      2
#> 6      Oak.Mead          2
#> 7      Church.Field      3
#> 10     Rookery.Slope      7
#> 12     North.Gravel      1
#> 13     South.Gravel      2
#> 14 Observatory.Ridge     0
#> 19     Gravel.Pit        1
```

Base R ile her şey yapılabilir; ancak işlemler uzadıkça `worms$...` tekrarları ve iç içe parantezler kodu okumayı zorlaştırır. Bu noktada **dplyr** devreye girer.

4.5 dplyr ile Veri Düzenleme

dplyr, veri düzenlemeyi “fil” gibi okunan fonksiyonlarla yapar. Her fonksiyonun ilk argümanı veri-
dir, sütunlar \$ yazmadan doğrudan adlarıyla kullanılır ve sonuç yine bir veri tablosudur.

Tablo 4.2: dplyr fonksiyonları ve base R karşılıkları

| Fonksiyon | Ne yapar? | Base R'daki karşılığı (yaklaşık) |
|--|--|--|
| <code>select()</code> | Sütun seçer | <code>veri[, c("a","b")]</code> |
| <code>filter()</code> | Koşula göre satır seçer | <code>veri[veri\$a > 5,]</code> |
| <code>arrange()</code> | Satırları sıralar | <code>veri[order(veri\$a),]</code> |
| <code>mutate()</code> | Yeni değişken türetir | <code>veri\$yeni <- ...</code> |
| <code>rename()</code> | Sütun adını değiştirir | <code>names(veri)[...] <- ...</code> |
| <code>summarise()</code> | Özet değer üretir | <code>mean(), sd() ...</code> |
| <code>group_by()</code> | Sonraki işlemleri gruplara göre yapar | <code>tapply(), aggregate()</code> |
| <code>count()</code> | Frekans sayar | <code>table()</code> |
| <code>distinct()</code> | Tekrarlanan satırları atar | <code>unique()</code> |
| <code>slice_max()</code> /
<code>slice_min()</code> | En büyük / küçük n satır | <code>head(veri[order(...),], n)</code> |

4.5.1 Pipe (Boru) Operatörü: |>

Birden çok işlemi art arda yapmak için **pipe** kullanılır. `x |> f(y)` ifadesi `f(x, y)` ile aynıdır ve “sonra” diye okunur. Eski kodlarda aynı işi gören `%>%` (magrittr) operatörünü de göreceksiniz.

```
# İç içe yazım (içten dışa okunur, zor)
round(mean(worms$Worm.density), 1)
```

```
#> [1] 4.3
```

```
# Pipe ile (soldan sağa okunur)
worms$Worm.density |> mean() |> round(1)
```

```
#> [1] 4.3
```

Bundan sonraki örneklerde penguins verisini kullanacağız.

4.5.2 select(): Sütun Seçme

```
penguins |> select(species, island, body_mass_g)
```

```
#> # A tibble: 344 x 3
#>   species island    body_mass_g
#>   <fct>   <fct>         <int>
#> 1 Adelie  Torgersen      3750
#> 2 Adelie  Torgersen      3800
#> 3 Adelie  Torgersen      3250
#> 4 Adelie  Torgersen         NA
#> 5 Adelie  Torgersen      3450
#> 6 Adelie  Torgersen      3650
#> # i 338 more rows
```

```
penguins |> select(species, starts_with("bill")) # "bill" ile başlayanlar
```

```
#> # A tibble: 344 x 3
#>   species bill_length_mm bill_depth_mm
#>   <fct>         <dbl>         <dbl>
#> 1 Adelie         39.1           18.7
#> 2 Adelie         39.5           17.4
#> 3 Adelie         40.3            18
#> 4 Adelie         NA             NA
#> 5 Adelie         36.7           19.3
#> 6 Adelie         39.3           20.6
#> # i 338 more rows
```

```
penguins |> select(where(is.numeric)) # Sayısal sütunlar
```

```
#> # A tibble: 344 x 5
#>   bill_length_mm bill_depth_mm flipper_length_mm body_mass_g year
#>   <dbl>         <dbl>         <int>         <int> <int>
#> 1         39.1         18.7           181         3750 2007
#> 2         39.5         17.4           186         3800 2007
#> 3         40.3            18           195         3250 2007
#> 4          NA             NA             NA          NA 2007
#> 5         36.7         19.3           193         3450 2007
#> 6         39.3         20.6           190         3650 2007
#> # i 338 more rows
```

```
penguins |> select(-year) # year hariç
```

```
#> # A tibble: 344 x 7
#>   species island   bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>   <fct>         <dbl>         <dbl>         <int>         <int>
#> 1 Adelie  Torgersen      39.1           18.7           181           3750
#> 2 Adelie  Torgersen      39.5           17.4           186           3800
#> 3 Adelie  Torgersen      40.3            18           195           3250
#> 4 Adelie  Torgersen      NA             NA             NA             NA
#> 5 Adelie  Torgersen      36.7           19.3           193           3450
#> 6 Adelie  Torgersen      39.3           20.6           190           3650
#> # i 338 more rows
#> # i 1 more variable: sex <fct>
```

4.5.3 filter(): Satır Süzme

```
penguins |> filter(species == "Gentoo", body_mass_g > 5500) # Virgül = VE
```

```
#> # A tibble: 28 x 8
#>   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>   <fct>         <dbl>         <dbl>         <int>         <int>
#> 1 Gentoo  Biscoe         50           16.3           230           5700
#> 2 Gentoo  Biscoe         50           15.2           218           5700
#> 3 Gentoo  Biscoe         49           16.1           216           5550
#> 4 Gentoo  Biscoe        48.4           14.6           213           5850
#> 5 Gentoo  Biscoe        49.3           15.7           217           5850
#> 6 Gentoo  Biscoe        49.2           15.2           221           6300
#> # i 22 more rows
#> # i 2 more variables: sex <fct>, year <int>
```

```
penguins |> filter(island %in% c("Dream", "Torgersen"), sex == "female")
```

```
#> # A tibble: 85 x 8
#>   species island   bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>   <fct>         <dbl>         <dbl>         <int>         <int>
#> 1 Adelie  Torgersen      39.5           17.4           186           3800
#> 2 Adelie  Torgersen      40.3            18           195           3250
#> 3 Adelie  Torgersen      36.7           19.3           193           3450
#> 4 Adelie  Torgersen      38.9           17.8           181           3625
#> 5 Adelie  Torgersen      41.1           17.6           182           3200
#> 6 Adelie  Torgersen      36.6           17.8           185           3700
#> # i 79 more rows
#> # i 2 more variables: sex <fct>, year <int>
```

```
penguins |> filter(is.na(body_mass_g)) # Ağırlığı eksik olanlar
```

```
#> # A tibble: 2 x 8
#>   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>   <fct>         <dbl>         <dbl>           <int>       <int>
#> 1 Adelie  Torgersen             NA             NA             NA         NA
#> 2 Gentoo  Biscoe              NA             NA             NA         NA
#> # i 2 more variables: sex <fct>, year <int>
```

```
penguins |> filter(!is.na(sex)) # Cinsiyeti bilinenler
```

```
#> # A tibble: 333 x 8
#>   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>   <fct>         <dbl>         <dbl>           <int>       <int>
#> 1 Adelie  Torgersen          39.1          18.7           181        3750
#> 2 Adelie  Torgersen          39.5          17.4           186        3800
#> 3 Adelie  Torgersen          40.3           18           195        3250
#> 4 Adelie  Torgersen          36.7          19.3           193        3450
#> 5 Adelie  Torgersen          39.3          20.6           190        3650
#> 6 Adelie  Torgersen          38.9          17.8           181        3625
#> # i 327 more rows
#> # i 2 more variables: sex <fct>, year <int>
```

4.5.4 arrange(): Sıralama

```
penguins |> arrange(body_mass_g) # Küçükten büyüğe
```

```
#> # A tibble: 344 x 8
#>   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>   <fct>         <dbl>         <dbl>           <int>       <int>
#> 1 Chinstrap Dream          46.9          16.6           192        2700
#> 2 Adelie    Biscoe          36.5          16.6           181        2850
#> 3 Adelie    Biscoe          36.4          17.1           184        2850
#> 4 Adelie    Biscoe          34.5          18.1           187        2900
#> 5 Adelie    Dream          33.1          16.1           178        2900
#> 6 Adelie    Torgersen          38.6           17           188        2900
#> # i 338 more rows
#> # i 2 more variables: sex <fct>, year <int>
```

```
penguins |> arrange(desc(flipper_length_mm)) # Büyükten küçüğe
```

```
#> # A tibble: 344 x 8
#>   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>   <fct>         <dbl>         <dbl>           <int>       <int>
#> 1 Gentoo  Biscoe          54.3          15.7           231        5650
#> 2 Gentoo  Biscoe          50           16.3           230        5700
#> 3 Gentoo  Biscoe          59.6           17           230        6050
#> 4 Gentoo  Biscoe          49.8          16.8           230        5700
#> 5 Gentoo  Biscoe          48.6           16           230        5800
#> 6 Gentoo  Biscoe          52.1           17           230        5550
```

```
#> # i 338 more rows
#> # i 2 more variables: sex <fct>, year <int>
```

```
penguins |> slice_max(body_mass_g, n = 3) # En ağır 3 penguen
```

```
#> # A tibble: 4 x 8
#>   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>    <fct>         <dbl>         <dbl>           <int>      <int>
#> 1 Gentoo  Biscoe         49.2          15.2            221       6300
#> 2 Gentoo  Biscoe         59.6           17            230       6050
#> 3 Gentoo  Biscoe         51.1          16.3            220       6000
#> 4 Gentoo  Biscoe         48.8          16.2            222       6000
#> # i 2 more variables: sex <fct>, year <int>
```

4.5.5 mutate(): Yeni Değişken Türetme

```
penguins |>
  mutate(
    body_mass_kg = body_mass_g / 1000,
    gaga_orani   = bill_length_mm / bill_depth_mm,
    buyuk       = if_else(body_mass_g > 4500, "Büyük", "Küçük")
  ) |>
  select(species, body_mass_kg, gaga_orani, buyuk)
```

```
#> # A tibble: 344 x 4
#>   species body_mass_kg gaga_orani buyuk
#>   <fct>         <dbl>     <dbl> <chr>
#> 1 Adelie         3.75         2.09 Küçük
#> 2 Adelie         3.8          2.27 Küçük
#> 3 Adelie         3.25         2.24 Küçük
#> 4 Adelie         NA          NA    <NA>
#> 5 Adelie         3.45         1.90 Küçük
#> 6 Adelie         3.65         1.91 Küçük
#> # i 338 more rows
```

`case_when()` birden fazla koşula göre değer atamak için kullanılır; `if_else()`'in çok koşullu hâli gibi düşünülebilir. En sondaki `.default` (ya da `TRUE ~ ...`) “yukarıdakilerin hiçbiri değilse” anlamına gelir:

```
penguins |>
  mutate(agirlik_sinifi = case_when(
    body_mass_g < 3500 ~ "Hafif",
    body_mass_g < 4500 ~ "Orta",
    body_mass_g >= 4500 ~ "Ağır",
    .default = NA_character_
  )) |>
  count(species, agirlik_sinifi)
```

```
#> # A tibble: 10 x 3
#>   species   agirlik_sinifi     n
#>   <fct>     <chr>         <int>
#> 1 Adelie   Ağır             8
#> 2 Adelie   Hafif            54
#> 3 Adelie   Orta             89
#> 4 Adelie   <NA>              1
#> 5 Chinstrap Ağır             3
#> 6 Chinstrap Hafif            17
#> # i 4 more rows
```

4.5.6 summarise() ve group_by(): Gruplara Göre Özet

```
penguins |>
  summarise(
    n = n(),
    ort_agirlik = mean(body_mass_g, na.rm = TRUE),
    ss_agirlik = sd(body_mass_g, na.rm = TRUE)
  )
```

```
#> # A tibble: 1 x 3
#>       n ort_agirlik ss_agirlik
#>   <int>     <dbl>     <dbl>
#> 1  344      4202.       802.
```

```
penguins |>
  group_by(species, sex) |>
  summarise(
    n = n(),
    ort_agirlik = mean(body_mass_g, na.rm = TRUE),
    ort_yuzgec = mean(flipper_length_mm, na.rm = TRUE),
    .groups = "drop"
  )
```

```
#> # A tibble: 8 x 5
#>   species sex      n ort_agirlik ort_yuzgec
#>   <fct>   <fct> <int>     <dbl>     <dbl>
#> 1 Adelie female   73      3369.      188.
#> 2 Adelie male     73      4043.      192.
#> 3 Adelie <NA>      6      3540       186.
#> 4 Chinstrap female  34      3527.      192.
#> 5 Chinstrap male   34      3939.      200.
#> 6 Gentoo  female  58      4680.      213.
#> # i 2 more rows
```

```
penguins |> count(species, island)
```

```
#> # A tibble: 5 x 3
#>   species island      n
#>   <fct>   <fct>   <int>
#> 1 Adelie Biscoe     44
#> 2 Adelie Dream      56
#> 3 Adelie Torgersen  52
#> 4 Chinstrap Dream     68
#> 5 Gentoo Biscoe    124
```

Aynı fonksiyonu birçok sütuna birden uygulamak için `across()` kullanılır:

```
penguins |>
  group_by(species) |>
  summarise(across(bill_length_mm:body_mass_g, \ (x) mean(x, na.rm = TRUE)))
```

```
#> # A tibble: 3 x 5
#>   species bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>         <dbl>         <dbl>         <dbl>         <dbl>
#> 1 Adelie         38.8           18.3           190.         3701.
#> 2 Chinstrap      48.8           18.4           196.         3733.
#> 3 Gentoo         47.5           15.0           217.         5076.
```

i Not

`\(x) mean(x, na.rm = TRUE)` ifadesi R 4.1 ile gelen kısa **anonim fonksiyon** yazımıdır ve `function(x) mean(x, na.rm = TRUE)` ile aynıdır. Fonksiyonları Bölüm 5'te ayrıntılı göreceğiz.

Worms verisinde eski notlardaki `aggregate()` örneğinin `dplyr` karşılığı:

```
aggregate(worms[, c(2, 3, 5, 7)], list(Vegetation = worms$Vegetation), mean) #
↪ base R
```

```
#>   Vegetation Area Slope Soil.pH Worm.density
#> 1   Arable 3.867 1.333  4.833      5.333
#> 2 Grassland 2.911 3.667  4.100      2.444
#> 3   Meadow 3.467 1.667  4.933      6.333
#> 4   Orchard 1.900 0.000  5.700      9.000
#> 5   Scrub 2.425 7.000  4.800      5.250
```

```
worms |> # dplyr
  group_by(Vegetation) |>
  summarise(across(c(Area, Slope, Soil.pH, Worm.density), mean))
```

```
#> # A tibble: 5 x 5
#>   Vegetation Area Slope Soil.pH Worm.density
#>   <chr>     <dbl> <dbl> <dbl>     <dbl>
#> 1 Arable    3.87  1.33  4.83      5.33
#> 2 Grassland 2.91  3.67  4.1       2.44
#> 3 Meadow    3.47  1.67  4.93      6.33
#> 4 Orchard   1.9   0     5.7       9
#> 5 Scrub     2.42  7     4.8      5.25
```

4.5.7 Eksik Değerlerle Çalışma

```
colSums(is.na(penguins)) # Her sütunda kaç eksik değer var?
```

```
#>      species      island bill_length_mm bill_depth_mm
#>      0          0          2          2
#> flipper_length_mm body_mass_g      sex      year
#>      2          2          11          0
```

```
penguins |> filter(if_any(everything(), is.na)) # En az bir eksik değeri olan
↪ satırlar
```

```
#> # A tibble: 11 x 8
#>   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#>   <fct>   <fct>         <dbl>         <dbl>         <int>         <int>
#> 1 Adelie Torgersen         NA           NA           NA           NA
#> 2 Adelie Torgersen        34.1         18.1         193         3475
#> 3 Adelie Torgersen        42          20.2         190         4250
#> 4 Adelie Torgersen        37.8         17.1         186         3300
#> 5 Adelie Torgersen        37.8         17.3         180         3700
#> 6 Adelie Dream          37.5         18.9         179         2975
#> # i 5 more rows
#> # i 2 more variables: sex <fct>, year <int>
```

```
penguenler <- penguins |> drop_na() # tidyr: eksik değeri olan satırları çıkar
nrow(penguins); nrow(penguenler)
```

```
#> [1] 344
```

```
#> [1] 333
```

Uyarı

Eksik değeri olan satırları körü körüne atmak her zaman doğru değildir. Önce **kaç** gözlemin ve **hangi gruptan** gözlemlerin atıldığına bakın. Eksiklik belirli bir grupta yoğunlaşıyorsa sonuçlar yanıltıcı olabilir.

4.6 Uygulama: Müşteri Verisini Analize Hazırlama

Gerçek veriler çoğu zaman “kirlidir”. `musteri` verisinde bunu görelim:

```
musteri |> count(Marital_Status) # "Absurd", "YOLO", "Alone" gibi garip
↪ kategoriler
```

```
#> # A tibble: 8 x 2
#>   Marital_Status     n
#>   <chr>          <int>
#> 1 Absurd          2
#> 2 Alone           3
#> 3 Divorced       232
#> 4 Married        864
#> 5 Single         480
#> 6 Together       580
#> # i 2 more rows
```

```
summary(musteri$Income) # 24 eksik gelir ve 666666 gibi aşırı bir
↪ değer
```

```
#>   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   NA's
#>   1730   35303   51382   52247   68522   666666    24
```

```
range(musteri$Year_Birth) # 1893 doğumlu müşteri?
```

```
#> [1] 1893 1996
```

Bu sorunları adım adım düzelterek analize hazır bir veri oluşturalım:

```
musteri_temiz <- muster_i |>
  filter(!is.na(Income), # Geliri eksik olanları çıkar
         Income < 200000, # Aşırı değeri çıkar
         Year_Birth >= 1940) |> # Mantıksız doğum yıllarını çıkar
  mutate(
    Yas = 2014 - Year_Birth, # Veri 2014 yılına ait
    Cocuk = Kidhome + Teenhome,
    Toplam_Harcama = MntWines + MntFruits + MntMeatProducts +
      MntFishProducts + MntSweetProducts + MntGoldProds,
    Medeni = if_else(Marital_Status %in% c("Married", "Together"), "Birlikte",
      ↪ "Yalnız"),
    Egitim = factor(Education,
      levels = c("Basic", "2n Cycle", "Graduation", "Master", "PhD"),
      ordered = TRUE)
  ) |>
  select(ID, Yas, Egitim, Medeni, Income, Cocuk, Toplam_Harcama, starts_with("Mnt"))

dim(musteri_temiz)
```

```
#> [1] 2212 13
```

```
musteri_temiz |>
  group_by(Egitim) |>
  summarise(n = n(), ort_gelir = mean(Income), ort_harcama = mean(Toplam_Harcama))
```

```
#> # A tibble: 5 x 4
#>   Egitim      n ort_gelir ort_harcama
#>   <ord>    <int>    <dbl>    <dbl>
#> 1 Basic      54  20306.    81.8
#> 2 2n Cycle   198  47625.   499.
#> 3 Graduation 1115  52170.   622.
#> 4 Master    365  52918.   610.
#> 5 PhD       480  56088.   674.
```

Temizlenmiş veriyi sonraki haftalarda kullanmak üzere kaydedebiliriz:

```
saveRDS(musteri_temiz, "veri/musteri_temiz.rds")
```

4.7 Veriyi Yeniden Şekillendirme: Uzun ve Geniş Biçim

Aynı veri iki şekilde tutulabilir. **Geniş** (*wide*) biçimde her ölçüm ayrı bir sütundadır; **uzun** (*long*) biçimde ise ölçümün adı bir sütunda, değeri başka bir sütundadır. **ggplot2** ve pek çok istatistiksel fonksiyon **uzun** biçimi bekler.

```
genis <- penguenerler |>
  select(species, bill_length_mm, bill_depth_mm, flipper_length_mm) |>
  slice_head(n = 3)
genis
```

```
#> # A tibble: 3 x 4
#>   species bill_length_mm bill_depth_mm flipper_length_mm
#>   <fct>      <dbl>      <dbl>      <int>
#> 1 Adelie      39.1       18.7       181
#> 2 Adelie      39.5       17.4       186
#> 3 Adelie      40.3       18        195
```

```
uzun <- genis |>
  pivot_longer(cols = -species, names_to = "olcum", values_to = "deger")
uzun
```

```
#> # A tibble: 9 x 3
#>   species olcum      deger
#>   <fct>   <chr>    <dbl>
#> 1 Adelie bill_length_mm  39.1
#> 2 Adelie bill_depth_mm  18.7
#> 3 Adelie flipper_length_mm 181
#> 4 Adelie bill_length_mm  39.5
#> 5 Adelie bill_depth_mm  17.4
#> 6 Adelie flipper_length_mm 186
#> # i 3 more rows
```

```
uzun |>
  group_by(species, olcum) |>
  summarise(ort = mean(deger), .groups = "drop") |>
  pivot_wider(names_from = olcum, values_from = ort)
```

```
#> # A tibble: 1 x 4
#>   species bill_depth_mm bill_length_mm flipper_length_mm
#>   <fct>         <dbl>         <dbl>         <dbl>
#> 1 Adelie         18.0           39.6           187.
```

4.8 Tabloları Birleştirme

İki tabloyu ortak bir **anahtar** sütun üzerinden birleştirmek için `*_join()` fonksiyonları kullanılır.

```
ada_bilgisi <- tibble(
  island = c("Biscoe", "Dream", "Torgersen"),
  ada_alani_km2 = c(1.8, 0.9, 0.3),          # Örnek amaçlı değerler
  arastirma_istasyonu = c("Evet", "Hayır", "Evet")
)

penguenler |>
  count(island) |>
  left_join(ada_bilgisi, by = "island")
```

```
#> # A tibble: 3 x 4
#>   island      n ada_alani_km2 arastirma_istasyonu
#>   <chr>   <int>         <dbl> <chr>
#> 1 Biscoe    163           1.8 Evet
#> 2 Dream     123           0.9 Hayır
#> 3 Torgersen  47           0.3 Evet
```

| Fonksiyon | Sonuç |
|-------------------------------|--|
| <code>left_join(x, y)</code> | x'in tüm satırları; y'de karşılığı olmayanlara NA |
| <code>inner_join(x, y)</code> | Yalnızca her iki tabloda da bulunan anahtarlar |
| <code>full_join(x, y)</code> | İki tablonun tüm satırları |
| <code>anti_join(x, y)</code> | x'te olup y'de olmayan satırlar (kontrol için çok kullanışlı) |

Kendini Sına

- 💡 1. worms verisinde ilk 5 satırın solucan yoğunluğunu yazdırıp worms3'e aktarın.

```
worms3 <- worms[1:5, "Worm.density"]  
# dplyr ile:  
worms3 <- worms |> slice(1:5) |> pull(Worm.density)
```

- 💡 2. worms verisinde ilk 5 ve son 3 satırı birleştirip son 3 değişkeni worms4'e aktarın.

```
worms4 <- worms[c(1:5, (nrow(worms) - 2):nrow(worms)), 5:7]
```

- 💡 3. Biscoe adasında yaşayan dişi penguenlerin ortalama yüzgeç uzunluğu nedir?

```
penguins |>  
  filter(island == "Biscoe", sex == "female") |>  
  summarise(ort = mean(flipper_length_mm, na.rm = TRUE))
```

- 💡 4. iris verisinde her tür için taç yaprak uzunluğunun ortalamasını ve standart sapmasını hesaplayıp ortalamaya göre büyükten küçüğe sıralayın.

```
iris |>  
  group_by(Species) |>  
  summarise(ort = mean(Petal.Length), ss = sd(Petal.Length)) |>  
  arrange(desc(ort))
```

- 💡 5. muster_i_temiz verisine gelir grubu (Düşük < 35000 Orta < 70000 Yüksek) ekleyip frekanslarını bulun.

```
muster_i_temiz |>  
  mutate(gelir_grubu = case_when(  
    Income < 35000 ~ "Düşük",  
    Income < 70000 ~ "Orta",  
    .default = "Yüksek"  
  )) |>  
  count(gelir_grubu)
```

5 Kontrol Yapıları ve Fonksiyon Yazma

i Bu haftanın hedefleri

- Koşula bağlı işlem yapmak (`if`, `else`, `ifelse`, `switch`),
- İşlemleri tekrar etmek (`for`, `while`, `repeat`) ve döngü sonuçlarını saklamak,
- Ne zaman döngü yerine **vektörel işlem** ya da `apply` ailesi kullanılacağını bilmek,
- Argümanları, varsayılan değerleri ve hata kontrolleri olan **kendi fonksiyonlarını** yazmak,
- İstatistiksel bir formülü (ortalama, ortanca, varyans) fonksiyona dökmek ve R'ın hazır fonksiyonuyla doğrulamak,
- Veri tablosu ve grafikte çalışan fonksiyonlar yazmak.

5.1 Koşul Yapıları

5.1.1 `if` / `else`

Bir koşul **doğruysa** bir işlemi, **yanlışsa** başka bir işlemi yapar:

```
if (koşul) {  
  koşul TRUE ise yapılacaklar  
} else {  
  koşul FALSE ise yapılacaklar  
}
```

```
x <- -5  
if (x < 0) {  
  print("Negatif sayıdır")  
} else {  
  print("Pozitif sayıdır")  
}
```

```
#> [1] "Negatif sayıdır"
```

`else if` ile istenildiği kadar koşul art arda eklenebilir. Koşullar **yukarıdan aşağıya** kontrol edilir; ilk sağlanan koşulun işlemi yapılır.

```
not <- 73  
if (not >= 90) {  
  harf <- "AA"  
} else if (not >= 80) {
```

```

  harf <- "BA"
} else if (not >= 70) {
  harf <- "BB"
} else if (not >= 60) {
  harf <- "CB"
} else {
  harf <- "FF"
}
harf

```

```
#> [1] "BB"
```

! Önemli

`if()` yalnızca **tek bir** TRUE/FALSE değeri kontrol eder. `if (c(3, -1) > 0)` gibi bir vektör verirsiniz R hata verir. Vektörün her elemanı için koşul kontrolü yapmak istiyorsanız `ifelse()` (ya da `dplyr::if_else()`) kullanın. Birden fazla koşulu `if` içinde birleştirirken `&` yerine tek değer döndüren `&&`, `|` yerine `||` kullanılır.

5.1.2 ifelse()

Vektörel çalışır: koşul sağlanan elemanlar için ikinci argümanı, sağlanmayanlar için üçüncü argümanı döndürür.

```

notlar <- c(45, 82, 67, 91, 30)
ifelse(notlar >= 60, "Geçti", "Kaldı")

```

```
#> [1] "Kaldı" "Geçti" "Geçti" "Geçti" "Kaldı"
```

```
ifelse(notlar >= 85, "Yüksek", ifelse(notlar >= 60, "Orta", "Düşük")) # İç içe
```

```
#> [1] "Düşük" "Orta" "Orta" "Yüksek" "Düşük"
```

5.1.3 switch()

Bir değişkenin **değerine göre** farklı işlemler yapmak için `switch()` kullanılır. Özellikle fonksiyonlarda “hangi yöntem?” seçimi için kullanışlıdır:

```

merkez <- function(x, yontem = "ortalama") {
  switch(yontem,
    ortalama = mean(x),
    ortanca = median(x),
    budanmis = mean(x, trim = 0.1),
    stop("Bilinmeyen yöntem: ", yontem))
}
merkez(penguenler$body_mass_g)

```

```
#> [1] 4207
```

```
merkez(penguenler$body_mass_g, "ortanca")
```

```
#> [1] 4050
```

5.2 Döngüler

5.2.1 for() Döngüsü

Bir vektörün **her elemanı için** aynı işlemi tekrar eder: `for (eleman in vektör) { işlemler }`.

```
for (i in 1:5) {  
  print(i^2)  
}
```

```
#> [1] 1  
#> [1] 4  
#> [1] 9  
#> [1] 16  
#> [1] 25
```

```
x <- c(2, 5, 3, 9, 8, 11, 6)  
count <- 0  
for (val in x) {  
  if (val %% 2 == 0) count <- count + 1 # Çift sayıları say  
}  
count
```

```
#> [1] 3
```

Döngü sonuçlarını saklamak için önce **boş bir vektör** oluşturup döngü içinde doldurmak iyi bir alışkanlıktır:

```
kareler <- numeric(5) # 5 elemanlı, sıfırlarla dolu vektör  
for (i in 1:5) {  
  kareler[i] <- i^2  
}  
kareler
```

```
#> [1] 1 4 9 16 25
```

```
(1:5)^2 # Aynı sonuç, vektörel işlemle tek satırda!
```

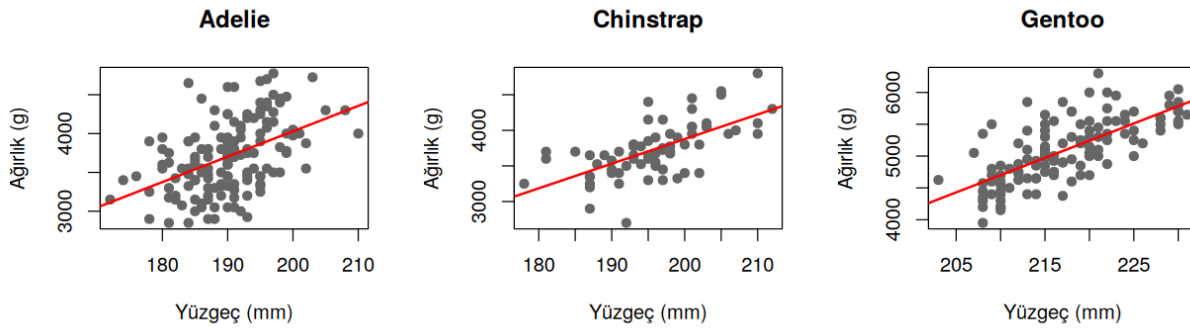
```
#> [1] 1 4 9 16 25
```

i Not

R'da pek çok işlem **vektörel** olduğu için döngüye gerek kalmaz. Döngü yazmadan önce “bunu vektörel yapabilir miyim?” diye sormak iyi bir alışkanlıktır. Yine de döngüler, **simülasyon** gibi her adımın tekrarlandığı durumlarda vazgeçilmezdir; Bölüm 10'de Merkezi Limit Teoremini döngülerle göstereceğiz.

Döngü ile gruplara göre grafik çizmek: eski notlarımızdaki prostat kanseri örneğinde (**islasso** paketi), seminal vezikül invazyonu (**svi**: 0/1) gruplarına göre iki değişken arasındaki ilişki ayrı ayrı çizdiriliyordu. Aynı fikri penguenlerle uygulayalım:

```
gruplar <- split(penguenler, penguenler$species) # Türlerle göre böl
for (tur in names(gruplar)) {
  d <- gruplar[[tur]]
  plot(d$flipper_length_mm, d$body_mass_g, main = tur,
       xlab = "Yüzgeç (mm)", ylab = "Ağırlık (g)", pch = 19, col = "grey40")
  abline(lm(body_mass_g ~ flipper_length_mm, data = d), col = "red", lwd = 2)
}
```



5.2.2 while() ve repeat

while koşul sağlandığı sürece tekrar eder. Kaç kez döneceği önceden bilinmediğinde kullanılır. Koşulu değiştiren bir adım unutulursa döngü **sonsuz** kadar döner (konsolda **Esc** ile durdurulur).

```
i <- 1
while (i < 6) {
  print(i)
  i <- i + 1
}
```

```
#> [1] 1
#> [1] 2
#> [1] 3
#> [1] 4
#> [1] 5
```

break döngüyü bitirir; **next** o adımı atlayıp bir sonrakine geçer. Aşağıdaki iki döngünün çıktılarını karşılaştırıp yorumlayınız:

```
i <- 1
while (i < 6) {
  print(i)
  if (i > 4) break
  i <- i + 1
}
```

```
#> [1] 1
#> [1] 2
#> [1] 3
#> [1] 4
#> [1] 5
```

```
i <- 1
while (i < 6) {
  if (i > 4) break
  print(i)
  i <- i + 1
}
```

```
#> [1] 1
#> [1] 2
#> [1] 3
#> [1] 4
```

```
for (i in 1:6) {
  if (i %% 2 == 0) next # Çiftleri atla
  print(i)
}
```

```
#> [1] 1
#> [1] 3
#> [1] 5
```

Bir simülasyon örneği: Adil bir zarı **ilk kez 6 gelene kadar** atalım. Kaç atış gerekir?

```
set.seed(347) # Rastgele sayıların tekrarlanabilir olması için
atis <- 0
repeat {
  atis <- atis + 1
  if (sample(1:6, 1) == 6) break
}
atis
```

```
#> [1] 5
```

5.3 Fonksiyon Yazma

Aynı kodu üç kez kopyalayıp yapıştırıyorsanız, onu bir **fonksiyon** hâline getirme zamanı gelmiştir. Fonksiyonun üç parçası vardır: **girdiler (argümanlar)**, **gövde** ve **çıktı** (döndürülen değer).

```
fonksiyon_adi <- function(arg1, arg2 = varsayilan_deger) {  
  # işlemler  
  return(sonuc)      # return yazılmazsa son satırın değeri döndürülür  
}
```

```
kare_al <- function(x) {  
  x^2  
}  
kare_al(4)
```

```
#> [1] 16
```

```
kare_al(1:5)      # Gövde vektörel olduğu için fonksiyon da vektörel
```

```
#> [1] 1 4 9 16 25
```

5.3.1 Argümanlar ve Varsayılan Değerler

```
standartlastir <- function(x, ortalama = mean(x), sapma = sd(x)) {  
  (x - ortalama) / sapma  
}  
standartlastir(c(10, 20, 30))
```

```
#> [1] -1 0 1
```

```
standartlastir(c(10, 20, 30), ortalama = 0, sapma = 10)
```

```
#> [1] 1 2 3
```

Argümanlar **sırasıyla** ya da **adıyla** verilebilir. Adıyla verildiğinde sıra önemli değildir:

```
fun1 <- function(data, data.frame, graph, limit) {  
  ## [fonksiyon gövdesi]  
}  
ans <- fun1(d, df, TRUE, 20)                # Sırayla  
ans <- fun1(d, df, graph = TRUE, limit = 20) # Karışık  
ans <- fun1(data = d, limit = 20, graph = TRUE, data.frame = df) # Adıyla
```

5.3.2 İstatistiksel Formülleri Fonksiyona Dökmek

Eski ders notlarımızda ortalama, ortanca ve varyans için kendi fonksiyonlarımızı yazmıştık. Bu, hem formülü hem R’ı öğrenmenin en iyi yoludur. Her zaman yazdığımız fonksiyonu R’in hazır fonksiyonuyla **karşılaştırarak doğrularız**.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

```
ort <- function(x) sum(x) / length(x)

set.seed(1)
veri <- rnorm(100, mean = 5, sd = 2)
ort(veri)
```

```
#> [1] 5.218
```

```
mean(veri)      # Doğrulama
```

```
#> [1] 5.218
```

Ortanca, sıralı verinin ortasındaki değerdir; gözlem sayısı çiftse ortadaki iki değer ortalamaştırılır:

```
ortanca <- function(x) {
  x <- sort(x)
  n <- length(x)
  if (n %% 2 == 1) {
    x[(n + 1) / 2]
  } else {
    (x[n / 2] + x[n / 2 + 1]) / 2
  }
}
ortanca(veri);      median(veri)
```

```
#> [1] 5.228
```

```
#> [1] 5.228
```

```
ortanca(veri[-1]); median(veri[-1]) # Tek sayıda gözlem
```

```
#> [1] 5.307
```

```
#> [1] 5.307
```

Örneklem varyansı, ortalamadan sapmaların kareler toplamının $n - 1$ 'e bölümüdür:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

```
varyans <- function(x) sum((x - mean(x))^2) / (length(x) - 1)
varyans(veri); var(veri)
```

```
#> [1] 3.227
```

```
#> [1] 3.227
```

5.3.3 Hata Kontrolü

İyi bir fonksiyon, yanlış girdi verildiğinde **anlaşılır** bir hata mesajı verir:

```
degisim_katsayisi <- function(x, na.rm = FALSE) {
  if (!is.numeric(x)) stop("x sayısal bir vektör olmalıdır.")
  if (na.rm) x <- x[!is.na(x)]
  if (length(x) < 2) stop("En az iki gözlem gerekir.")
  sd(x) / mean(x)
}
degisim_katsayisi(penguins$body_mass_g, na.rm = TRUE)
```

```
#> [1] 0.1909
```

```
degisim_katsayisi(penguins$species)
```

```
#> Error in degisim_katsayisi(penguins$species): x sayısal bir vektör olmalıdır.
```

5.3.4 Birden Fazla Sonuç Döndürmek

Bir fonksiyon tek bir nesne döndürür. Birden fazla sonuç gerekiyorsa bunlar bir **liste** ya da **data frame** içinde döndürülür:

```
ozet <- function(x, na.rm = TRUE) {
  if (na.rm) x <- x[!is.na(x)]
  list(n = length(x), ortalama = mean(x), ortanca = median(x),
       std_sapma = sd(x), dk = sd(x) / mean(x))
}
str(ozet(penguins$body_mass_g))
```

```
#> List of 5
#> $ n          : int 342
#> $ ortalama   : num 4202
#> $ ortanca    : num 4050
#> $ std_sapma  : num 802
#> $ dk         : num 0.191
```

5.3.5 Örnek: Harf Notu Fonksiyonu

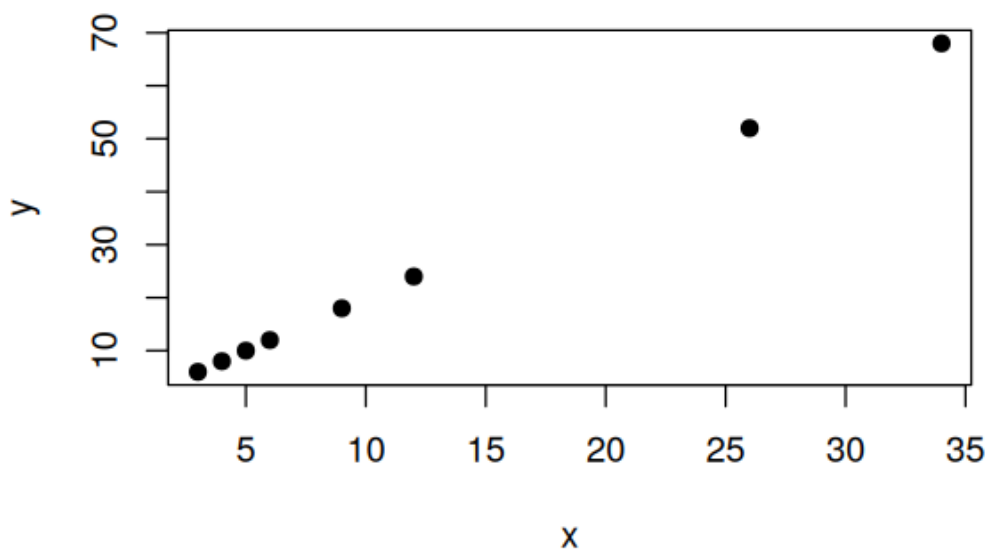
Koşul yapısını fonksiyonla birleştirelim. `if` yerine vektörel çalışan `cut()` kullanırsak fonksiyon tüm sınıf için bir kerede çalışır:

```
harf_notu <- function(vize, final, vize_agirlik = 0.4) {  
  ort <- vize_agirlik * vize + (1 - vize_agirlik) * final  
  harf <- cut(ort, breaks = c(-Inf, 60, 70, 80, 90, Inf),  
             labels = c("FF", "CB", "BB", "BA", "AA"), right = FALSE)  
  data.frame(vize, final, ortalama = ort, harf)  
}  
harf_notu(vize = c(55, 80, 40, 90), final = c(60, 85, 50, 88))
```

```
#>   vize final ortalama harf  
#> 1  55    60    58.0   FF  
#> 2  80    85    83.0   BA  
#> 3  40    50    46.0   FF  
#> 4  90    88    88.8   BA
```

5.3.6 Örnek: Grafik Çizen Fonksiyon

```
buyenifonksiyon <- function(x, grafik = TRUE) {  
  y <- 2 * x  
  if (grafik) plot(x, y, pch = 19)  
  invisible(y)      # Sonucu döndür ama konsola yazdırma  
}  
sonuc <- buyenifonksiyon(c(3, 5, 6, 9, 4, 12, 26, 34))
```



```
sonuc
```

```
#> [1] 6 10 12 18 8 24 52 68
```

5.3.7 Örnek: Banka Hesabı ve Kapsam (Scope)

```
acik.hesap <- function(toplam) {          # Başlangıç bakiyesi ile hesap açılıyor
  list(
    mevduat = function(miktar) {
      if (miktar <= 0) stop("Mevduat pozitif olmalı!")
      toplam <- toplam + miktar
      cat(miktar, "yatırıldı. Bakiyeniz:", toplam, "\n")
    },
    cekilen = function(miktar) {
      if (miktar > toplam) stop("0 kadar para yok!")
      toplam <- toplam - miktar
      cat(miktar, "çekildi. Kalan bakiyeniz:", toplam, "\n")
    },
    bakiye = function() cat("Kalan bakiyeniz:", toplam, "\n")
  )
}
ali <- acik.hesap(100)
burak <- acik.hesap(200)
ali$cekilen(30)
```

```
#> 30 çekildi. Kalan bakiyeniz: 70
```

```
ali$mevduat(50)
```

```
#> 50 yatırıldı. Bakiyeniz: 120
```

```
ali$bakiye()
```

```
#> Kalan bakiyeniz: 120
```

```
burak$bakiye()
```

```
#> Kalan bakiyeniz: 200
```

i <- ile <<- farkı

Fonksiyon içinde <- ile yapılan atama yalnızca **fonksiyonun içinde** geçerlidir (yerel değişken); fonksiyon bitince silinir. <<- ise bir üst ortamdaki değişkeni değiştirir. Yukarıda her hesabın bakiyesinin ayrı ayrı hatırlanmasını sağlayan budur. Günlük kullanımda <<-’dan kaçınmak

önerilir: fonksiyonlar dışarıdaki nesneleri değiştirmemeli, sonucu **döndürmelidir**.

5.4 Döngü Yerine: apply Ailesi

apply ailesi, “her eleman / satır / sütun / grup için bu fonksiyonu uygula” işini tek satırda yapar.

| Fonksiyon | Neye uygulanır? | Ne döndürür? |
|--------------------------|--|--------------------------|
| sapply(x, f) | Vektör / listenin her elemanına | Mümkünse vektör / matris |
| lapply(x, f) | Vektör / listenin her elemanına | Her zaman liste |
| vapply(x, f, numeric(1)) | sapply gibi, çıktı tipini garanti eder | Vektör |
| apply(m, 1 ya da 2, f) | Matrisin satırlarına (1) / sütunlarına (2) | Vektör / matris |
| tapply(x, grup, f) | Bir vektöre, grup grup | Vektör / dizi |
| mapply(f, x, y) | Birden fazla vektöre paralel | Vektör / liste |

```
sapply(1:5, kare_al)
```

```
#> [1] 1 4 9 16 25
```

```
sapply(iris[, 1:4], mean)
```

```
# Her sütunun ortalaması
```

```
#> Sepal.Length Sepal.Width Petal.Length Petal.Width  
#> 5.843 3.057 3.758 1.199
```

```
apply(as.matrix(iris[, 1:4]), 2, sd)  
↪ sapması
```

```
# Her sütunun standart
```

```
#> Sepal.Length Sepal.Width Petal.Length Petal.Width  
#> 0.8281 0.4359 1.7653 0.7622
```

```
tapply(iris$Petal.Length, iris$Species, median)
```

```
# Türlerine göre ortanca
```

```
#> setosa versicolor virginica  
#> 1.50 4.35 5.55
```

```
sapply(penguenler[, 3:6], \(x) unlist(ozet(x))) |> round(2) # Kendi fonksiyonumuzu  
↪ her sütuna
```

```
#> bill_length_mm bill_depth_mm flipper_length_mm body_mass_g  
#> n 333.00 333.00 333.00 333.00  
#> ortalama 43.99 17.16 200.97 4207.06  
#> ortanca 44.50 17.30 197.00 4050.00  
#> std_sapma 5.47 1.97 14.02 805.22  
#> dk 0.12 0.11 0.07 0.19
```

💡 İpucu

tidyverse içindeki purrr paketi aynı işi `map()`, `map_dbl()` gibi fonksiyonlarla yapar: `purrr::map_dbl(iris[, 1:4], mean)`. Tutarlı adlandırması sayesinde büyük projelerde tercih edilir.

5.5 Veri Tablosuyla Çalışan Fonksiyonlar

dplyr fonksiyonlarını kendi fonksiyonumuzun içinde kullanırken, sütun adını argüman olarak almak için **çift süslü parantez** `{ }` (*embrace*) kullanılır:

```
grup_ozeti <- function(veri, grup, degisken) {  
  veri |>  
  group_by({{ grup }}) |>  
  summarise(  
    n          = n(),  
    ortalama   = mean({{ degisken }}, na.rm = TRUE),  
    ss         = sd({{ degisken }}, na.rm = TRUE),  
    .groups    = "drop"  
  )  
}  
grup_ozeti(penguins, species, body_mass_g)
```

```
#> # A tibble: 3 x 4  
#>   species      n ortalama    ss  
#>   <fct>    <int>   <dbl> <dbl>  
#> 1 Adelie    152   3701.  459.  
#> 2 Chinstrap  68   3733.  384.  
#> 3 Gentoo   124   5076.  504.
```

```
grup_ozeti(iris, Species, Sepal.Width)
```

```
#> # A tibble: 3 x 4  
#>   Species      n ortalama    ss  
#>   <fct>    <int>   <dbl> <dbl>  
#> 1 setosa    50    3.43 0.379  
#> 2 versicolor 50    2.77 0.314  
#> 3 virginica  50    2.97 0.322
```

```
grup_ozeti(worms, Vegetation, Worm.density)
```

```
#> # A tibble: 5 x 4  
#>   Vegetation      n ortalama    ss  
#>   <chr>    <int>   <dbl> <dbl>  
#> 1 Arable      3    5.33  1.53  
#> 2 Grassland   9    2.44  2.07  
#> 3 Meadow      3    6.33  1.53  
#> 4 Orchard     1     9    NA  
#> 5 Scrub       4    5.25  2.22
```

Aynı fonksiyon üç farklı veri setinde, hiçbir değişiklik yapılmadan çalıştı. Fonksiyon yazmanın asıl gücü budur. Aynı fikri Bölüm 6'de **grafik çizen** fonksiyonlarda da kullanacağız.

Kendini Sına

- 💡 1. Bir sayının tek mi çift mi olduğunu döndüren, vektörlerle de çalışan bir fonksiyon yazın.

```
tek_cift <- function(x) ifelse(x %% 2 == 0, "Çift", "Tek")
tek_cift(1:6)
```

if kullanılsaydı fonksiyon yalnızca tek bir sayı için çalışırdı.

- 💡 2. for döngüsü ile 1'den 100'e kadar olan sayıların toplamını bulun.

```
toplam <- 0
for (i in 1:100) toplam <- toplam + i
toplam          # 5050
sum(1:100)      # Vektörel karşılığı
```

- 💡 3. Bir yazı-tura atışında art arda 3 kez yazı gelene kadar kaç atış gerektiğini while ile bulun.

```
set.seed(1)
art_arda <- 0; atis <- 0
while (art_arda < 3) {
  atis <- atis + 1
  if (sample(c("Y", "T"), 1) == "Y") art_arda <- art_arda + 1 else art_arda <- 0
}
atis
```

- 💡 4. Ortalamaya göre ortalama mutlak sapmayı (OMS) hesaplayan bir fonksiyon yazın:
$$OMS = \frac{1}{n} \sum |x_i - \bar{x}|.$$

```
oms <- function(x) mean(abs(x - mean(x)))
oms(penguenler$body_mass_g)
```

- 💡 5. grup_ozeti() fonksiyonuna ortancayı da ekleyin ve mtcars verisinde silindir sayısına (cyl) göre mpg'yi özetleyin.

summarise() içine ortanca = median({{ degisken }}, na.rm = TRUE) satırı eklenir; ardından grup_ozeti(mtcars, cyl, mpg).

Kısım II

Kısım II — Veriyi Görmek ve Özetlemek

6 Görselleştirme: base R, ggplot2 ve plotly

i Bu haftanın hedefleri

- Sorulan soruya ve değişkenlerin türüne göre **doğru grafiği seçmek**,
- base R ile hızlı keşif grafikleri çizmek,
- ggplot2'nin **grafik dilbilgisini** (veri + estetik eşleme + geometri + katmanlar) kavramak,
- Eşleme (*mapping*) ile sabit değer (*setting*) farkını bilmek,
- Paneller, ölçekler, temalar ve grafik birleştirme ile yayın kalitesinde grafik hazırlamak,
- Grafik çizen **fonksiyonlar** yazmak,
- plotly ile etkileşimli grafikler hazırlamak (ggplotly() ve plot_ly()).

Amacımız süslü grafikler değil; **hangi soruya hangi grafiğin** cevap verdiğini bilmek ve grafiği analize hazırlık olarak kullanmaktır.

6.1 Neden Önce Görselleştirme?

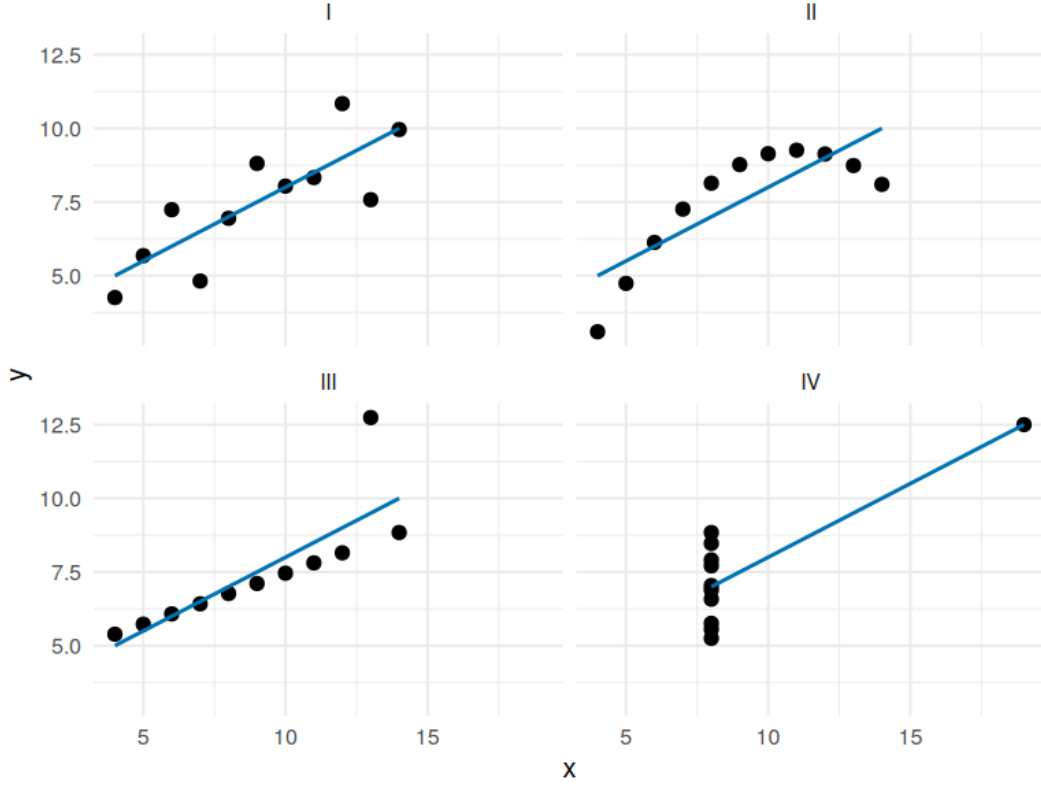
Aşağıdaki dört veri setinin (Anscombe dörtlüsü) ortalamaları, varyansları, korelasyonları ve regresyon doğruları **neredeyse aynıdır**:

```
anscombe_uzun <- data.frame(  
  set = rep(c("I", "II", "III", "IV"), each = 11),  
  x = c(anscombe$x1, anscombe$x2, anscombe$x3, anscombe$x4),  
  y = c(anscombe$y1, anscombe$y2, anscombe$y3, anscombe$y4)  
)  
anscombe_uzun |>  
  group_by(set) |>  
  summarise(ort_x = mean(x), ort_y = mean(y), ss_y = sd(y), r = cor(x, y))
```

```
#> # A tibble: 4 x 5  
#>   set   ort_x ort_y ss_y    r  
#>   <chr> <dbl> <dbl> <dbl> <dbl>  
#> 1 I       9  7.50  2.03 0.816  
#> 2 II      9  7.50  2.03 0.816  
#> 3 III     9  7.5  2.03 0.816  
#> 4 IV      9  7.50  2.03 0.817
```

Ama grafikleri birbirinden tamamen farklıdır:

```
ggplot(anscombe_uzun, aes(x, y)) +
  geom_point(size = 2.5) +
  geom_smooth(method = "lm", se = FALSE, colour = "#0072B2", linewidth = 0.8) +
  facet_wrap(~ set)
```



Şekil 6.1: Anscombe dördlüsü: aynı özet istatistikler, çok farklı veriler

Ders: Özet istatistiklere bakmadan önce veriye bakın. Bir regresyon ya da ANOVA'nın varsayımlarının sağlanıp sağlanmadığını görmenin ilk yolu da grafiklerdir.

6.2 Hangi Soruya Hangi Grafik?

Tablo 6.1: Soruya göre grafik seçimi

| Soru | Değişken türü | Grafik | ggplot2 |
|---------------------------------------|---------------|-----------------------------------|--|
| Bir sayısal değişken nasıl dağılıyor? | 1 sayısal | Histogram, yoğunluk, kutu grafiği | geom_histogram(),
geom_density(),
geom_boxplot() |
| Kategorilerin frekansları nedir? | 1 kategorik | Çubuk grafiği | geom_bar() |
| İki sayısal değişken ilişkili mi? | 2 sayısal | Serpilme grafiği | geom_point() +
geom_smooth() |

| Soru | Değişken türü | Grafik | ggplot2 |
|---|-------------------------|-------------------------------|---|
| Sayısal değişken gruplara göre farklı mı? | 1 sayısal + 1 kategorik | Yan yana kutu / keman grafiği | <code>geom_boxplot()</code> ,
<code>geom_violin()</code> |
| İki kategorik değişken ilişkili mi? | 2 kategorik | Gruplanmış / yığılmış çubuk | <code>geom_bar(position = "dodge" / "fill")</code> |
| Zamanla nasıl değişiyor? | Zaman + sayısal | Çizgi grafiği | <code>geom_line()</code> |
| Veri normal dağılıma uyuyor mu? | 1 sayısal | Q-Q grafiği | <code>geom_qq()</code> +
<code>geom_qq_line()</code> |

⚠ Kaçınılması gerekenler

- **Pasta grafiği:** Göz açıları ve alanları karşılaştırmakta kötüdür; çubuk grafiği hemen her zaman daha iyidir.
- **Çift y eksenli grafikler:** Ölçekler keyfi seçilebildiği için yanıltıcıdır; iki ayrı grafik çizin.
- **3B çubuk grafikleri** ve gereksiz gölgeler: Veri eklemeyi, okumayı zorlaştırır.
- **Sıfırdan başlamayan çubuk grafiği:** Çubuğun boyu değeri temsil eder; eksen sıfırdan başlamalıdır.

6.3 base R Grafikleri

base R grafikleri **hızlıdır** ve veriye ilk bakış için idealdir. Her grafik tek bir fonksiyonla çizilir; başlık, eksen adı ve renk argümanlarla verilir. Bir grafik çizildikten sonra üzerine `points()`, `lines()`, `abline()`, `text()`, `legend()` ile ekleme yapılabilir.

| Fonksiyon | Grafik |
|-----------------------------------|---|
| <code>plot(x, y)</code> | Serpilme grafiği (x faktörse kutu grafiği!) |
| <code>hist(x)</code> | Histogram |
| <code>boxplot(y ~ grup)</code> | Gruplara göre kutu grafiği |
| <code>barplot(table(x))</code> | Çubuk grafiği |
| <code>pairs(veri)</code> | Serpilme grafiği matrisi |
| <code>qqnorm(x); qqline(x)</code> | Normal Q-Q grafiği |
| <code>coplot(y ~ x z)</code> | Koşullu serpilme grafiği |

Sık kullanılan argümanlar: `main` (başlık), `xlab/ylab` (eksen adları), `col` (renk), `pch` (nokta şekli, *plotting character*), `bg` (dolgu rengi), `cex` (boyut), `lty/lwd` (çizgi tipi/kalınlığı), `xlim/ylim` (eksen sınırları).

```
par(mfrow = c(2, 2)) # Grafik alanını 2x2'ye böl

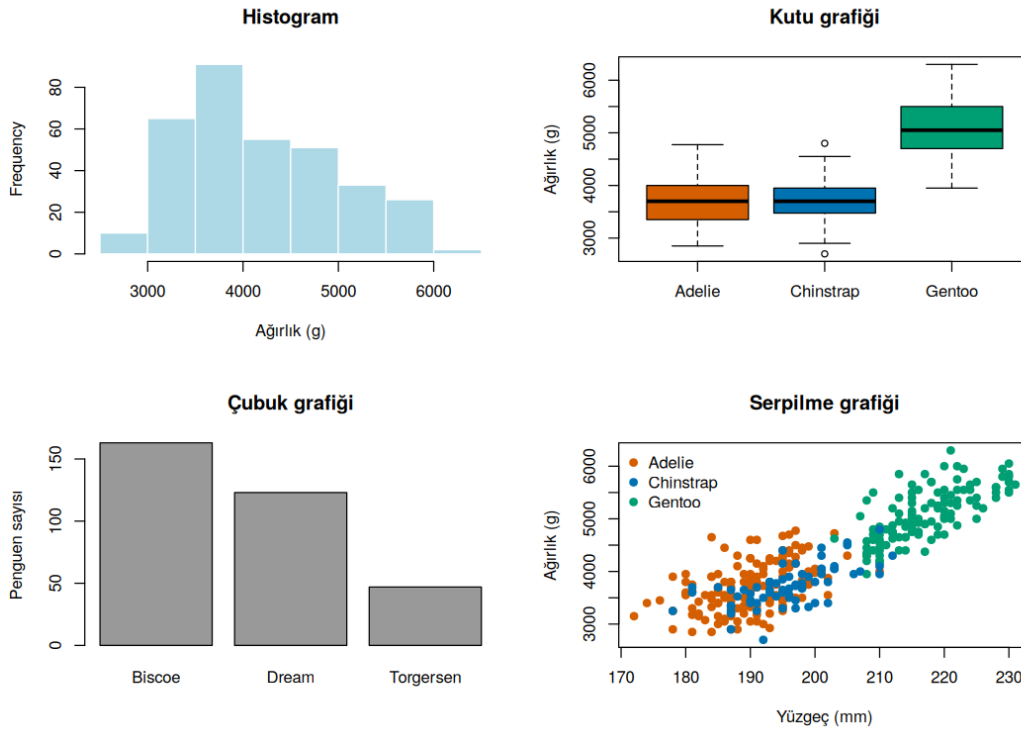
hist(penguenler$body_mass_g, main = "Histogram", xlab = "Ağırlık (g)",
     col = "lightblue", border = "white")
boxplot(body_mass_g ~ species, data = penguenler, main = "Kutu grafiği",
        xlab = "", ylab = "Ağırlık (g)", col = tur_renkleri)
```

```

barplot(table(penguenler$island), main = "Çubuk grafiği", col = "grey60",
        ylab = "Penguen sayısı")
plot(penguenler$flipper_length_mm, penguenler$body_mass_g, main = "Serpilme
  ↪ grafiği",
      xlab = "Yüzgeç (mm)", ylab = "Ağırlık (g)", pch = 19,
      col = tur_renkleri[penguenler$species])
legend("topleft", legend = names(tur_renkleri), col = tur_renkleri, pch = 19, bty =
  ↪ "n")

par(mfrow = c(1, 1)) # Eski hâline döndür

```



Şekil 6.2: base R ile dört temel grafik

6.3.1 Eski Notlardan: Worms Verisiyle Çizimler

```

plot(worms$Soil.pH, worms$Worm.density, pch = 21, bg = "red",
     xlab = "Toprak pH", ylab = "Solucan yoğunluğu")
plot(factor(worms$Vegetation), worms$Worm.density, main = "Bitki örtüsü",
     col = "lightgrey") # x faktör olduğu için kutu grafiği çizildi

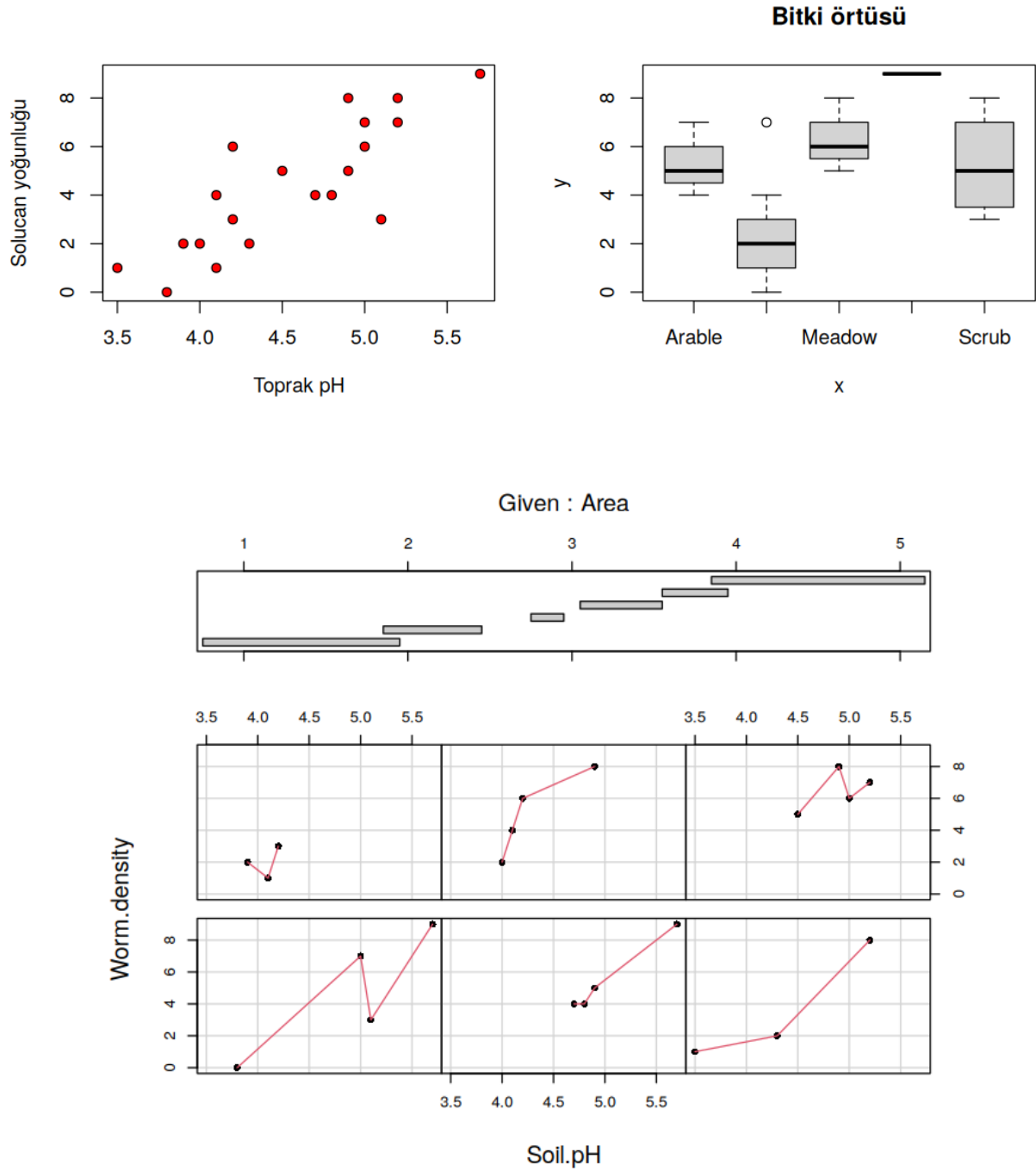
```

`coplot()` bir değişkenin aralıklarına (burada alan) göre koşullu serpilme grafikleri çizer. `panel.smooth` parametrik olmayan bir eğilim eğrisi ekler; `overlap` koşul aralıklarının ne kadar iç içe geçeceğini belirler:

```

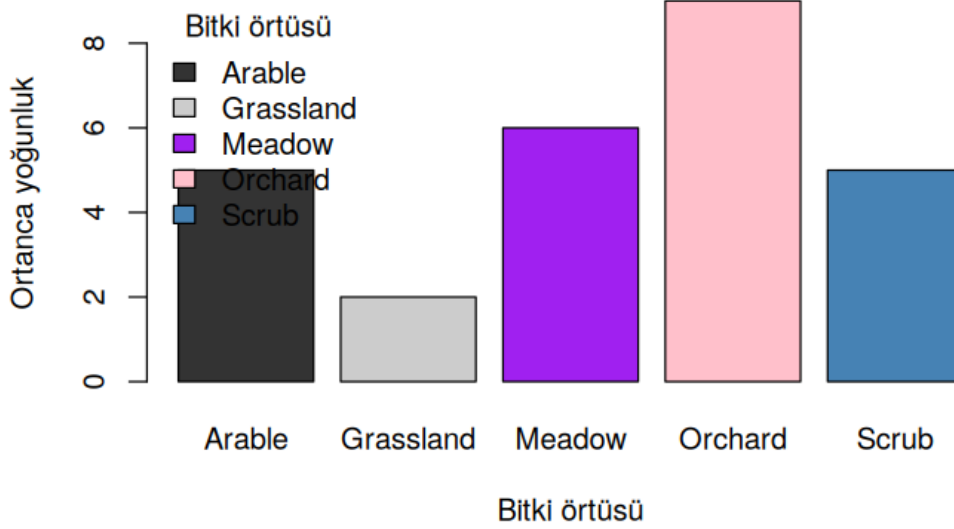
coplot(Worm.density ~ Soil.pH | Area, data = worms, pch = 16,
       panel = panel.smooth, overlap = 0.1)

```



Ortanca yoğunlukların çubuk grafiği ve açıklama kutusu:

```
orta_yogunluk <- tapply(worms$Worm.density, worms$Vegetation, median)
renkler <- c("grey20", "grey80", "purple", "pink", "steelblue")
barplot(orta_yogunluk, xlab = "Bitki örtüsü", ylab = "Ortanca yoğunluk", col =
  renkler)
legend("topleft", legend = names(orta_yogunluk), fill = renkler,
  title = "Bitki örtüsü", bty = "n")
```



i Not

Eski notlarda açıklama kutusunun yeri `legend(locator(1), ...)` ile fare tıklamasıyla seçiliyordu. `locator()` yalnızca etkileşimli oturumda çalışır; Quarto belgesinde `"topleft"`, `"topright"` gibi konum adları ya da koordinat kullanılmalıdır.

Grafiği dosyaya kaydetmek için RStudio'daki **Plots** → **Export** menüsü ya da kodla `png()` ... `dev.off()` kullanılır:

```
png("agirlik_histogram.png", width = 1600, height = 1200, res = 200)
hist(penguenler$body_mass_g)
dev.off()
```

6.4 ggplot2: Grafiklerin Dilbilgisi

ggplot2, grafiği katman katman kuran bir sistemdir (*Grammar of Graphics*, Wilkinson 2005). Her ggplot grafiği şu bileşenlerden oluşur:

Tablo 6.3: ggplot2'nin bileşenleri

| Bileşen | Soru | Örnek |
|-------------------------------|---|--|
| Veri (data) | Hangi veri tablosu? | <code>penguins</code> |
| Estetik eşleme (aes()) | Hangi değişken hangi görsel özelliğe gidecek? | <code>x = flipper_length_mm,</code>
<code>colour = species</code> |
| Geometri (geom_*()) | Veri hangi şekille çizilecek? | nokta, çubuk, kutu, çizgi |
| İstatistik (stat_*()) | Çizmeden önce veri dönüştürülecek mi? | histogram için sayma, eğilim doğrusu |

| Bileşen | Soru | Örnek |
|--------------------------------------|---|---|
| Ölçek (<code>scale_*()</code>) | Değerler görsel özelliklere nasıl çevrilecek? | renkler, eksen sınırları, log ölçek |
| Panel (<code>facet_*()</code>) | Grafik gruplara göre bölünecek mi? | her tür için ayrı panel |
| Koordinat (<code>coord_*()</code>) | Hangi koordinat sistemi? | <code>coord_flip()</code> ,
<code>coord_cartesian()</code> |
| Tema (<code>theme_*()</code>) | Veri dışı görünüm | yazı tipi, arka plan, ızgara |

Katmanlar **+** işareti ile eklenir:

```
ggplot(data = VERİ, aes(x = DEĞİŞKEN1, y = DEĞİŞKEN2)) +
  geom_XXX() +           # geometri
  labs(...) +           # başlık ve eksen adları
  theme_XXX()            # görünüm
```

6.4.1 Adım Adım Bir ggplot

```
# 1. Sadece veri ve eşleme: eksenler var, veri yok
ggplot(penguenler, aes(x = flipper_length_mm, y = body_mass_g))

# 2. Geometri ekleyince noktalar çizilir
ggplot(penguenler, aes(x = flipper_length_mm, y = body_mass_g)) +
  geom_point()

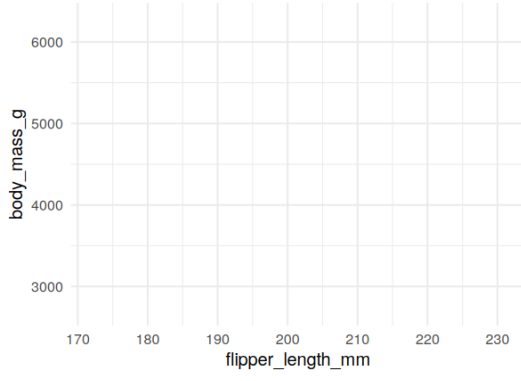
# 3. Rengi üçüncü bir değişkene eşleyelim
ggplot(penguenler, aes(x = flipper_length_mm, y = body_mass_g, colour = species)) +
  geom_point(alpha = 0.7)

# 4. Eğilim doğruları, başlık, eksen adları ve sabit renkler
ggplot(penguenler, aes(x = flipper_length_mm, y = body_mass_g, colour = species)) +
  geom_point(alpha = 0.7) +
  geom_smooth(method = "lm", se = FALSE) +
  scale_colour_manual(values = tur_renkleri) +
  labs(title = "Yüzgeç uzunluğu ve ağırlık",
       x = "Yüzgeç uzunluğu (mm)", y = "Ağırlık (g)", colour = "Tür")
```

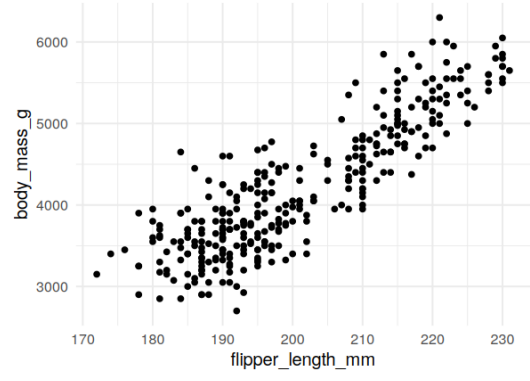
! Eşleme (mapping) ile sabit değer (setting) farkı

- `aes(colour = species)` → Renk **veriye göre değişir**. Değişken adı **aes()** içine yazılır.
- `geom_point(colour = "blue")` → **Tüm noktalar** mavi olur. Sabit değer **aes()** dışına yazılır.

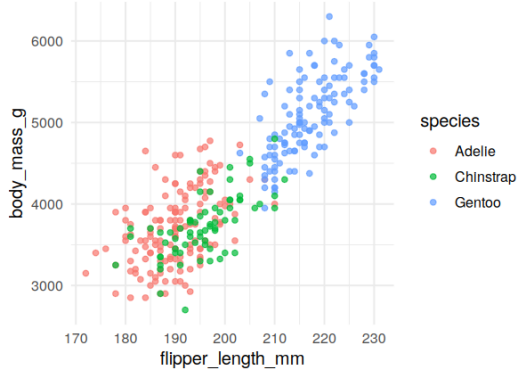
`aes(colour = "blue")` yazmak yaygın bir hatadır: R “blue” adında tek kategorili bir değişken varmış gibi davranır ve noktalar pembemsi bir renkte, “blue” yazan bir açıklama kutusuyla çıkar.



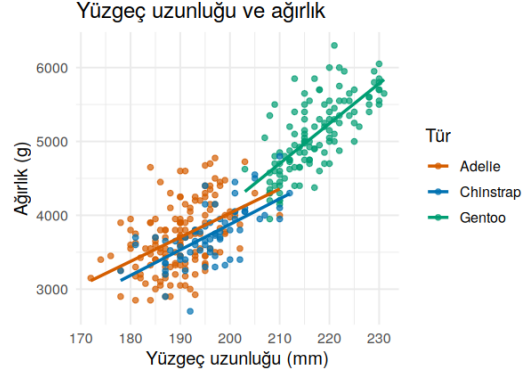
(a) Veri ve eşleme: boş tuval



(b) + geom_point()



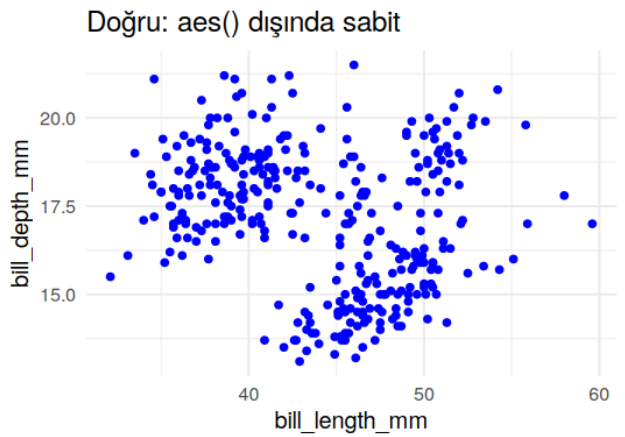
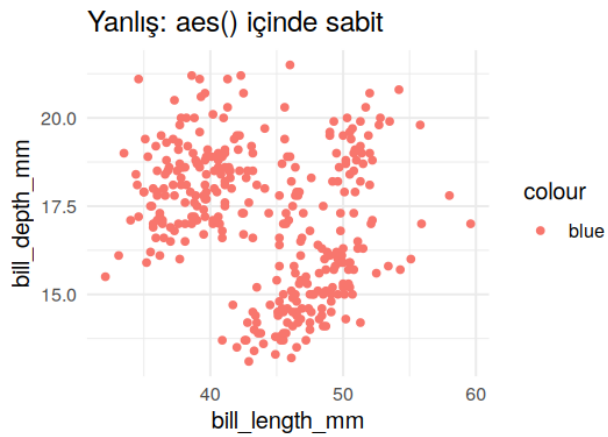
(c) + renk eşlemesi



(d) + eğilim doğruları, etiketler, ölçek

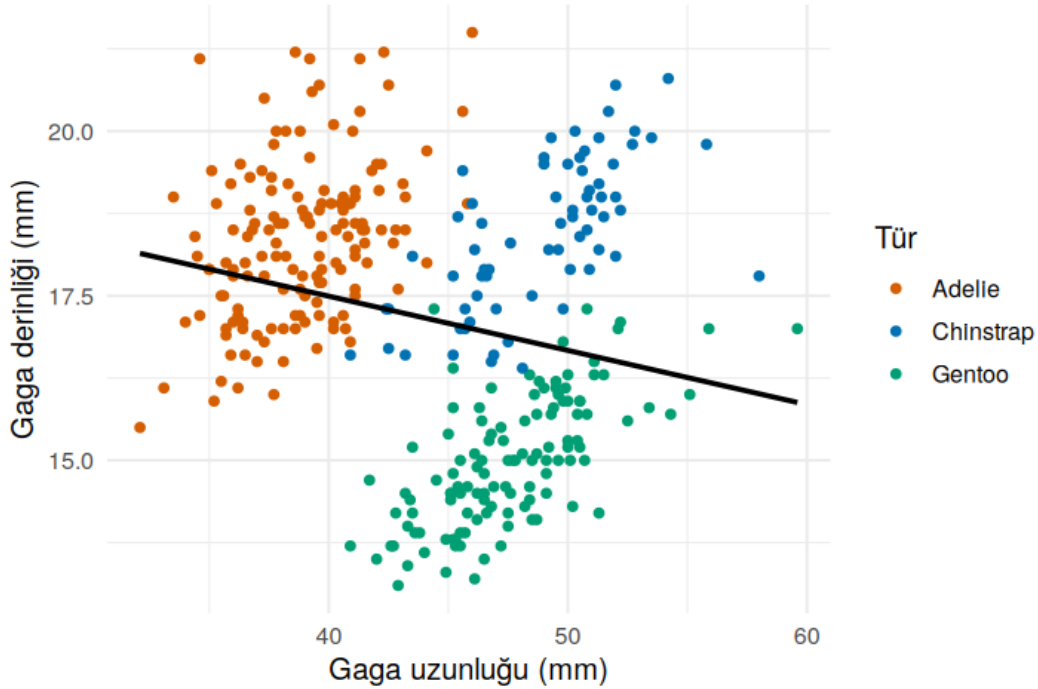
Şekil 6.3: Bir ggplot grafiğinin katman katman kurulması

```
ggplot(penguenler, aes(bill_length_mm, bill_depth_mm)) +
  geom_point(aes(colour = "blue")) + labs(title = "Yanlış: aes() içinde sabit")
ggplot(penguenler, aes(bill_length_mm, bill_depth_mm)) +
  geom_point(colour = "blue") + labs(title = "Doğru: aes() dışında sabit")
```



`aes()` içine `ggplot()`'a yazılan eşlemeler **bütün katmanlara**, bir `geom`'a yazılanlar yalnızca **o katmana** uygulanır. Aşağıda renk yalnızca noktalara verildiği için tek bir (genel) eğilim doğrusu çizilir:

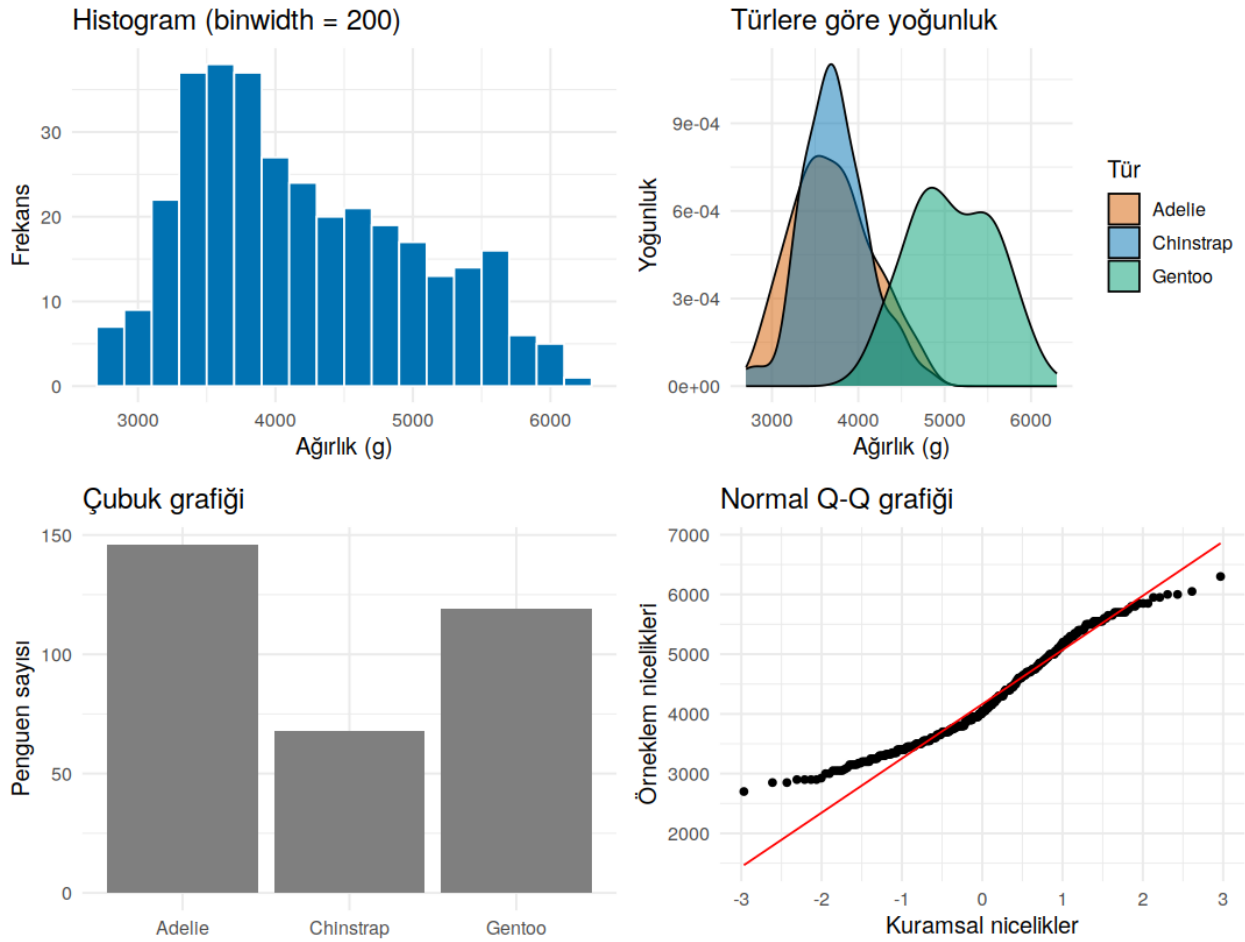
```
ggplot(penguenler, aes(bill_length_mm, bill_depth_mm)) +
  geom_point(aes(colour = species)) +
  geom_smooth(method = "lm", se = FALSE, colour = "black") +
  scale_colour_manual(values = tur_renkleri) +
  labs(x = "Gaga uzunluğu (mm)", y = "Gaga derinliği (mm)", colour = "Tür")
```



Bu grafik önemli bir şey gösteriyor: Tüm penguenlere bakınca gaga uzunluğu ile derinliği arasında **negatif** bir ilişki görünür, oysa her türün içinde ilişki **pozitif**tir. Buna **Simpson paradoksu** denir. Gruplayıcı bir değişkeni göz ardı etmek yanlış sonuca götürebilir.

6.4.2 Tek Değişkenin Dağılımı

```
ggplot(penguenler, aes(x = body_mass_g)) +
  geom_histogram(binwidth = 200, fill = "#0072B2", colour = "white") +
  labs(title = "Histogram (binwidth = 200)", x = "Ağırlık (g)", y = "Frekans")
ggplot(penguenler, aes(x = body_mass_g, fill = species)) +
  geom_density(alpha = 0.5) +
  scale_fill_manual(values = tur_renkleri) +
  labs(title = "Türlere göre yoğunluk", x = "Ağırlık (g)", y = "Yoğunluk", fill =
    ↪ "Tür")
ggplot(penguenler, aes(x = species)) +
  geom_bar(fill = "grey50") +
  labs(title = "Çubuk grafiği", x = NULL, y = "Penguen sayısı")
ggplot(penguenler, aes(sample = body_mass_g)) +
  geom_qq() + geom_qq_line(colour = "red") +
  labs(title = "Normal Q-Q grafiği", x = "Kuramsal nicelikler", y = "Örneklem
    ↪ nicelikleri")
```



💡 Histogramda kutu genişliği

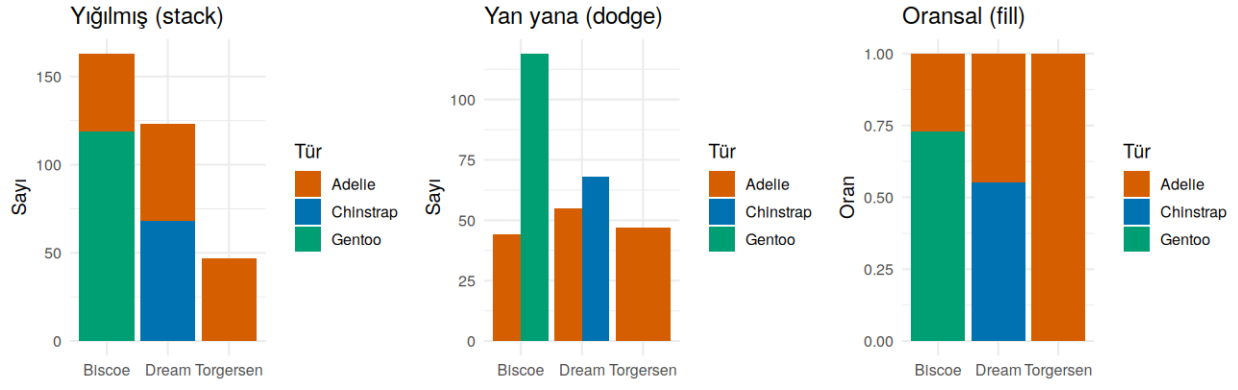
Histogramın görünümü kutu genişliğine (binwidth) ya da kutu sayısına (bins) çok bağlıdır. Varsayılan bins = 30 her veri için uygun değildir; birkaç farklı değer deneyin.

6.4.3 İki Kategorik Değişken

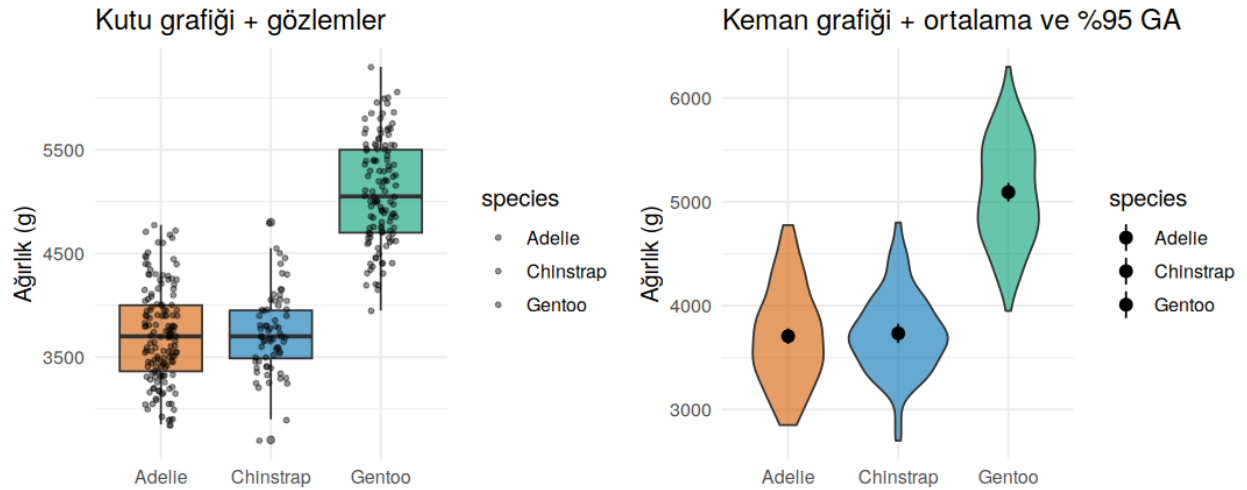
```
p <- ggplot(penguenler, aes(x = island, fill = species)) +
  scale_fill_manual(values = tur_renkleri) +
  labs(x = NULL, fill = "Tür")
p + geom_bar() + labs(title = "Yığılmış (stack)", y = "Sayı")
p + geom_bar(position = "dodge") + labs(title = "Yan yana (dodge)", y = "Sayı")
p + geom_bar(position = "fill") + labs(title = "Oransal (fill)", y = "Oran")
```

Gentoo penguenleri yalnızca Biscoe adasında, Chinstrap penguenleri yalnızca Dream adasında yaşar. Bu tür ilişkileri Bölüm 11’te ki-kare testi ile sınayacağız.

6.4.4 Sayısal Değişken ile Kategorik Değişken



```
ggplot(penguenler, aes(x = species, y = body_mass_g, fill = species)) +
  geom_boxplot(show.legend = FALSE, alpha = 0.6) +
  geom_jitter(width = 0.15, alpha = 0.4, size = 1) + # Gözlemleri de göster
  scale_fill_manual(values = tur_renkleri) +
  labs(title = "Kutu grafiği + gözlemler", x = NULL, y = "Ağırlık (g)")
ggplot(penguenler, aes(x = species, y = body_mass_g, fill = species)) +
  geom_violin(show.legend = FALSE, alpha = 0.6) +
  stat_summary(fun.data = mean_cl_normal, colour = "black") + # Ortalama ± %95 GA
  scale_fill_manual(values = tur_renkleri) +
  labs(title = "Keman grafiği + ortalama ve %95 GA", x = NULL, y = "Ağırlık (g)")
```



i Kutu grafiğini okumak

Kutunun alt ve üst kenarları 1. ve 3. çeyrekliği (Q_1 , Q_3), ortadaki çizgi **ortancayı** gösterir. Bıyıklar kutudan en fazla $1.5 \times \text{ÇAG}$ (çeyrekler arası genişlik) uzaklıktaki en uç gözleme kadar uzanır; bunun dışındaki noktalar **aykırı değer adayıdır** (Bölüm 7).

6.4.5 Paneller (facet)

Aynı grafiği bir kategorik değişkenin her düzeyi için ayrı panelde çizmek (*small multiples*):

```
ggplot(penguenler, aes(x = flipper_length_mm, y = body_mass_g, colour = species)) +
  geom_point(alpha = 0.7, show.legend = FALSE) +
  scale_colour_manual(values = tur_renkleri) +
  facet_grid(sex ~ species) +
  labs(x = "Yüzgeç uzunluğu (mm)", y = "Ağırlık (g)")
```



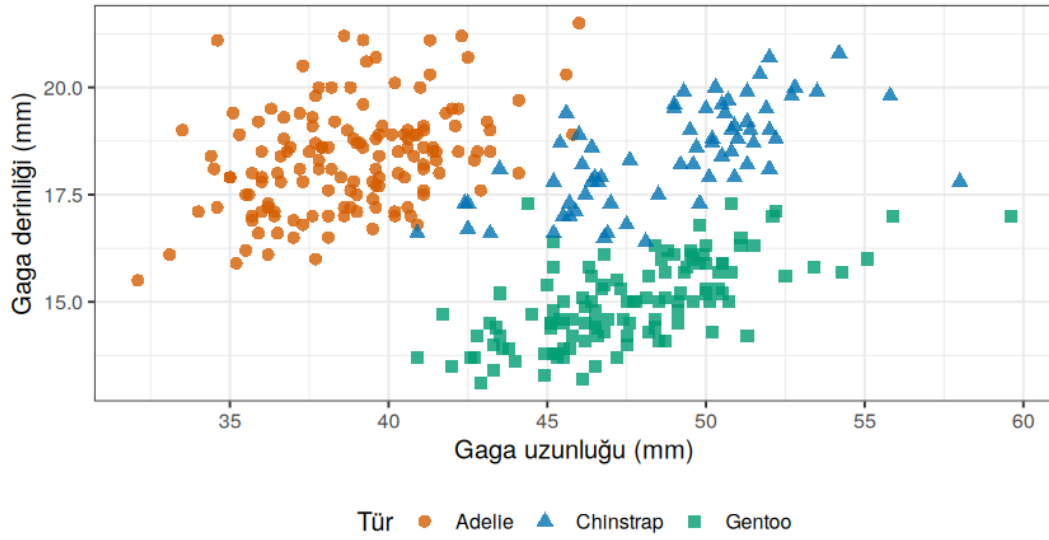
`facet_wrap(~ değişken)` tek bir değişkene göre panelleri satırlara sarar; `facet_grid(satır ~ sütun)` iki değişkene göre bir ızgara oluşturur.

6.4.6 Ölçekler, Etiketler ve Temalar

```
ggplot(penguenler, aes(x = bill_length_mm, y = bill_depth_mm,
  colour = species, shape = species)) +
  geom_point(size = 2.5, alpha = 0.8) +
  scale_colour_manual(values = tur_renkleri) +
  scale_x_continuous(breaks = seq(30, 60, by = 5)) +
  labs(
    title = "Gaga ölçüleri türleri iyi ayırıyor",
    subtitle = "Palmer Takımadaları, 2007-2009",
    x = "Gaga uzunluğu (mm)", y = "Gaga derinliği (mm)",
    colour = "Tür", shape = "Tür",
    caption = "Kaynak: palmerpenguins paketi"
  ) +
  theme_bw(base_size = 12) +
  theme(legend.position = "bottom", plot.title = element_text(face = "bold"))
```

Gaga ölçüleri türleri iyi ayırıyor

Palmer Takımadaları, 2007–2009



Kaynak: palmerpenguins paketi

Renk ile birlikte **şekil** (shape) eşlemek, grafiği renk körü okuyucular ve siyah-beyaz baskı için de okunur kılar.

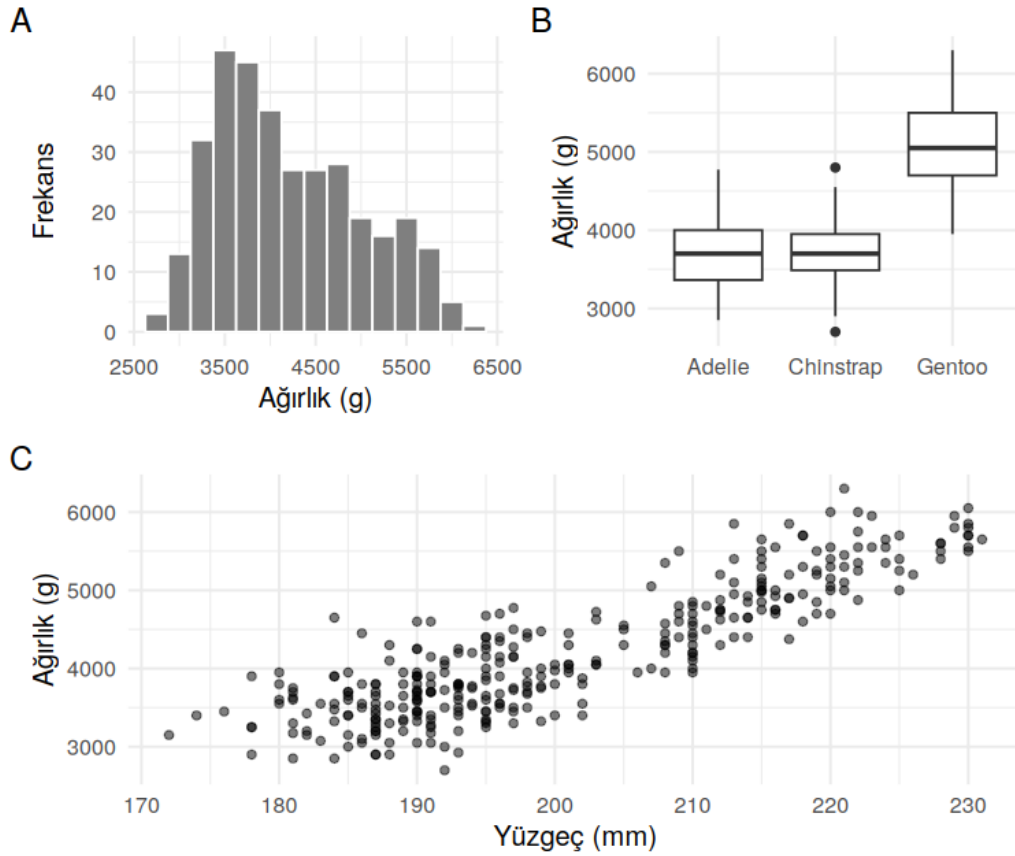
| Amaç | Fonksiyon |
|--|---|
| Renkleri elle belirleme | <code>scale_colour_manual(values = ...)</code> , <code>scale_fill_manual()</code> |
| Renk körlüğüne uygun hazır palet | <code>scale_colour_viridis_d()</code> (kategorik),
<code>scale_colour_viridis_c()</code> (sürekli) |
| Eksen sınırları (veriyi atmadan yakınlaştırma) | <code>coord_cartesian(xlim = ..., ylim = ...)</code> |
| Logaritmik eksen | <code>scale_x_log10()</code> |
| Eksenleri çevirme | <code>coord_flip()</code> ya da x ile y'yi yer değiştirme |
| Hazır temalar | <code>theme_minimal()</code> , <code>theme_bw()</code> , <code>theme_classic()</code> |
| Açıklama kutusunun yeri | <code>theme(legend.position = "bottom")</code> ya da "none" |

6.4.7 Grafikleri Birleştirme

patchwork paketi ile ggplot nesneleri + (yan yana) ve / (alt alta) ile birleştirilir:

```
p1 <- ggplot(penguenler, aes(body_mass_g)) +
  geom_histogram(binwidth = 250, fill = "grey50", colour = "white") +
  labs(x = "Ağırlık (g)", y = "Frekans")
p2 <- ggplot(penguenler, aes(species, body_mass_g)) +
  geom_boxplot() + labs(x = NULL, y = "Ağırlık (g)")
p3 <- ggplot(penguenler, aes(flipper_length_mm, body_mass_g)) +
  geom_point(alpha = 0.5) + labs(x = "Yüzgeç (mm)", y = "Ağırlık (g)")

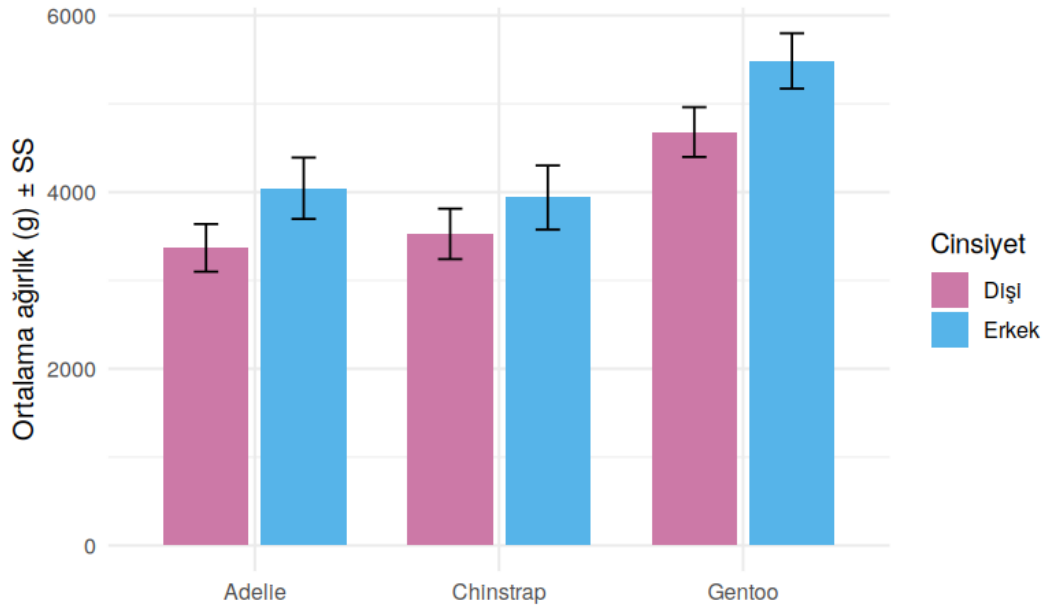
(p1 + p2) / p3 + plot_annotation(tag_levels = "A")
```



6.4.8 dplyr + ggplot2 Birlikte

Özet tabloyu dplyr ile üretip doğrudan ggplot2'ye aktarabiliriz. Dikkat: dplyr adımları |> ile, ggplot katmanları + ile bağlanır.

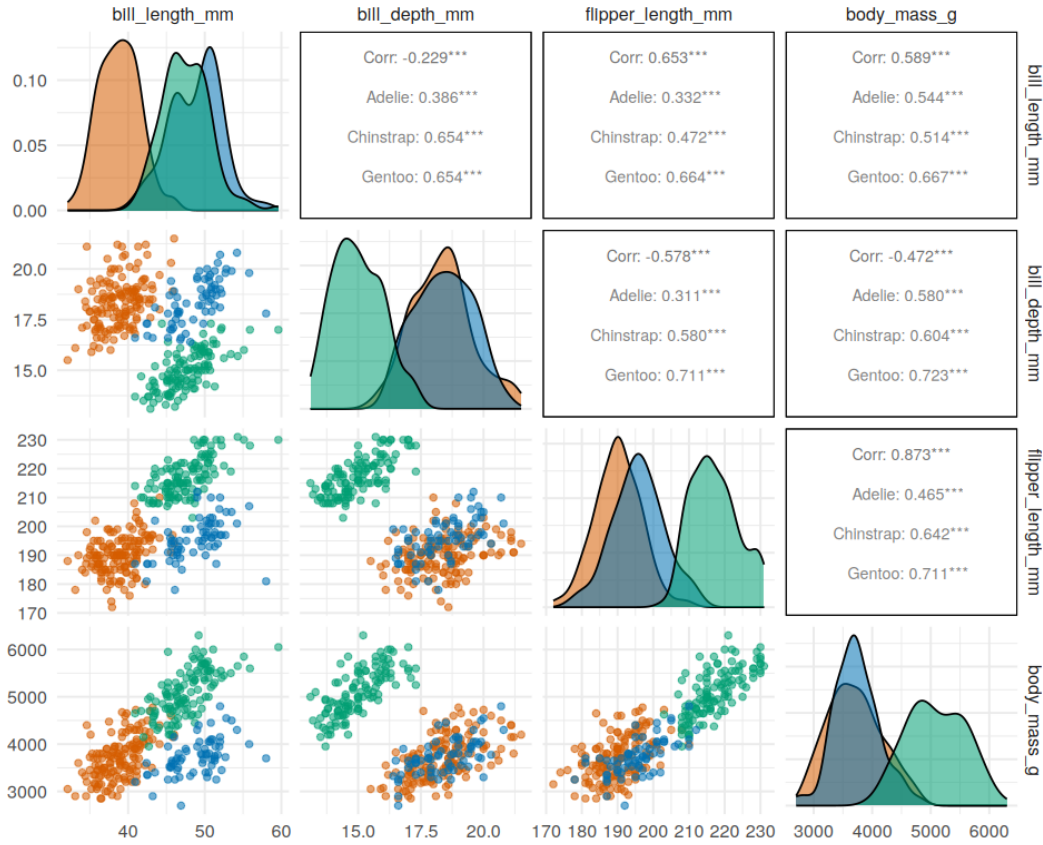
```
penguenler |>
  group_by(species, sex) |>
  summarise(ort = mean(body_mass_g), ss = sd(body_mass_g), .groups = "drop") |>
  ggplot(aes(x = species, y = ort, fill = sex)) +
  geom_col(position = position_dodge(width = 0.8), width = 0.7) +
  geom_errorbar(aes(ymin = ort - ss, ymax = ort + ss),
               position = position_dodge(width = 0.8), width = 0.2) +
  scale_fill_manual(values = c(female = "#CC79A7", male = "#56B4E9"),
                   labels = c("Dişi", "Erkek")) +
  labs(x = NULL, y = "Ortalama ağırlık (g) ± SS", fill = "Cinsiyet")
```



6.4.9 Çok Değişkene Birden Bakmak

`pairs()` (base R) ve `GGally::ggpairs()` tüm sayısal değişken çiftlerinin serpilme grafiklerini ve korelasyonlarını tek bir matriste gösterir. Regresyondan önce çok kullanışlıdır (Bölüm 13):

```
GGally::ggpairs(  
  penguenler, columns = 3:6, aes(colour = species, alpha = 0.5),  
  upper = list(continuous = GGally::wrap("cor", size = 3))  
) +  
  scale_colour_manual(values = tur_renkleri) +  
  scale_fill_manual(values = tur_renkleri)
```



6.4.10 Grafiği Kaydetme

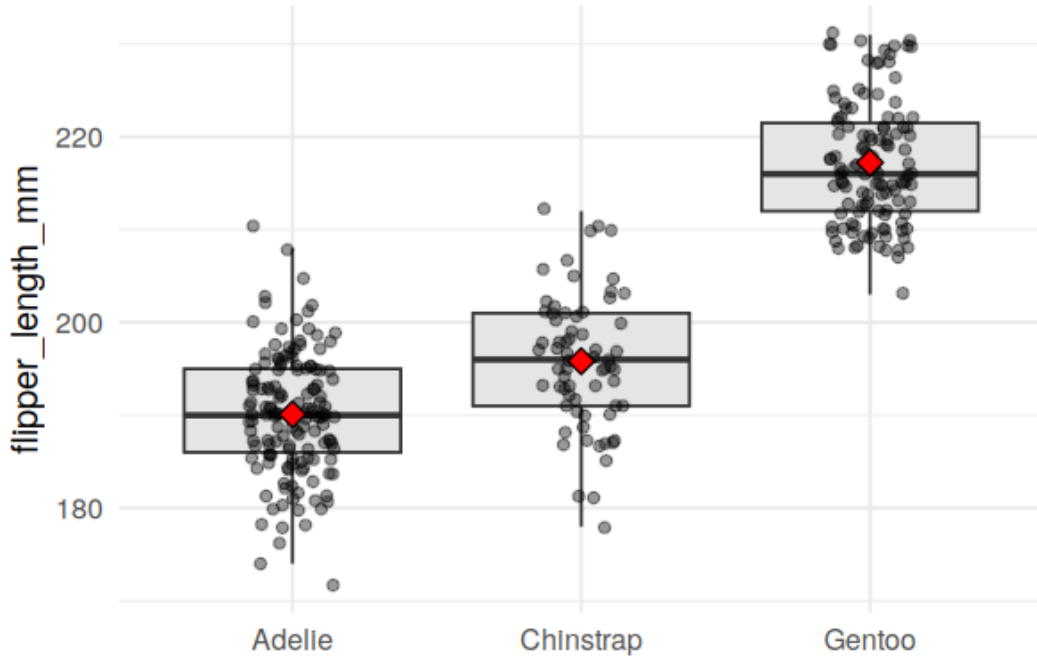
```
g <- ggplot(penguenler, aes(flipper_length_mm, body_mass_g)) + geom_point()
ggsave("yuzgec_agirlik.png", plot = g, width = 6, height = 4, dpi = 300)
ggsave("yuzgec_agirlik.pdf", plot = g, width = 6, height = 4)
```

6.5 Grafik Çizen Fonksiyonlar Yazmak

Aynı grafiği farklı değişkenler için tekrar tekrar çiziyorsanız bir fonksiyon yazın. Bölüm 5'te gördüğümüz { } burada da çalışır:

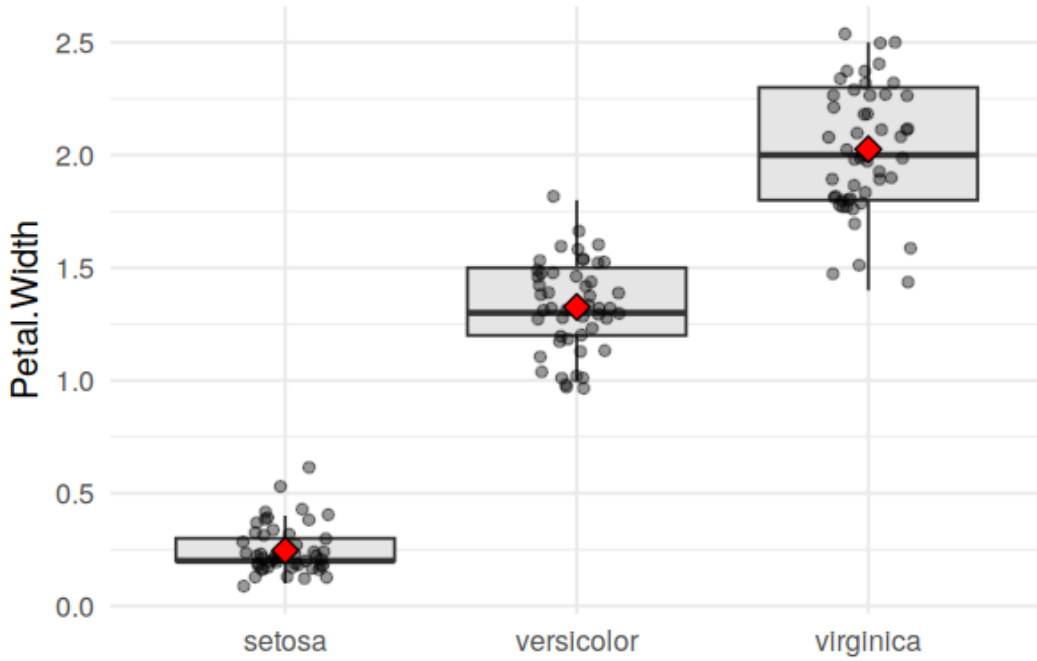
```
grup_kutu <- function(veri, grup, degisken, baslik = NULL) {
  ggplot(veri, aes(x = {{ grup }}, y = {{ degisken }})) +
    geom_boxplot(fill = "grey90", outlier.shape = NA) +
    geom_jitter(width = 0.15, alpha = 0.4) +
    stat_summary(fun = mean, geom = "point", shape = 23, size = 3, fill = "red") +
    labs(title = baslik, x = NULL)
}
grup_kutu(penguenler, species, flipper_length_mm, "Türlere göre yüzgeç uzunluğu")
```

Türlere göre yüzgeç uzunluğu



```
grup_kutu(iris, Species, Petal.Width, "iris: taç yaprak genişliği")
```

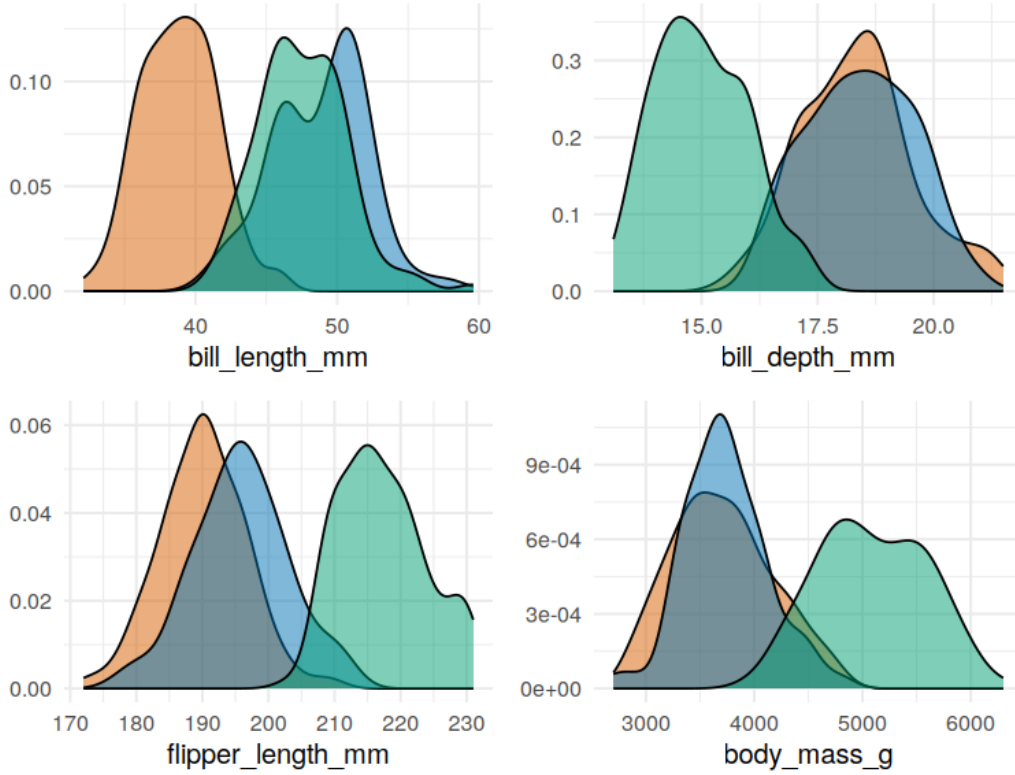
iris: taç yaprak genişliği



Değişken adı bir **karakter** olarak geliyorsa (örneğin bir döngüde) `.data[[ ]]` kullanılır. Böylece bir veri setinin bütün sayısal değişkenleri için histogram çizen bir döngü yazılabilir:

```
degiskenler <- c("bill_length_mm", "bill_depth_mm", "flipper_length_mm",  
  ↪ "body_mass_g")  
grafikler <- list()
```

```
for (d in degiskenler) {
  grafikler[[d]] <- ggplot(penguenler, aes(x = .data[[d]], fill = species)) +
    geom_density(alpha = 0.5, show.legend = FALSE) +
    scale_fill_manual(values = tur_renkleri) +
    labs(y = NULL)
}
wrap_plots(grafikler, ncol = 2)
```



6.6 plotly: Etkileşimli Grafikler

plotly ile çizilen grafiklerde fareyle üzerine gelindiğinde değerler görünür, yakınlaştırma yapılabilir, açıklama kutusundaki gruplar tıklanarak açılıp kapatılabilir. Veriyi **keşfetmek**, aykırı gözlemleri bulmak ve HTML raporları zenginleştirmek için çok kullanışlıdır. Etkileşimli grafikler RStudio'nun **Viewer** penceresinde ve **HTML** çıktılarda çalışır (Word/PDF'te çalışmaz).

plotly'yi iki şekilde kullanabiliriz.

i Not

Bu kitabın PDF sürümünde etkileşimli plotly grafikleri gösterilmez; kodlar aşağıdadır. Grafikleri görmek için kodu RStudio'da çalıştırın ya da kitabın HTML sürümüne bakın.

6.6.1 1) ggplotly(): Hazır ggplot'u Etkileşimli Yapmak

En kolay yol: Bildiğimiz ggplot grafiğini bir nesneye atayıp **ggplotly()** içine vermek. **text** estetiği fareyle üzerine gelindiğinde görünecek bilgiyi belirler:

```
g <- ggplot(penguenler, aes(x = flipper_length_mm, y = body_mass_g, colour = species,
                           text = paste("Ada:", island, "<br>Cinsiyet:", sex))) +
  geom_point(alpha = 0.7) +
  scale_colour_manual(values = tur_renkleri) +
  labs(x = "Yüzgeç uzunluğu (mm)", y = "Ağırlık (g)", colour = "Tür")
ggplotly(g)
```

6.6.2 2) plot_ly(): Doğrudan plotly Sözdizimi

plot_ly() fonksiyonunda değişkenler başlarına **tilde (~)** konularak verilir; grafik türü **type** (ve gerekirse **mode**) argümanı ile seçilir. Katmanlar **+** ile değil, **pipe (|>)** ile eklenir.

Tablo 6.5: ggplot2 ve plotly sözdizimi karşılaştırması

| ggplot2 | plotly |
|-----------------------------------|--|
| aes(x = body_mass_g) | x = ~body_mass_g |
| aes(colour = species) | color = ~species |
| scale_colour_manual(values = ...) | colors = ... |
| geom_point() | type = "scatter", mode = "markers" |
| geom_line() | type = "scatter", mode = "lines" |
| geom_histogram() | type = "histogram" |
| geom_boxplot() | type = "box" |
| geom_violin() | type = "violin" |
| geom_col() | type = "bar" |
| labs(...) | layout(title = ..., xaxis = list(title = ...)) |
| + | > |

```
plot_ly(penguenler, x = ~flipper_length_mm, y = ~body_mass_g, color = ~species,
        colors = tur_renkleri, type = "scatter", mode = "markers",
        text = ~paste("Ada:", island), hoverinfo = "text+x+y") |>
  layout(title = "Yüzgeç uzunluğu ve ağırlık",
        xaxis = list(title = "Yüzgeç uzunluğu (mm)"),
        yaxis = list(title = "Ağırlık (g)"))
```

```
plot_ly(penguenler, x = ~species, y = ~body_mass_g, color = ~species,
        colors = tur_renkleri, type = "box", boxpoints = "all", jitter = 0.3) |>
  layout(showlegend = FALSE, xaxis = list(title = ""), yaxis = list(title = "Ağırlık
↵ (g)"))
```

```
plot_ly(penguenler, x = ~body_mass_g, color = ~species, colors = tur_renkleri,
        type = "histogram", alpha = 0.6) |>
  layout(barmode = "overlay", xaxis = list(title = "Ağırlık (g)"),
        yaxis = list(title = "Frekans"))
```

Üç sayısal değişkeni aynı anda incelemek için plotly'nin **3B serpilme grafiği** kullanılabilir. Grafiği fareyle döndürün:

```
plot_ly(penguenler, x = ~bill_length_mm, y = ~bill_depth_mm, z = ~flipper_length_mm,
        color = ~species, colors = tur_renkleri, type = "scatter3d", mode = "markers",
        marker = list(size = 3)) |>
  layout(scene = list(xaxis = list(title = "Gaga uz."),
                        yaxis = list(title = "Gaga der."),
                        zaxis = list(title = "Yüzgeç")))
```

Birden çok plotly grafiğini yan yana koymak için `subplot()` kullanılır:

```
s1 <- plot_ly(iris, x = ~Species, y = ~Sepal.Length, type = "box", name = "Çanak")
s2 <- plot_ly(iris, x = ~Species, y = ~Petal.Length, type = "box", name = "Taç")
subplot(s1, s2, shareY = TRUE) |> layout(title = "iris: çanak ve taç yaprak
↪ uzunlukları")
```

💡 Hangisini ne zaman kullanalım?

- **Rapor / makale / sunum / PDF** için statik grafik → **ggplot2**
- **Veriyi keşfetmek**, aykırı gözlemleri bulmak, HTML rapor ya da Bölüm 14'te göreceğimiz **Shiny** uygulamaları → **plotly** (çoğu zaman `ggplotly()` yeterlidir)
- Hızlı bir kontrol grafiği → **base R**

Kendini Sına

💡 1. `mtcars` verisinde ağırlık (`wt`) ile yakıt tüketimi (`mpg`) arasındaki ilişkiyi silindir sayısına göre renklendirerek çizin.

```
ggplot(mtcars, aes(wt, mpg, colour = factor(cyl))) +
  geom_point(size = 2.5) +
  labs(x = "Ağırlık (1000 lb)", y = "Mil/galon", colour = "Silindir")
```

`cyl` sayısal olduğu için `factor()` yazılmazsa renkler sürekli bir ölçekle (açık-koyu mavi) verilir.

💡 2. Aşağıdaki kodda noktalar neden mavi değil?

```
ggplot(iris, aes(Sepal.Length, Petal.Length, colour = "blue")) + geom_point()
```

"blue" bir değişken değil sabit bir değerdir ve `aes()` içine yazılmıştır. `ggplot` onu tek düzeyli bir kategori sanar ve varsayılan paletin ilk rengini kullanır. Doğrusu `geom_point(colour = "blue")`.

💡 3. `iris` verisinde dört ölçümün türlere göre dağılımını tek bir grafikte gösterin.

Önce veriyi uzun biçime çevirin (Bölüm 4), sonra panellere ayırın:

```
iris |>
  tidyr::pivot_longer(-Species, names_to = "olcum", values_to = "deger") |>
  ggplot(aes(Species, deger, fill = Species)) +
  geom_boxplot(show.legend = FALSE) +
  facet_wrap(~ olcum, scales = "free_y")
```

💡 4. Önceki grafiği `ggplotly()` ile etkileşimli yapın. Hangi tür hangi ölçümde en çok örtüşüyor?

Grafiği bir nesneye atayıp `ggplotly(nesne)` çalıştırın. *versicolor* ve *virginica* çanak yaprak ölçümlerinde (**Sepal.***) çok örtüşür; taç yaprak ölçümleri (**Petal.***) türleri çok daha iyi ayırır.

💡 5. `grup_kutu()` fonksiyonuna, kutuları ortancaya göre sıralayan bir seçenek ekleyin.

`aes()` içinde `x = reorder({{ grup }}, {{ degisken }}, FUN = median)` kullanılabilir. `reorder()` bir faktörün düzeylerini başka bir değişkenin özetine göre sıralar.

7 Betimsel İstatistikler

i Bu haftanın hedefleri

- **Konum** ölçülerini (ortalama, ortanca, tepe değeri, budanmış ortalama, çeyreklikler) hesaplamak ve ne zaman hangisinin kullanılacağına karar vermek,
- **Değişim** ölçülerini (değişim genişliği, varyans, standart sapma, ÇAG, MAD, değişim katsayısı) hesaplamak; varyansın neden $n - 1$ 'e bölüldüğünü tartışmak,
- **Şekil** ölçülerini (çarpıklık ve basıklık) kendi fonksiyonlarımızla hesaplamak,
- Standart (z) değerleri ve aykırı değerleri belirlemek,
- Kategorik değişkenler için frekans ve çapraz tablolar oluşturmak,
- Gruplara göre özet tabloları base R ve **dplyr** ile üretip raporlamak.

Betimsel istatistikler, veriyi **özetleyen** sayılardır. Bir değişkenin dağılımını üç soruyla özetleriz:

1. **Merkez nerede?** → Konum ölçüleri
2. **Değerler ne kadar yayılmış?** → Değişim ölçüleri
3. **Dağılımın şekli nasıl?** → Çarpıklık, basıklık, aykırı değerler

Her zaman olduğu gibi önce grafiğe, sonra sayılara bakarız (Bölüm 6).

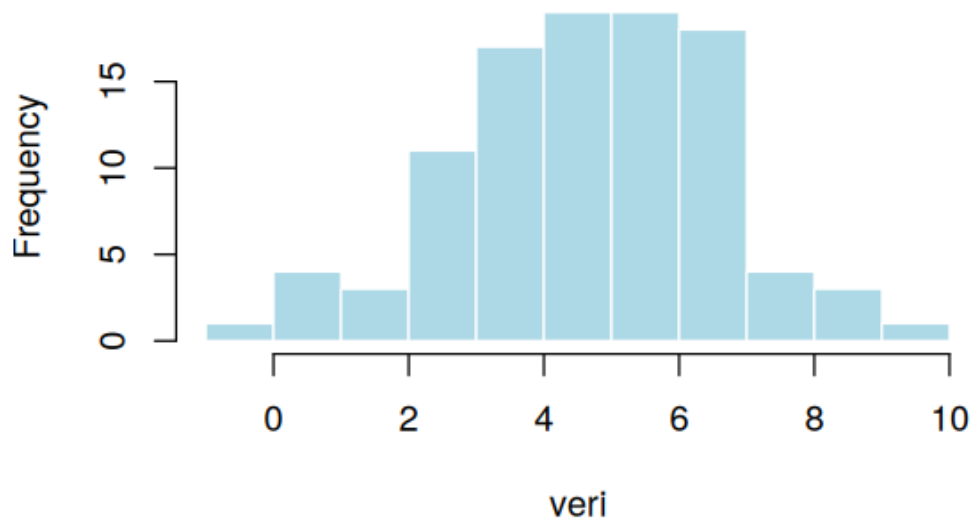
7.1 Konum Ölçüleri

7.1.1 Aritmetik Ortalama

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Eski ders notlarımızdaki gibi, ortalamayı önce adım adım hesaplayıp sonra `mean()` ile doğrulayalım:

```
set.seed(347)
veri <- rnorm(100, mean = 5, sd = 2) # Normal dağılımdan 100 gözlem (@sec-olasilik)
hist(veri, col = "lightblue", border = "white", main = "", xlab = "veri")
```



```
toplam <- sum(veri)
n <- length(veri)
toplam / n
```

```
#> [1] 4.67
```

```
mean(veri)
```

```
#> [1] 4.67
```

7.1.2 Ortanca (Medyan)

Sıralanmış verinin ortasındaki değerdir. Bölüm 5'te yazdığımız `ortanca()` fonksiyonu ile R'nin `median()` fonksiyonu aynı sonucu verir. Ortanca, **aykırı değerlerden etkilenmez**:

```
maaslar <- c(28, 30, 31, 33, 35, 36, 38, 40, 42, 400) # bin TL, biri genel müdür
mean(maaslar)
```

```
#> [1] 71.3
```

```
median(maaslar)
```

```
#> [1] 35.5
```

Tek bir aykırı değer ortalamayı 33.5'ten 71.3'e çıkarırken ortanca neredeyse hiç değişmedi.

7.1.3 Tepe Değer (Mod)

En sık görülen değerdir. R'da tepe değer için hazır bir fonksiyon yoktur (`mode()` başka bir iş yapar: nesnenin depolama tipini verir). Kendimiz yazalım:

```
tepe_deger <- function(x) {  
  frekans <- table(x)  
  names(frekans)[frekans == max(frekans)]  
}  
tepe_deger(worms$Vegetation)
```

```
#> [1] "Grassland"
```

```
tepe_deger(c(2, 3, 3, 5, 5, 7))    # İki tepe değer
```

```
#> [1] "3" "5"
```

Tepe değer özellikle **kategorik** değişkenler için anlamlıdır.

7.1.4 Budanmış Ortalama

Verinin her iki ucundan belirli bir oranda gözlem atılarak hesaplanan ortalamadır; ortalama ile ortanca arasında bir uzlaşmadır:

```
mean(maaslar, trim = 0.1)    # Her iki uçtan %10 (biri gözlem) atıldı
```

```
#> [1] 35.62
```

7.1.5 Çeyreklikler ve Yüzdelikler

p . yüzdelik, verinin % p 'sinin altında kaldığı değerdir. Q_1 (%25), Q_2 (%50 = ortanca) ve Q_3 (%75) çeyrekliklerdir.

```
quantile(penguenler$body_mass_g)
```

```
#>   0%   25%   50%   75%  100%  
#> 2700 3550 4050 4775 6300
```

```
quantile(penguenler$body_mass_g, probs = c(0.05, 0.95))
```

```
#>   5%   95%  
#> 3150 5670
```

```
fivenum(penguenler$body_mass_g)    # Tukey'in beş sayı özeti
```

```
#> [1] 2700 3550 4050 4775 6300
```

```
summary(penguenler$body_mass_g)
```

```
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>   2700   3550   4050   4207   4775   6300
```

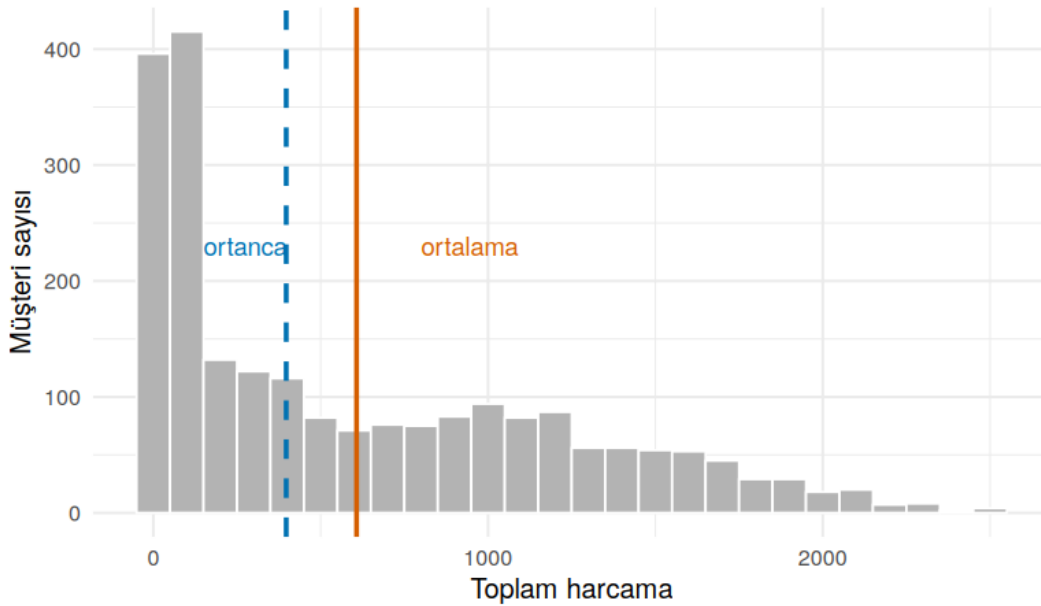
i Not

Yüzdelikleri hesaplamının birden fazla yöntemi vardır; `quantile()` varsayılan olarak 7. yöntemi kullanır (`type = 7`). SPSS ve Excel farklı yöntemler kullanabildiği için küçük veri setlerinde sonuçlar biraz farklı çıkabilir.

7.1.6 Ortalama mı, Ortanca mı?

Simetrik dağılımlarda ortalama ile ortanca birbirine yakındır. **Sağa çarpık** (uzun kuyruğu sağda olan) dağılımlarda ortalama ortancadan büyüktür. Gelir, harcama, bekleme süresi gibi değişkenler genellikle sağa çarpıktır:

```
ggplot(musteri_temiz, aes(Toplam_Harcama)) +
  geom_histogram(binwidth = 100, fill = "grey70", colour = "white") +
  geom_vline(xintercept = mean(musteri_temiz$Toplam_Harcama), colour = "#D55E00",
    ↪ linewidth = 1) +
  geom_vline(xintercept = median(musteri_temiz$Toplam_Harcama), colour = "#0072B2",
    ↪ linewidth = 1,
    linetype = "dashed") +
  annotate("text", x = c(800, 150), y = 230, label = c("ortalama", "ortanca"),
    colour = c("#D55E00", "#0072B2"), hjust = 0) +
  labs(x = "Toplam harcama", y = "Müşteri sayısı")
```



Çarpık verilerde ve aykırı değer varken **ortanca**, simetrik verilerde **ortalama** raporlanması daha uygundur. “Ortalama müşteri” dediğimizde aslında çoğu müşteriye temsil etmeyen bir sayıdan söz ediyor olabiliriz.

7.2 Değişim Ölçüleri

Aynı ortalamaya sahip iki veri çok farklı olabilir. Değişim ölçüleri, değerlerin merkezin etrafında ne kadar yayıldığını gösterir.

7.2.1 Değişim Genişliği

```
range(veri)
```

```
#> [1] -0.03448 9.38262
```

```
diff(range(veri)) # En büyük - en küçük
```

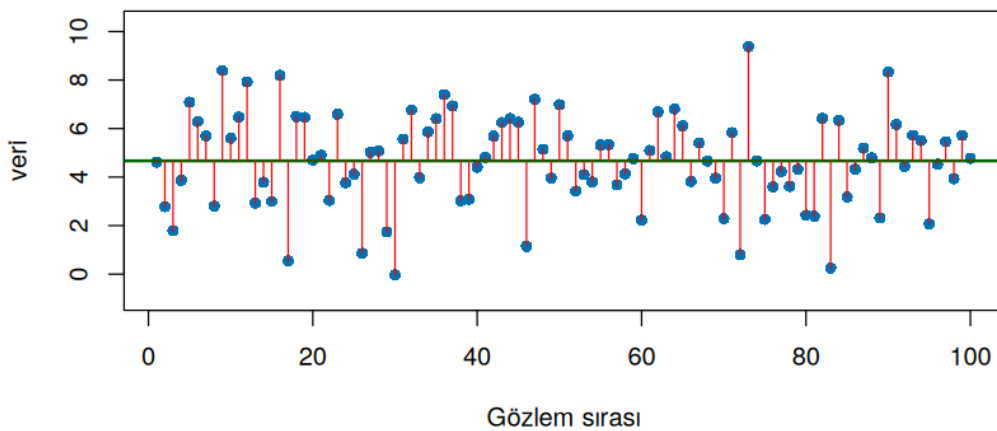
```
#> [1] 9.417
```

Değişim genişliği yalnızca iki uç değere bağlı olduğu için aykırı değerlere çok duyarlıdır.

7.2.2 Ortalamadan Sapmalar ve Varyans

Eski notlarımızdaki “kırmızı çizgiler” sorusunu hatırlayalım: Her gözlemin ortalamadan uzaklığını bir çizgiyle gösterirsek, bu çizgilerin uzunluklarının toplamı nedir?

```
plot(veri, ylim = c(min(veri) - 1, max(veri) + 1), pch = 16, col = "#0072B2",  
      xlab = "Gözlem sırası", ylab = "veri")  
abline(h = mean(veri), col = "darkgreen", lwd = 2)  
for (i in seq_along(veri)) lines(c(i, i), c(mean(veri), veri[i]), col = "red")
```



Şekil 7.1: Her gözlemin ortalamadan sapması (kırmızı çizgiler)

```
sum(veri - mean(veri))          # Sapmaların toplamı her zaman 0'dır!
```

```
#> [1] 3.997e-14
```

```
sum(abs(veri - mean(veri)))      # Mutlak sapmaların toplamı
```

```
#> [1] 151
```

```
sum((veri - mean(veri))^2)       # Sapma kareleri toplamı (SKT)
```

```
#> [1] 359.4
```

Sapmaların toplamı artı ve eksiler birbirini götürdüğü için her zaman sıfırdır. Bu yüzden sapmaların **karelerini** alırız. Sapma kareleri toplamının serbestlik derecesine bölümü **örneklem varyansıdır**:

$$s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1} \quad s = \sqrt{s^2}$$

```
varyans <- function(x) sum((x - mean(x))^2) / (length(x) - 1)
varyans(veri)
```

```
#> [1] 3.63
```

```
var(veri)
```

```
#> [1] 3.63
```

```
sd(veri)          # Standart sapma: verinin kendi biriminde
```

```
#> [1] 1.905
```

Varyansın birimi verinin biriminin **karesidir** (g^2 gibi). Bu yüzden yorumlarda verinin kendi biriminde olan **standart sapma** kullanılır.

7.2.3 Neden $n - 1$ 'e Bölüyoruz? Serbestlik Derecesi

n gözlemin ortalamasını bildiğimizde, $n - 1$ tanesini serbestçe seçebiliriz ama sonuncusu artık **belirlenmiştir** (çünkü sapmaların toplamı 0 olmak zorundadır). Ortalamayı veriden tahmin ettiğimiz için bir serbestlik derecesi “harcamış” oluruz.

Daha önemlisi: n 'e bölersek kitle varyansını sistematik olarak **küçük** tahmin ederiz. Bunu bir simülasyonla görelim. Varyansı $\sigma^2 = 4$ olan bir kitleden $n = 5$ büyüklüğünde 10.000 örneklem çekip her birinde iki tahmini hesaplayalım:

```
set.seed(1)
tahmin_n <- numeric(10000)
tahmin_n_1 <- numeric(10000)
for (i in 1:10000) {
  x <- rnorm(5, mean = 10, sd = 2)
  tahmin_n[i] <- sum((x - mean(x))^2) / 5
  tahmin_n_1[i] <- sum((x - mean(x))^2) / 4
}
mean(tahmin_n)      # Ortalamada 4'ün altında: yanlış
```

```
#> [1] 3.242
```

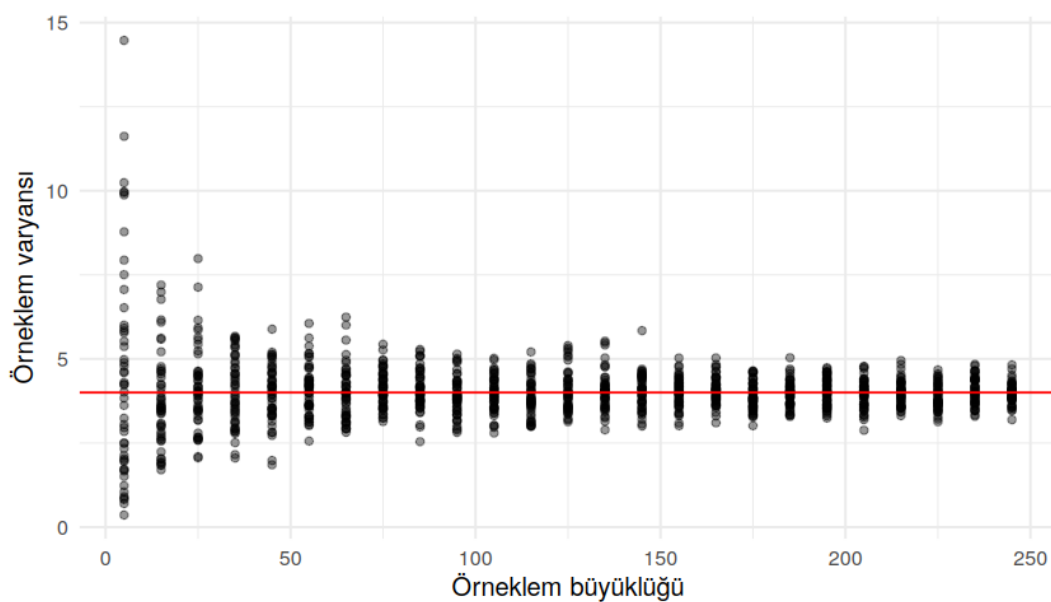
```
mean(tahmin_n_1)    # Ortalamada 4'e çok yakın: yansız
```

```
#> [1] 4.053
```

7.2.4 Varyans ve Örneklem Büyüklüğü

Örneklem büyüdükçe varyans tahmini değişmez ama tahminlerimiz kitle değerinin ($\sigma^2 = 4$) etrafında **daha az dalgalanır**:

```
set.seed(2)
sonuc <- expand.grid(n = seq(5, 250, by = 10), tekrar = 1:50)
sonuc$varyans <- sapply(sonuc$n, function(n) var(rnorm(n, mean = 10, sd = 2)))
ggplot(sonuc, aes(n, varyans)) +
  geom_point(alpha = 0.4) +
  geom_hline(yintercept = 4, colour = "red") +
  labs(x = "Örneklem büyüklüğü", y = "Örneklem varyansı")
```



7.2.5 Worms Verisinde Gruplara Göre Varyans

Eski notlarımızdaki örnek: nemli ve nemsiz tarlalarda solucan yoğunluğunun varyansını adım adım hesaplayalım.

```
nemli <- worms$Worm.density[worms$Damp == TRUE]
nemsiz <- worms$Worm.density[worms$Damp == FALSE]
nemli - mean(nemli) # Sapmalar
```

```
#> [1] -0.5  1.5  0.5  2.5 -1.5 -2.5
```

```
sum((nemli - mean(nemli))^2) # Sapma kareleri toplamı
```

```
#> [1] 17.5
```

```
sum((nemli - mean(nemli))^2) / (length(nemli) - 1)
```

```
#> [1] 3.5
```

```
var(nemli); var(nemsiz)
```

```
#> [1] 3.5
```

```
#> [1] 7.824
```

Bu iki varyansın birbirinden farklı olup olmadığını Bölüm 11’te **F testi** ile sınayacağız.

7.2.6 Çeyrekler Arası Genişlik, MAD ve Değişim Katsayısı

| Ölçü | Formül | R | Ne zaman? |
|--------------------------------------|-------------------------------------|------------------------------|---|
| Çeyrekler arası genişlik (ÇAG / IQR) | $Q_3 - Q_1$ | <code>IQR(x)</code> | Çarpık veri, aykırı değer varken |
| Ortancadan mutlak sapma (MAD) | $\text{ortanca}(x_i - \tilde{x})$ | <code>mad(x)</code> | Aykırı değerlere en dayanıklı ölçü |
| Değişim katsayısı (DK) | $s/\bar{x}$ | <code>sd(x) / mean(x)</code> | Farklı birim / ölçekteki değişkenleri karşılaştırma |

```
IQR(penguenler$body_mass_g)
```

```
#> [1] 1225
```

```
mad(penguenler$body_mass_g) # Normal dağılımla tutarlı olması için 1.4826 ile
↪ çarpılır
```

```
#> [1] 889.6
```

```
# Değişim katsayısı: hangi ölçüm göreceli olarak daha değişken?
sapply(penguenler[, 3:6], \(x) sd(x) / mean(x)) |> round(3)
```

```
#>      bill_length_mm      bill_depth_mm flipper_length_mm      body_mass_g
#>                0.124                0.115                0.070                0.191
```

Gaga uzunluğu milimetre, ağırlık gram cinsinden olduğu için standart sapmaları karşılaştırılmaz; değişim katsayısı birimsiz olduğu için karşılaştırılabilir. Penguenlerin ağırlığı, diğer ölçümlerine göre göreceli olarak daha değişkendir.

7.3 Şekil Ölçüleri: Çarpıklık ve Basıklık

Çarpıklık (*skewness*) dağılımın simetriden sapmasını, **basıklık** (*kurtosis*) kuyrukların normal dağılıma göre kalınlığını ölçer. Ortalamaya göre 3. ve 4. momentlerle tanımlanırlar:

$$\text{çarpıklık} = \frac{m_3}{s^3}, \quad m_3 = \frac{1}{n} \sum (x_i - \bar{x})^3 \quad \text{basıklık} = \frac{m_4}{s^4} - 3, \quad m_4 = \frac{1}{n} \sum (x_i - \bar{x})^4$$

| Değer | Çarpıklık | Basıklık (fazlalık) |
|-------|-------------------------------|-------------------------------|
| 0 | Simetrik | Normal dağılım kadar kuyruklu |
| > 0 | Sağa çarpık (uzun sağ kuyruk) | Kalın kuyruklu, sivri |
| < 0 | Sola çarpık (uzun sol kuyruk) | İnce kuyruklu, basık |

Eski ders notlarımızdaki fonksiyonlar:

```
carpiklik <- function(x) {
  m3 <- sum((x - mean(x))^3) / length(x)
  s3 <- sqrt(var(x))^3
  m3 / s3
}
basiklik <- function(x) {
  m4 <- sum((x - mean(x))^4) / length(x)
  s4 <- var(x)^2
  m4 / s4 - 3
}
carpiklik(worms$Worm.density)
```

```
#> [1] 0.1196
```

```
basiklik(worms$Worm.density)
```

```
#> [1] -1.258
```

```
carpiklik(musteri_temiz$Toplam_Harcama) # Belirgin sağa çarpık
```

```
#> [1] 0.8562
```

Büyük örneklerde çarpıklığın standart hatası yaklaşık $\sqrt{6/n}$, basıklığinki $\sqrt{24/n}$ 'dir. Değerin standart hatasına oranı yaklaşık olarak bir t istatistiği gibi yorumlanabilir:

```
n <- length(worms$Worm.density)
t_carpiklik <- carpiklik(worms$Worm.density) / sqrt(6 / n)
t_carpiklik
```

```
#> [1] 0.2183
```

```
1 - pt(t_carpiklik, df = n - 2) # Tek yönlü p değeri (@sec-hipotez)
```

```
#> [1] 0.4148
```

p değeri büyük olduğu için solucan yoğunluğunun simetrik bir dağılımdan geldiği düşüncesini reddetmek için bir kanıt yoktur.

i Not

`moments` ve `e1071` paketlerinde `skewness()` ve `kurtosis()` fonksiyonları vardır. Paketler arasında küçük formül farklılıkları (örneğin n ya da $n - 1$ kullanımı, 3'ün çıkarılıp çıkarılmaması) olduğu için hangi formülün kullanıldığını yardım sayfasından kontrol edin.

7.4 Standart Değerler ve Aykırı Değerler

7.4.1 z Değerleri

Bir gözlemin ortalamadan kaç standart sapma uzakta olduğunu gösterir:

$$z_i = \frac{x_i - \bar{x}}{s}$$

```
z <- scale(penguenler$body_mass_g)[, 1] # scale() bir matris döndürür
round(c(ortalama = mean(z), ss = sd(z)), 3)
```

```
#> ortalama      ss
#>         0         1
```

```
penguenler[which.max(z), c("species", "sex", "body_mass_g")] # En ağır penguen
```

```
#> # A tibble: 1 x 3  
#>   species sex   body_mass_g  
#>   <fct>   <fct>         <int>  
#> 1 Gentoo male           6300
```

```
max(z)
```

```
#> [1] 2.599
```

z değerleri farklı ölçeklerdeki gözlemleri karşılaştırmayı sağlar: “Bu penguen ağırlığıyla mı, gaga uzunluğuyla mı daha sıra dışı?”

7.4.2 Aykırı Değerler

İki yaygın kural:

1. **Tukey (kutu grafiği) kuralı:** $Q_1 - 1.5 \text{ ÇAG}$ 'den küçük ya da $Q_3 + 1.5 \text{ ÇAG}$ 'den büyük değerler.
2. **z kuralı:** $|z| > 3$ olan değerler (yaklaşık normal veriler için).

```
aykiri_bul <- function(x) {  
  q <- quantile(x, c(0.25, 0.75), na.rm = TRUE)  
  cag <- q[2] - q[1]  
  which(x < q[1] - 1.5 * cag | x > q[2] + 1.5 * cag)  
}  
gelir <- readr::read_tsv("veri/dataset_ex.csv", show_col_types = FALSE)$Income  
gelir[aykiri_bul(gelir)]
```

```
#> [1] 157243 162397 153924 160803 157733 157146 156924 666666
```

```
boxplot.stats(gelir)$out # base R'in kutu grafiği için kullandığı aynı kural
```

```
#> [1] 157243 162397 153924 160803 157733 157146 156924 666666
```

Uyarı

Aykırı değer **otomatik olarak silinmez**. Önce nedenini araştırın: Veri giriş hatası mı (666666 TL gelir), ölçüm hatası mı, yoksa gerçek ama nadir bir gözlem mi? Silme ya da düzeltme kararı ve gerekçesi rapora yazılmalıdır.

7.5 Kategorik Değişkenler: Frekans ve Çapraz Tablolar

```
table(penguenler$species) # Frekans
```

```
#>
#>   Adelie Chinstrap   Gentoo
#>    146      68     119
```

```
prop.table(table(penguenler$species)) |> round(3) # Oran
```

```
#>
#>   Adelie Chinstrap   Gentoo
#>  0.438   0.204   0.357
```

```
tablo <- table(penguenler$species, penguenler$island)
tablo
```

```
#>
#>           Biscoe Dream Torgersen
#> Adelie      44     55         47
#> Chinstrap    0     68          0
#> Gentoo     119     0          0
```

```
addmargins(tablo) # Satır ve sütun toplamaları
```

```
#>
#>           Biscoe Dream Torgersen Sum
#> Adelie      44     55         47 146
#> Chinstrap    0     68          0  68
#> Gentoo     119     0          0 119
#> Sum        163    123         47 333
```

```
prop.table(tablo, margin = 1) |> round(2) # Satır yüzdeleri: her türün
↪ adalara dağılımı
```

```
#>
#>           Biscoe Dream Torgersen
#> Adelie      0.30  0.38         0.32
#> Chinstrap    0.00  1.00         0.00
#> Gentoo      1.00  0.00         0.00
```

```
prop.table(tablo, margin = 2) |> round(2) # Sütun yüzdeleri: her adanın tür
↪ dağılımı
```

```
#>
#>           Biscoe Dream Torgersen
#> Adelie      0.27  0.45         1.00
#> Chinstrap    0.00  0.55         0.00
#> Gentoo      0.73  0.00         0.00
```

dplyr ile aynı tablo:

```
penguenler |>
  count(species, island) |>
  group_by(species) |>
  mutate(oran = n / sum(n))
```

```
#> # A tibble: 5 x 4
#> # Groups:   species [3]
#>   species island      n oran
#>   <fct>   <fct>   <int> <dbl>
#> 1 Adelie  Biscoe      44 0.301
#> 2 Adelie  Dream      55 0.377
#> 3 Adelie  Torgersen   47 0.322
#> 4 Chinstrap Dream      68 1
#> 5 Gentoo  Biscoe     119 1
```

7.6 Gruplara Göre Özetler

Eski notlarımızdaki base R fonksiyonları:

```
summary(worms)
```

```
#>   Field.Name      Area      Slope      Vegetation
#> Length:20      Min.   :0.80   Min.   : 0.00   Length:20
#> Class :character 1st Qu.:2.17   1st Qu.: 0.75   Class :character
#> Mode  :character Median :3.00   Median : 2.00   Mode  :character
#>              Mean   :2.99   Mean   : 3.50
#>              3rd Qu.:3.73   3rd Qu.: 5.25
#>              Max.   :5.10   Max.   :11.00
#>   Soil.pH      Damp      Worm.density
#> Min.   :3.50   Mode :logical   Min.   :0.00
#> 1st Qu.:4.10   FALSE:14        1st Qu.:2.00
#> Median :4.60   TRUE :6         Median :4.00
#> Mean   :4.55                Mean   :4.35
#> 3rd Qu.:5.00                3rd Qu.:6.25
#> Max.   :5.70                Max.   :9.00
```

```
aggregate(worms[, c(2, 3, 5, 7)], by = list(Bitki = worms$Vegetation), FUN = mean)
```

```
#>      Bitki  Area Slope Soil.pH Worm.density
#> 1  Arable 3.867 1.333  4.833      5.333
#> 2 Grassland 2.911 3.667  4.100      2.444
#> 3  Meadow 3.467 1.667  4.933      6.333
#> 4  Orchard 1.900 0.000  5.700      9.000
#> 5   Scrub 2.425 7.000  4.800      5.250
```

```
aggregate(worms[, c(2, 3, 5, 7)],
          by = list(Nem = worms$Damp, Bitki = worms$Vegetation), FUN = mean)
```

```
#>      Nem      Bitki Area Slope Soil.pH Worm.density
#> 1 FALSE   Arable 3.867 1.333  4.833      5.333
#> 2 FALSE Grassland 3.087 3.625  3.987      1.875
#> 3 TRUE  Grassland 1.500 4.000  5.000      7.000
#> 4 TRUE   Meadow 3.467 1.667  4.933      6.333
#> 5 FALSE  Orchard 1.900 0.000  5.700      9.000
#> 6 FALSE   Scrub 3.350 5.000  4.700      7.000
#> 7 TRUE    Scrub 1.500 9.000  4.900      3.500
```

```
with(worms, tapply(Worm.density, Vegetation, mean))
```

```
#>      Arable Grassland      Meadow      Orchard      Scrub
#>      5.333      2.444      6.333      9.000      5.250
```

```
with(worms, tapply(Slope, list(Damp, Vegetation), mean)) # İki faktöre göre
```

```
#>      Arable Grassland Meadow Orchard Scrub
#> FALSE 1.333      3.625      NA      0      5
#> TRUE  NA      4.000 1.667      NA      9
```

`aggregate(x, by, FUN)` yapısında önce veri, sonra gruplama değişkenleri (birden fazlaysa `list()` içinde), en son uygulanacak fonksiyon yazılır.

`dplyr` ile raporlanmaya hazır bir özet tablosu:

```
penguenler |>
  group_by(Tür = species) |>
  summarise(
    n          = n(),
    Ortalama   = mean(body_mass_g),
    SS         = sd(body_mass_g),
    Ortanca    = median(body_mass_g),
    ÇAG        = IQR(body_mass_g),
    Min        = min(body_mass_g),
    Maks       = max(body_mass_g),
    Çarpıklık  = carpiklik(body_mass_g)
  ) |>
  knitr::kable(digits = 1)
```

Tablo 7.3: Türlere göre penguen ağırlığının betimsel istatistikleri (g)

| Tür | n | Ortalama | SS | Ortanca | ÇAG | Min | Maks | Çarpıklık |
|-----------|-----|----------|-------|---------|-------|------|------|-----------|
| Adelie | 146 | 3706 | 458.6 | 3700 | 637.5 | 2850 | 4775 | 0.3 |
| Chinstrap | 68 | 3733 | 384.3 | 3700 | 462.5 | 2700 | 4800 | 0.2 |

Tablo 7.3: Türlerle göre penguen ağırlığının betimsel istatistikleri (g)

| Tür | n | Ortalama | SS | Ortanca | ÇAG | Min | Maks | Çarpıklık |
|--------|-----|----------|-------|---------|-------|------|------|-----------|
| Gentoo | 119 | 5092 | 501.5 | 5050 | 800.0 | 3950 | 6300 | 0.0 |

`psych` paketindeki `describe()` ve `describeBy()` fonksiyonları tek satırda çok sayıda betimsel istatistik verir:

```
psych::describe(penguenler[, 3:6]) |> round(2)
```

```
#>               vars    n   mean    sd median trimmed   mad   min   max
#> bill_length_mm     1 333  43.99  5.47  44.5   43.98   6.97  32.1  59.6
#> bill_depth_mm      2 333  17.16  1.97  17.3   17.19   2.22  13.1  21.5
#> flipper_length_mm  3 333  200.97 14.02 197.0  200.36  16.31 172.0 231.0
#> body_mass_g        4 333 4207.06 805.22 4050.0 4159.46 889.56 2700.0 6300.0
#>               range skew kurtosis    se
#> bill_length_mm   27.5  0.04   -0.90  0.30
#> bill_depth_mm     8.4 -0.15   -0.91  0.11
#> flipper_length_mm 59.0  0.36   -0.98  0.77
#> body_mass_g    3600.0  0.47   -0.75 44.13
```

7.7 Raporlama İlkeleri

- Ortalama ile birlikte **standart sapmayı**, ortanca ile birlikte **ÇAG**'ı (ya da min–maks) verin.
- Her istatistiğin **kaç gözlemle** (n) hesaplandığını belirtin; eksik değer varsa söyleyin.
- Ondalık basamak sayısını verinin ölçüm duyarlılığına göre seçin (ağırlık 1 g duyarlılıkla ölçül-müşse ortalamayı 4201.75381 değil 4201.8 yazın).
- Sayıları rapora elle değil, **satır içi kod** ile yazın (Bölüm 3). Örneğin: Penguenlerin ortalama ağırlığı 4207 g (SS = 805 g, n = 333) olarak bulunmuştur.

Kendini Sına

💡 1. `iris` verisinde hangi ölçüm türlerle göre en farklı ortalamalara sahip? Her tür için dört ölçümün ortalamasını tek bir tabloda gösterin.

```
iris |> group_by(Species) |> summarise(across(everything(), mean))
```

Petal.Length: *setosa* 1.5, *virginica* 5.6 cm.

💡 2. Bir veride ortalama 52, ortanca 38'dir. Dağılımın şekli hakkında ne söylenebilir? Hangi merkez ölçüsü raporlanmalı?

Ortalama ortancadan belirgin biçimde büyük olduğu için dağılım büyük olasılıkla **sağa çarpıktır** (ya da büyük aykırı değerler vardır). Ortanca ve ÇAG raporlanmalıdır.

💡 3. Neden `sum(x - mean(x))` her zaman (yuvarlama hatası dışında) sıfırdır?

$\sum (x_i - \bar{x}) = \sum x_i - n\bar{x} = n\bar{x} - n\bar{x} = 0$. Bu yüzden değişimi ölçmek için sapmaların mutlak değeri ya da karesi alınır.

💡 4. `penguins` (eksik değerli) verisinde `mean(penguins$body_mass_g)` neden NA döndürür? Kaç penguenin ağırlığı eksiktir?

İçinde NA olan hesaplamaların sonucu NA'dır. `mean(penguins$body_mass_g, na.rm = TRUE)` kullanılmalı. `sum(is.na(penguins$body_mass_g)) → 2`.

💡 5. Tukey kuralına göre `mtcars$hp` değişkeninde aykırı değer var mı? Hangi araba?

```
mtcars[aykiri_bul(mtcars$hp), "hp", drop = FALSE] # Maserati Bora, 335 hp
```

8 Ara Sınav: Tekrar ve Hazırlık

Sekizinci hafta ara sınav haftasıdır. Bu kısa bölüm sınav kapsamını ve hazırlık için bir kontrol listesini özetler. Bölüm numaralarının haftalarla aynı kalması için kitapta bu hafta da bir bölüm olarak yer alır.

8.1 Kapsam

Tablo 8.1: Ara sınav kapsamı

| Hafta | Konu | Gözden geçirilecek |
|-------|----------------------------------|--|
| 1 | R ve RStudio | Paket kurma/yükleme farkı, çalışma dizini ve görelî yol, veri tipleri, NA |
| 2 | Veri yapıları | Vektör, matris, faktör, liste, data frame; <code>[]</code> , <code>[[]]</code> , <code>\$</code> ile seçim; ordinal faktör |
| 3 | Quarto | YAML başlığı, kod parçası seçenekleri, satır içi kod, çapraz referans |
| 4 | Veri okuma ve <code>dplyr</code> | Ayırıcı seçimi, <code>select/filter/arrange/mutate/summarise/group_by</code> , <code>pipe</code> , eksik değerler |
| 5 | Kontrol ve fonksiyon | <code>if/else</code> ile <code>ifelse</code> farkı, döngüde sonuç saklama, fonksiyon argümanları, <code>{ }</code> |
| 6 | Görselleştirme | Soruya göre grafik seçimi, <code>aes()</code> içi/dışı, <code>facet</code> , <code>ggplotly()</code> ve <code>plot_ly()</code> |
| 7 | Betimsel istatistik | Ortalama/ortanca seçimi, varyans ve $n - 1$, ÇAG, DK, çarpıklık, aykırı değer kuralları |

8.2 Hazırlık Listesi

- Her bölümün sonundaki **Kendini Sına** sorularını cevaplara bakmadan yanıtlayın.
- Bir Quarto belgesini sıfırdan, şablon kullanmadan oluşturup içine veri okuma, özet tablo ve bir ggplot grafiğı koyarak derleyebildiğinizden emin olun.
- `penguins` ve `iris` verileri için “her tür için ... ortalamasını bul” türünden soruları hem base R (`apply`, `aggregate`) hem `dplyr` ile çözün.
- Bir kod parçasının çıktısını **çalıştırmadan önce** tahmin edin, sonra çalıştırıp kontrol edin.
- Hata mesajlarını okuyun: hangi satırda, hangi nesne, ne bekleniyordu?

8.3 Örnek Sorular

💡 1. `x <- c(4, 8, NA, 2)` için `mean(x)`, `mean(x, na.rm = TRUE)` ve `sum(is.na(x))` ne döndürür?

NA, 4.667 ve 1.

💡 2. Aşağıdaki kodun çıktısı nedir?

```
f <- factor(c("orta", "az", "çok", "orta"), levels = c("az", "orta", "çok"))
as.integer(f)
```

2 1 3 2. Faktör düzeyleri `levels` sırasıyla 1, 2, 3 olarak saklanır.

💡 3. `penguins |> filter(species == "Adelie") |> summarise(m = mean(body_mass_g))` neden NA döndürür? Nasıl düzeltilir?

Adelie penguenlerinden birinin ağırlığı eksiktir. `mean(body_mass_g, na.rm = TRUE)` yazılmalıdır.

💡 4. Bir fonksiyon içinde `if (x > 0)` kullanıldı ve fonksiyon `x = c(-1, 2, 3)` ile çağrıldığında hata verdi. Neden? Nasıl düzeltilir?

`if` tek bir mantıksal değer bekler; vektör verildiğinde R 4.2'de hata verir. Vektörel çalışması için `ifelse(x > 0, ..., ...)` kullanılmalıdır.

💡 5. Aşağıdaki grafikte noktalar neden türlere göre renklenmiyor?

```
ggplot(penguins, aes(flipper_length_mm, body_mass_g)) +
  geom_point(colour = species)
```

`species` bir değişkendir ve **eşleme** yapılmak isteniyor; bu yüzden `aes()` içine yazılmalıdır: `geom_point(aes(colour = species))`. Mevcut hâliyle R `species` adında bir nesne arar ve “object ‘species’ not found” hatası verir.

💡 6. Bir değişkende ortalama 120, standart sapma 30; başka bir değişkende ortalama 12, standart sapma 4'tür. Hangisi görece olarak daha değişkendir?

Değişim katsayıları: $30/120 = 0.25$ ve $4/12 = 0.33$. İkinci değişken görece olarak daha değişkendir.

💡 7. Bir Quarto belgesinde kodun görünmesini ama çalıştırılmamasını istiyorsunuz. Hangi seçenek?

```
#| eval: false.
```

Kısım III

Kısım III — İstatistiksel Çıkarım

9 Olasılık Dağılımları

i Bu haftanın hedefleri

- Rastgele değişken, olasılık fonksiyonu (pmf), olasılık yoğunluk fonksiyonu (pdf) ve birikimli dağılım fonksiyonu (cdf) kavramlarını hatırlamak,
- R'in **d**, **p**, **q**, **r** fonksiyon ailesini her dağılım için kullanabilmek,
- Binom ve Poisson (kesikli); düzgün, normal, üstel, t , χ^2 ve F (sürekli) dağılımlarını tanımak ve çizmek,
- Normal dağılımda olasılık hesaplamak ve bu olasılıkları grafikte göstermek,
- `set.seed()` ile tekrarlanabilir simülasyon yapmak,
- Bir verinin normal dağılıma uyup uymadığını grafikte incelemek.

9.1 Temel Kavramlar

Bir deneyin sonucunu sayıya çeviren değişkene **rastgele değişken** denir. Alabileceği değerler sayılabiliyorsa **kesikli** (zar, başarı sayısı), bir aralıktaki her değeri alabiliyorsa **sürekli** (boy, ağırlık, süre) rastgele değişkendir.

| Kavram | Kesikli | Sürekli |
|----------------------------|---|--|
| Olasılığı veren fonksiyon | Olasılık fonksiyonu $P(X = x)$ | Olasılık yoğunluk fonksiyonu $f(x)$;
$P(a < X < b) = \int_a^b f(x) dx$ |
| Birikimli dağılım | $F(x) = P(X \leq x) = \sum_{t \leq x} P(X = t)$ | $F(x) = P(X \leq x) = \int_{-\infty}^x f(t) dt$ |
| Tek bir noktanın olasılığı | Sıfırdan büyük olabilir | Her zaman sıfırdır :
$P(X = x) = 0$ |

9.2 R'da Dağılım Fonksiyonları: d, p, q, r

R'da her dağılım için aynı mantıkla adlandırılmış dört fonksiyon vardır. Dağılımın kısaltmasının önüne bir harf gelir:

Tablo 9.2: d, p, q, r fonksiyon ailesi

| Ön ek | Ne verir? | Normal dağılım örneği | Soru |
|-------|---|---------------------------------|--------------------------------------|
| d | Yoğunluk / olasılık:
$f(x)$ ya da $P(X = x)$ | <code>dnorm(170, 170, 8)</code> | Bu noktada eğrinin yüksekliği nedir? |

| Ön ek | Ne verir? | Normal dağılım örneği | Soru |
|-------|--|----------------------------------|---|
| p | Birikimli olasılık:
$P(X \leq q)$ | <code>pnorm(160, 170, 8)</code> | 160 cm'den kısa olma olasılığı nedir? |
| q | Nicelik (p'nin tersi):
$P(X \leq q) = p$ olan q | <code>qnorm(0.95, 170, 8)</code> | En uzun %5'lik dilim hangi boydan başlar? |
| r | Rastgele sayı üretir | <code>rnorm(100, 170, 8)</code> | Bu dağılımdan 100 kişi seçsek? |

Tablo 9.3: R'daki yaygın dağılımlar

| Dağılım | R adı | Parametreler |
|-------------|--------------------|---------------------------------|
| Binom | <code>binom</code> | <code>size (n), prob (p)</code> |
| Poisson | <code>pois</code> | <code>lambda (λ)</code> |
| Düzgün | <code>unif</code> | <code>min, max</code> |
| Normal | <code>norm</code> | <code>mean (μ), sd (σ)</code> |
| Üstel | <code>exp</code> | <code>rate (λ)</code> |
| Student t | <code>t</code> | <code>df</code> |
| Ki-kare | <code>chisq</code> | <code>df</code> |
| F | <code>f</code> | <code>df1, df2</code> |

Örneğin F dağılımı için: `df(x, df1, df2)`, `pf(q, df1, df2)`, `qf(p, df1, df2)`, `rf(n, df1, df2)`.

! Önemli

p fonksiyonları varsayılan olarak **sol kuyruk** olasılığını, yani $P(X \leq q)$ 'yu verir. Sağ kuyruk için `1 - pnorm(q)` ya da `pnorm(q, lower.tail = FALSE)` kullanılır.

9.3 Rastgele Sayı Üretmek ve `set.seed()`

Bilgisayar gerçekten rastgele değil, **sözde rastgele** sayılar üretir: belirli bir başlangıç değerinden (*seed*) başlayan deterministik bir dizi. `set.seed()` ile başlangıç değerini sabitlesek aynı “rastgele” sayıları her seferinde yeniden üretebiliriz. **Simülasyon içeren her raporda `set.seed()` kullanılmalıdır.**

```
set.seed(347); rnorm(3)
```

```
#> [1] -0.1924 -1.1089 -1.6042
```

```
set.seed(347); rnorm(3)      # Aynı sayılar
```

```
#> [1] -0.1924 -1.1089 -1.6042
```

```
sample(1:6, 10, replace = TRUE)      # 10 zar atışı
```

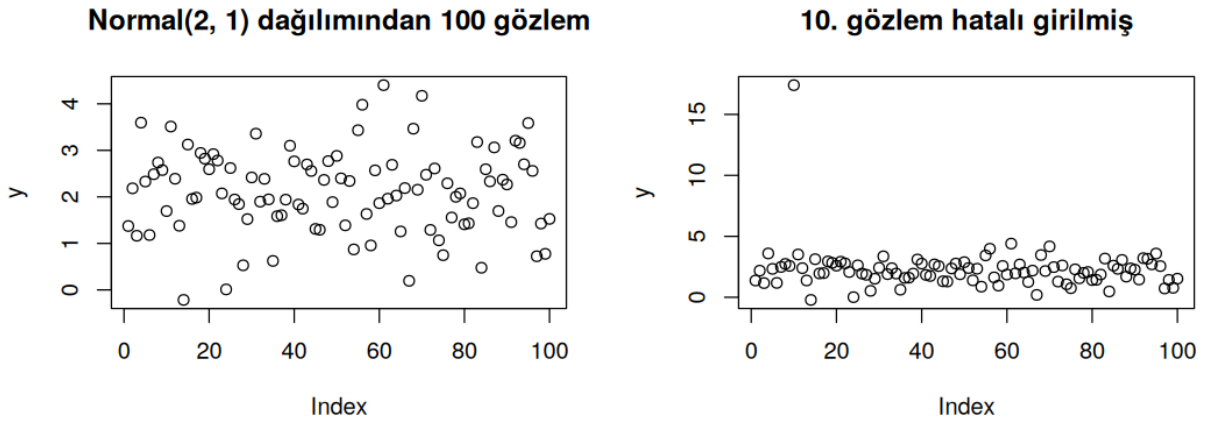
```
#> [1] 6 2 1 1 1 3 1 1 2 4
```

```
sample(c("Yazı", "Tura"), 5, replace = TRUE)
```

```
#> [1] "Tura" "Tura" "Yazı" "Tura" "Yazı"
```

Eski ders notlarımızdaki örnek: Normal dağılımdan 100 sayı üretip bir tanesini bilerek bozalım ve grafikte görelim:

```
set.seed(1)
y <- rnorm(100, mean = 2, sd = 1)
plot(y, main = "Normal(2, 1) dağılımından 100 gözlem")
y[10] <- 17.4      # Bir veri giriş hatası
plot(y, main = "10. gözlem hatalı girilmiş")
which(y > 15)      # Hatalı gözlemi bul
```



```
#> [1] 10
```

9.4 Kesikli Dağılımlar

9.4.1 Binom Dağılımı

Başarı olasılığı p olan bağımsız n denemedeki başarı sayısı $X \sim \text{Binom}(n, p)$:

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad E(X) = np, \quad V(X) = np(1-p)$$

Örnek: 20 soruluk, her sorusu 5 seçenekli bir testte tüm soruları rastgele işaretleyen bir öğrenci düşünelim ($n = 20$, $p = 0.2$).

```
dbinom(4, size = 20, prob = 0.2) # Tam 4 doğru
```

```
#> [1] 0.2182
```

```
pbinom(4, size = 20, prob = 0.2) # En fazla 4 doğru
```

```
#> [1] 0.6296
```

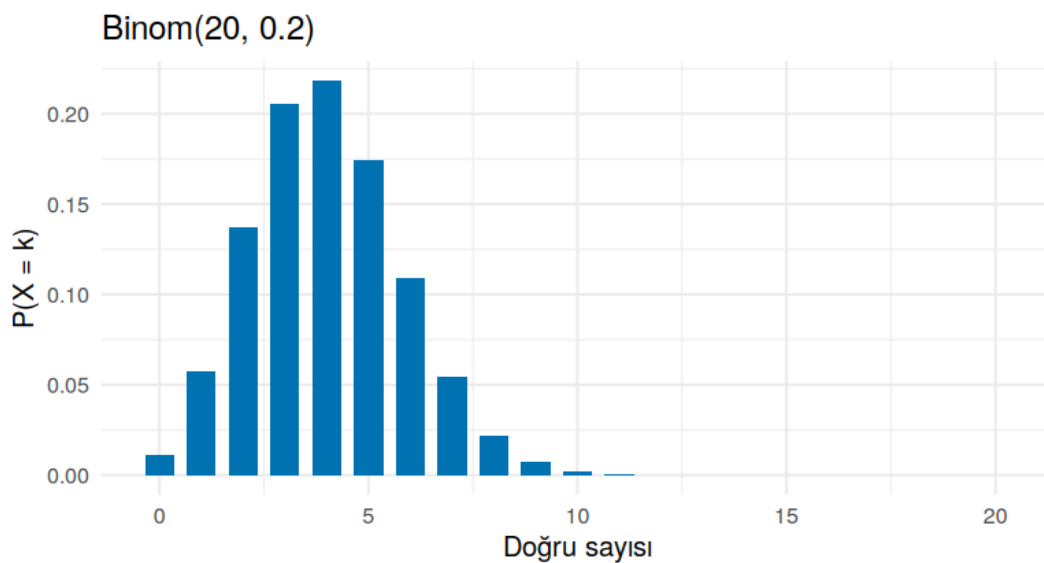
```
1 - pbinom(9, size = 20, prob = 0.2) # En az 10 doğru (geçme)
```

```
#> [1] 0.002595
```

```
qbinom(0.95, size = 20, prob = 0.2) # Öğrencilerin %95'i en fazla kaç doğru yapar?
```

```
#> [1] 7
```

```
data.frame(k = 0:20, olasilik = dbinom(0:20, 20, 0.2)) |>  
  ggplot(aes(k, olasilik)) +  
  geom_col(fill = "#0072B2", width = 0.7) +  
  labs(x = "Doğru sayısı", y = "P(X = k)", title = "Binom(20, 0.2)")
```



9.4.2 Poisson Dağılımı

Belirli bir zaman ya da alan biriminde **nadir** olarak gerçekleşen olayların sayısı. Ortalaması ve varyansı λ 'dır:

$$P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}$$

Örnek: Bir çağrı merkezine dakikada ortalama 3 çağrı geliyor.

```
dpois(0, lambda = 3) # Bir dakikada hiç çağrı gelmemesi
```

```
#> [1] 0.04979
```

```
1 - ppois(5, lambda = 3) # 5'ten fazla çağrı gelmesi
```

```
#> [1] 0.08392
```

```
set.seed(2)
cagrilar <- rpois(1000, lambda = 3)
c(ortalama = mean(cagrilar), varyans = var(cagrilar)) # İkisi de  $\approx 3$ 
```

```
#> ortalama varyans
#> 2.994 3.169
```

9.5 Sürekli Dağılımlar

9.5.1 Düzgün (Uniform) Dağılım

Bir aralıktaki her değerın eşit olasılıklı olduğu dağılım. `runif()` simülasyonların temel taşıdır.

```
punif(0.3, min = 0, max = 1)
```

```
#> [1] 0.3
```

```
set.seed(3); runif(5, min = 10, max = 20)
```

```
#> [1] 11.68 18.08 13.85 13.28 16.02
```

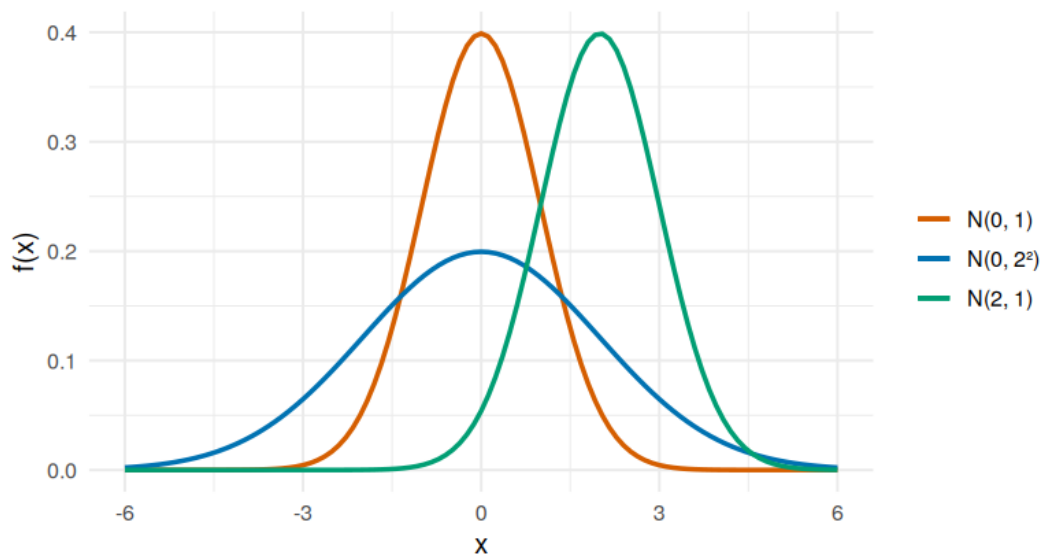
9.5.2 Normal Dağılım

İstatistiğin en önemli dağılımı. Ortalaması μ , standart sapması σ olan çan eğrisi:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

$\mu = 0$, $\sigma = 1$ olan normal dağılıma **standart normal dağılım** denir. Her normal değişken $Z = (X - \mu)/\sigma$ dönüşümüyle standart normale çevrilebilir.

```
ggplot() +
  geom_function(fun = dnorm, args = list(mean = 0, sd = 1), aes(colour = "N(0, 1)",
    ↪ linewidth = 1) +
  geom_function(fun = dnorm, args = list(mean = 0, sd = 2), aes(colour = "N(0, 2²)",
    ↪ linewidth = 1) +
  geom_function(fun = dnorm, args = list(mean = 2, sd = 1), aes(colour = "N(2, 1)",
    ↪ linewidth = 1) +
  xlim(-6, 6) +
  scale_colour_manual(values = c("#D55E00", "#0072B2", "#009E73")) +
  labs(x = "x", y = "f(x)", colour = NULL)
```



μ eğrinin yerini, σ eğrinin genişliğini belirler.

9.5.2.1 68–95–99.7 Kuralı ve “6 Sigma”

Normal dağılımda gözlemlerin yaklaşık %68'i ortalamadan 1, %95'i 2, %99.7'si 3 standart sapma uzaklık içindedir:

```
pnorm(1) - pnorm(-1)
```

```
#> [1] 0.6827
```

```
pnorm(2) - pnorm(-2)
```

```
#> [1] 0.9545
```

```
pnorm(3) - pnorm(-3)
```

```
#> [1] 0.9973
```

```
qnorm(c(0.025, 0.975))      # Ortadaki %95'in sınırları: ±1.96
```

```
#> [1] -1.96  1.96
```

Eski notlarımızdaki tartışma sorusu: **6 sigma kuralı nedir?** Kalite kontrolde, süreç ortalamasının her iki yanında 6 standart sapmalı bir alanın tolerans sınırları içinde kalması hedeflenir; yani hatalı ürün olasılığı neredeyse sıfırdır. $\mu \pm 3\sigma$ toplam 6σ genişliğinde bir aralıktır ve gözlemlerin %99.7'sini içerir. “Altı Sigma” metodolojisi ise süreç kaymalarını hesaba katarak milyonda 3.4 hata hedefler:

```
1 - pnorm(3)                # Bir kuyrukta 3 sigma dışı
```

```
#> [1] 0.00135
```

```
(1 - pnorm(6 - 1.5)) * 1e6   # 1.5 sigma kayma ile milyonda hata ( $\approx 3.4$ )
```

```
#> [1] 3.398
```

9.5.2.2 Normal Dağılımda Olasılık Hesaplama ve Gösterme

Örnek: Boy uzunlukları $\mu = 170$ cm, $\sigma = 8$ cm olan normal dağılıma uysun.

```
pnorm(160, mean = 170, sd = 8)      # P(X < 160)
```

```
#> [1] 0.1056
```

```
pnorm(-1.25)                       # Aynı: z = (160 - 170) / 8 = -1.25
```

```
#> [1] 0.1056
```

```
1 - pnorm(185, mean = 170, sd = 8)  # P(X > 185), z = 1.875
```

```
#> [1] 0.0304
```

```
pnorm(180, 170, 8) - pnorm(160, 170, 8)  # P(160 < X < 180)
```

```
#> [1] 0.7887
```

```
qnorm(0.90, 170, 8)              # En uzun %10'un alt sınırı
```

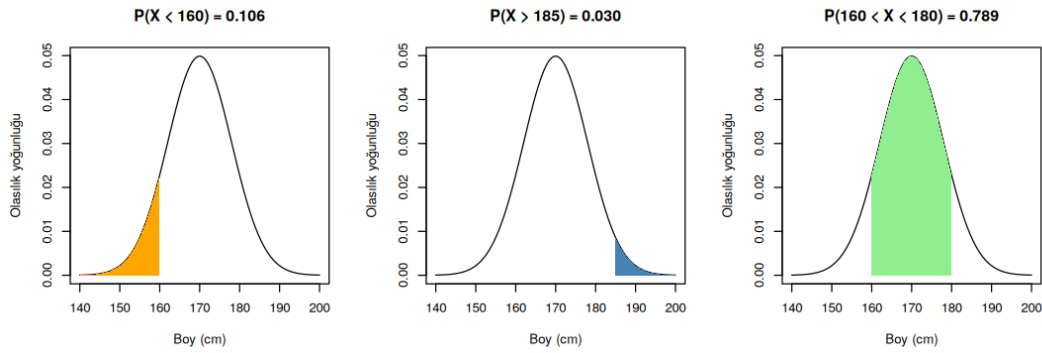
```
#> [1] 180.3
```

Eski notlarımızda bu olasılıkları `polygon()` ile boyamıştık:

```

par(mfrow = c(1, 3))
boy <- seq(140, 200, by = 0.1)
yog <- dnorm(boy, 170, 8)
alan_ciz <- function(alt, ust, renk, baslik) {
  plot(boy, yog, type = "l", xlab = "Boy (cm)", ylab = "Olasılık yoğunluğu", main =
    ↪ baslik)
  secim <- boy >= alt & boy <= ust
  polygon(c(alt, boy[secim], ust), c(0, yog[secim], 0), col = renk, border = NA)
}
alan_ciz(140, 160, "orange", "P(X < 160) = 0.106")
alan_ciz(185, 200, "steelblue", "P(X > 185) = 0.030")
alan_ciz(160, 180, "lightgreen", "P(160 < X < 180) = 0.789")
par(mfrow = c(1, 1))

```



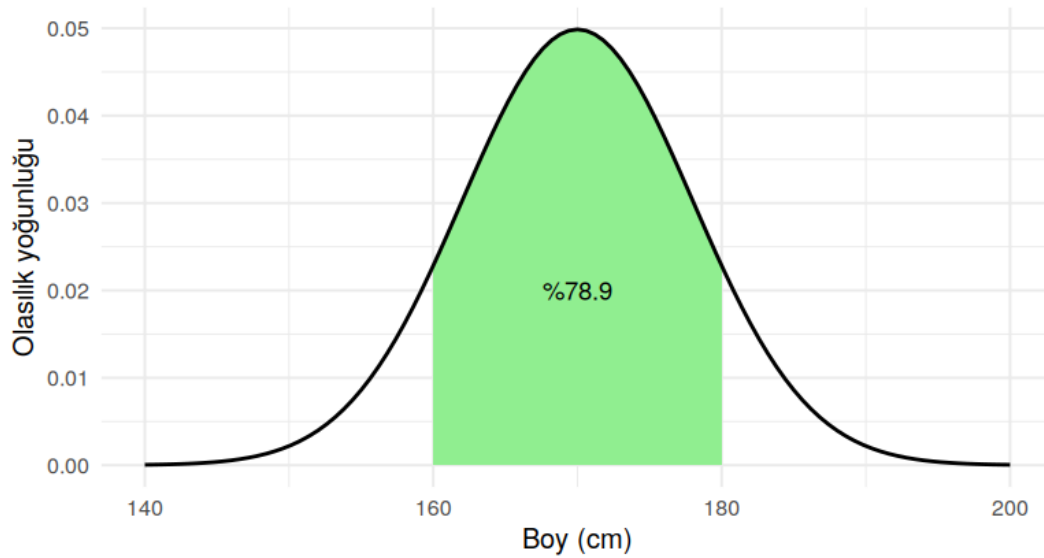
Şekil 9.1: $N(170, 8^2)$ dağılımında üç olasılık

Aynı grafik ggplot2 ile:

```

ggplot(data.frame(x = c(140, 200)), aes(x)) +
  stat_function(fun = dnorm, args = list(170, 8), geom = "area",
    xlim = c(160, 180), fill = "lightgreen") +
  stat_function(fun = dnorm, args = list(170, 8), linewidth = 0.8) +
  annotate("text", x = 170, y = 0.02, label = "%78.9") +
  labs(x = "Boy (cm)", y = "Olasılık yoğunluğu")

```



9.5.3 Üstel Dağılım

Poisson sürecinde iki olay arasındaki **bekleme süresi**. Belirgin biçimde **sağa çarpıktır**; Bölüm 10'de Merkezi Limit Teoreminin normal olmayan bir dağılımda da çalıştığını bu dağılımla göreceğiz.

```
pexp(1, rate = 3)      # Dakikada 3 çağrı varken 1 dakikadan kısa bekleme olasılığı
```

```
#> [1] 0.9502
```

```
set.seed(4)
bekleme <- rexp(1000, rate = 3)
mean(bekleme)      # ≈ 1/3 dakika
```

```
#> [1] 0.3431
```

9.6 Örneklem Dağılımları: t, Ki-kare ve F

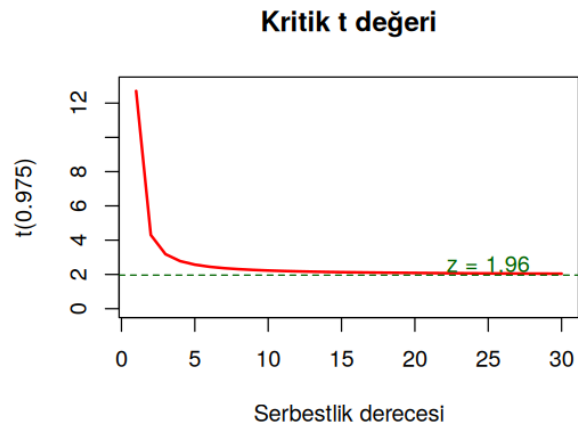
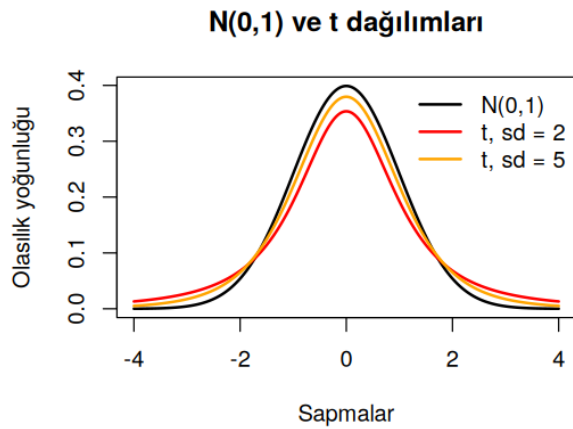
Bu üç dağılım, normal dağılımdan çekilen örneklemelerden hesaplanan istatistiklerin dağılımlarıdır ve hipotez testlerinin temelini oluşturur (Bölüm 11, Bölüm 12).

| Dağılım | Nereden çıkar? | Kullanıldığı yer |
|--------------------|---|---|
| t_{n-1} | $\frac{\bar{X}-\mu}{S/\sqrt{n}}$ (σ bilinmediğinde) | Ortalamalar için güven aralığı ve t -testleri |
| χ^2_{n-1} | $\frac{(n-1)S^2}{\sigma^2}$ | Varyans için çıkarım, ki-kare testleri |
| F_{n_1-1, n_2-1} | $\frac{S_1^2/\sigma_1^2}{S_2^2/\sigma_2^2}$ | İki varyansın karşılaştırılması, ANOVA, regresyon |

9.6.1 Student t Dağılımı

t dağılımı normal dağılım gibi simetrik ve çan biçimlidir ama kuyrukları **daha kalındır**; serbestlik derecesi arttıkça standart normale yaklaşır. Eski notlarımızdaki iki grafiği birleştirelim:

```
xs <- seq(-4, 4, by = 0.01)
plot(xs, dnorm(xs), type = "l", lwd = 2, xlab = "Sapmalar", ylab = "Olasılık
  yoğunluğu",
      main = "N(0,1) ve t dağılımları")
lines(xs, dt(xs, df = 2), col = "red", lwd = 2)
lines(xs, dt(xs, df = 5), col = "orange", lwd = 2)
legend("topright", c("N(0,1)", "t, sd = 2", "t, sd = 5"),
      col = c("black", "red", "orange"), lwd = 2, bty = "n")
plot(1:30, qt(0.975, df = 1:30), type = "l", col = "red", lwd = 2, ylim = c(0, 13),
      xlab = "Serbestlik derecesi", ylab = "t(0.975)", main = "Kritik t değeri")
abline(h = qnorm(0.975), lty = 2, col = "darkgreen")
text(25, 2.6, "z = 1.96", col = "darkgreen")
```



```
qnorm(0.975)
```

```
#> [1] 1.96
```

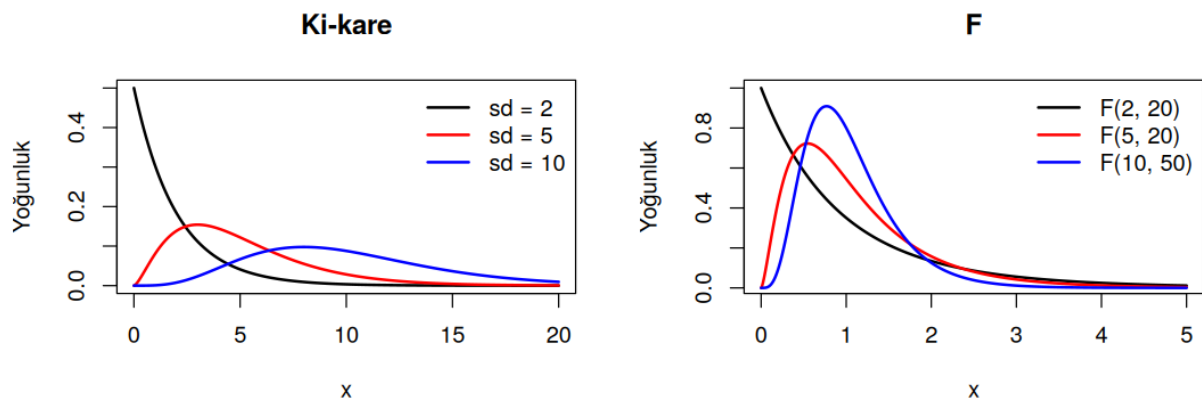
```
qt(0.975, df = c(2, 5, 10, 30, 105))
```

```
#> [1] 4.303 2.571 2.228 2.042 1.983
```

Küçük örneklemelerde kritik değer 1.96'dan belirgin biçimde büyüktür; bu yüzden küçük örneklem-lerle kurulan güven aralıkları daha geniştir. $n \approx 30$ 'dan sonra fark küçülür.

9.6.2 Ki-kare ve F Dağılımları

```
x <- seq(0, 20, by = 0.05)
plot(x, dchisq(x, df = 2), type = "l", lwd = 2, ylab = "Yoğunluk", main = "Ki-kare")
lines(x, dchisq(x, df = 5), col = "red", lwd = 2)
lines(x, dchisq(x, df = 10), col = "blue", lwd = 2)
legend("topright", paste("sd =", c(2, 5, 10)), col = c("black", "red", "blue"), lwd
      = 2, bty = "n")
x <- seq(0, 5, by = 0.01)
plot(x, df(x, df1 = 2, df2 = 20), type = "l", lwd = 2, ylab = "Yoğunluk", main = "F")
lines(x, df(x, df1 = 5, df2 = 20), col = "red", lwd = 2)
lines(x, df(x, df1 = 10, df2 = 50), col = "blue", lwd = 2)
legend("topright", c("F(2, 20)", "F(5, 20)", "F(10, 50)"),
      col = c("black", "red", "blue"), lwd = 2, bty = "n")
```



```
qchisq(0.95, df = 1) # Ki-kare tablosu: 3.84
```

```
#> [1] 3.841
```

```
qf(0.975, df1 = 9, df2 = 9) # F tablosu
```

```
#> [1] 4.026
```

```
1 - pf(2.5, df1 = 3, df2 = 20) # F = 2.5 için sağ kuyruk olasılığı
```

```
#> [1] 0.08884
```

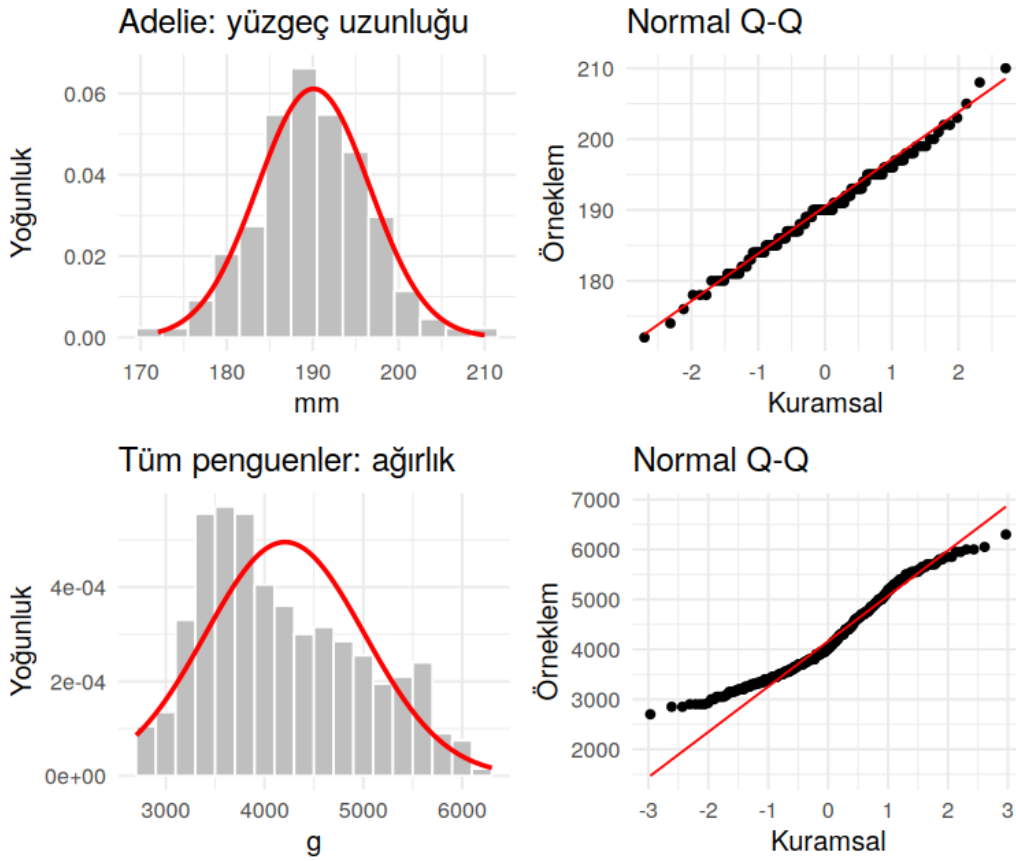
Artık istatistik kitaplarının arkasındaki tablolara ihtiyacımız yok: Her tablo değeri bir **q** fonksiyonu, her tablo olasılığı bir **p** fonksiyonudur.

9.7 Veri Normal Dağılıma Uyuyor mu?

Pek çok yöntem (t-testi, ANOVA, regresyon) normallik varsayımına dayanır. Bir verinin normal dağılıma uyup uymadığını önce **grafikle** inceleriz:

1. **Histogram + normal eğri:** Verinin ortalaması ve standart sapmasıyla çizilmiş normal eğri histograma uyuyor mu?
2. **Q-Q grafiği:** Verinin nicelikleri ile normal dağılımın kuramsal nicelikleri karşılaştırılır. Noktalar doğru üzerinde ise veri normaldir.

```
adelie <- penguinler |> filter(species == "Adelie")
p1 <- ggplot(adelie, aes(flipper_length_mm)) +
  geom_histogram(aes(y = after_stat(density)), binwidth = 3, fill = "grey75", colour
  ↪ = "white") +
  stat_function(fun = dnorm, colour = "red", linewidth = 1,
    args = list(mean(adelie$flipper_length_mm),
    ↪ sd(adelie$flipper_length_mm))) +
  labs(title = "Adelie: yüzgeç uzunluğu", x = "mm", y = "Yoğunluk")
p2 <- ggplot(adelie, aes(sample = flipper_length_mm)) +
  geom_qq() + geom_qq_line(colour = "red") +
  labs(title = "Normal Q-Q", x = "Kuramsal", y = "Örneklem")
p3 <- ggplot(penguinler, aes(body_mass_g)) +
  geom_histogram(aes(y = after_stat(density)), binwidth = 200, fill = "grey75",
  ↪ colour = "white") +
  stat_function(fun = dnorm, colour = "red", linewidth = 1,
    args = list(mean(penguinler$body_mass_g),
    ↪ sd(penguinler$body_mass_g))) +
  labs(title = "Tüm penguinler: ağırlık", x = "g", y = "Yoğunluk")
p4 <- ggplot(penguinler, aes(sample = body_mass_g)) +
  geom_qq() + geom_qq_line(colour = "red") +
  labs(title = "Normal Q-Q", x = "Kuramsal", y = "Örneklem")
library(patchwork)
(p1 + p2) / (p3 + p4)
```



Adelie penguenlerinin yüzgeç uzunlukları normal dağılıma çok iyi uyar. Bütün penguenlerin ağırlıkları ise sağa çarpıktır: Q-Q grafiğinde noktalar iki uçta doğrudan sapar. Bunun nedeni farklı türlerin bir arada olmasıdır (Gentoo'lar çok daha ağırdır). Normalliği **gruplar içinde** incelemek gerekir.

Normalliğin biçimsel testleri (Shapiro–Wilk) Bölüm 11'te ele alınacaktır.

Kendini Sına

- 💡 1. Bir sınavın puanları $N(65, 12^2)$ dağılımına uyuyor. 80'in üzerinde puan alanların oranı nedir? En başarılı %10'a girmek için kaç puan gerekir?

```
1 - pnorm(80, 65, 12) # 0.106
qnorm(0.90, 65, 12)  # 80.4
```

- 💡 2. Adil bir para 10 kez atılıyor. En az 8 tura gelme olasılığı nedir?

```
1 - pbinom(7, 10, 0.5) # 0.0547
sum(dbinom(8:10, 10, 0.5))
```

- 💡 3. `dnorm(0)` neden 0.399 verir de bir olasılık değildir?

Sürekli dağılımlarda **d** fonksiyonu yoğunluk eğrisinin **yüksekliğini** verir; olasılık, eğrinin altındaki **alandır**. Tek bir noktanın olasılığı sıfırdır. Yoğunluk 1'den büyük bile olabilir: `dnorm(0,`

`sd = 0.1) = 3.99.`

💡 4. Serbestlik derecesi 10 olan bir t dağılımında çift yönlü %5 kritik değer nedir? Neden 1.96'dan büyüktür?

`qt(0.975, 10) = 2.228.` t dağılımının kuyrukları normal dağılımdan kalın olduğu için aynı kuyruk alanını bırakmak için daha uzağa gitmek gerekir.

💡 5. `rnorm(1000, 50, 10)` ile veri üretip ortalamasını, standart sapmasını ve 60'tan büyük gözlemlerin oranını hesaplayın. Kuramsal değerlerle karşılaştırın.

```
set.seed(1); x <- rnorm(1000, 50, 10)
mean(x); sd(x); mean(x > 60)      # ≈ 50, ≈ 10, ≈ 0.159
1 - pnorm(60, 50, 10)             # Kuramsal: 0.1587
```

10 Örneklem, Merkezi Limit Teoremi ve Güven Aralıkları

i Bu haftanın hedefleri

- Kitle ile örneklem, parametre ile istatistik arasındaki farkı açıklamak,
- Bir istatistiğin **örneklem dağılımını** simülasyonla üretmek,
- **Merkezi Limit Teoremini** (MLT) simülasyonla göstermek ve neden bu kadar önemli olduğunu açıklamak,
- Standart sapma ile **standart hata** arasındaki farkı bilmek,
- Ortalama ve oran için **güven aralığı** kurmak ve doğru yorumlamak,
- Örneklem büyüklüğü ile güven aralığı genişliği arasındaki ilişkiyi göstermek,
- **Bootstrap** yöntemiyle, formül kullanmadan güven aralığı oluşturmak.

10.1 Kitle, Örneklem, Parametre, İstatistik

| | Kitle (anakütle) | Örneklem |
|----------------|---------------------------------------|--|
| Nedir? | İlgilenilen tüm birimler | Kitleden seçilen birimler |
| Özeti | Parametre (bilinmez, sabittir) | İstatistik (hesaplanır, örneklemden örneklem değişir) |
| Ortalama | μ | $\bar{x}$ |
| Standart sapma | σ | s |
| Oran | p | $\hat{p}$ |

İstatistiksel çıkarımın temel sorusu şudur: **Elimizdeki tek bir örneklemden kitle hakkında ne kadar emin konuşabiliriz?** Bunu cevaplamak için, örnekleme tekrar tekrar çekseydik istatistiğimizin nasıl değişeceğini, yani **örneklem dağılımını** bilmemiz gerekir.

10.2 Örneklem Dağılımı

Bir an için penguenler verisindeki 333 penguenin bütün kitle olduğunu varsayalım. Kitle ortalaması bilinsin:

```
mu <- mean(penguenler$body_mass_g)
mu
```

```
#> [1] 4207
```

Bu kitleden 10 penguenlik bir örneklem çekelim. Her örneklem ortalaması farklı çıkar:

```
set.seed(10)
mean(sample(penguenler$body_mass_g, 10))
```

```
#> [1] 4092
```

```
mean(sample(penguenler$body_mass_g, 10))
```

```
#> [1] 4112
```

```
mean(sample(penguenler$body_mass_g, 10))
```

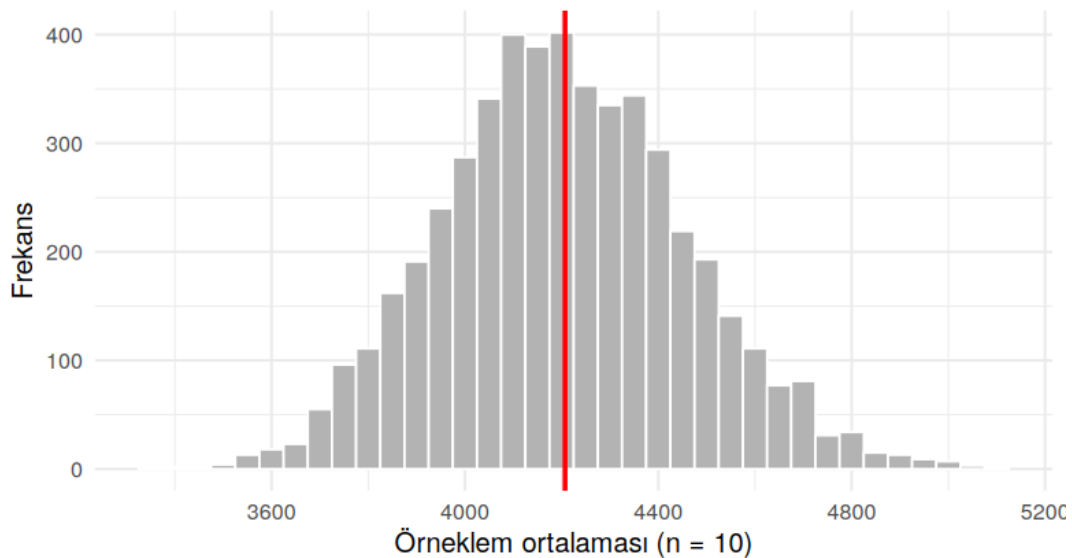
```
#> [1] 4160
```

Bunu 5000 kez tekrarlayıp örneklem ortalamalarının dağılımına bakalım. Bu dağılım, **örneklem ortalamasının örnekleme dağılımıdır**:

```
set.seed(10)
ortalamalar <- replicate(5000, mean(sample(penguenler$body_mass_g, 10)))
c(ortalamalarin_ortalatasi = mean(ortalamalar), ortalamalarin_ss = sd(ortalamalar))
```

```
#> ortalamalarin_ortalatasi      ortalamalarin_ss
#>                4205.0                254.6
```

```
ggplot(data.frame(ort = ortalamalar), aes(ort)) +
  geom_histogram(binwidth = 50, fill = "grey70", colour = "white") +
  geom_vline(xintercept = mu, colour = "red", linewidth = 1) +
  labs(x = "Örneklem ortalaması (n = 10)", y = "Frekans")
```



i Not

`replicate(k, ifade)` bir ifadeyi k kez çalıştırıp sonuçları bir vektörde toplar. Bir `for` döngüsünün kısa yoludur ve simülasyonlarda çok kullanılır.

Örneklem ortalamalarının ortalaması kitle ortalamasına eşittir (örneklem ortalaması **yansızdır**). Ortalamaların standart sapmasına **standart hata** denir.

10.3 Merkezi Limit Teoremi

Kitlenin dağılımı ne olursa olsun (sonlu varyansı olduğu sürece), örneklem büyüklüğü n yeterince büyük olduğunda örneklem ortalamasının dağılımı yaklaşık olarak normaldir:

$$\bar{X} \sim N\left(\mu, \frac{\sigma^2}{n}\right)$$

10.3.1 Eski Notlardan: Dart Oyunu

Bir dart tahtasında puanlar 0–10 arasında ve ortadaki puanlara isabet etme olasılığı daha yüksek olsun:

```
dartpuan <- 0:10
dart_frekans <- c(1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1)
oyunlar <- rep(dartpuan, dart_frekans)
oyunlar
```

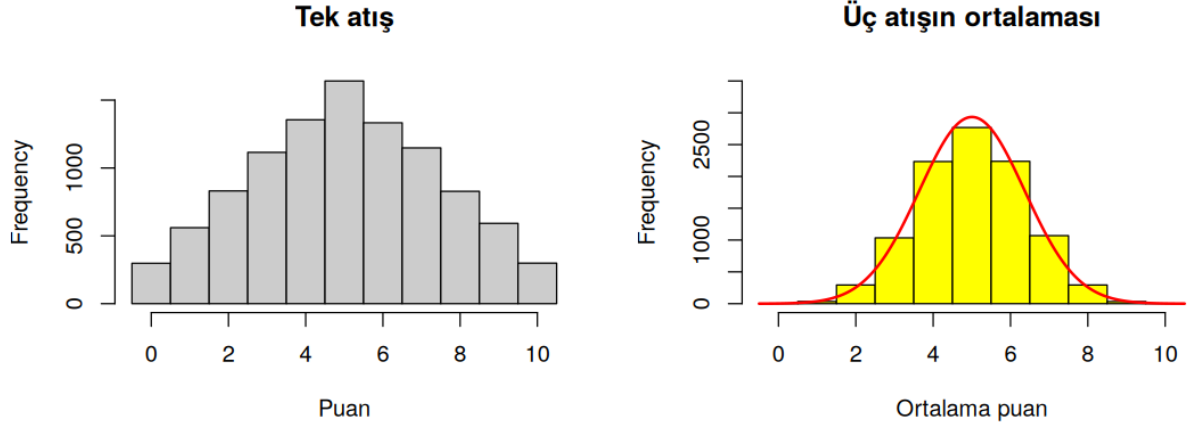
```
#> [1] 0 1 1 2 2 2 3 3 3 3 4 4 4 4 4 5 5 5 5 5 5 6 6 6 6
#> [26] 6 7 7 7 7 8 8 8 9 9 10
```

Tek bir atışın dağılımı üçgen biçimlidir. Üç atışın **ortalamasının** dağılımı ise normal dağılıma çok benzer:

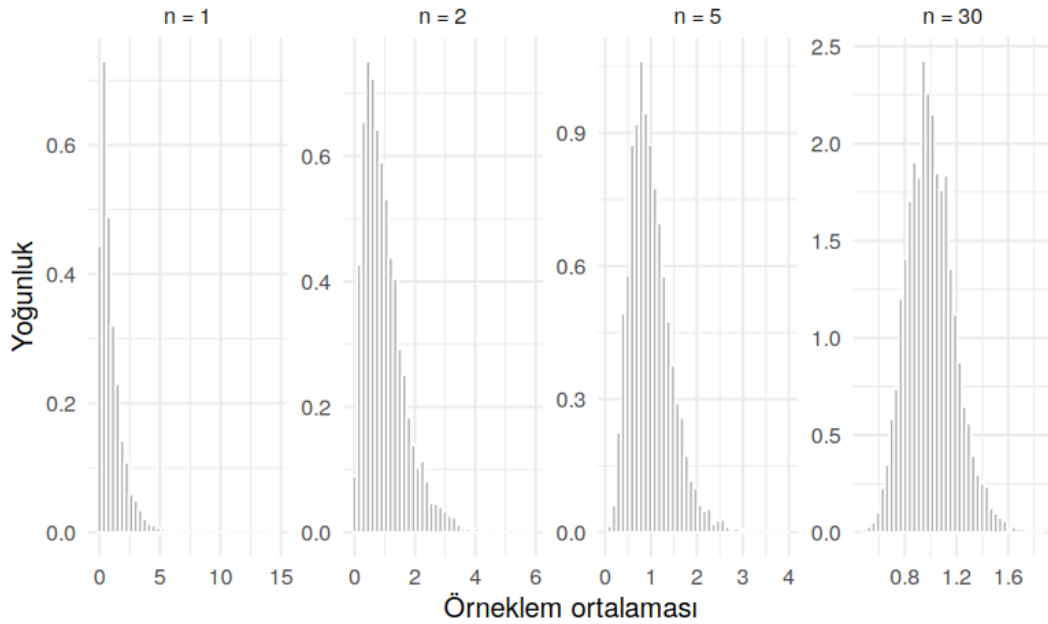
```
set.seed(5)
secim <- sample(oyunlar, 10000, replace = TRUE)
hist(secim, breaks = seq(-0.5, 10.5, 1), col = "grey80", main = "Tek atış", xlab =
  ↪ "Puan")
ort_dartpuan <- replicate(10000, mean(sample(oyunlar, 3)))
hist(ort_dartpuan, breaks = seq(-0.5, 10.5, 1), col = "yellow", ylim = c(0, 3500),
  main = "Üç atışın ortalaması", xlab = "Ortalama puan")
xx <- seq(-0.5, 10.5, 0.01)
lines(xx, 10000 * dnorm(xx, mean(ort_dartpuan), sd(ort_dartpuan)), col = "red", lwd
  ↪ = 2)
```

10.3.2 Çarpık Bir Kitleyle MLT

MLT'nin asıl gücü, kitle **normal olmasa bile** çalışmasıdır. Belirgin biçimde sağa çarpık olan üstel dağılımdan ($\lambda = 1$, yani $\mu = \sigma = 1$) farklı büyüklükte örneklem çekeriz:



```
set.seed(6)
mlt <- do.call(rbind, lapply(c(1, 2, 5, 30), function(n) {
  data.frame(n = paste("n =", n), ort = replicate(5000, mean(rexp(n, rate = 1))))
}))
mlt$n <- factor(mlt$n, levels = paste("n =", c(1, 2, 5, 30)))
ggplot(mlt, aes(ort)) +
  geom_histogram(aes(y = after_stat(density)), bins = 40, fill = "grey70", colour =
    ↪ "white") +
  facet_wrap(~ n, scales = "free", nrow = 1) +
  labs(x = "Örneklem ortalaması", y = "Yoğunluk")
```



Şekil 10.1: Üstel dağılımdan çekilen örneklemelerin ortalamaları: n büyüdükçe dağılım normalleşir ve daralır

$n = 30$ olduğunda örneklem ortalamalarının dağılımı neredeyse simetrik ve çan biçimlidir; yayılımı da $\sigma/\sqrt{n} = 1/\sqrt{30} \approx 0.18$ 'e düşmüştür:

```
mlt |> group_by(n) |> summarise(ortalama = mean(ort), standart_hata = sd(ort))
```

```
#> # A tibble: 4 x 3
#>   n      ortalama standart_hata
#>   <fct>      <dbl>      <dbl>
#> 1 n = 1      0.974      0.986
#> 2 n = 2      0.985      0.693
#> 3 n = 5      0.995      0.447
#> 4 n = 30     1.00      0.185
```

! MLT neden bu kadar önemli?

Kitlenin dağılımını bilmesek bile, örneklem ortalamasının dağılımını **biliyoruz**. Bu sayede güven aralıkları ve t-testleri, veri tam normal olmasa da (yeterince büyük örneklerde) çalışır. “Yeterince büyük” ne kadar? Simetrik dağılımlarda $n \approx 15-30$ çoğu zaman yeterlidir; çok çarpık dağılımlarda daha büyük örneklemeler gerekir.

10.4 Standart Hata

| | Standart sapma (s) | Standart hata (SH) |
|----------------------------------|---|---|
| Neyin yayılımı?
n büyüdükçe | Gözlemlerin
Değişmez (kitle σ 'sını tahmin eder) | Örneklem ortalamalarının
Küçülür |
| Formül | $s = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n-1}}$ | $SH = \frac{s}{\sqrt{n}} = \sqrt{\frac{s^2}{n}}$ |
| Soru | Penguenler birbirinden ne kadar farklı? | Ortalama tahminimiz ne kadar kesin? |

Eski notlarımızdaki worms örneği:

```
sh <- sqrt(var(worms$Worm.density) / length(worms$Worm.density))
sh
```

```
#> [1] 0.5861
```

10.5 Ortalama İçin Güven Aralığı

σ bilinmediğinde (pratikte hep böyledir) ortalama için $(1 - \alpha)$ güven aralığı:

$$\bar{x} \pm t_{1-\alpha/2, n-1} \cdot \frac{s}{\sqrt{n}}$$

Worms verisinde solucan yoğunluğu ortalaması için %95 güven aralığını adım adım kuralım:

```
x <- worms$Worm.density
n <- length(x)
t_kritik <- qt(0.975, df = n - 1)
c(alt = mean(x) - t_kritik * sd(x) / sqrt(n),
  ust = mean(x) + t_kritik * sd(x) / sqrt(n))
```

```
#> alt ust
#> 3.123 5.577
```

```
t.test(x)$conf.int # R'ın hazır fonksiyonu ile doğrulama
```

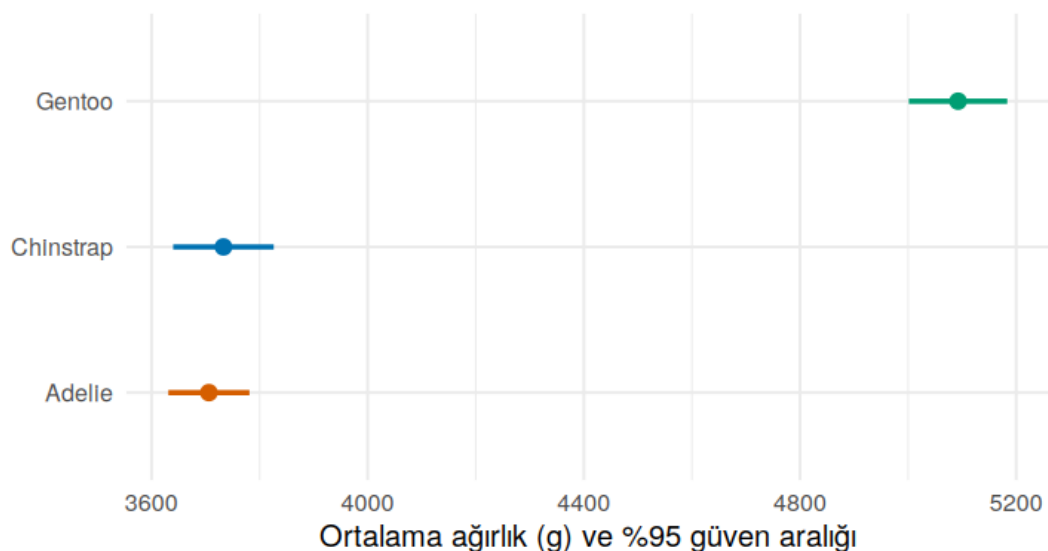
```
#> [1] 3.123 5.577
#> attr(,"conf.level")
#> [1] 0.95
```

Penguenlerde türlere göre ortalama ağırlık ve %95 güven aralıkları:

```
ga_tablo <- penguenler |>
  group_by(species) |>
  summarise(n = n(), ort = mean(body_mass_g), sh = sd(body_mass_g) / sqrt(n),
            alt = ort - qt(0.975, n - 1) * sh, ust = ort + qt(0.975, n - 1) * sh)
ga_tablo
```

```
#> # A tibble: 3 x 6
#>   species      n  ort    sh  alt  ust
#>   <fct>    <int> <dbl> <dbl> <dbl> <dbl>
#> 1 Adelie    146 3706.  38.0 3631. 3781.
#> 2 Chinstrap  68 3733.  46.6 3640. 3826.
#> 3 Gentoo   119 5092.  46.0 5001. 5183.
```

```
ggplot(ga_tablo, aes(x = ort, y = species, colour = species)) +
  geom_pointrange(aes(xmin = alt, xmax = ust), linewidth = 1, show.legend = FALSE) +
  scale_colour_manual(values = tur_renkleri) +
  labs(x = "Ortalama ağırlık (g) ve %95 güven aralığı", y = NULL)
```



10.5.1 Güven Aralığını Doğru Yorumlamak

⚠ Yanlış: “Kitle ortalamasının bu aralıkta olma olasılığı %95’tir.”

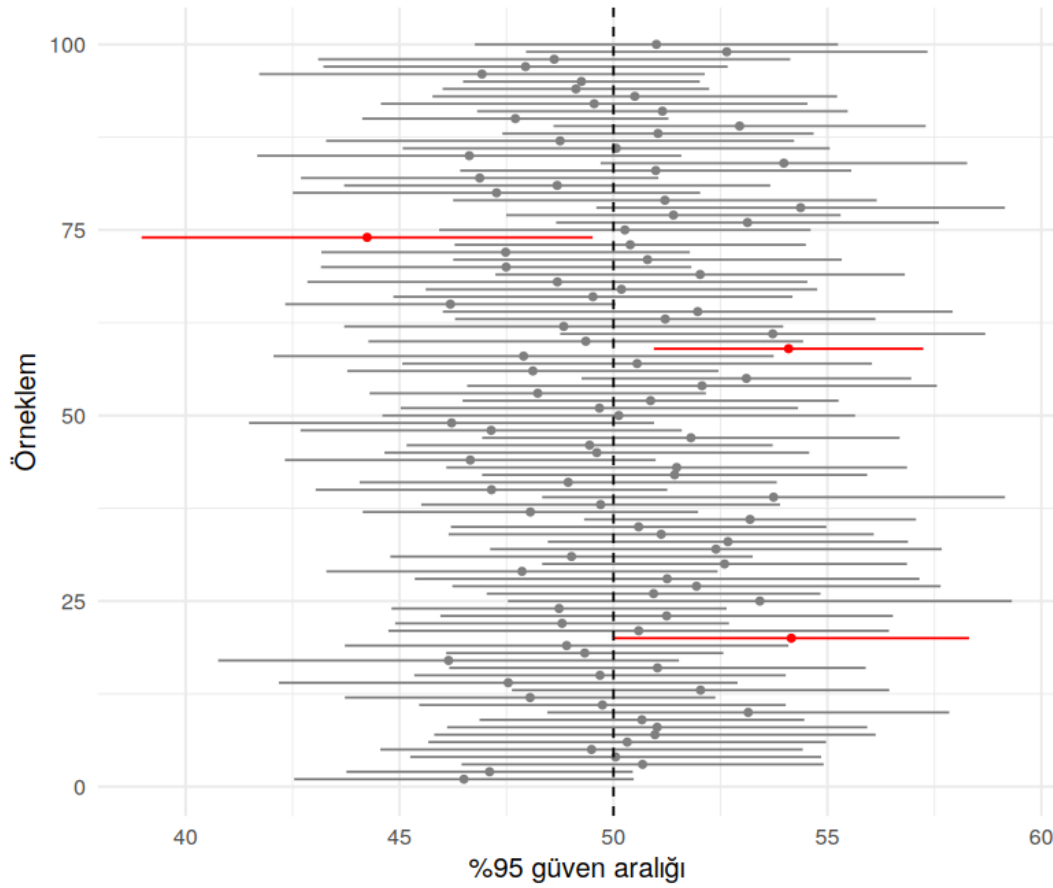
Kitle ortalaması sabit bir sayıdır; ya aralıktadır ya değildir. **Doğru yorum:** Bu yöntemle çok sayıda örneklem çekip her birinden aralık kursaydık, aralıkların yaklaşık **%95’i** kitle ortalamasını içerirdi. “%95” yöntemin güvenilirliğidir, tek bir aralığın değil.

Bunu simülasyonla görelim: $N(50, 10^2)$ kitlesinden $n = 20$ büyüklüğünde 100 örneklem çekip her birinden %95 güven aralığı kuralım:

```
set.seed(11)
ga <- do.call(rbind, lapply(1:100, function(i) {
  ornek <- rnorm(20, mean = 50, sd = 10)
  ci <- t.test(ornek)$conf.int
  data.frame(no = i, alt = ci[1], ust = ci[2], ort = mean(ornek))
}))
ga$kapsiyor <- ga$alt < 50 & ga$ust > 50
mean(ga$kapsiyor)
```

```
#> [1] 0.97
```

```
ggplot(ga, aes(y = no, x = ort, xmin = alt, xmax = ust, colour = kapsiyor)) +
  geom_pointrange(size = 0.15) +
  geom_vline(xintercept = 50, linetype = "dashed") +
  scale_colour_manual(values = c(`TRUE` = "grey50", `FALSE` = "red"), guide =
    ↪ "none") +
  labs(x = "%95 güven aralığı", y = "Örneklem")
```

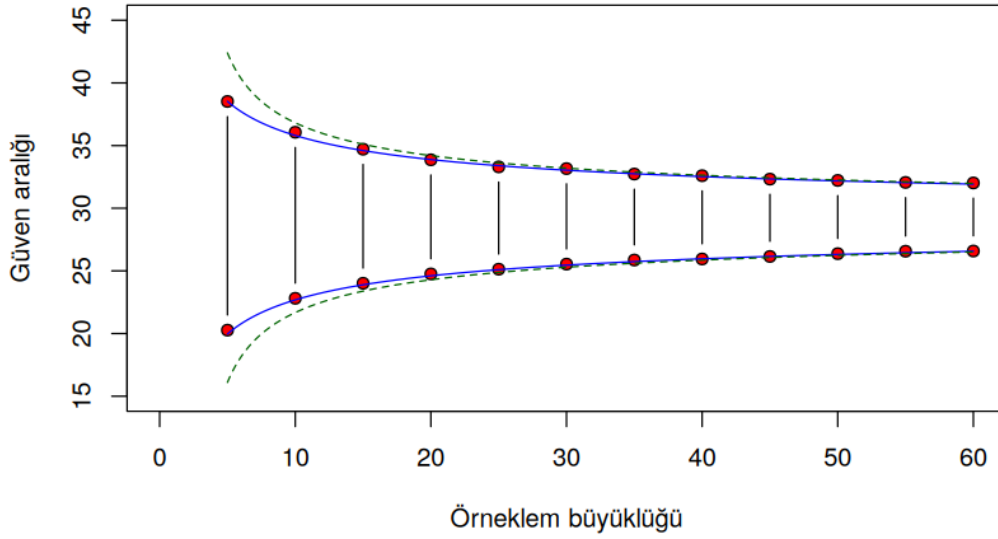


Şekil 10.2: 100 güven aralığından kitle ortalamasını (kesikli çizgi) içermeyenler kırmızı

10.5.2 Örneklem Büyüklüğü Arttıkça Güven Aralığı Daralır

Eski notlarımızdaki simülasyon: Bir veriden farklı büyüklükte (yerine koyarak) örneklem çekip ortalamaların %2.5 ve %97.5 yüzdelerlerini çizelim; üzerine normal (mavi) ve t (yeşil) dağılımına göre kuramsal sınırları ekleyelim:

```
set.seed(30)
values <- rnorm(100, mean = 30, sd = 10)
plot(c(0, 60), c(15, 45), type = "n", xlab = "Örneklem büyüklüğü", ylab = "Güven
  ↪ aralığı")
for (k in seq(5, 60, 5)) {
  a <- replicate(5000, mean(sample(values, k, replace = TRUE)))
  points(c(k, k), quantile(a, c(0.025, 0.975)), type = "b", pch = 21, bg = "red")
}
xler <- seq(5, 60, 0.1)
lines(xler, mean(values) + 1.96 * sqrt(var(values) / xler), col = "blue")
lines(xler, mean(values) - 1.96 * sqrt(var(values) / xler), col = "blue")
lines(xler, mean(values) + qt(0.975, xler - 1) * sqrt(var(values) / xler), lty = 2,
  ↪ col = "darkgreen")
lines(xler, mean(values) - qt(0.975, xler - 1) * sqrt(var(values) / xler), lty = 2,
  ↪ col = "darkgreen")
```



Genişlik $1/\sqrt{n}$ ile azalır: Aralığı **yarıya** indirmek için örnekleme **dört katına** çıkarmak gerekir.

10.5.3 Gerekli Örneklem Büyüklüğü

Ortalamayı en fazla E hata payıyla tahmin etmek için gereken örneklem büyüklüğü (hakkında bir ön bilgiyle):

$$n = \left(\frac{z_{1-\alpha/2} \sigma}{E} \right)^2$$

```
sigma <- 460      # Önceki çalışmadan penguen ağırlığının standart sapması
E <- 50           # ±50 g hata payı istiyoruz
ceiling((qnorm(0.975) * sigma / E)^2)
```

```
#> [1] 326
```

10.6 Oran İçin Güven Aralığı

Büyük örneklerde örneklem oranı $\hat{p}$ yaklaşık normal dağılır ve standart hatası $\sqrt{\hat{p}(1-\hat{p})/n}$ 'dir:

$$\hat{p} \pm z_{1-\alpha/2} \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

Penguenlerin ne kadarı Adelie türündendir?

```
x_adelie <- sum(penguenler$species == "Adelie"); n <- nrow(penguenler)
p_sapka <- x_adelie / n
p_sapka + c(-1, 1) * qnorm(0.975) * sqrt(p_sapka * (1 - p_sapka) / n) # Wald aralığı
```

```
#> [1] 0.3851 0.4917
```

```
prop.test(x_adelie, n)$conf.int      # Wilson aralığı (daha iyi davranır)
```

```
#> [1] 0.3847 0.4936  
#> attr(,"conf.level")  
#> [1] 0.95
```

```
binom.test(x_adelie, n)$conf.int     # Kesin (Clopper-Pearson) aralık
```

```
#> [1] 0.3844 0.4936  
#> attr(,"conf.level")  
#> [1] 0.95
```

Küçük örneklerde ya da oran 0'a veya 1'e yakinken Wald aralığı kötü sonuç verir; `prop.test()` ya da `binom.test()` tercih edilmelidir.

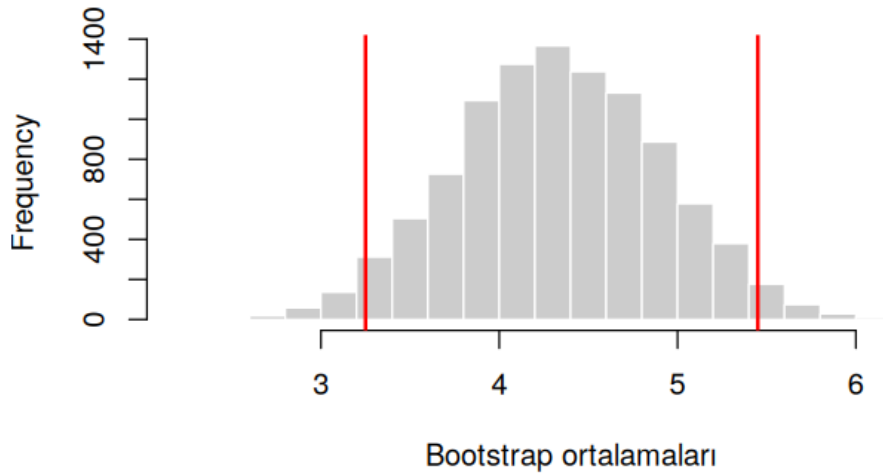
10.7 Bootstrap

Ya örnekleme dağılımı için bir formülümüz yoksa (örneğin ortanca ya da korelasyon için), ya da veri normal değilse ve örneklem küçükse? **Bootstrap** (Efron, 1979) bu durumda bilgisayar gücünü kullanır:

1. Elimizdeki örnekleme **kitle gibi** kabul et.
2. Bu örneklemden **yerine koyarak**, aynı büyüklükte çok sayıda (örneğin 10.000) yeni örneklem çek.
3. Her yeni örnekleme ilgilenilen istatistiği hesapla.
4. Bu istatistiklerin dağılımı, örnekleme dağılımının bir tahminidir. %2.5 ve %97.5 yüzdelikleri bir %95 güven aralığı verir (**yüzdelik bootstrap aralığı**).

Eski notlarımızdaki worms örneği:

```
set.seed(12)  
boot_ort <- numeric(10000)  
for (i in 1:10000) {  
  boot_ort[i] <- mean(sample(worms$Worm.density, replace = TRUE))  
}  
hist(boot_ort, col = "grey80", border = "white", main = "", xlab = "Bootstrap  
↪ ortalamaları")  
abline(v = quantile(boot_ort, c(0.025, 0.975)), col = "red", lwd = 2)
```



```
quantile(boot_ort, c(0.025, 0.975))      # Bootstrap %95 GA
```

```
#> 2.5% 97.5%
#> 3.25 5.45
```

```
t.test(worms$Worm.density)$conf.int      # t dağılımına dayalı %95 GA
```

```
#> [1] 3.123 5.577
#> attr(,"conf.level")
#> [1] 0.95
```

İki aralık birbirine çok yakındır. Eski notlarımızda bu bootstrap dağılımı, ortalamanın 6.5 olamayacağını göstermek için de kullanılmıştı: 6.5 değeri bootstrap aralığının dışında kalıyor. Bu fikir Bölüm 11'teki hipotez testlerine bir köprüdür.

Bootstrap'ın asıl gücü, formülü olmayan istatistiklerde ortaya çıkar. Gentoo penguenlerinin ağırlık **ortancası** için %95 güven aralığı:

```
gentoo <- penguenler$body_mass_g[penguenler$species == "Gentoo"]
set.seed(13)
boot_ortanca <- replicate(10000, median(sample(gentoo, replace = TRUE)))
median(gentoo)
```

```
#> [1] 5050
```

```
quantile(boot_ortanca, c(0.025, 0.975))
```

```
#> 2.5% 97.5%
#> 4925 5200
```

i Not

`boot` paketi bootstrap için daha gelişmiş yöntemler (BCa aralığı gibi) sunar. Bu derste mantığı görmek için döngü ve `replicate()` ile yazmak yeterlidir.

Kendini Sına

💡 1. Standart sapma ile standart hata arasındaki farkı bir cümleyle açıklayın.

Standart sapma tek tek **gözlemlerin** ortalama etrafındaki yayılımını, standart hata ise **örneklem ortalamasının** (tekrarlanan örneklerde) kitle ortalaması etrafındaki yayılımını ölçer; $SH = s/\sqrt{n}$.

💡 2. iris verisinde *setosa* türünün çanak yaprak uzunluğu için %95 ve %99 güven aralıklarını kurun. Hangisi daha geniş? Neden?

```
s <- iris$Sepal.Length[iris$Species == "setosa"]
t.test(s, conf.level = 0.95)$conf.int
t.test(s, conf.level = 0.99)$conf.int
```

%99 aralık daha geniştir: daha emin olmak için daha geniş bir aralık vermek gerekir.

💡 3. MLT simülasyonunu düzgün dağılım (`runif`) için tekrarlayın. Hangi n değerinde ortalamaların dağılımı normale benzemeye başlıyor?

Düğüün dağılım simetrik olduğı için $n = 5$ civarında bile ortalamaların dağılımı çan biçimine çok yaklaşı; üstel dağılımda bunun için daha büyük n gerekir.

💡 4. Chinstrap penguenlerinin gaga uzunluğu ile gaga derinliği arasındaki korelasyon için bootstrap %95 güven aralığı kurun.

```
ch <- penguenler |> filter(species == "Chinstrap")
set.seed(1)
boot_r <- replicate(5000, {
  i <- sample(nrow(ch), replace = TRUE)
  cor(ch$bill_length_mm[i], ch$bill_depth_mm[i])
})
quantile(boot_r, c(0.025, 0.975))
```

Satırları (gözlem çiftlerini) birlikte örneklemek gerekir; değişkenleri ayrı ayrı örneklemek ilişkiyi bozar.

11 Hipotez Testleri

Bu haftanın hedefleri

- Hipotez testinin mantığını (sıfır ve alternatif hipotez, test istatistiği, p değeri, anlamlılık düzeyi) açıklamak,
- I. tip ve II. tip hataları, testin gücünü ve **etki büyüklüğünü** yorumlamak,
- Varsayımları (normallik, varyans homojenliği) grafik ve testlerle denetlemek,
- Tek örneklem, iki bağımsız örneklem ve eşleştirilmiş örneklem için **t-testlerini** uygulamak,
- Varsayımlar sağlanmadığında **parametrik olmayan** karşılıklarını (işaret testi, Wilcoxon işaretli sıra testi, Mann–Whitney U testi) kullanmak,
- İki varyansı **F testi** ile karşılaştırmak,
- Oranlar için binom ve oran testlerini, kategorik değişkenler için **ki-kare** ve **Fisher kesin** testlerini uygulamak,
- Sonuçları istatistiksel olarak doğru ve okunabilir biçimde raporlamak.

11.1 Hipotez Testinin Mantığı

Hipotez testi, bir mahkemeye benzer: Sanık (sıfır hipotezi) **suçsuz kabul edilir**; ancak aksine güçlü kanıt varsa reddedilir. Kanıt yetersizse “suçsuz” değil, “**suçlu olduğu kanıtlanamadı**” deriz.

| Adım | Açıklama | Örnek |
|----------------------|---|--|
| 1. Hipotezler | H_0 : “fark/etki yok”; H_1 : araştırma iddiası | $H_0 : \mu = 190, H_1 : \mu \neq 190$ |
| 2. Anlamlılık düzeyi | Yanlışlıkla H_0 ’ı reddetmeye razı olduğumuz olasılık | $\alpha = 0.05$ |
| 3. Test istatistiği | Verinin H_0 ’dan ne kadar uzak olduğunun ölçüsü | $t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$ |
| 4. p değeri | H_0 doğruyken , gözlenen kadar ya da daha uç bir sonuç elde etme olasılığı | $p = 0.03$ |
| 5. Karar | $p < \alpha$ ise H_0 reddedilir | H_0 reddedilir |
| 6. Yorum | Kararı bağlamda ve etki büyüklüğüyle birlikte açıkla | “Ortalama 190 mm’den farklıdır; fark ... mm” |

p değeri ne DEĞİLDİR?

- H_0 ’ın doğru olma olasılığı **değildir**.
- Etkinin büyüklüğünü ya da önemini **göstermez**. Çok büyük örneklemelerde önemsiz farklar bile “anlamlı” çıkar.
- $p > 0.05$ “etki yok” demek **değildir**; yalnızca “etkiyi gösterecek kadar kanıt yok” demektir.

Bu yüzden p değeri her zaman **güven aralığı** ve **etki büyüklüğü** ile birlikte raporlanmalıdır.

11.1.1 Hatalar ve Güç

Tablo 11.2: Hipotez testinde karar tablosu

| | H_0 gerçekte doğru | H_0 gerçekte yanlış |
|--------------------|---------------------------------|---|
| H_0 reddedilmedi | Doğru karar ($1 - \alpha$) | II. tip hata (β) |
| H_0 reddedildi | I. tip hata (α) | Doğru karar: güç ($1 - \beta$) |

α 'yı küçültmek (daha temkinli olmak) β 'yı büyütür. Gücü artırmanın en etkili yolu **örneklem büyüklüğünü** artırmaktır.

11.1.2 p Değerini Simülasyonla Görmek

H_0 doğruyken p değerleri 0 ile 1 arasında **düzgün** dağılır; yani testlerin yaklaşık %5'i yine de $p < 0.05$ verir (I. tip hata). H_0 yanlışken ise p değerleri sıfıra yığılır:

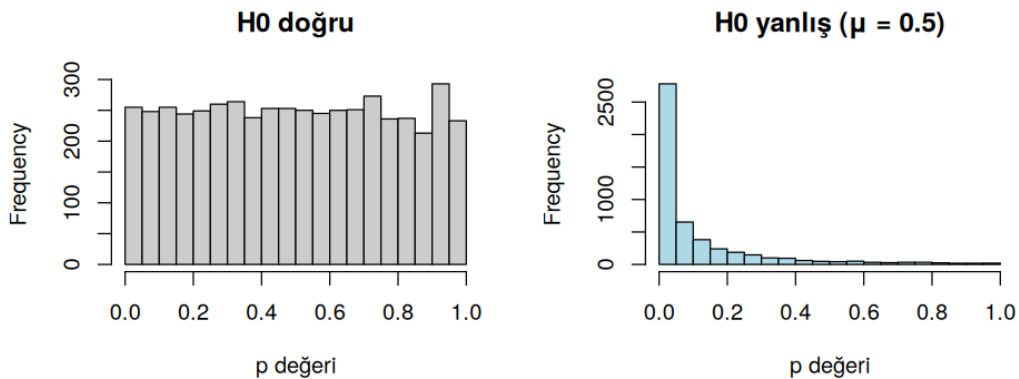
```
set.seed(21)
p_h0_dogru <- replicate(5000, t.test(rnorm(20, mean = 0))$p.value)
p_h0_yanlis <- replicate(5000, t.test(rnorm(20, mean = 0.5))$p.value)
mean(p_h0_dogru < 0.05) # I. tip hata oranı ≈ 0.05
```

```
#> [1] 0.051
```

```
mean(p_h0_yanlis < 0.05) # Güç: gerçek fark 0.5 SS iken n = 20 ile
```

```
#> [1] 0.5562
```

```
par(mfrow = c(1, 2))
hist(p_h0_dogru, breaks = 20, col = "grey80", main = "H0 doğru", xlab = "p değeri")
hist(p_h0_yanlis, breaks = 20, col = "lightblue", main = "H0 yanlış (μ = 0.5)", xlab = "p değeri")
```



```
par(mfrow = c(1, 1))
```

Gerçek fark yarım standart sapma iken $n = 20$ ile testin gücü yalnızca %56 civarındadır. `power.t.test()` gereken örneklem büyüklüğünü hesaplar:

```
power.t.test(delta = 0.5, sd = 1, sig.level = 0.05, power = 0.80, type =  
  ↪ "one.sample")
```

```
#>  
#>      One-sample t test power calculation  
#>  
#>              n = 33.37  
#>            delta = 0.5  
#>              sd = 1  
#>    sig.level = 0.05  
#>        power = 0.8  
#> alternative = two.sided
```

11.2 Parametrik ve Parametrik Olmayan Testler

Parametrik testler verinin belirli bir dağılımdan (genellikle normal) geldiğini varsayar ve ortalamalar üzerinden çalışır. **Parametrik olmayan** (dağılımdan bağımsız) testler ise verinin **sıralarını** (rank) kullanır; normallik varsaymaz, aykırı değerlerden daha az etkilenir ve sıralı (ordinal) veriyle de çalışır. Karşılığında, varsayımlar sağlandığında biraz daha **düşük güce** sahiptirler.

Tablo 11.3: Parametrik testler ve parametrik olmayan karşılıkları

| Durum | Parametrik test | Parametrik olmayan karşılığı | R |
|-------------------------|------------------------------------|---|--|
| Tek örneklem, merkez | Tek örneklem t-testi | İşaret testi / Wilcoxon işaretli sıra testi | <code>t.test(x, mu =) / wilcox.test(x, mu =)</code> |
| İki bağımsız grup | Bağımsız t-testi (Student / Welch) | Mann–Whitney U (Wilcoxon sıra toplamı) | <code>t.test(y ~ g) / wilcox.test(y ~ g)</code> |
| İki eşleştirilmiş ölçüm | Eşleştirilmiş t-testi | Wilcoxon işaretli sıra testi | <code>t.test(x, y, paired = TRUE) / wilcox.test(..., paired = TRUE)</code> |

| Durum | Parametrik test | Parametrik olmayan karşılığı | R |
|---------------------------------|---------------------|----------------------------------|--|
| İkiden fazla bağımsız grup | Tek yönlü ANOVA | Kruskal–Wallis | <code>aov()</code>
/
<code>kruskal.test()</code>
(Bölüm 12) |
| İki varyans | F testi | Ansari–Bradley / Fligner–Killeen | <code>var.test()</code>
/
<code>fligner.test()</code> |
| İki sayısal değişkenin ilişkisi | Pearson korelasyonu | Spearman / Kendall korelasyonu | <code>cor.test(method =)</code>
(Bölüm 13) |
| İki kategorik değişken | — | Ki-kare / Fisher kesin testi | <code>chisq.test()</code>
/
<code>fisher.test()</code> |

Hangi testin seçileceğine dair bir akış şeması Ek A’de verilmiştir.

11.3 Varsayımların Denetlenmesi

11.3.1 Normallik

Normalliği önce **grafikle** (histogram, Q-Q grafiği; Bölüm 9), sonra gerekirse **Shapiro–Wilk testi** ile inceleriz. H_0 : “Veri normal dağılımdan gelmektedir.”

```
shapiro.test(penguenler$flipper_length_mm[penguenler$species == "Adelie"])
```

```
#>
#> Shapiro-Wilk normality test
#>
#> data: penguenler$flipper_length_mm[penguenler$species == "Adelie"]
#> W = 0.99, p-value = 0.7
```

```
shapiro.test(penguenler$body_mass_g)      # Türler bir arada: normal değil
```

```
#>
#> Shapiro-Wilk normality test
#>
#> data: penguenler$body_mass_g
#> W = 0.96, p-value = 4e-08
```

Gruplara göre topluca:

```
penguenler |>
  group_by(species) |>
  summarise(W = shapiro.test(body_mass_g)$statistic,
            p = shapiro.test(body_mass_g)$p.value)
```

```
#> # A tibble: 3 x 3
#>   species      W      p
#>   <fct>    <dbl> <dbl>
#> 1 Adelie    0.981 0.0423
#> 2 Chinstrap 0.984 0.561
#> 3 Gentoo    0.986 0.261
```

💡 Normallik testlerine körü körüne güvenmeyin

- **Küçük** örneklerde normallik testlerinin gücü düşüktür: veri normal olmasa da $p > 0.05$ çıkabilir.
- **Büyük** örneklerde önemsiz sapmalar bile $p < 0.05$ verir; oysa MLT sayesinde t-testi bu durumda zaten iyi çalışır.

Karar verirken **Q-Q grafiğine**, örneklem büyüklüğüne ve aykırı değerlere birlikte bakın.

11.4 Tek Örneklem Testleri

11.4.1 Tek Örneklem t-Testi

Soru: Adelie penguenlerinin ortalama yüzgeç uzunluğu 190 mm'den farklı mıdır?

$$H_0 : \mu = 190 \quad H_1 : \mu \neq 190 \quad t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}} \sim t_{n-1}$$

Önce elle, sonra `t.test()` ile:

```
x <- penguenler$flipper_length_mm[penguenler$species == "Adelie"]
n <- length(x)
t_ist <- (mean(x) - 190) / (sd(x) / sqrt(n))
p_deger <- 2 * pt(-abs(t_ist), df = n - 1)
c(ortalama = mean(x), t = t_ist, sd = n - 1, p = p_deger)
```

```
#> ortalama      t      sd      p
#> 190.1027  0.1903 145.0000  0.8493
```

```
t.test(x, mu = 190)
```

```
#>
#> One Sample t-test
#>
#> data:  x
#> t = 0.19, df = 145, p-value = 0.8
#> alternative hypothesis: true mean is not equal to 190
#> 95 percent confidence interval:
#>  189.0 191.2
#> sample estimates:
#> mean of x
#>    190.1
```

Yorum: Adelie penguenlerinin ortalama yüzgeç uzunluğu (190.1 mm) 190 mm'den istatistiksel olarak farklı değildir ($t_{145} = 0.19$, $p = 0.85$). Ortalama için %95 güven aralığı [189.0, 191.2] mm'dir ve 190'ı içerir.

Tek yönlü hipotezler için `alternative = "greater"` ya da `"less"` kullanılır:

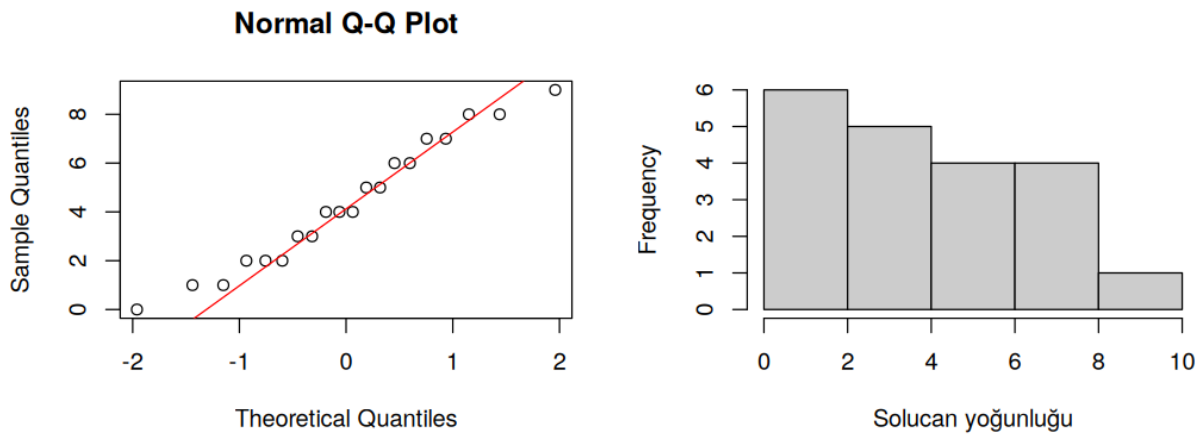
```
t.test(x, mu = 185, alternative = "greater")$p.value # H1:  $\mu > 185$ 
```

```
#> [1] 4.024e-17
```

11.4.2 İşaret Testi ve Wilcoxon İşaretli Sıra Testi

Eski notlarımızda worms verisinin solucan yoğunluğunun ortalamasının 6.5 olup olmadığı soruluyordu. Önce veriye bakalım:

```
qqnorm(worms$Worm.density); qqline(worms$Worm.density, col = "red")
hist(worms$Worm.density, col = "grey80", main = "", xlab = "Solucan yoğunluğu")
```



Yalnızca 20 gözlem var ve değerler kesikli (tam sayı). Böyle durumlarda parametrik olmayan testler tercih edilebilir.

İşaret testi en basit testtir: H_0 altında gözlemlerin yarısı ortancanın üstünde, yarısı altında olmalıdır. Bu yüzden 6.5'ten büyük gözlem sayısı $\text{Binom}(n, 0.5)$ dağılır:

```
ustunde <- sum(worms$Worm.density > 6.5)
toplam <- sum(worms$Worm.density != 6.5)
c(ustunde = ustunde, toplam = toplam)
```

```
#> ustunde toplam
#>      5      20
```

```
binom.test(ustunde, toplam, p = 0.5)
```

```
#>
#> Exact binomial test
#>
#> data: ustunde and toplam
#> number of successes = 5, number of trials = 20, p-value = 0.04
#> alternative hypothesis: true probability of success is not equal to 0.5
#> 95 percent confidence interval:
#>  0.08657 0.49105
#> sample estimates:
#> probability of success
#>                0.25
```

Wilcoxon işaretli sıra testi yalnızca işaretleri değil, farkların **büyüklik sıralarını** da kullandığı için daha güçlüdür. Varsayımı: farkların dağılımı simetriktir. H_0 : ortanca = 6.5.

```
wilcox.test(worms$Worm.density, mu = 6.5, exact = FALSE)
```

```
#>
#> Wilcoxon signed rank test with continuity correction
#>
#> data: worms$Worm.density
#> V = 28, p-value = 0.004
#> alternative hypothesis: true location is not equal to 6.5
```

```
mean(worms$Worm.density); median(worms$Worm.density)
```

```
#> [1] 4.35
```

```
#> [1] 4
```

Mantığını adım adım görelim:

```
farklar <- worms$Worm.density - 6.5
siralar <- rank(abs(farklar))
V <- sum(siralar[farklar > 0]) # Pozitif farkların sıra toplamı
V
```

```
#> [1] 28.5
```

`wilcox.test()` çıktısındaki V istatistiği budur. H_0 altında pozitif ve negatif sıra toplamaları birbirine yakın olmalıdır.

i Not

Eşit değerler (*ties*) olduğunda kesin p değeri hesaplanamaz ve R bir uyarı verir; `exact = FALSE` normal yaklaşımı kullanır. Worms verisindeki gibi tam sayılı verilerde eşitlikler çok yaygındır.

11.5 İki Varyansın Karşılaştırılması: F Testi

Bağımsız iki örneklem t-testinin klasik (Student) sürümü iki grubun varyanslarının eşit olduğunu varsayar. İki varyansı **F testi** ile karşılaştırırız:

$$H_0 : \sigma_1^2 = \sigma_2^2 \quad F = \frac{s_1^2}{s_2^2} \sim F_{n_1-1, n_2-1}$$

Eski notlarımızdaki worms örneği: nemli ve nemsiz tarlalarda solucan yoğunluğunun varyansları eşit mi?

```
nemsiz <- worms$Worm.density[worms$Damp == FALSE]
nemli <- worms$Worm.density[worms$Damp == TRUE]
n1 <- length(nemsiz); n2 <- length(nemli)
F_orani <- var(nemsiz) / var(nemli)
c(var_nemsiz = var(nemsiz), var_nemli = var(nemli), F = F_orani)
```

```
#> var_nemsiz var_nemli      F
#>      7.824      3.500    2.235
```

```
2 * min(pf(F_orani, n1 - 1, n2 - 1), 1 - pf(F_orani, n1 - 1, n2 - 1)) # Çift yönlü p
```

```
#> [1] 0.3843
```

```
qf(c(0.025, 0.975), n1 - 1, n2 - 1) # Kritik değerler
```

```
#> [1] 0.2655 6.4876
```

```
var.test(nemsiz, nemli)
```

```
#>
#> F test to compare two variances
#>
#> data: nemsiz and nemli
#> F = 2.2, num df = 13, denom df = 5, p-value = 0.4
#> alternative hypothesis: true ratio of variances is not equal to 1
#> 95 percent confidence interval:
#>  0.3446 8.4203
#> sample estimates:
#> ratio of variances
#>                2.235
```

F testi normallığe **çok duyarlıdır**. Normallikten şüphe edildiğinde parametrik olmayan **Fligner–Killeen** testi ya da Bölüm 12’de göreceğimiz **Levene** testi kullanılır:

```
fligner.test(Worm.density ~ Damp, data = worms)
```

```
#>
#> Fligner-Killeen test of homogeneity of variances
#>
#> data: Worm.density by Damp
#> Fligner-Killeen:med chi-squared = 1.3, df = 1, p-value = 0.3
```

11.6 İki Bağımsız Örneklem

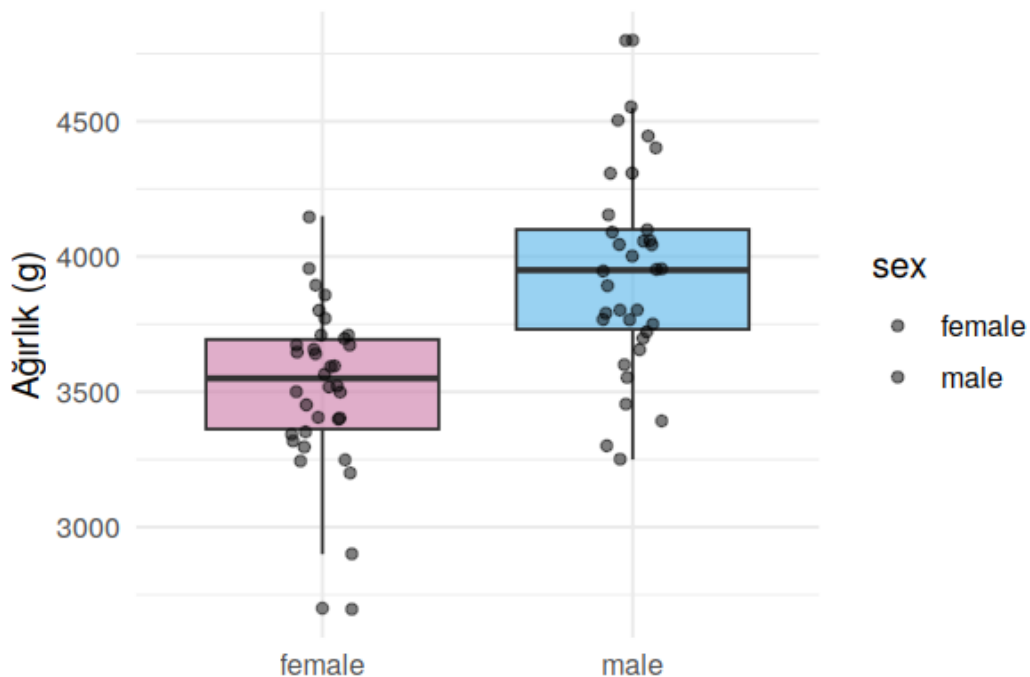
11.6.1 Bağımsız İki Örneklem t-Testi

Soru: Chinstrap penguenlerinde erkeklerin ve dişilerin ortalama ağırlıkları farklı mıdır?

```
chinstrap <- penguinler |> filter(species == "Chinstrap")
chinstrap |>
  group_by(sex) |>
  summarise(n = n(), ort = mean(body_mass_g), ss = sd(body_mass_g))
```

```
#> # A tibble: 2 x 4
#>   sex      n  ort  ss
#>   <fct> <int> <dbl> <dbl>
#> 1 female    34 3527. 285.
#> 2 male     34 3939. 362.
```

```
ggplot(chinstrap, aes(sex, body_mass_g, fill = sex)) +
  geom_boxplot(alpha = 0.6, show.legend = FALSE) +
  geom_jitter(width = 0.1, alpha = 0.5) +
  scale_fill_manual(values = c(female = "#CC79A7", male = "#56B4E9")) +
  labs(x = NULL, y = "Ağırlık (g)")
```



İki sürüm vardır:

| | Student t-testi | Welch t-testi |
|---------------------|--|--|
| Varsayım | Varyanslar eşit | Varyanslar eşit olmak zorunda değil |
| Serbestlik derecesi | $n_1 + n_2 - 2$ | Welch–Satterthwaite yaklaşımı |
| R | <code>t.test(..., var.equal = TRUE)</code> | <code>t.test(...)</code> (varsayılan) |

```
t.test(body_mass_g ~ sex, data = chinstrap) # Welch
```

```
#>
#> Welch Two Sample t-test
#>
#> data: body_mass_g by sex
#> t = -5.2, df = 63, p-value = 2e-06
#> alternative hypothesis: true difference in means between group female and group male is not equal
#> 95 percent confidence interval:
#> -569.8 -253.7
#> sample estimates:
#> mean in group female mean in group male
#> 3527 3939
```

```
t.test(body_mass_g ~ sex, data = chinstrap, var.equal = TRUE) # Student
```

```
#>
#> Two Sample t-test
#>
#> data: body_mass_g by sex
#> t = -5.2, df = 66, p-value = 2e-06
#> alternative hypothesis: true difference in means between group female and group male is not equal
#> 95 percent confidence interval:
#> -569.6 -253.9
#> sample estimates:
#> mean in group female mean in group male
#> 3527 3939
```

💡 İpucu

Welch testi varyanslar eşit olduğunda da Student testi kadar iyi çalışır, eşit olmadığında ise çok daha güvenilirdir. Bu yüzden R'ın varsayılanı Welch'tir ve önce F testi yapıp sonra test seçmek yerine **doğrudan Welch testi** kullanılması önerilir.

Eski notlarımızdaki worms örneğinde test istatistiğini elle hesaplamıştık:

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_1^2/n_1 + s_2^2/n_2}}$$

```
t_h <- (mean(nemsiz) - mean(nemli)) / sqrt(var(nemsiz) / n1 + var(nemli) / n2)
t_h
```

```
#> [1] -1.537
```

```
t.test(nemsiz, nemli)$statistic
```

```
#>      t
#> -1.537
```

11.6.2 Etki Büyüklüğü: Cohen'in d'si

p değeri farkın **var olup olmadığını**, etki büyüklüğü ise **ne kadar büyük** olduğunu söyler. İki ortalama arasındaki farkın ortak standart sapmaya oranına **Cohen'in d'si** denir:

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_p}, \quad s_p = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

```
cohen_d <- function(x, y) {
  n1 <- length(x); n2 <- length(y)
  sp <- sqrt(((n1 - 1) * var(x) + (n2 - 1) * var(y)) / (n1 + n2 - 2))
  (mean(x) - mean(y)) / sp
}
with(chinstrap, cohen_d(body_mass_g[sex == "male"], body_mass_g[sex == "female"]))
```

```
#> [1] 1.263
```

Kabaca: $|d| \approx 0.2$ küçük, 0.5 orta, 0.8 büyük etki. Chinstrap erkekleri dişilerden **çok büyük** bir etkiyle daha ağırdır. `rstatix::cohens_d()` aynı hesaplamayı yapar.

11.6.3 Mann–Whitney U (Wilcoxon Sıra Toplamı) Testi

İki bağımsız grubun dağılımlarının konumunu karşılaştırır; normallik varsaymaz. Mantığı: İki gruptaki tüm gözlemler birlikte sıralanır; H_0 doğruysa iki grubun sıra ortalamaları birbirine yakın olacaktır.

Eski notlarımızdaki worms örneğini adım adım yapalım:

```
yogunluk <- c(nemsiz, nemli)
etiket <- c(rep("Nemsiz", n1), rep("Nemli", n2))
birlesik_siralar <- rank(yogunluk)
birlesik_siralar
```

```
#> [1] 10.0 16.5  5.0 14.5  5.0  7.5 10.0 20.0 18.5  2.5  5.0  1.0 12.5  2.5 12.5
#> [16] 16.5 14.5 18.5 10.0  7.5
```

```
tapply(birlesik_siralar, etiket, sum) # Sıra toplamları
```

```
#> Nemli Nemsiz  
#> 79.5 130.5
```

```
tapply(birlesik_siralar, etiket, mean) # Sıra ortalamaları
```

```
#> Nemli Nemsiz  
#> 13.250 9.321
```

```
wilcox.test(nemsiz, nemli, exact = FALSE)
```

```
#>  
#> Wilcoxon rank sum test with continuity correction  
#>  
#> data: nemsiz and nemli  
#> W = 26, p-value = 0.2  
#> alternative hypothesis: true location shift is not equal to 0
```

W istatistiği, ilk grubun sıra toplamından $n_1(n_1 + 1)/2$ çıkarılarak elde edilir:

```
sum(birlesik_siralar[etiket == "Nemsiz"]) - n1 * (n1 + 1) / 2
```

```
#> [1] 25.5
```

Nemli tarlalarda solucan yoğunluğu nemsiz tarlalardan anlamlı biçimde yüksektir. Penguen örneğinde de iki testin olduğu sonuç aynıdır:

```
wilcox.test(body_mass_g ~ sex, data = chinstrap, exact = FALSE)
```

```
#>  
#> Wilcoxon rank sum test with continuity correction  
#>  
#> data: body_mass_g by sex  
#> W = 206, p-value = 5e-06  
#> alternative hypothesis: true location shift is not equal to 0
```

i Not

Mann–Whitney testi çoğu zaman “ortancaları karşılaştırır” diye anlatılır. Bu yalnızca iki grubun dağılımlarının **aynı şekilde** olduğu (yalnızca kaydı) durumunda doğrudur. Genel olarak test, bir gruptan rastgele seçilen bir gözlemin diğer gruptan seçilenden büyük olma olasılığının 0.5 olup olmadığını sınar.

11.7 Eşleştirilmiş (Bağımlı) Örneklemeler

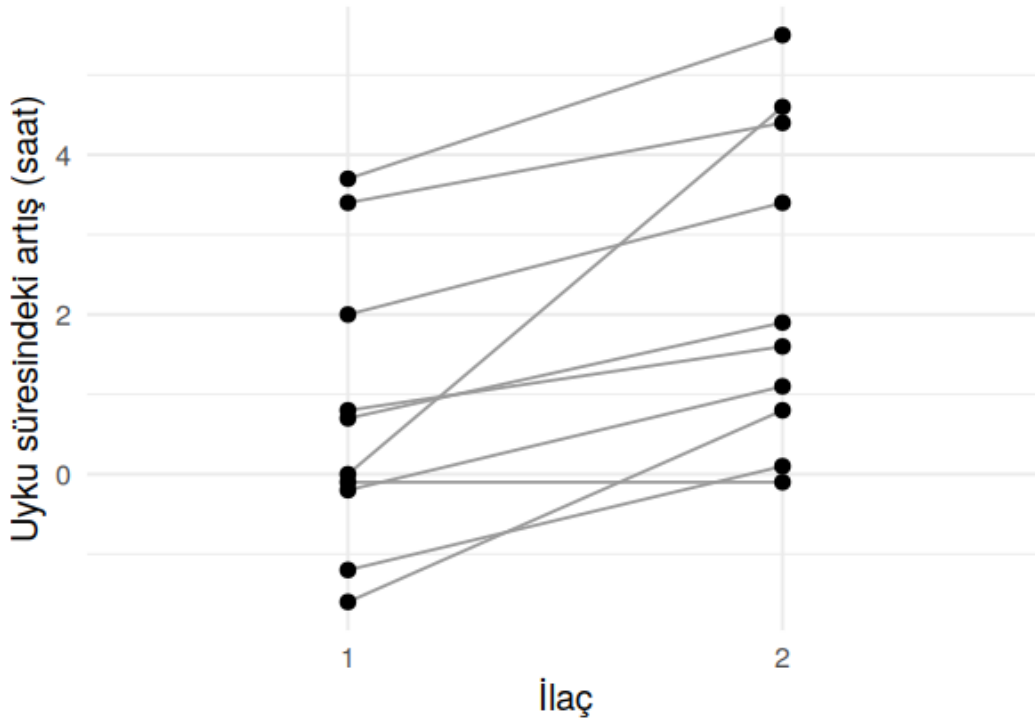
Aynı birimler üzerinde iki ölçüm yapıldığında (önce–sonra, iki ilaç, sağ–sol göz) gözlemler **bağımsız değildir**. Bu durumda her birimin iki ölçümü arasındaki **farklar** analiz edilir.

sleep verisi: 10 hastaya iki farklı uyku ilacı verilmiş ve uyku süresindeki artış (saat) ölçülmüştür.

```
uyku <- tidyr::pivot_wider(sleep, names_from = group, values_from = extra,  
                           names_prefix = "ilac")  
uyku
```

```
#> # A tibble: 10 x 3  
#>   ID   ilac1 ilac2  
#>   <fct> <dbl> <dbl>  
#> 1 1      0.7   1.9  
#> 2 2     -1.6   0.8  
#> 3 3     -0.2   1.1  
#> 4 4     -1.2   0.1  
#> 5 5     -0.1  -0.1  
#> 6 6      3.4   4.4  
#> # i 4 more rows
```

```
ggplot(sleep, aes(group, extra, group = ID)) +  
  geom_line(colour = "grey60") + geom_point(size = 2) +  
  labs(x = "İlaç", y = "Uyku süresindeki artış (saat)")
```



Her hasta (çizgi) ikinci ilaçla daha fazla uyumuş görünüyor.

```
t.test(uyku$ilac2, uyku$ilac1, paired = TRUE)
```

```
#>
#> Paired t-test
#>
#> data: uyku$ilac2 and uyku$ilac1
#> t = 4.1, df = 9, p-value = 0.003
#> alternative hypothesis: true mean difference is not equal to 0
#> 95 percent confidence interval:
#>  0.7001 2.4599
#> sample estimates:
#> mean difference
#>          1.58
```

```
t.test(uyku$ilac2 - uyku$ilac1, mu = 0) # Aynı şey: farkların tek örneklem
↪ testi
```

```
#>
#> One Sample t-test
#>
#> data: uyku$ilac2 - uyku$ilac1
#> t = 4.1, df = 9, p-value = 0.003
#> alternative hypothesis: true mean is not equal to 0
#> 95 percent confidence interval:
#>  0.7001 2.4599
#> sample estimates:
#> mean of x
#>          1.58
```

```
wilcox.test(uyku$ilac2, uyku$ilac1, paired = TRUE, exact = FALSE)
```

```
#>
#> Wilcoxon signed rank test with continuity correction
#>
#> data: uyku$ilac2 and uyku$ilac1
#> V = 45, p-value = 0.009
#> alternative hypothesis: true location shift is not equal to 0
```

Uyarı

Aynı veriyi yanlışlıkla **bağımsız** iki örneklem gibi analiz edersek, kişiler arasındaki büyük farklılık ilaç etkisini “gizler”:

```
t.test(extra ~ group, data = sleep)$p.value # YANLIŞ analiz: p = 0.08
```

```
#> [1] 0.07939
```

11.8 Oranlar İçin Testler

11.8.1 Tek Oran: Binom Testi

Eski notlarımızdaki örnek: 9 denemede 1 başarı. Başarı olasılığı 0.5'ten farklı mı?

```
binom.test(1, 9) # binom.test(başarı sayısı, deneme sayısı, p = 0.5)
```

```
#>
#> Exact binomial test
#>
#> data: 1 and 9
#> number of successes = 1, number of trials = 9, p-value = 0.04
#> alternative hypothesis: true probability of success is not equal to 0.5
#> 95 percent confidence interval:
#> 0.002809 0.482497
#> sample estimates:
#> probability of success
#> 0.1111
```

Penguen verisinde dişi oranı %50'den farklı mı?

```
binom.test(sum(penguenler$sex == "female"), nrow(penguenler), p = 0.5)
```

```
#>
#> Exact binomial test
#>
#> data: sum(penguenler$sex == "female") and nrow(penguenler)
#> number of successes = 165, number of trials = 333, p-value = 0.9
#> alternative hypothesis: true probability of success is not equal to 0.5
#> 95 percent confidence interval:
#> 0.4405 0.5505
#> sample estimates:
#> probability of success
#> 0.4955
```

11.8.2 İki Oranın Karşılaştırılması

Eski notlarımızdaki örnek: 40 kişiden 4'ü ile 3270 kişiden 196'sı arasındaki oran farkı:

```
prop.test(c(4, 196), c(40, 3270))
```

```
#>
#> 2-sample test for equality of proportions with continuity correction
#>
#> data: c(4, 196) out of c(40, 3270)
#> X-squared = 0.52, df = 1, p-value = 0.5
#> alternative hypothesis: two.sided
```

```
#> 95 percent confidence interval:
#> -0.06592 0.14604
#> sample estimates:
#> prop 1 prop 2
#> 0.10000 0.05994
```

Gözlenen oranlar %10 ve %6 olsa da, ilk grup çok küçük olduğu için bu fark anlamlı değildir.

11.9 Kategorik Değişkenler: Ki-kare Testleri

11.9.1 Uyum İyiliği Testi

Gözlenen frekansların belirli bir dağılıma (örneğin eşit oranlara) uyup uymadığını sınar:

$$\chi^2 = \sum \frac{(G - B)^2}{B}, \quad G : \text{gözlenen}, \quad B : \text{beklenen}$$

Soru: Üç penguen türü örnekleme eşit oranda mı?

```
gozlenen <- table(penguenler$species)
gozlenen
```

```
#>
#>   Adelie Chinstrap   Gentoo
#>    146         68    119
```

```
chisq.test(gozlenen) # H0: oranlar 1/3, 1/3, 1/3
```

```
#>
#> Chi-squared test for given probabilities
#>
#> data: gozlenen
#> X-squared = 28, df = 2, p-value = 7e-07
```

```
chisq.test(gozlenen, p = c(0.45, 0.20, 0.35)) # Belirli oranlar
```

```
#>
#> Chi-squared test for given probabilities
#>
#> data: gozlenen
#> X-squared = 0.18, df = 2, p-value = 0.9
```

11.9.2 Bağımsızlık Testi

İki kategorik değişkenin bağımsız olup olmadığını sınırlar. H_0 : “Değişkenler bağımsızdır.” Beklenen frekanslar $B_{ij} = \frac{\text{satır toplam}_i \times \text{sütun toplam}_j}{n}$ ile hesaplanır.

Eski notlarımızdaki 2×2 örnek:

```
qchisq(0.95, df = 1)      # Kritik değer
```

```
#> [1] 3.841
```

```
count <- matrix(c(38, 14, 11, 51), nrow = 2)
count
```

```
#>      [,1] [,2]
#> [1,]   38   11
#> [2,]   14   51
```

```
chisq.test(count)          # Yates süreklilik düzeltmesiyle (2x2 için
↪ varsayılan)
```

```
#>
#> Pearson's Chi-squared test with Yates' continuity correction
#>
#> data:  count
#> X-squared = 33, df = 1, p-value = 9e-09
```

```
chisq.test(count, correct = FALSE)  # Düzeltmesiz
```

```
#>
#> Pearson's Chi-squared test
#>
#> data:  count
#> X-squared = 35, df = 1, p-value = 3e-09
```

```
chisq.test(count)$expected      # Beklenen frekanslar
```

```
#>      [,1] [,2]
#> [1,] 22.35 26.65
#> [2,] 29.65 35.35
```

Soru: Penguenlerde cinsiyet ile tür bağımsız mıdır? Peki tür ile ada?

```
chisq.test(table(penguenler$species, penguenler$sex))
```

```
#>
#> Pearson's Chi-squared test
#>
#> data:  table(penguenler$species, penguenler$sex)
#> X-squared = 0.049, df = 2, p-value = 1
```

```
tur_ada <- table(penguenler$species, penguenler$island)
tur_ada
```

```
#>
#>           Biscoe Dream Torgersen
#> Adelie      44      55         47
#> Chinstrap    0      68          0
#> Gentoo     119      0          0
```

```
test <- suppressWarnings(chisq.test(tur_ada))
test
```

```
#>
#> Pearson's Chi-squared test
#>
#> data:  tur_ada
#> X-squared = 285, df = 4, p-value <2e-16
```

```
test$expected |> round(1)
```

```
#>
#>           Biscoe Dream Torgersen
#> Adelie      71.5  53.9      20.6
#> Chinstrap   33.3  25.1       9.6
#> Gentoo     58.2  44.0      16.8
```

Cinsiyet ile tür arasında bir ilişki yoktur (her türde dişi/erkek oranı benzerdir). Tür ile ada arasında ise çok güçlü bir ilişki vardır. Ancak bazı beklenen frekanslar 5'ten küçük olduğu için R bir uyarı verir.

! Ki-kare testinin varsayımı

Beklenen frekansların en az %80'i 5'ten büyük olmalı, hiçbiri 1'den küçük olmamalıdır. Aksi hâlde **Fisher kesin testi** kullanılmalıdır.

11.9.3 Fisher Kesin Testi

Küçük örneklemelerde çapraz tablolar için kesin p değeri verir. Eski notlarımızdaki örnek:

```
x <- matrix(c(6, 4, 2, 8), nrow = 2)
x
```

```
#>      [,1] [,2]
#> [1,]    6    2
#> [2,]    4    8
```

```
fisher.test(x)
```

```
#>
#> Fisher's Exact Test for Count Data
#>
#> data:  x
#> p-value = 0.2
#> alternative hypothesis: true odds ratio is not equal to 1
#> 95 percent confidence interval:
#>  0.6027 79.8309
#> sample estimates:
#> odds ratio
#>      5.43
```

```
fisher.test(tur_ada, simulate.p.value = TRUE, B = 10000) # Büyük tablolar için
↳ simülasyonla
```

```
#>
#> Fisher's Exact Test for Count Data with simulated p-value (based on
#> 10000 replicates)
#>
#> data:  tur_ada
#> p-value = 1e-04
#> alternative hypothesis: two.sided
```

11.9.4 İlişkinin Gücü: Cramér'in V'si

Ki-kare testi ilişkinin var olup olmadığını söyler; gücünü **Cramér'in V'si** ile ölçeriz (0: ilişki yok, 1: tam ilişki):

$$V = \sqrt{\frac{\chi^2}{n(\min(r, c) - 1)}}$$

```
cramer_v <- function(tablo) {
  chi2 <- suppressWarnings(chisq.test(tablo, correct = FALSE)$statistic)
  unname(sqrt(chi2 / (sum(tablo) * (min(dim(tablo)) - 1))))
}
cramer_v(tur_ada)
```

```
#> [1] 0.6537
```

```
cramer_v(table(penguenler$species, penguenler$sex))
```

```
#> [1] 0.01208
```

11.10 Çoklu Karşılaştırma Sorunu

Her testte $\alpha = 0.05$ olasılıkla I. tip hata yapıyorsak, **20 bağımsız test** yaptığımızda en az bir tanesinde yanlışlıkla “anlamlı” sonuç bulma olasılığı $1 - 0.95^{20} \approx 0.64$ ’tür! Bu yüzden üç ya da daha fazla grubu ikişer ikişer t-testleriyle karşılaştırmak yerine önce **ANOVA** (Bölüm 12) yapılır, ardından düzeltilmiş çoklu karşılaştırmalar kullanılır.

```
1 - 0.95^c(1, 3, 10, 20)
```

```
#> [1] 0.0500 0.1426 0.4013 0.6415
```

```
p.adjust(c(0.01, 0.02, 0.03, 0.04), method = "bonferroni")
```

```
#> [1] 0.04 0.08 0.12 0.16
```

```
p.adjust(c(0.01, 0.02, 0.03, 0.04), method = "holm")
```

```
#> [1] 0.04 0.06 0.06 0.06
```

11.11 Sonuçları Raporlamak

Bir test sonucu raporlanırken şunlar verilmelidir: **testin adı**, **betimsel istatistikler** (ortalama/SS ya da ortanca/ÇAG), **test istatistiği ve serbestlik derecesi**, **p değeri**, **güven aralığı** ve **etki büyüklüğü**, ve en önemlisi **bağlamdaki yorum**.

```
tt <- t.test(body_mass_g ~ sex, data = chinstrap)
d <- with(chinstrap, cohen_d(body_mass_g[sex == "male"], body_mass_g[sex ==
  ↪ "female"]))
```

Chinstrap penguenlerinde erkeklerin ortalama ağırlığı (3939 g) dişilerden (3527 g) anlamlı biçimde yüksektir (Welch $t_{62.6} = 5.21$, $p < 0.001$, fark için %95 GA: [254, 570] g, Cohen’s $d = 1.26$).

Yukarıdaki cümledeki bütün sayılar satır içi kodla yazılmıştır (Bölüm 3).

`broom::tidy()` test sonuçlarını bir data frame’e çevirir; birden fazla testi tablo hâlinde raporlamak için çok kullanışlıdır:

```
broom::tidy(tt)
```

```
#> # A tibble: 1 x 10
#>   estimate estimate1 estimate2 statistic    p.value parameter conf.low conf.high
#>   <dbl>     <dbl>     <dbl>     <dbl>    <dbl>     <dbl>     <dbl>     <dbl>
#> 1   -412.     3527.     3939.     -5.21 0.00000226     62.6     -570.     -254.
#> # i 2 more variables: method <chr>, alternative <chr>
```

Kendini Sına

💡 1. iris verisinde *versicolor* ve *virginica* türlerinin çanak yaprak genişlikleri (Sepal.Width) farklı mı? Hem t-testi hem Mann-Whitney testi ile sınavın.

```
iki_tur <- droplevels(subset(iris, Species != "setosa"))
t.test(Sepal.Width ~ Species, data = iki_tur)
wilcox.test(Sepal.Width ~ Species, data = iki_tur, exact = FALSE)
```

İki test de anlamlı fark gösterir ($p < 0.01$); *virginica*'nın çanak yaprakları biraz daha geniştir.

💡 2. Bir araştırmacı aynı 15 öğrencinin dönem başı ve dönem sonu puanlarını bağımsız t-testi ile karşılaştırmış. Hata nedir?

Ölçümler aynı öğrencilerden alındığı için **eşleştirilmiştir**. `t.test(son, bas, paired = TRUE)` ya da `wilcox.test(son, bas, paired = TRUE)` kullanılmalıdır. Bağımsız test öğrenciler arası değişkenliği hata olarak sayar ve gücü düşürür.

💡 3. $p = 0.0001$ ve Cohen's $d = 0.05$ çıkan bir sonucu nasıl yorumlarsınız?

Fark istatistiksel olarak anlamlıdır ama **pratik olarak önemsizdir**. Büyük olasılıkla örneklem çok büyüktür ve çok küçük bir fark bile anlamlı çıkmıştır.

💡 4. mtcars verisinde vites türü (am) ile silindir sayısı (cyl) bağımsız mıdır? Hangi test uygun?

```
tab <- table(mtcars$am, mtcars$cyl)
chisq.test(tab)$expected # Bazı beklenen frekanslar < 5
fisher.test(tab)
```

Beklenen frekansların bazıları 5'ten küçük olduğu için Fisher kesin testi kullanılmalıdır; iki değişken bağımsız değildir ($p \approx 0.009$).

💡 5. Adelie penguenlerinin gaga uzunluğu ortalaması 40 mm'den büyük müdür? Tek yönlü test yapın ve sonucu bir cümleyle raporlayın.

```
a <- penguenler$bill_length_mm[penguenler$species == "Adelie"]
t.test(a, mu = 40, alternative = "greater")
```

Ortalama 38.8 mm olduğu için $H_1 : \mu > 40$ desteklenmez ($p \approx 1$).

12 Varyans Analizi (ANOVA)

i Bu haftanın hedefleri

- İkiiden fazla grubun ortalamalarını karşılaştırırken neden ikili t-testleri yerine **ANOVA** kullanıldığını açıklamak,
- Toplam değişimin **gruplar arası** ve **gruplar içi** değişime ayrılmasını grafikte ve elle hesaplayarak göstermek,
- Tek yönlü ANOVA tablosunu okumak ve **etki büyüklüğünü** (η^2) hesaplamak,
- Varsayımları (normallik, varyans homojenliği, bağımsızlık) denetlemek; varsayımlar sağlanmadığında **Welch ANOVA** ya da **Kruskal–Wallis** testini kullanmak,
- **Çoklu karşılaştırma** (post-hoc) testlerini (Tukey HSD, Bonferroni/Holm, Dunn) uygulamak,
- İki yönlü ANOVA ile iki faktörün ve **etkileşimlerinin** etkisini incelemek,
- ANOVA sonuçlarını grafik ve tabloyla raporlamak.

12.1 Neden ANOVA?

Üç penguen türünün ortalama ağırlıklarını karşılaştırmak istiyoruz. Üç ayrı t-testi yapabildik (Adelie–Chinstrap, Adelie–Gentoo, Chinstrap–Gentoo) ama Bölüm 11’te gördüğümüz gibi her ek test I. tip hata olasılığını artırır. **Varyans analizi** (*Analysis of Variance*, ANOVA) tüm grupları **tek bir testle** karşılaştırır:

$$H_0 : \mu_1 = \mu_2 = \dots = \mu_k \quad H_1 : \text{En az bir grup ortalaması diğerlerinden farklıdır}$$

Adı “varyans analizi” olsa da ANOVA **ortalamları** karşılaştırır; bunu varyansları (değişimi) parçalara ayırarak yapar.

12.2 Değişimin Parçalanması

Eski ders notlarımızdaki fikri worms verisiyle görelim. Solucan yoğunluğunun **genel ortalamadan** sapmaları (yeşil çizgiler) toplam değişimi gösterir. Tarlaları neme göre iki gruba ayırıp sapmaları **kendi grup ortalamalarından** ölçersek (sağdaki grafik) çizgiler kısalmır. Çizgiler ne kadar kısalmırsa, gruplama değişkeni değişimin o kadar büyük kısmını **açıklıyor** demektir.

```
y <- worms$Worm.density
grup <- worms$Damp
sira <- seq_along(y)
par(mfrow = c(1, 2))

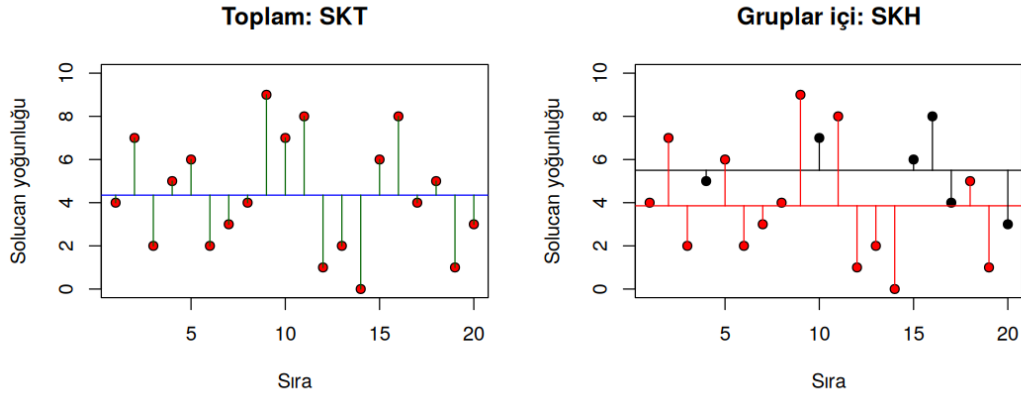
plot(sira, y, ylim = c(0, 10), pch = 21, bg = "red", xlab = "Sıra", ylab = "Solucan
↪ yoğunluğu",
```

```

    main = "Toplam: SKT")
abline(h = mean(y), col = "blue")
for (i in sira) lines(c(i, i), c(mean(y), y[i]), col = "darkgreen")

plot(sira, y, ylim = c(0, 10), pch = 21, bg = ifelse(grup, "black", "red"),
     xlab = "Sıra", ylab = "Solucan yoğunluğu", main = "Gruplar içi: SKH")
abline(h = mean(y[grup]), col = "black")
abline(h = mean(y[!grup]), col = "red")
for (i in sira) lines(c(i, i), c(mean(y[grup == grup[i]]), y[i]), col =
  ↪ ifelse(grup[i], "black", "red"))
par(mfrow = c(1, 1))

```



Şekil 12.1: Solda toplam değişim (genel ortalamadan sapmalar), sağda gruplar içi değişim (grup ortalamalarından sapmalar)

Matematiksel olarak:

$$\underbrace{\sum_{i,j} (y_{ij} - \bar{y})^2}_{\text{SKT: toplam}} = \underbrace{\sum_j n_j (\bar{y}_j - \bar{y})^2}_{\text{SKG: gruplar arası}} + \underbrace{\sum_{i,j} (y_{ij} - \bar{y}_j)^2}_{\text{SKH: gruplar içi (hata)}}$$

Tablo 12.1: Tek yönlü ANOVA tablosu

| Kaynak | Kareler toplamı | sd | Kareler ortalaması | F |
|--------------------|-----------------|---------|-------------------------------------|---------------------------|
| Gruplar arası | SKG | $k - 1$ | $\text{KOG} = \text{SKG} / (k - 1)$ | KOG / KOH |
| Gruplar içi (hata) | SKH | $n - k$ | $\text{KOH} = \text{SKH} / (n - k)$ | |
| Toplam | SKT | $n - 1$ | | |

H_0 doğruysa KOG ile KOH aynı varyansı tahmin eder ve $F \approx 1$ olur. Gruplar arası farklar büyükçe F büyür. F istatistiği H_0 altında $F_{k-1, n-k}$ dağılımına uyar (Bölüm 9).

12.2.1 Elle Hesaplama: Penguenler

```

y <- penguinler$body_mass_g
g <- penguinler$species
k <- nlevels(g); n <- length(y)
grup_ort <- tapply(y, g, mean)
grup_n <- tapply(y, g, length)

SKT <- sum((y - mean(y))^2)
SKG <- sum(grup_n * (grup_ort - mean(y))^2)
SKH <- sum((y - grup_ort[g])^2)
c(SKT = SKT, SKG = SKG, SKH = SKH, SKG_arti_SKH = SKG + SKH)

```

```

#>          SKT          SKG          SKH SKG_arti_SKH
#> 215259666 145190219 70069447 215259666

```

```

F_ist <- (SKG / (k - 1)) / (SKH / (n - k))
p_deger <- 1 - pf(F_ist, k - 1, n - k)
c(F = F_ist, p = p_deger)

```

```

#>      F      p
#> 341.9 0.0

```

12.2.2 aov() ile Tek Yönlü ANOVA

```

model <- aov(body_mass_g ~ species, data = penguinler)
summary(model)

```

```

#>          Df    Sum Sq Mean Sq F value Pr(>F)
#> species     2 1.45e+08 72595110    342 <2e-16 ***
#> Residuals  330 7.01e+07  212332
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Elle bulduğumuz değerlerle aynı. $p < 0.001$ olduğu için en az bir türün ortalama ağırlığı diğerlerinden farklıdır. Hangi türlerin farklı olduğunu **çoklu karşılaştırma** testleri söyleyecektir.

i Not

İki grup olduğunda tek yönlü ANOVA, varyansların eşit olduğu (Student) t-testi ile **aynıdır**: $F = t^2$.

```

summary(aov(Worm.density ~ Damp, data = worms))

```

```

#>          Df Sum Sq Mean Sq F value Pr(>F)
#> Damp       1  11.3    11.34    1.71  0.21
#> Residuals 18 119.2     6.62

```

```
t.test(Worm.density ~ Damp, data = worms, var.equal = TRUE)$statistic^2
```

```
#>      t  
#> 1.712
```

12.2.3 Etki Büyüklüğü: Eta Kare

Toplam değişimin ne kadarının gruplar tarafından açıklandığını gösterir:

$$\eta^2 = \frac{SKG}{SKT}$$

```
SKG / SKT
```

```
#> [1] 0.6745
```

Penguen ağırlığındaki değişimin yaklaşık **%67'si** türler arasındaki farklardan kaynaklanmaktadır. Kabaca $\eta^2 \approx 0.01$ küçük, 0.06 orta, 0.14 ve üzeri büyük etki olarak yorumlanır. `effectsize::eta_squared(model)` aynı değeri güven aralığıyla verir.

12.3 Varsayımlar

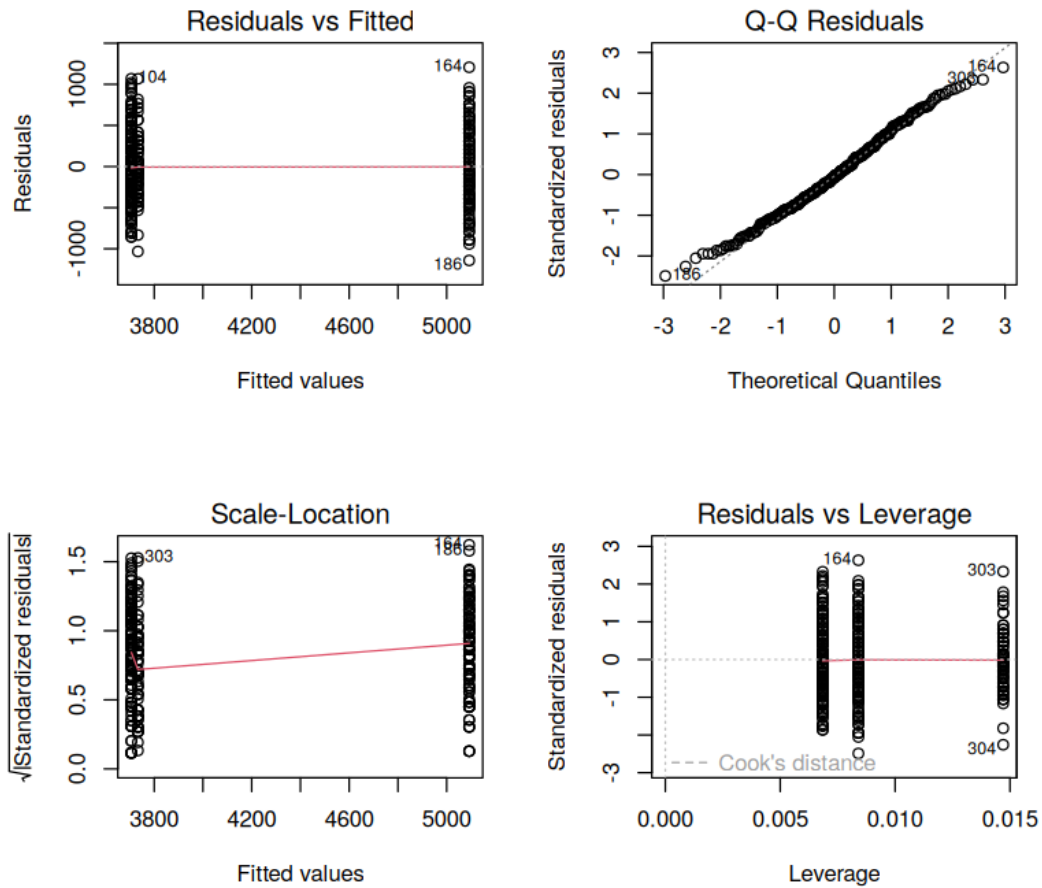
Tablo 12.2: ANOVA varsayımları

| Varsayım | Nasıl denetlenir? | Sağlanmazsa |
|---|--|---|
| Bağımsızlık:
gözlemler birbirinden
bağımsız | Veri toplama tasarımı | Tekrarlı ölçümler ANOVA /
karma modeller |
| Normallik: her
grupta (ya da
artıklarda) normal
dağılım | Artıkların Q-Q grafiği, Shapiro–Wilk | Büyük n 'de genellikle sorun
değil; küçük n 'de
Kruskal–Wallis |
| Varyans
homojenliği: grup
varyansları eşit | Kutu grafikleri, Levene / Bartlett testi | Welch ANOVA
(<code>oneway.test()</code>) |

12.3.1 Grafikle Denetleme

`plot()` bir `aov` ya da `lm` nesnesine uygulandığında dört tane grafiği çizer:

```
par(mfrow = c(2, 2))  
plot(model)
```



```
par(mfrow = c(1, 1))
```

| Grafik | Ne aranır? |
|-----------------------|---|
| Residuals vs Fitted | Her grubun artıkları sıfır çevresinde benzer genişlikte mi? (Varyans homojenliği) |
| Normal Q-Q | Artıklar doğru üzerinde mi? (Normallik) |
| Scale-Location | Kırmızı çizgi yatay mı? (Varyans homojenliği) |
| Residuals vs Leverage | Sonucu tek başına etkileyen gözlem var mı? |

12.3.2 Testle Denetleme

```
shapiro.test(residuals(model)) # Artıkların normalliği
```

```
#>
#> Shapiro-Wilk normality test
#>
#> data: residuals(model)
#> W = 0.99, p-value = 0.08
```

```
bartlett.test(body_mass_g ~ species, data = penguenler) # Normalliğe duyarlı
```

```
#>
#> Bartlett test of homogeneity of variances
#>
#> data: body_mass_g by species
#> Bartlett's K-squared = 5.7, df = 2, p-value = 0.06
```

```
car::leveneTest(body_mass_g ~ species, data = penguenler) # Normalliğe daha
↳ dayanıklı
```

```
#> Levene's Test for Homogeneity of Variance (center = median)
#>      Df F value Pr(>F)
#> group 2    5.13 0.0064 **
#>      330
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Artıkların normalliği reddedilmiyor ($p = 0.08$); Q-Q grafiği de yalnızca uçlarda küçük sapmalar gösteriyor ve her grupta 68'den fazla gözlem olduğu için ANOVA bu konuda dayanıklıdır. Ancak Levene testi varyansların **homojen olmadığını** gösteriyor ($p = 0.006$): Gentoo penguenlerinin ağırlıkları diğer türlere göre daha değişkendir. Bu durumda sonucu **Welch ANOVA** ile doğrulamak gerekir.

12.3.3 Welch ANOVA

Varyanslar homojen değilse, Welch t-testinin ikiden fazla grup için genellemesi olan **Welch ANOVA** kullanılır:

```
oneway.test(body_mass_g ~ species, data = penguenler) # Welch
↳ (varsayılan)
```

```
#>
#> One-way analysis of means (not assuming equal variances)
#>
#> data: body_mass_g and species
#> F = 317, num df = 2, denom df = 188, p-value <2e-16
```

```
oneway.test(body_mass_g ~ species, data = penguenler, var.equal = TRUE) # Klasik
↳ ANOVA
```

```
#>
#> One-way analysis of means
#>
#> data: body_mass_g and species
#> F = 342, num df = 2, denom df = 330, p-value <2e-16
```

Welch ANOVA da aynı sonuca varıyor: türlerin ortalama ağırlıkları farklıdır. Varyanslar eşit olmadığında ikili karşılaştırmalar için de varyans eşitliği varsaymayan yöntemler (Games-Howell ya da `pool.sd = FALSE` ile Holm düzeltmeli t-testleri) tercih edilir.

12.4 Çoklu Karşılaştırmalar (Post-hoc Testler)

ANOVA anlamlı çıktığında **hangi** grupların birbirinden farklı olduğunu bulmak için, I. tip hatayı kontrol eden çoklu karşılaştırma testleri kullanılır.

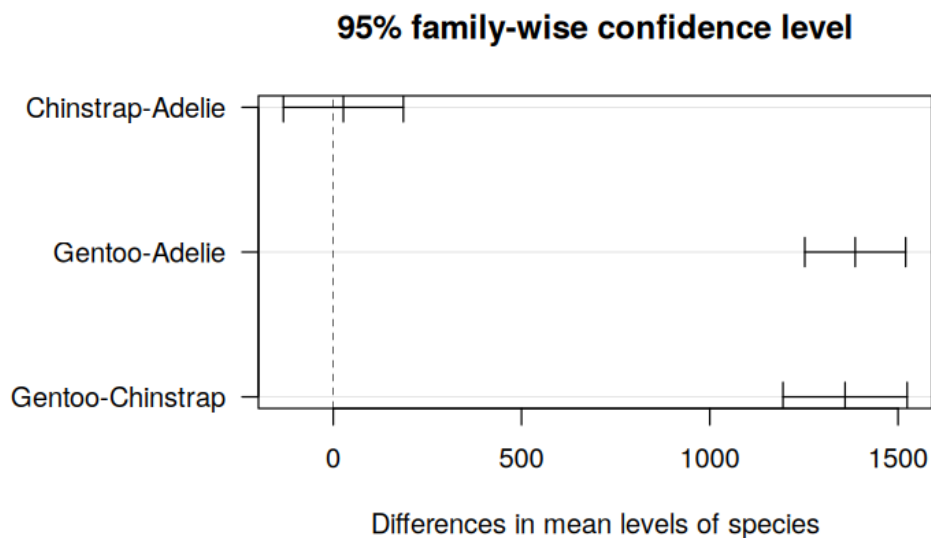
12.4.1 Tukey HSD

Tüm ikili karşılaştırmalar için, aile bazında hata oranını α 'da tutan güven aralıkları ve düzeltilmiş p değerleri verir:

```
tukey <- TukeyHSD(model)
tukey
```

```
#> Tukey multiple comparisons of means
#> 95% family-wise confidence level
#>
#> Fit: aov(formula = body_mass_g ~ species, data = penguinler)
#>
#> $species
#>
#>      diff      lwr      upr    p adj
#> Chinstrap-Adelie  26.92 -132.4  186.2 0.9164
#> Gentoo-Adelie    1386.27 1252.3 1520.3 0.0000
#> Gentoo-Chinstrap 1359.35 1194.4 1524.3 0.0000
```

```
par(mar = c(5, 11, 3, 1))
plot(tukey, las = 1)
```



```
par(mar = c(5, 4, 4, 2) + 0.1)
```

Sıfırı içermeyen güven aralıkları anlamlı farkları gösterir: Gentoo, hem Adelie'den hem Chinstrap'ten yaklaşık 1350–1400 g daha ağırdır; Adelie ile Chinstrap arasında anlamlı bir fark yoktur.

12.4.2 Bonferroni ve Holm Düzeltmeli t-Testleri

```
pairwise.t.test(penguenler$body_mass_g, penguenler$species, p.adjust.method =  
  ↪ "bonferroni")
```

```
#>  
#> Pairwise comparisons using t tests with pooled SD  
#>  
#> data: penguenler$body_mass_g and penguenler$species  
#>  
#>          Adelie Chinstrap  
#> Chinstrap 1      -  
#> Gentoo    <2e-16 <2e-16  
#>  
#> P value adjustment method: bonferroni
```

```
pairwise.t.test(penguenler$body_mass_g, penguenler$species, p.adjust.method =  
  ↪ "holm",  
                pool.sd = FALSE) # Varyanslar eşit varsayılmadan (Welch)
```

```
#>  
#> Pairwise comparisons using t tests with non-pooled SD  
#>  
#> data: penguenler$body_mass_g and penguenler$species  
#>  
#>          Adelie Chinstrap  
#> Chinstrap 0.7      -  
#> Gentoo    <2e-16 <2e-16  
#>  
#> P value adjustment method: holm
```

```
rstatix::games_howell_test(penguenler, body_mass_g ~ species)
```

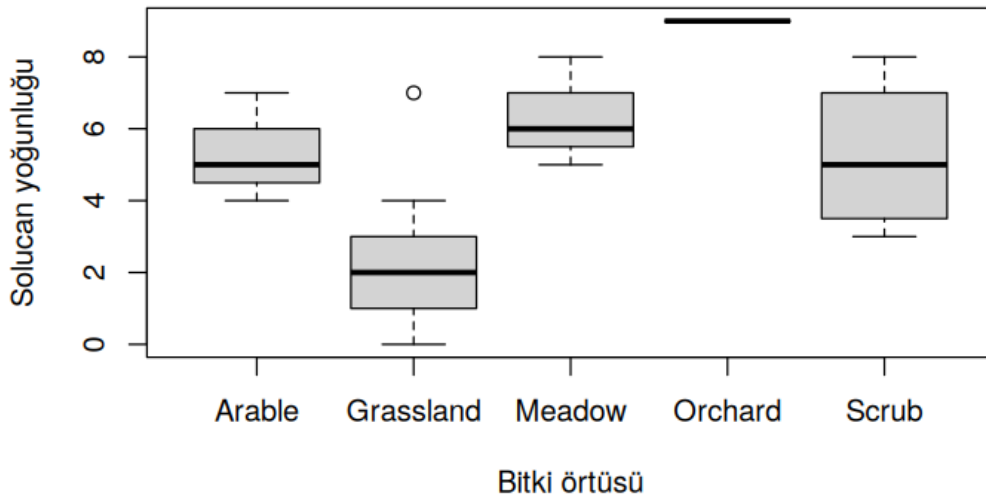
```
#> # A tibble: 3 x 8  
#>   .y.      group1      group2 estimate conf.low conf.high p.adj p.adj.signif  
#> * <chr>      <chr>      <chr>      <dbl>   <dbl>   <dbl> <dbl> <chr>  
#> 1 body_mass_g Adelie    Chinstrap    26.9   -115.    169. 0.895 ns  
#> 2 body_mass_g Adelie    Gentoo     1386.   1246.   1527. 0      ****  
#> 3 body_mass_g Chinstrap Gentoo     1359.   1205.   1514. 0      ****
```

| Yöntem | Ne zaman? |
|---------------------|--|
| Tukey HSD | Tüm ikili karşılaştırmalar, varyanslar eşit (en yaygın) |
| Bonferroni | Az sayıda planlı karşılaştırma; en tutucu yöntem |
| Holm | Bonferroni'den her zaman daha güçlü, aynı güvence |
| Games–Howell | Varyanslar eşit değilken tüm ikili karşılaştırmalar
(<code>rstatix::games_howell_test()</code>) |
| Dunnett | Her grubu yalnızca bir kontrol grubuyla karşılaştırma |

12.5 ANOVA Sonuçlarını Görselleştirmek

Eski notlarımızda iki grafik önerilmişti: **kutu grafiği** ve **hata çubuklu çubuk grafiği**. Worms verisinde bitki örtüsüne göre solucan yoğunluğu:

```
plot(factor(worms$Vegetation), worms$Worm.density, xlab = "Bitki örtüsü",
      ylab = "Solucan yoğunluğu", col = "lightgrey")
```

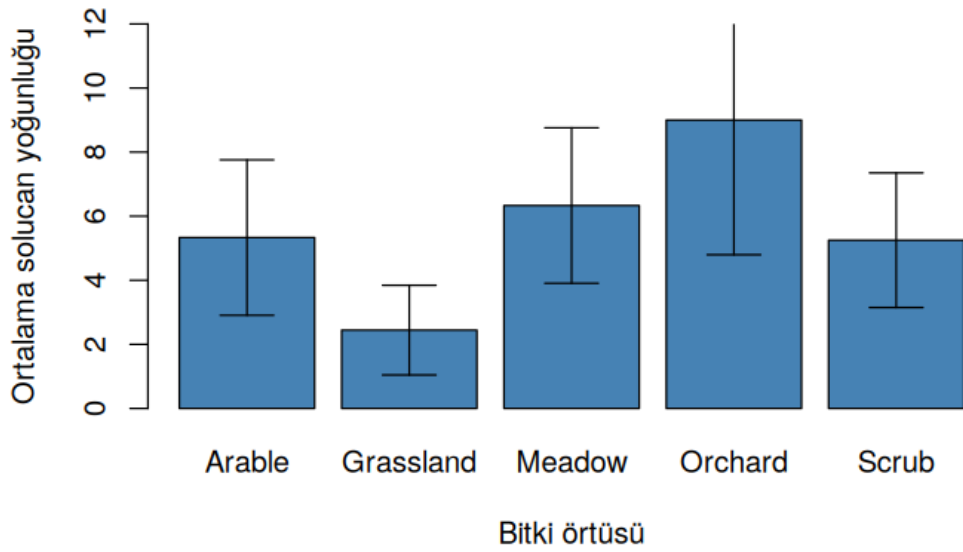


Eski notlarımızdaki hata çubuğu fonksiyonu, `arrows()` ile çubukların üzerine dikey çizgiler çizer. Güven aralıklarını ise ANOVA'nın hata kareler ortalamasından (KOH) hesaplıyorduk. Bu hesabı her gruba uygun n ve serbestlik derecesiyle **kodla** yapalım:

```
hata_cubuk <- function(y, z) {
  x <- barplot(y, plot = FALSE)
  for (i in seq_along(y)) {
    arrows(x[i], y[i] - z[i], x[i], y[i] + z[i], code = 3, angle = 90, length = 0.15)
  }
}
worms_model <- aov(Worm.density ~ Vegetation, data = worms)
KOH <- sum(residuals(worms_model)^2) / df.residual(worms_model)
yogunluk <- tapply(worms$Worm.density, worms$Vegetation, mean)
n_grup <- tapply(worms$Worm.density, worms$Vegetation, length)
ga <- qt(0.975, df.residual(worms_model)) * sqrt(KOH / n_grup)
KOH
```

```
#> [1] 3.887
```

```
barplot(yogunluk, col = "steelblue", ylim = c(0, 12), xlab = "Bitki örtüsü",
        ylab = "Ortalama solucan yoğunluğu")
hata_cubuk(yogunluk, ga)
```

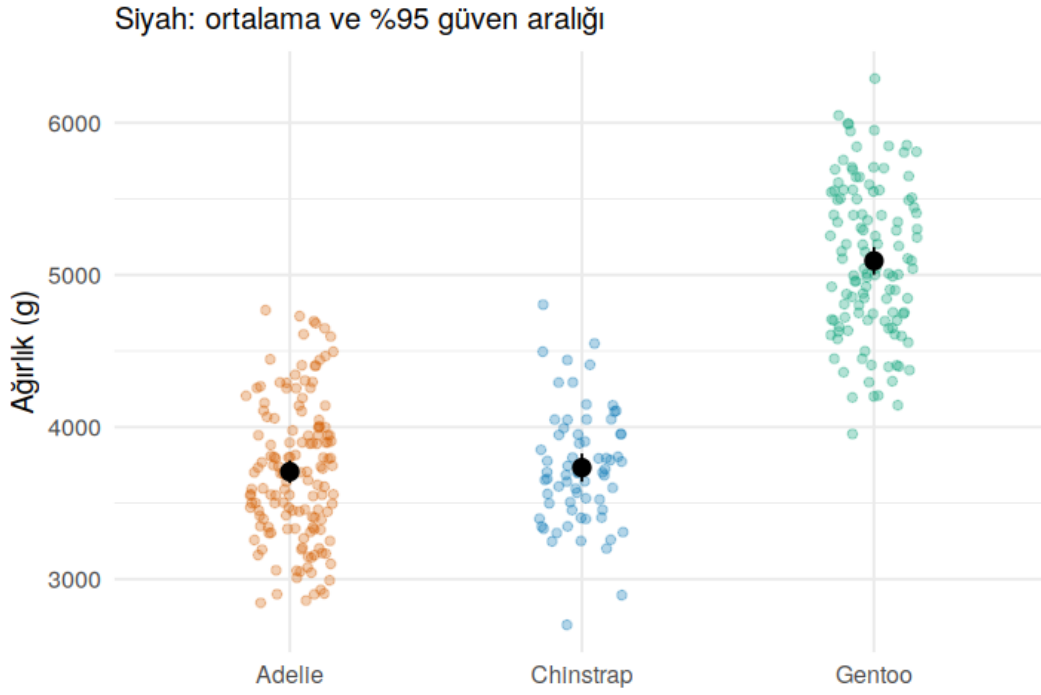


i Not

Eski notlarda standart hatalar `sqrt(3.887/3)` gibi tablodan okunan sayılarla elle yazılıyordu. Kodla hesaplamak hem hatayı önler hem de veri değiştiğinde grafiğin kendiliğinden güncellenmesini sağlar. Arable ve Meadow gruplarında 3'er, Scrub grubunda 4, Orchard grubunda ise yalnızca 1 tarla olduğu için bu grupların güven aralıkları geniştir. Tek gözlemlili bir grubun ortalaması hakkında güvenilir bir şey söylemek mümkün değildir.

`ggplot2` ile aynı fikir: gözlemler, grup ortalamaları ve %95 güven aralıkları tek grafikte:

```
ggplot(penguenler, aes(species, body_mass_g, colour = species)) +
  geom_jitter(width = 0.15, alpha = 0.3) +
  stat_summary(fun.data = mean_cl_normal, geom = "pointrange", colour = "black",
    ↪ size = 0.6) +
  scale_colour_manual(values = tur_renkleri, guide = "none") +
  labs(x = NULL, y = "Ağırlık (g)", subtitle = "Siyah: ortalama ve %95 güven aralığı")
```



12.6 Kruskal–Wallis Testi

Tek yönlü ANOVA'nın **parametrik olmayan** karşılığıdır. Normallik varsaymaz; gözlemlerin tümünü birlikte sıralayıp grupların sıra ortalamalarını karşılaştırır. Küçük örneklerde, aykırı değerler varken ya da sıralı (ordinal) verilerde tercih edilir.

$$H = \frac{12}{n(n+1)} \sum_{j=1}^k \frac{R_j^2}{n_j} - 3(n+1) \sim \chi_{k-1}^2$$

Burada R_j , j . grubun sıra toplamıdır. Worms verisinde gruplar çok küçük ve veri kesikli olduğu için Kruskal–Wallis uygun bir seçimdir:

```
siralar <- rank(worms$Worm.density)
R <- tapply(siralar, worms$Vegetation, sum)
nj <- tapply(siralar, worms$Vegetation, length)
N <- nrow(worms)
H <- 12 / (N * (N + 1)) * sum(R^2 / nj) - 3 * (N + 1)
c(H = H, p = 1 - pchisq(H, df = length(nj) - 1))
```

```
#>      H      p
#> 10.45020 0.03349
```

```
kruskal.test(Worm.density ~ Vegetation, data = worms)
```

```
#>
#> Kruskal-Wallis rank sum test
#>
#> data: Worm.density by Vegetation
#> Kruskal-Wallis chi-squared = 11, df = 4, p-value = 0.03
```

(R'ın sonucu, eşit değerler için düzeltme yaptığından küçük bir fark gösterir.)

Penguenlerde yüzgeç uzunluğu:

```
kruskal.test(flipper_length_mm ~ species, data = penguinler)
```

```
#>
#> Kruskal-Wallis rank sum test
#>
#> data: flipper_length_mm by species
#> Kruskal-Wallis chi-squared = 237, df = 2, p-value <2e-16
```

12.6.1 Kruskal–Wallis Sonrası Çoklu Karşılaştırma

Kruskal–Wallis anlamlı çıktığında, düzeltilmiş ikili Mann–Whitney testleri ya da **Dunn testi** kullanılır:

```
pairwise.wilcox.test(penguinler$flipper_length_mm, penguinler$species,
                      p.adjust.method = "holm", exact = FALSE)
```

```
#>
#> Pairwise comparisons using Wilcoxon rank sum test with continuity correction
#>
#> data: penguinler$flipper_length_mm and penguinler$species
#>
#>          Adelie Chinstrap
#> Chinstrap 7e-08  -
#> Gentoo    <2e-16 <2e-16
#>
#> P value adjustment method: holm
```

```
rstatix::dunn_test(penguinler, flipper_length_mm ~ species, p.adjust.method = "holm")
```

```
#> # A tibble: 3 x 9
#>   .y.      group1 group2    n1    n2 statistic      p    p.adj p.adj.signif
#> * <chr>      <chr> <chr> <int> <int>      <dbl>    <dbl>    <dbl> <chr>
#> 1 flipper_le~ Adelie Chins~   146    68      3.55 3.85e- 4 3.85e- 4 ***
#> 2 flipper_le~ Adelie Gentoo   146   119     15.2 2.54e-52 7.61e-52 ****
#> 3 flipper_le~ Chins~ Gentoo     68   119      8.94 3.98e-19 7.96e-19 ****
```

12.7 İki Yönlü ANOVA

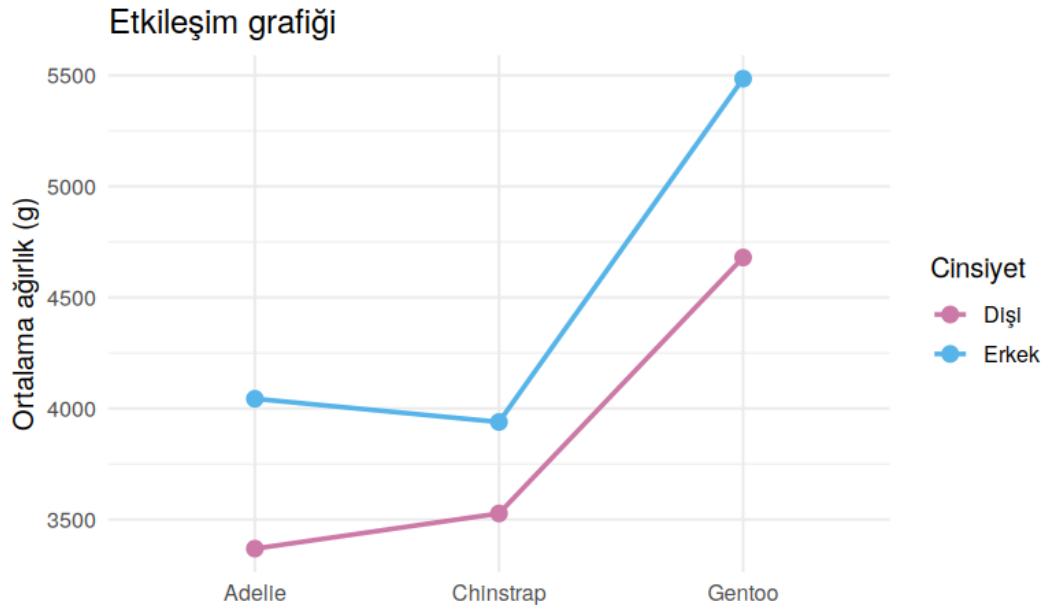
Bağımlı değişkeni **iki faktörün** birlikte nasıl etkilediğini inceler. Üç hipotez sınanır:

1. Birinci faktörün ana etkisi (tür)
2. İkinci faktörün ana etkisi (cinsiyet)
3. **Etkileşim**: Bir faktörün etkisi, diğer faktörün düzeyine göre değişiyor mu?

```

penguenler |>
  group_by(species, sex) |>
  summarise(ort = mean(body_mass_g), .groups = "drop") |>
  ggplot(aes(species, ort, colour = sex, group = sex)) +
  geom_point(size = 3) + geom_line(linewidth = 1) +
  scale_colour_manual(values = c(female = "#CC79A7", male = "#56B4E9"),
    labels = c("Dişi", "Erkek")) +
  labs(x = NULL, y = "Ortalama ağırlık (g)", colour = "Cinsiyet",
    title = "Etkileşim grafiği")

```



Çizgiler **paralel** olsaydı etkileşim olmazdı: cinsiyetin etkisi her türde aynı olurdu. Burada Gentoo'larda erkek-dişi farkı Chinstrap'lere göre daha büyük görünüyor.

```

model2 <- aov(body_mass_g ~ species * sex, data = penguenler)
summary(model2)

```

```

#>           Df    Sum Sq Mean Sq F value Pr(>F)
#> species      2 1.45e+08 72595110   758.36 <2e-16 ***
#> sex          1 3.71e+07 37090262   387.46 <2e-16 ***
#> species:sex   2 1.68e+06  838278    8.76 2e-04 ***
#> Residuals   327 3.13e+07   95727
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

`species * sex` yazımı, `species + sex + species:sex` (iki ana etki ve etkileşim) ile aynıdır. Etkileşim anlamlıdır ($p < 0.001$): Cinsiyetin ağırlık üzerindeki etkisi türe göre değişmektedir. Etkileşim anlamlı olduğunda ana etkileri tek başına yorumlamak yanıltıcı olabilir; cinsiyet etkisini **her tür için ayrı ayrı** inceleriz:

```
penguenler |>
  group_by(species) |>
  summarise(fark = mean(body_mass_g[sex == "male"]) - mean(body_mass_g[sex ==
    ↪ "female"]),
            p = t.test(body_mass_g ~ sex)$p.value)
```

```
#> # A tibble: 3 x 3
#>   species   fark      p
#>   <fct>     <dbl>   <dbl>
#> 1 Adelie   675. 6.40e-26
#> 2 Chinstrap 412. 2.26e- 6
#> 3 Gentoo   805. 1.87e-28
```

i Dengesiz tasarımlar

Gruplardaki gözlem sayıları eşit değilse (dengesiz tasarım), `summary(aov())` **sıralı** (I. tip) kareler toplamalarını verir ve sonuç faktörlerin yazılış sırasına bağlı olabilir. Bu durumda `car::Anova(model, type = 2)` (ya da etkileşim varsa `type = 3`) kullanılması önerilir.

```
car::Anova(lm(body_mass_g ~ species * sex, data = penguenler), type = 2)
```

```
#> Anova Table (Type II tests)
#>
#> Response: body_mass_g
#>           Sum Sq Df F value Pr(>F)
#> species    1.43e+08  2  749.02 <2e-16 ***
#> sex         3.71e+07  1  387.46 <2e-16 ***
#> species:sex 1.68e+06  2    8.76 2e-04 ***
#> Residuals   3.13e+07 327
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

12.7.1 Eski Notlardan: Toprak Asitliği

Eski notlarımızda pH değeri 4.6'nın altındaki tarlalar “asidik” olarak gruplanıp aynı analiz yapılmıştı. Asitlik ve nemi birlikte ele alalım:

```
worms$asidik <- worms$Soil.pH < 4.6
summary(aov(Worm.density ~ asidik, data = worms))
```

```
#>           Df Sum Sq Mean Sq F value Pr(>F)
#> asidik      1   61.2    61.2    15.9 0.00086 ***
#> Residuals   18   69.3     3.9
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(Worm.density ~ asidik * Damp, data = worms))
```

```
#>               Df Sum Sq Mean Sq F value    Pr(>F)
#> asidik         1   61.2    61.2    16.29 0.00086 ***
#> Damp           1    5.4     5.4     1.44 0.24713
#> Residuals     17   63.9     3.8
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

12.8 Sonuçları Raporlamak

```
s <- summary(model)[[1]]
eta2 <- s$`Sum Sq`[1] / sum(s$`Sum Sq`)
```

Penguenlerin ortalama ağırlığı türlere göre anlamlı farklılık göstermektedir (tek yönlü ANOVA, $F(2, 330) = 341.9$, $p < 0.001$, $\eta^2 = 0.67$). Tukey HSD testine göre Gentoo penguenleri hem Adelie (fark = 1386 g) hem Chinstrap penguenlerinden (fark = 1359 g) daha ağırdır ($p < 0.001$); Adelie ile Chinstrap arasında anlamlı bir fark yoktur ($p = 0.92$).

Kendini Sına

💡 1. iris verisinde taç yaprak uzunluğu türlere göre farklı mı? ANOVA, varsayımlar ve Tukey testini uygulayın.

```
m <- aov(Petal.Length ~ Species, data = iris)
summary(m)
car::leveneTest(Petal.Length ~ Species, data = iris) # Varyanslar homojen
↪ değil!
oneway.test(Petal.Length ~ Species, data = iris)    # Welch ANOVA
TukeyHSD(m)
```

Levene testi varyansların eşit olmadığını gösterdiği için Welch ANOVA raporlanmalıdır; tüm türler birbirinden farklıdır.

💡 2. Aynı soruyu Kruskal–Wallis testi ile cevaplayın. Sonuç değişiyor mu?

```
kruskal.test(Petal.Length ~ Species, data = iris)
pairwise.wilcox.test(iris$Petal.Length, iris$Species, p.adjust.method = "holm",
↪ exact = FALSE)
```

Değişmez: fark o kadar büyüktür ki hem parametrik hem parametrik olmayan testler aynı sonuca varır.

💡 3. Neden üç gruba ikili t-testleriyle karşılaştırmak yerine ANOVA yapıyoruz?

Üç ikili test yapıldığında, H_0 doğru olsa bile en az birinde yanlışlıkla anlamlı sonuç bulma olasılığı yaklaşık $1 - 0.95^3 = 0.14$ 'e çıkar. ANOVA tüm grupları tek testle, $\alpha = 0.05$ düzeyinde karşılaştırır; ikili karşılaştırmalar daha sonra düzeltilmiş post-hoc testlerle yapılır.

💡 4. ToothGrowth verisinde (len: diş uzunluğu, supp: takviye türü, dose: doz) iki yönlü ANOVA yapın. Etkileşim var mı?

```
tg <- ToothGrowth; tg$dose <- factor(tg$dose)
summary(aov(len ~ supp * dose, data = tg))
interaction.plot(tg$dose, tg$supp, tg$len)
```

Etkileşim anlamlıdır ($p \approx 0.02$): Portakal suyunun (OJ) üstünlüğü düşük dozlarda belirgin, en yüksek dozda yok gibidir.

13 Korelasyon ve Basit Doğrusal Regresyon

i Bu haftanın hedefleri

- Kovaryans ve korelasyonu hesaplamak; **Pearson**, **Spearman** ve **Kendall** korelasyonlarını ne zaman kullanacağını bilmek,
- Korelasyonun nedensellik olmadığını ve aykırı değerlerden nasıl etkilendiğini göstermek,
- Basit doğrusal regresyon modelini kurmak, **en küçük kareler** yönteminin mantığını kavramak,
- `lm()` çıktısını (katsayılar, standart hatalar, t-testleri, R^2 , F testi) okumak ve yorumlamak,
- Ortalama yanıt için **güven aralığı** ile yeni bir gözlem için **kestirim aralığı** arasındaki farkı bilmek,
- Artık grafikleriyle model **varsayımlarını** denetlemek,
- Regresyonun ANOVA ile ilişkisini görmek ve çoklu regresyona kısa bir giriş yapmak.

13.1 İki Sayısal Değişken Arasındaki İlişki

13.1.1 Kovaryans ve Korelasyon

Kovaryans, iki değişkenin birlikte nasıl değiştiğini ölçer:

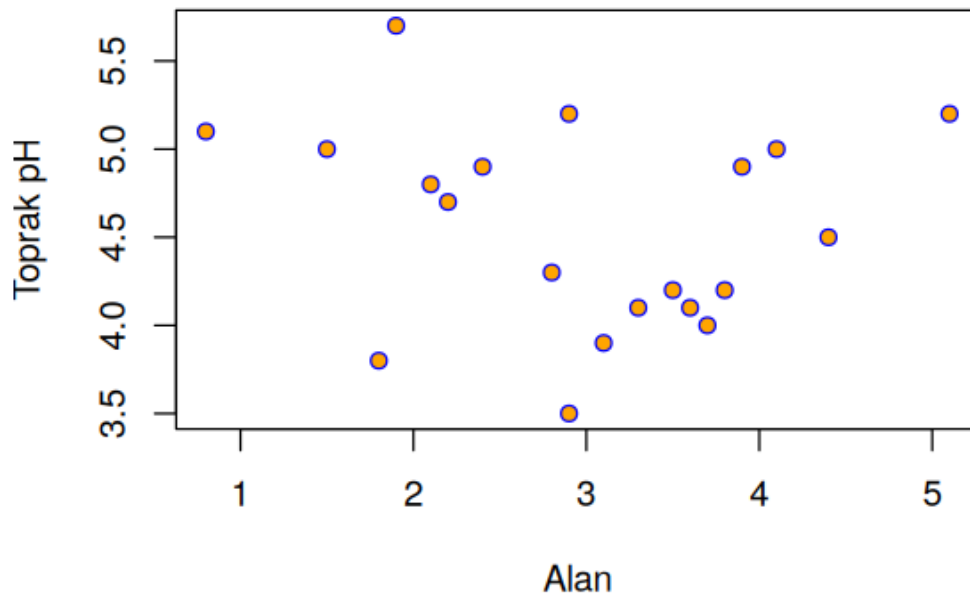
$$\text{Kov}(x, y) = s_{xy} = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

Kovaryansın birimi değişkenlerin birimlerinin çarpımıdır, bu yüzden büyüklüğü yorumlanamaz. Standart sapmalara bölerek birimsiz **Pearson korelasyon katsayısını** elde ederiz:

$$r = \frac{s_{xy}}{s_x s_y}, \quad -1 \leq r \leq 1$$

Eski notlarımızdaki worms örneği: tarla alanı ile toprak pH'ı arasındaki ilişki.

```
with(worms, {  
  plot(Area, Soil.pH, pch = 21, col = "blue", bg = "orange",  
        xlab = "Alan", ylab = "Toprak pH")  
  print(c(var_alan = var(Area), var_ph = var(Soil.pH), kov = var(Area, Soil.pH)))  
  print(var(Area, Soil.pH) / sqrt(var(Area) * var(Soil.pH))) # Elle korelasyon  
  print(cor(Area, Soil.pH))  
})
```



```
#> var_alan   var_ph      kov
#>   1.1494   0.3321  -0.1010
#> [1] -0.1635
#> [1] -0.1635
```

13.1.2 Korelasyon Katsayısının Testi

$H_0 : \rho = 0$ hipotezi $t = r\sqrt{\frac{n-2}{1-r^2}} \sim t_{n-2}$ istatistiği ile sınanır:

```
cor.test(worms$Area, worms$Soil.pH)
```

```
#>
#> Pearson's product-moment correlation
#>
#> data: worms$Area and worms$Soil.pH
#> t = -0.7, df = 18, p-value = 0.5
#> alternative hypothesis: true correlation is not equal to 0
#> 95 percent confidence interval:
#>  -0.5651  0.3008
#> sample estimates:
#>      cor
#> -0.1635
```

13.1.3 Pearson, Spearman ve Kendall

Tablo 13.1: Korelasyon katsayıları

| Katsayı | Neyi ölçer? | Ne zaman? | R |
|-----------------------|---|---|---|
| Pearson r | Doğrusal ilişkinin gücü | İki değişken sürekli, ilişki doğrusal, büyük aykırı değer yok | <code>cor(x, y)</code> |
| Spearman r_s | Monoton ilişki (sıralar üzerinden Pearson) | Sıralı veri, doğrusal olmayan ama monoton ilişki, aykırı değerler | <code>cor(x, y, method = "spearman")</code> |
| Kendall τ | Uyumlu-uyumsuz çiftlerin farkı | Küçük örneklem, çok sayıda eşit değer | <code>cor(x, y, method = "kendall")</code> |

Spearman ve Kendall katsayıları **parametrik olmayan** korelasyon ölçüleridir.

```
cor.test(worms$Area, worms$Soil.pH, method = "spearman", exact = FALSE)
```

```
#>
#> Spearman's rank correlation rho
#>
#> data: worms$Area and worms$Soil.pH
#> S = 1473, p-value = 0.7
#> alternative hypothesis: true rho is not equal to 0
#> sample estimates:
#>      rho
#> -0.1074
```

```
cor.test(worms$Area, worms$Soil.pH, method = "kendall", exact = FALSE)
```

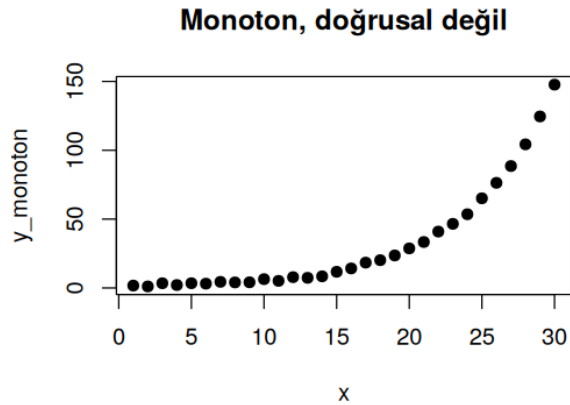
```
#>
#> Kendall's rank correlation tau
#>
#> data: worms$Area and worms$Soil.pH
#> z = -0.33, p-value = 0.7
#> alternative hypothesis: true tau is not equal to 0
#> sample estimates:
#>      tau
#> -0.05348
```

Aykırı değerlerin etkisini görelim: Doğrusal olmayan ama monoton bir ilişkide ve tek bir aykırı değer eklendiğinde üç katsayı nasıl değişiyor?

```

set.seed(13)
x <- 1:30
y_monoton <- exp(x / 6) + rnorm(30)
plot(x, y_monoton, pch = 19, main = "Monoton, doğrusal değil")
c(pearson = cor(x, y_monoton), spearman = cor(x, y_monoton, method = "spearman"))
x2 <- c(rnorm(29), 8); y2 <- c(rnorm(29), 8)
plot(x2, y2, pch = 19, main = "İlişki yok + tek aykırı değer")
c(pearson = cor(x2, y2), spearman = cor(x2, y2, method = "spearman"))

```



```

#>   pearson spearman
#>   0.8582   0.9951

```



```

#>   pearson spearman
#>   0.6365  -0.1631

```

Tek bir aykırı değer, aslında ilişkisiz iki değişken arasında güçlü bir Pearson korelasyonu yaratabilir. Önce grafiğe bakın.

13.1.4 Korelasyon Matrisi

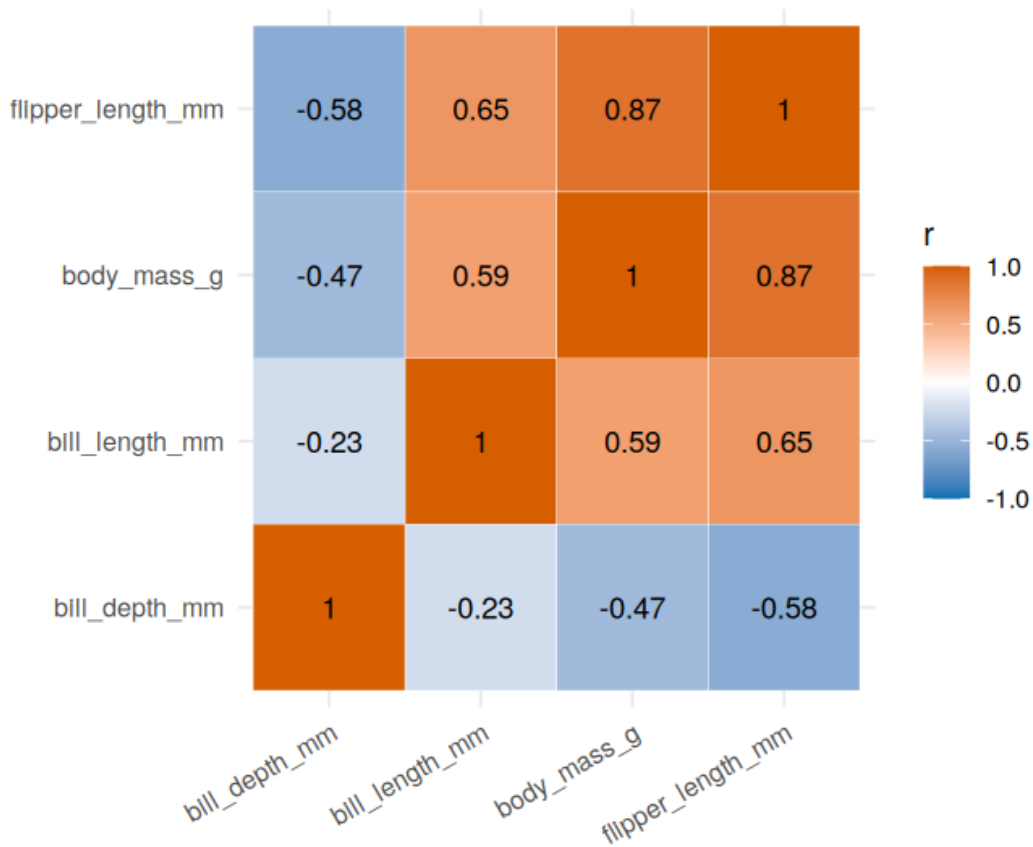
```
cor(penguenler[, 3:6]) |> round(2)
```

```

#>               bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
#> bill_length_mm             1.00         -0.23              0.65          0.59
#> bill_depth_mm             -0.23             1.00             -0.58         -0.47
#> flipper_length_mm          0.65          -0.58              1.00          0.87
#> body_mass_g               0.59          -0.47              0.87          1.00

```

```
cor(penguenler[, 3:6]) |>
  as.data.frame() |>
  tibble::rownames_to_column("d1") |>
  tidyr::pivot_longer(-d1, names_to = "d2", values_to = "r") |>
  ggplot(aes(d1, d2, fill = r)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = round(r, 2))) +
  scale_fill_gradient2(low = "#0072B2", mid = "white", high = "#D55E00", limits =
    ↪ c(-1, 1)) +
  labs(x = NULL, y = NULL, fill = "r") +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
```



⚠ Korelasyon nedensellik değildir

Dondurma satışları ile boğulma vakaları arasında güçlü bir pozitif korelasyon vardır; çünkü ikisi de **sıcak havadan** etkilenir. Üçüncü bir değişkenin iki değişkeni birlikte etkilemesine **karıştırıcı değişken** denir. Gaga uzunluğu ve derinliği örneğinde (Bölüm 6) gördüğümüz Simpson paradoksu da bunun bir örneğidir. Nedensellik ancak deney tasarımıyla ya da çok dikkatli gözlemsel çalışmalarla gösterilebilir.

13.2 Basit Doğrusal Regresyon

Korelasyon ilişkinin **gücünü** ölçer; regresyon ise bir değişkeni (bağımlı, y) diğerinden (bağımsız, x) **tahmin etmek** için bir model kurar:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2)$$

| Simge | Adı | Anlamı |
|---------------|--------------------------------|---|
| y | Bağımlı (yanıt) değişken | Tahmin etmek istediğimiz |
| x | Bağımsız (açıklayıcı) değişken | Tahmin için kullandığımız |
| β_0 | Sabit (kesim noktası) | $x = 0$ iken y 'nin beklenen değeri |
| β_1 | Eğim (regresyon katsayısı) | x bir birim arttığında y 'deki ortalama değişim |
| ε | Hata terimi | Modelin açıklayamadığı rastgele sapma |

13.2.1 Veri: Old Faithful Gayzeri

Yellowstone Ulusal Parkı'ndaki **Old Faithful** gayzeri, düzenli aralıklarla püskürmesiyle ünlüdür. Park görevlileri ziyaretçilere bir sonraki püskürmenin ne zaman olacağını söylemek için, son püskürmenin **ne kadar sürdüğünü** kullanırlar. R ile birlikte gelen **faithful** verisi 272 püskürmenin süresini (**eruptions**, dakika) ve bir sonraki püskürmeye kadar geçen bekleme süresini (**waiting**, dakika) içerir.

Soru: Bir püskürmenin süresinden, bir sonraki püskürmeye kadar ne kadar bekleneceğini tahmin edebilir miyiz?

```
str(faithful)
```

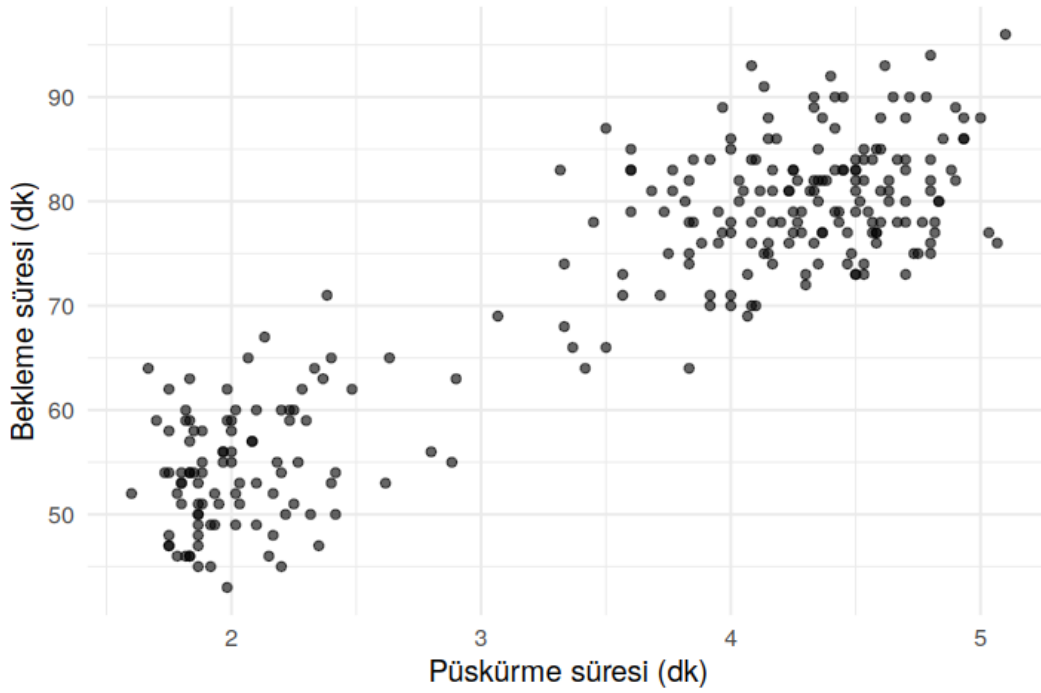
```
#> 'data.frame':   272 obs. of  2 variables:
#> $ eruptions: num  3.6 1.8 3.33 2.28 4.53 ...
#> $ waiting : num  79 54 74 62 85 55 88 85 51 85 ...
```

```
summary(faithful)
```

```
#>   eruptions      waiting
#> Min.   :1.60   Min.   :43.0
#> 1st Qu.:2.16   1st Qu.:58.0
#> Median :4.00   Median :76.0
#> Mean   :3.49   Mean   :70.9
#> 3rd Qu.:4.45   3rd Qu.:82.0
#> Max.   :5.10   Max.   :96.0
```

```
ggplot(faithful, aes(eruptions, waiting)) +
  geom_point(alpha = 0.6) +
  labs(x = "Püskürme süresi (dk)", y = "Bekleme süresi (dk)")
cor(faithful$eruptions, faithful$waiting)
```

```
#> [1] 0.9008
```



Şekil 13.1: Old Faithful: püskürme süresi ve bir sonraki püskürmeye kadar bekleme süresi

Güçlü, pozitif ve doğrusal bir ilişki var ($r = 0.90$): Uzun süren püskürmelerden sonra daha uzun beklenir. Veride iki küme (kısa ve uzun püskürmeler) olduğu da dikkat çekiyor.

13.2.2 En Küçük Kareler Yöntemi

Veriye en iyi uyan doğru hangisidir? **En küçük kareler** yöntemi, gözlenen değerler ile doğrunun tahminleri arasındaki farkların (**artıkların**) kareleri toplamını en küçük yapan doğruyu seçer:

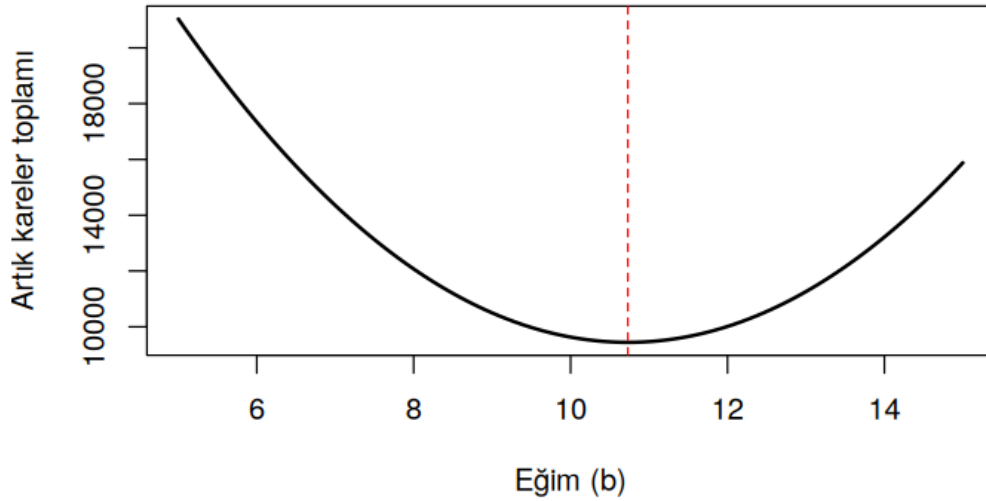
$$AKT = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - b_0 - b_1 x_i)^2 \rightarrow \min$$

Eski ders notlarımızdaki fikir: Eğim için çok sayıda aday değer deneyip her biri için artık kareler toplamını (AKT) hesaplayalım. (Her eğim için en iyi sabit $b_0 = \bar{y} - b_1 \bar{x}$ 'tir.)

```
x <- faithful$eruptions; y <- faithful$waiting
b <- seq(5, 15, by = 0.01)
akt <- numeric(length(b))
for (i in seq_along(b)) {
  a <- mean(y) - b[i] * mean(x)
  artiklar <- y - a - b[i] * x
  akt[i] <- sum(artiklar^2)
}
en_iyi_b <- b[which.min(akt)]
en_iyi_b
```

```
#> [1] 10.73
```

```
plot(b, akt, type = "l", lwd = 2, xlab = "Eğim (b)", ylab = "Artık kareler toplamı")
abline(v = en_iyi_b, col = "red", lty = 2)
```



Bu arama yerine türev alarak kapalı bir çözüm elde edilir:

$$b_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = \frac{s_{xy}}{s_x^2} = r \frac{s_y}{s_x}, \quad b_0 = \bar{y} - b_1 \bar{x}$$

```
b1 <- sum((x - mean(x)) * (y - mean(y))) / sum((x - mean(x))^2)
b0 <- mean(y) - b1 * mean(x)
c(b0 = b0, b1 = b1)
```

```
#>      b0      b1
#> 33.47 10.73
```

```
cor(x, y) * sd(y) / sd(x)    # Aynı eğim
```

```
#> [1] 10.73
```

13.2.3 lm() ile Model Kurma

```
model <- lm(waiting ~ eruptions, data = faithful)
model
```

```
#>
#> Call:
#> lm(formula = waiting ~ eruptions, data = faithful)
#>
#> Coefficients:
#> (Intercept)      eruptions
#>          33.5          10.7
```

Formül yazımı $y \sim x$ “y’yi x ile modelle” diye okunur. Kestirilen model:

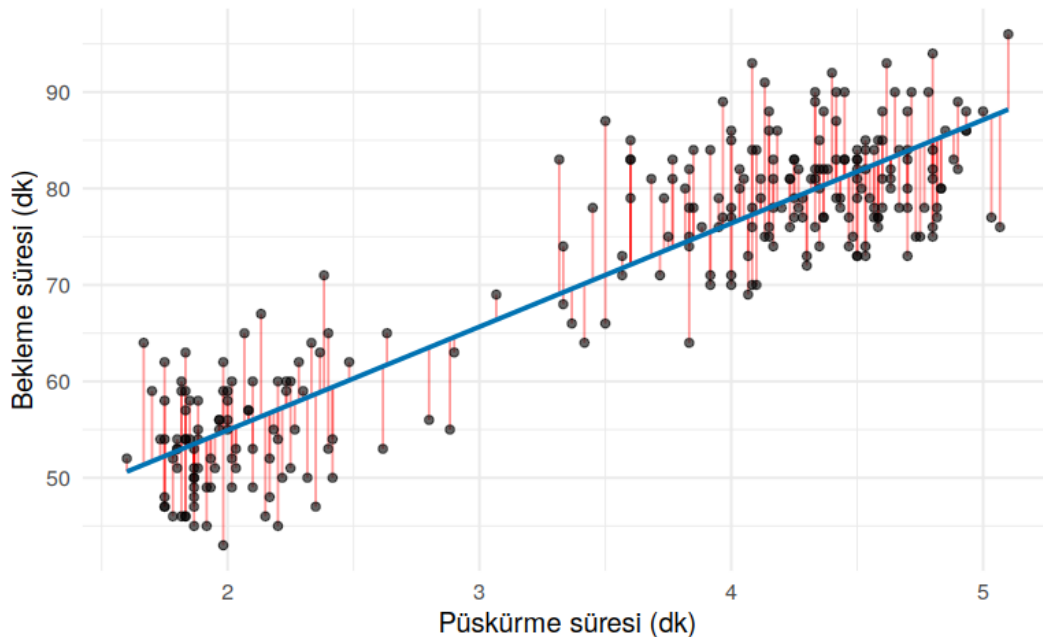
$$\widehat{\text{bekleme}} = 33.47 + 10.73 \times \text{püskürme süresi}$$

Yorum: Püskürme süresi **1 dakika** uzadığında, bir sonraki püskürmeye kadar beklenen süre **ortalama 10.7 dakika** artar. Sabit terim ($x = 0$, yani hiç sürmeyen bir püskürme) bu veride anlamlı bir yorum taşımaz; yalnızca doğrunun konumunu belirler.

13.2.4 Artıkları Görmek

Eski notlarımızda her gözlemden regresyon doğrusuna kırmızı çizgiler çizmiştik. Bu çizgiler **artıklardır**: $e_i = y_i - \hat{y}_i$.

```
faithful_tahmin <- faithful |>
  mutate(tahmin = fitted(model), artik = residuals(model))
ggplot(faithful_tahmin, aes(eruptions, waiting)) +
  geom_segment(aes(xend = eruptions, yend = tahmin), colour = "red", alpha = 0.4) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", se = FALSE, colour = "#0072B2") +
  labs(x = "Püskürme süresi (dk)", y = "Bekleme süresi (dk)")
```



```
sum(faithful_tahmin$artik) # Artıkların toplamı her zaman 0
```

```
#> [1] 3.775e-14
```

13.2.5 Model Özetini Okumak

```
summary(model)
```

```
#>
#> Call:
#> lm(formula = waiting ~ eruptions, data = faithful)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -12.080  -4.483   0.212   3.925  15.972
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept)    33.474      1.155    29.0   <2e-16 ***
#> eruptions     10.730      0.315    34.1   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 5.91 on 270 degrees of freedom
#> Multiple R-squared:  0.811, Adjusted R-squared:  0.811
#> F-statistic: 1.16e+03 on 1 and 270 DF, p-value: <2e-16
```

Tablo 13.3: lm() özet çıktısının bileşenleri

| Çıktı | Anlamı | Bu modelde |
|--------------------------------|--|------------------------------|
| Estimate | Katsayı tahminleri b_0, b_1 | $b_1 = 10.73$ |
| Std. Error | Katsayıların standart hataları | $SH(b_1) = 0.31$ |
| t value, Pr(> t) | $H_0 : \beta = 0$ testi ($t = b/SH$) | Eğim anlamlı ($p < 0.001$) |
| Residual standard error | Artıkların standart sapması | 5.9 dk |
| Multiple R-squared | $\hat{\sigma}$: tipik tahmin hatası
R^2 : y 'deki değişimin model tarafından açıklanan oranı | 0.81 |
| F-statistic | Modelin bütün olarak anlamlılığı (basit regresyonda $F = t^2$) | $p < 0.001$ |

Belirlilik katsayısı R^2 :

$$R^2 = \frac{\text{SKR}}{\text{SKT}} = 1 - \frac{\text{AKT}}{\text{SKT}}, \quad \text{basit regresyonda } R^2 = r^2$$

```
cor(x, y)^2
```

```
#> [1] 0.8115
```

```
summary(model)$r.squared
```

```
#> [1] 0.8115
```

Bekleme süresindeki değişimin yaklaşık **%81**'i püskürme süresiyle açıklanmaktadır.

13.2.6 Regresyon ve ANOVA

Eski notlarımızdaki `summary.aov(model)` çıktısı, regresyonun da değişimi parçalara ayırdığını gösterir: toplam değişim = regresyonla açıklanan + artık. Bölüm 12'deki ANOVA tablosunun aynısıdır:

```
anova(model)
```

```
#> Analysis of Variance Table
#>
#> Response: waiting
#>           Df Sum Sq Mean Sq F value Pr(>F)
#> eruptions  1  40644   40644    1162 <2e-16 ***
#> Residuals 270   9443     35
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Aslında ANOVA, bağımsız değişkeni **kategorik** olan bir regresyondur:

```
anova(lm(body_mass_g ~ species, data = penguinler)) # @sec-anova'daki aov() ile
↪ aynı
```

```
#> Analysis of Variance Table
#>
#> Response: body_mass_g
#>           Df Sum Sq Mean Sq F value Pr(>F)
#> species     2 1.45e+08 72595110    342 <2e-16 ***
#> Residuals 330 7.01e+07  212332
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
coef(lm(body_mass_g ~ species, data = penguinler)) # Sabit: Adelie ortalaması;
↪ diğerleri: farklar
```

```
#>           (Intercept) speciesChinstrap speciesGentoo
#>           3706.16           26.92           1386.27
```

13.2.7 Katsayılar İçin Güven Aralıkları

$$b_1 \pm t_{0.975, n-2} \cdot SH(b_1)$$

```
confint(model)
```

```
#>           2.5 % 97.5 %  
#> (Intercept) 31.20  35.75  
#> eruptions   10.11  11.35
```

Püskürme süresindeki 1 dakikalık artışın bekleme süresine etkisi, %95 güvenle 10.1 ile 11.3 dakika arasındadır.

13.3 Tahmin: Güven Aralığı ve Kestirim Aralığı

Modeli kullanarak yeni değerler için tahmin yapabiliriz. Ancak iki farklı soru vardır:

| Soru | Aralık | R |
|---|--------------------------|-------------------------|
| 3 dakika süren bütün püskürmelerden sonra ortalama ne kadar beklenir? | Güven aralığı (dar) | interval = "confidence" |
| Az önce 3 dakika süren bu püskürmeden sonra ne kadar bekleyeceğiz? | Kestirim aralığı (geniş) | interval = "prediction" |

```
yeni <- data.frame(eruptions = c(2, 3, 4.5))  
predict(model, newdata = yeni)
```

```
#>      1      2      3  
#> 54.93 65.66 81.76
```

```
predict(model, newdata = yeni, interval = "confidence")
```

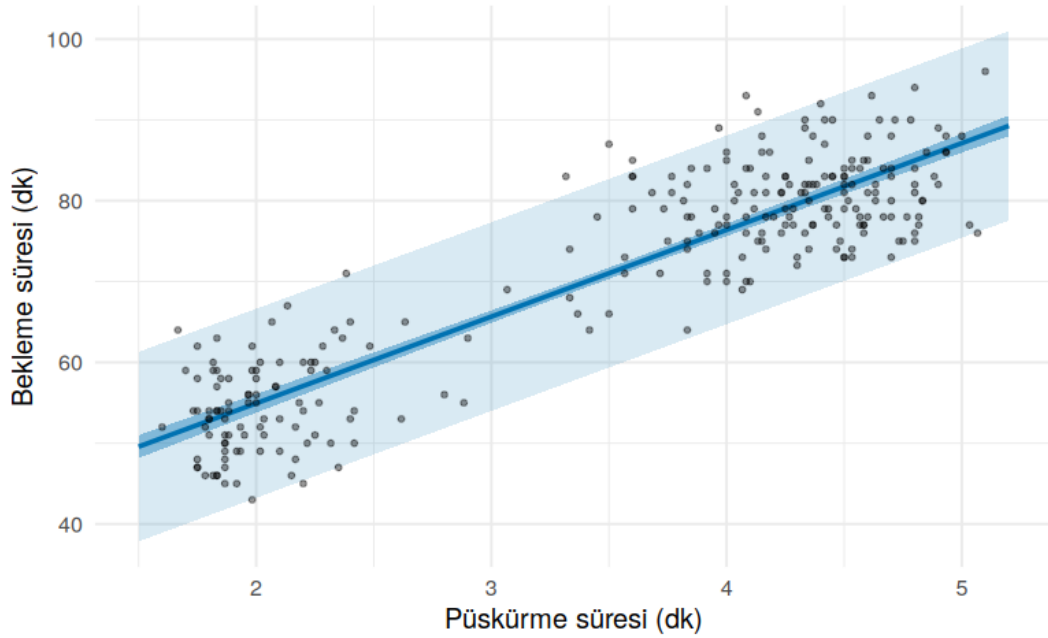
```
#>      fit   lwr   upr  
#> 1 54.93 53.77 56.09  
#> 2 65.66 64.90 66.43  
#> 3 81.76 80.81 82.70
```

```
predict(model, newdata = yeni, interval = "prediction")
```

```
#>      fit   lwr   upr  
#> 1 54.93 43.23 66.63  
#> 2 65.66 53.99 77.33  
#> 3 81.76 70.08 93.44
```

Kestirim aralığı her zaman daha geniştir, çünkü hem doğrunun belirsizliğini hem de tek tek gözlemlerin doğru etrafındaki dağılımını içerir. Bir park görevlisi ziyaretçilere “3 dakikalık bir püskürmeden sonra yaklaşık 54 ile 77 dakika arasında bir sonraki püskürme olur” demelidir.

```
izgara <- data.frame(eruptions = seq(1.5, 5.2, length.out = 100))
bantlar <- cbind(izgara,
                 predict(model, izgara, interval = "prediction"),
                 guven = predict(model, izgara, interval = "confidence")[, 2:3])
ggplot(bantlar, aes(eruptions)) +
  geom_ribbon(aes(ymin = lwr, ymax = upr), fill = "#0072B2", alpha = 0.15) +
  geom_ribbon(aes(ymin = guven.lwr, ymax = guven.upr), fill = "#0072B2", alpha = 0.4)
  +
  geom_line(aes(y = fit), colour = "#0072B2", linewidth = 1) +
  geom_point(data = faithful, aes(y = waiting), alpha = 0.4, size = 1) +
  labs(x = "Püskürme süresi (dk)", y = "Bekleme süresi (dk)")
```



Şekil 13.2: Regresyon doğrusu, %95 güven bandı (koyu) ve %95 kestirim bandı (açık)

⚠ Dışkestirim (extrapolation) yapmayın

Model yalnızca verinin aralığında (burada 1.6–5.1 dakika) geçerlidir. 10 dakikalık bir püskürme için tahmin yapmak, ilişkinin bu aralığın dışında da doğrusal kaldığını varsaymak demektir.

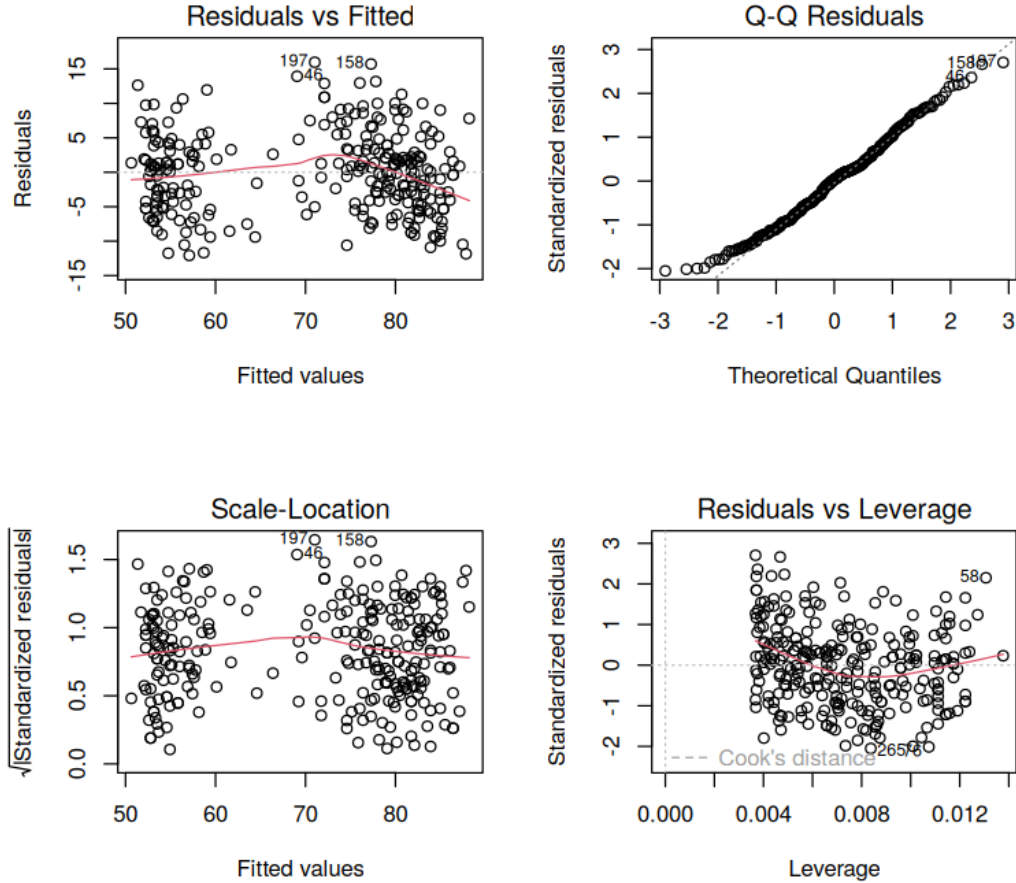
13.4 Model Varsayımları ve Artık Analizi

Doğrusal regresyonun varsayımları (kısaca **LINE**):

Tablo 13.5: Doğrusal regresyonun varsayımları

| Varsayım | Anlamı | Nasıl denetlenir? |
|-----------------------------|---|---|
| Linearity (doğrusallık) | x ile y arasındaki ilişki doğrusaldır | Serpilme grafiği, artıklar–tahminler grafiğinde örüntü olmaması |
| Independence (bağımsızlık) | Hatalar birbirinden bağımsızdır | Veri toplama tasarımı; zaman sırasıyla artık grafiği |
| Normality (normallik) | Hatalar normal dağılır | Artıkların Q-Q grafiği |
| Equal variance (eş varyans) | Hataların varyansı x 'e göre değişmez | Artıklar–tahminler grafiğinde huni biçimi olmaması |

```
par(mfrow = c(2, 2))
plot(model)
```



```
par(mfrow = c(1, 1))
```

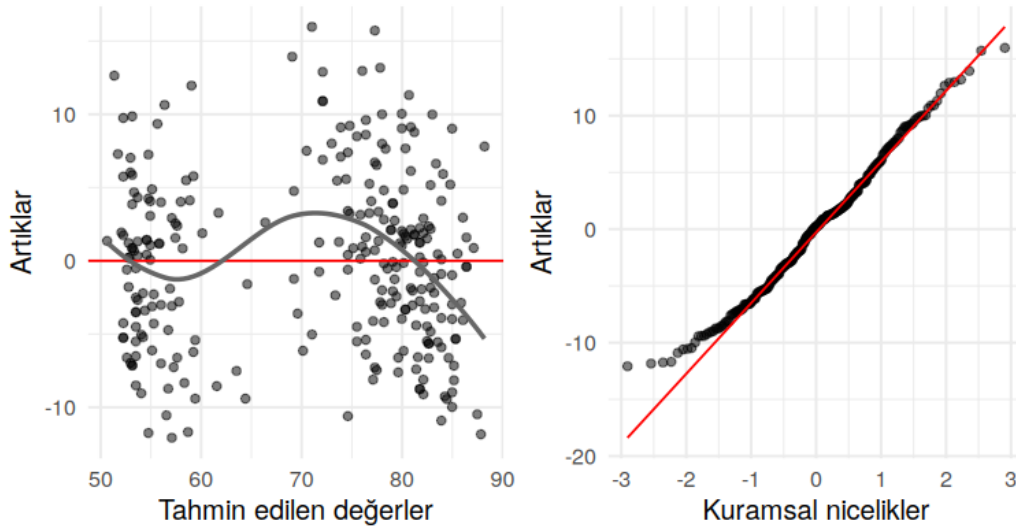
Artıklar sıfır çevresinde rastgele dağılıyor, Q-Q grafiği doğruya yakın ve belirgin bir huni biçimi yok. Model varsayımları makul ölçüde sağlanıyor.

ggplot2 ile aynı grafiklerin ilk ikisi:

```

a1 <- ggplot(faithful_tahmin, aes(tahmin, artik)) +
  geom_point(alpha = 0.5) + geom_hline(yintercept = 0, colour = "red") +
  geom_smooth(se = FALSE, colour = "grey40", method = "loess") +
  labs(x = "Tahmin edilen deęerler", y = "Artıklar")
a2 <- ggplot(faithful_tahmin, aes(sample = artik)) +
  geom_qq(alpha = 0.5) + geom_qq_line(colour = "red") +
  labs(x = "Kuramsal nicelikler", y = "Artıklar")
a1 + a2

```

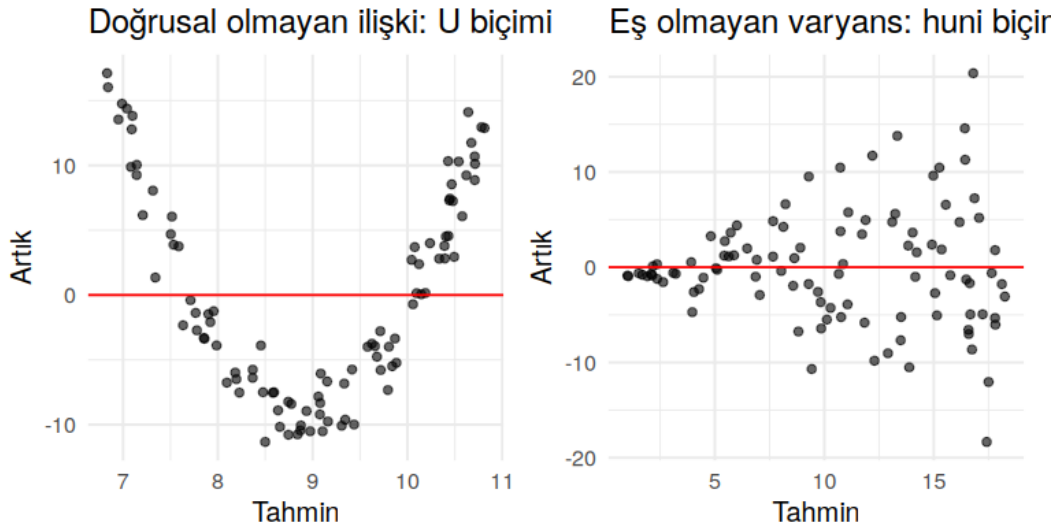


13.4.1 Varsayımlar Sağlanmadığında Nasıl Görünür?

```

set.seed(4)
xx <- runif(100, 0, 10)
egri <- data.frame(x = xx, y = (xx - 5)^2 + rnorm(100, sd = 2)) # Doğrusal deęil
huni <- data.frame(x = xx, y = 2 * xx + rnorm(100, sd = xx)) # Varyans artıyor
artik_grafik <- function(d, baslik) {
  m <- lm(y ~ x, data = d)
  ggplot(data.frame(t = fitted(m), e = residuals(m)), aes(t, e)) +
    geom_point(alpha = 0.6) + geom_hline(yintercept = 0, colour = "red") +
    labs(title = baslik, x = "Tahmin", y = "Artık")
}
artik_grafik(egri, "Doęrusal olmayan iliřki: U bięimi") +
  artik_grafik(huni, "Eř olmayan varyans: huni bięimi")

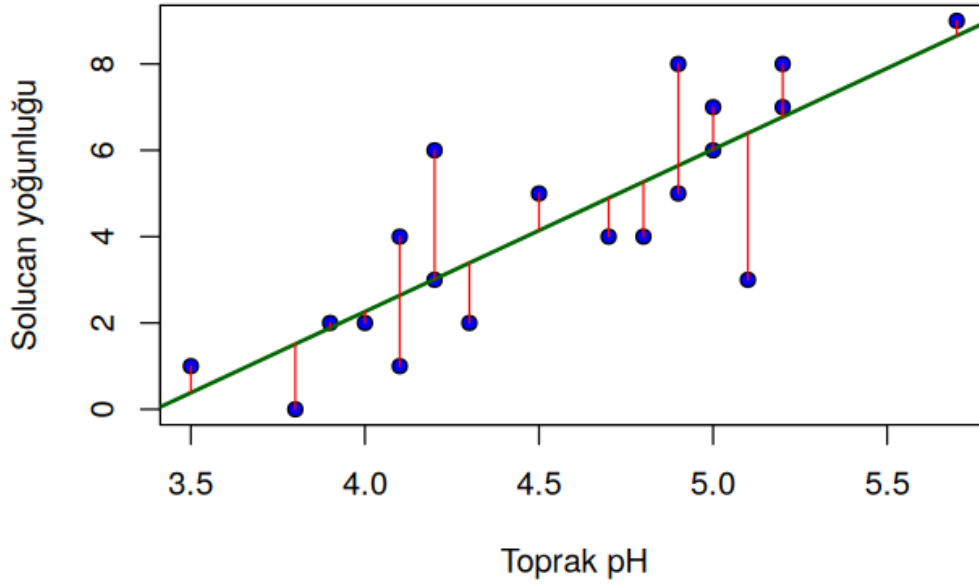
```



Böyle durumlarda değişkenlere dönüşüm uygulamak (örneğin $\log y$), modele terim eklemek (x^2) ya da farklı modeller kullanmak gerekir.

13.5 Eski Notlardan: Worms Verisinde Regresyon

```
worms_model <- lm(Worm.density ~ Soil.pH, data = worms)
plot(worms$Soil.pH, worms$Worm.density, pch = 21, bg = "blue",
     xlab = "Toprak pH", ylab = "Solucan yoğunluğu")
abline(worms_model, col = "darkgreen", lwd = 2)
tahmin <- predict(worms_model)
for (i in seq_len(nrow(worms))) {
  lines(c(worms$Soil.pH[i], worms$Soil.pH[i]), c(worms$Worm.density[i], tahmin[i]),
       col = "red")
}
```



```
summary(worms_model)
```

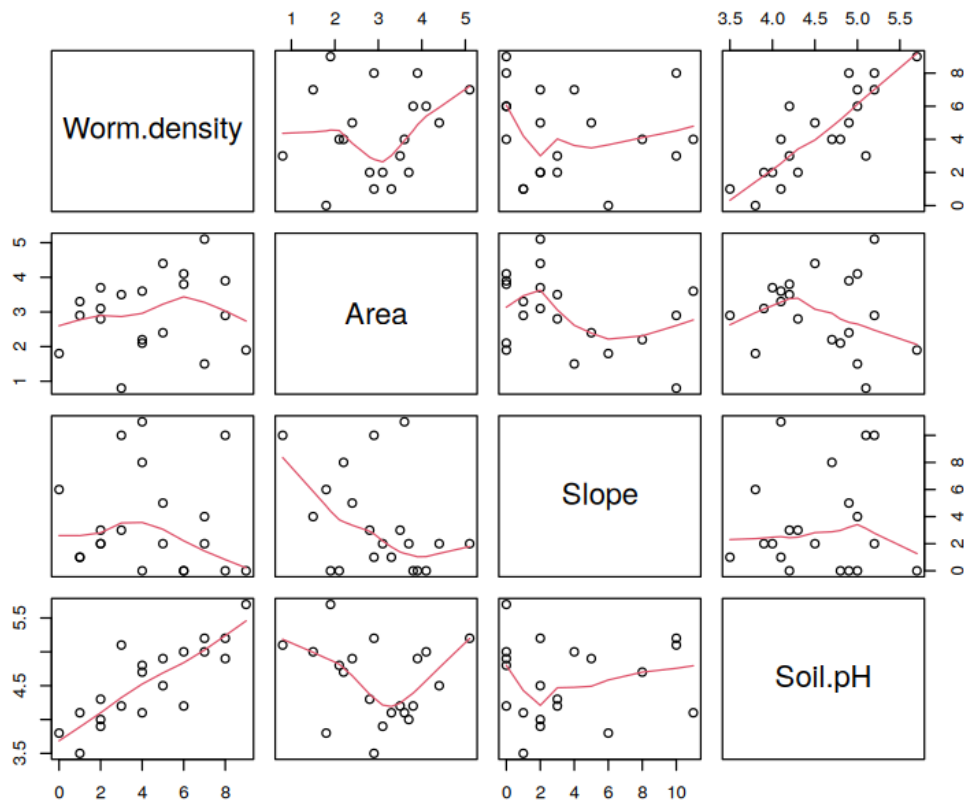
```
#>
#> Call:
#> lm(formula = Worm.density ~ Soil.pH, data = worms)
#>
#> Residuals:
#>    Min       1Q   Median       3Q      Max
#> -3.398 -0.989  0.048  0.887  2.984
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept)  -12.771      2.771   -4.61  0.00022 ***
#> Soil.pH         3.759      0.604    6.22  7.1e-06 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 1.52 on 18 degrees of freedom
#> Multiple R-squared:  0.683, Adjusted R-squared:  0.665
#> F-statistic: 38.7 on 1 and 18 DF,  p-value: 7.14e-06
```

Toprak pH'ı bir birim arttığında solucan yoğunluğu ortalama 3.76 birim artmaktadır ($R^2 = 0.68$).

13.6 Çoklu Regresyona Kısa Bir Bakış

Gerçek problemlerde bağımlı değişkeni genellikle birden fazla değişken etkiler. Eski notlarımızda worms verisi için birden çok değişkenli bir model kurulmuştu:

```
pairs(worms[, c(7, 2, 3, 5)], panel = panel.smooth)
```



```
reg_cok <- lm(Worm.density ~ Area + Slope + Soil.pH, data = worms)
summary(reg_cok)
```

```
#>
#> Call:
#> lm(formula = Worm.density ~ Area + Slope + Soil.pH, data = worms)
#>
#> Residuals:
#>    Min       1Q   Median       3Q      Max
#> -1.815 -0.818 -0.260  0.948  2.401
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept) -15.9209     2.8623  -5.56  4.3e-05 ***
#> Area          0.7285     0.3166   2.30   0.035 *
#> Slope        -0.0215     0.0919  -0.23   0.818
#> Soil.pH       3.9885     0.5446   7.32  1.7e-06 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 1.35 on 16 degrees of freedom
#> Multiple R-squared:  0.777, Adjusted R-squared:  0.735
#> F-statistic: 18.6 on 3 and 16 DF, p-value: 1.84e-05
```

Çoklu regresyonda her katsayı, **diğer değişkenler sabit tutulduğunda** o değişkendeki bir birimlik artışın etkisidir.

Penguenlerde ağırlığı yüzgeç uzunluğu **ve** türle açıklayalım. Kategorik bir değişken modele eklendiğinde her tür için ayrı bir sabit (paralel doğrular) elde edilir:

```
m_tur <- lm(body_mass_g ~ flipper_length_mm + species, data = penguenler)
coef(m_tur)
```

```
#>      (Intercept) flipper_length_mm speciesChinstrap speciesGentoo
#>      -4013.18      40.61      -205.38      284.52
```

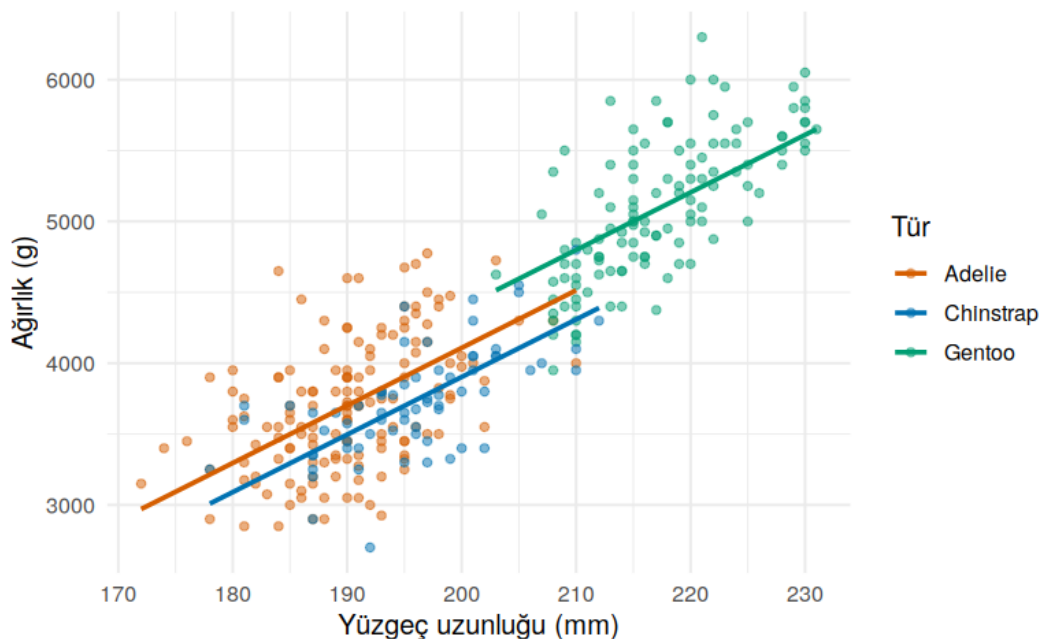
```
summary(lm(body_mass_g ~ flipper_length_mm, data = penguenler))$r.squared
```

```
#> [1] 0.7621
```

```
summary(m_tur)$r.squared
```

```
#> [1] 0.787
```

```
penguenler |>
  mutate(tahmin = fitted(m_tur)) |>
  ggplot(aes(flipper_length_mm, body_mass_g, colour = species)) +
  geom_point(alpha = 0.5) +
  geom_line(aes(y = tahmin), linewidth = 1) +
  scale_colour_manual(values = tur_renkleri) +
  labs(x = "Yüzgeç uzunluğu (mm)", y = "Ağırlık (g)", colour = "Tür")
```



Çoklu regresyon, değişken seçimi, çoklu doğrusal bağlantı ve etkileşim terimleri sonraki derslerin (regresyon çözümlemesi) konusudur.

13.7 Sonuçları Raporlamak

broom paketi model çıktılarını düzenli tablolara çevirir:

```
broom::tidy(model, conf.int = TRUE)
```

```
#> # A tibble: 2 x 7
#>   term          estimate std.error statistic    p.value conf.low conf.high
#>   <chr>          <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1 (Intercept)    33.5      1.15     29.0 7.14e- 85     31.2     35.7
#> 2 eruptions     10.7      0.315    34.1 8.13e-100     10.1     11.3
```

```
broom::glance(model)
```

```
#> # A tibble: 1 x 12
#>   r.squared adj.r.squared sigma statistic    p.value    df logLik   AIC   BIC
#>   <dbl>         <dbl> <dbl>    <dbl>    <dbl> <dbl> <dbl> <dbl> <dbl>
#> 1   0.811         0.811  5.91    1162. 8.13e-100     1 -868. 1743. 1754.
#> # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

```
broom::tidy(model, conf.int = TRUE) |>
  mutate(term = c("Sabit", "Püskürme süresi (dk)")) |>
  knitr::kable(digits = 2, col.names = c("Terim", "Tahmin", "SH", "t", "p",
    "%95 GA alt", "%95 GA üst"))
```

Tablo 13.6: Bekleme süresinin püskürme süresi ile basit doğrusal regresyon modeli

| Terim | Tahmin | SH | t | p | %95 GA alt | %95 GA üst |
|----------------------|--------|------|-------|---|------------|------------|
| Sabit | 33.47 | 1.15 | 28.99 | 0 | 31.20 | 35.75 |
| Püskürme süresi (dk) | 10.73 | 0.31 | 34.09 | 0 | 10.11 | 11.35 |

Püskürme süresi ile bir sonraki püskürmeye kadar geçen bekleme süresi arasında güçlü, pozitif ve doğrusal bir ilişki bulunmuştur ($r = 0.9$). Basit doğrusal regresyon modeline göre püskürme süresindeki her 1 dakikalık artış, bekleme süresini ortalama 10.7 dakika (%95 GA: 10.1–11.3) artırmaktadır ($t_{270} = 34.1$, $p < 0.001$). Model, bekleme süresindeki değişimin %81'ini açıklamaktadır; tipik tahmin hatası yaklaşık 5.9 dakikadır.

Kendini Sına

💡 1. cars verisinde hız (speed) ile fren mesafesi (dist) arasında bir regresyon modeli kurun. Eğimi yorumlayın.

```
m <- lm(dist ~ speed, data = cars)
coef(m) # Hız 1 mph arttığında fren mesafesi ortalama 3.9 ft artar
```

💡 2. Önceki modelin artık grafiklerine bakın. Varsayımlarda bir sorun görüyor musunuz?

`plot(m)` çıktısında artıklar–tahminler grafiği hafif bir **huni** biçimi gösterir: Yüksek hızlarda fren mesafesindeki değişkenlik artmaktadır. Bu, eş varyans varsayımının ihlal edildiğini düşündürür; `lm(sqrt(dist) ~ speed)` gibi bir dönüşüm denenebilir.

💡 3. $r = -0.95$ olan iki değişken için R^2 nedir? Bu, x 'in y 'ye neden olduğunu gösterir mi?

$R^2 = 0.9025$: y 'deki değişimin %90'ı x ile (doğrusal olarak) açıklanabilir. Bu bir **nedensellik** kanıtı değildir.

💡 4. Adelie penguenlerinde yüzgeç uzunluğu 200 mm olan **bir** penguenin ağırlığı için %95 kestirim aralığı nedir?

```
a <- penguenler |> filter(species == "Adelie")
m <- lm(body_mass_g ~ flipper_length_mm, data = a)
predict(m, data.frame(flipper_length_mm = 200), interval = "prediction")
```

💡 5. `iris` verisinde taç yaprak uzunluğu ile genişliği arasındaki Pearson ve Spearman korelasyonlarını türlere göre ayrı ayrı hesaplayın. Tüm veriyle hesaplanan korelasyonla karşılaştırın.

```
cor(iris$Petal.Length, iris$Petal.Width)
iris |> group_by(Species) |>
  summarise(pearson = cor(Petal.Length, Petal.Width),
            spearman = cor(Petal.Length, Petal.Width, method = "spearman"))
```

Tüm veride $r \approx 0.96$ iken türlerin içinde korelasyonlar çok daha düşüktür (0.32–0.79). Yüksek genel korelasyonun büyük kısmı türler arasındaki farktan kaynaklanır.

Kısım IV

Kısım IV — Sunmak

14 Shiny: Etkileşimli Uygulamalar

Bu haftanın hedefleri

- Bir Shiny uygulamasının iki parçasını (**ui** ve **server**) tanımak,
- Girdi (*input*) ve çıktı (*output*) fonksiyonlarını kullanmak,
- **Reaktiflik** kavramını ve **reactive()** ile ortak hesaplamaları anlamak,
- Dönem boyunca öğrenilen görselleştirme, betimsel istatistik ve hipotez testlerini tek bir uygulamada birleştirmek,
- Uygulamayı çalıştırmak ve paylaşmak.

14.1 Shiny Nedir?

Shiny, yalnızca R kodu yazarak etkileşimli **web uygulamaları** oluşturmayı sağlayan bir pakettir. HTML, CSS ya da JavaScript bilmeye gerek yoktur. Bir analiz raporunu okuyan kişi yalnızca sizin seçtiğiniz grafikleri görür; bir Shiny uygulamasında ise kullanıcı değişkeni, grubu, testi kendisi seçer ve sonuçlar anında güncellenir.

Bu ders boyunca öğrendiklerimizin hepsi burada bir araya gelir: **veri düzenleme** (Bölüm 4), **fonsiyonlar** (Bölüm 5), **ggplot2** ve **plotly** (Bölüm 6), **betimsel istatistik** (Bölüm 7) ve **hipotez testleri** (Bölüm 11, Bölüm 12).

Önemli

Shiny uygulamaları çalışan bir R oturumuna ihtiyaç duyar. Bu yüzden bu bölümdeki kodlar kitabın içinde **çalıştırılmamıştır**. Uygulamaları kendi bilgisayarınızda **shiny_ornekleri/** klasöründen çalıştırın.

14.2 Bir Shiny Uygulamasının Yapısı

Her Shiny uygulaması üç parçadan oluşur:

```
library(shiny)

ui <- fluidPage(
  # Kullanıcının gördüğü arayüz: girdiler ve çıktıların yeri
)

server <- function(input, output, session) {
  # Girdilere göre çıktıları üreten R kodu
}

shinyApp(ui, server)
```

| Parça | Görevi | Benzetme |
|------------------------------|---|-------------------------------|
| ui (<i>user interface</i>) | Sayfanın düzeni; girdi kontrolleri ve çıktıların nereye konacağı | Restoranın menüsü ve masaları |
| server | Girdileri alıp çıktıları hesaplayan R kodu | Mutfak |
| shinyApp() | İkisini birleştirip uygulamayı başlatır | Restoranı açmak |

Bir uygulama genellikle **app.R** adlı tek bir dosyaya yazılır. RStudio'da **File → New File → Shiny Web App...** ile hazır bir şablon oluşturulabilir; **app.R** açıkken editörün üstünde **Run App** düğmesi görünür.

14.3 İlk Uygulama

shiny_ornekleri/01-histogram/app.R: Kullanıcı bir kaydırma çubuğuyla histogramın kutu sayısını seçer.

```
# İST347 - 14. hafta: İlk Shiny uygulaması
# Çalıştırmak için: RStudio'da bu dosyayı açıp "Run App" düğmesine basın
# ya da konsolda shiny::runApp("shiny_ornekleri/01-histogram")

library(shiny)

ui <- fluidPage(
  titlePanel("Old Faithful: bekleme süreleri"),
  sidebarLayout(
    sidebarPanel(
      sliderInput("kutu", "Kutu sayısı:", min = 5, max = 50, value = 20),
      checkboxInput("yogunluk", "Yoğunluk eğrisini göster", value = FALSE)
    ),
    mainPanel(
      plotOutput("histogram"),
      verbatimTextOutput("ozet")
    )
  )
)

server <- function(input, output, session) {
  output$histogram <- renderPlot({
    x <- faithful$waiting
    hist(x, breaks = input$kutu, col = "steelblue", border = "white",
         freq = !input$yogunluk, main = "", xlab = "Bekleme süresi (dk)")
    if (input$yogunluk) lines(density(x), lwd = 2, col = "red")
  })
  output$ozet <- renderPrint(summary(faithful$waiting))
}
```

```
shinyApp(ui, server)
```

Bu kısa uygulamada Shiny'nin bütün temel fikirleri vardır:

1. `sliderInput("kutu", ...)` adı **kutu** olan bir girdi oluşturur. Kullanıcının seçtiği değer server'da **input\$kutu** ile okunur.
2. `plotOutput("histogram")` ui'da bir grafik için **yer ayırır**. Server'da **output\$histogram**'a atanan grafik buraya çizilir.
3. `renderPlot({ ... })` içindeki kod `input$kutu` ya da `input$yogunluk` her değiştiğinde **kendiliğinden yeniden çalışır**. Buna **reaktiflik** denir.

14.4 Girdiler ve Çıktılar

Her çıktı fonksiyonunun ui'da bir `*Output()`, server'da bir `render*()` eşi vardır:

| Girdi fonksiyonu | Ne oluşturur? |
|--|---------------------------|
| <code>sliderInput()</code> | Kaydırma çubuğu |
| <code>numericInput()</code> | Sayı kutusu |
| <code>selectInput()</code> | Açılır liste |
| <code>radioButtons()</code> | Tek seçimli düğmeler |
| <code>checkboxInput()</code> / <code>checkboxGroupInput()</code> | Onay kutusu / çoklu seçim |
| <code>textInput()</code> | Metin kutusu |
| <code>fileInput()</code> | Dosya yükleme |
| <code>actionButton()</code> | Düğme |

Tablo 14.3: Shiny girdi ve çıktı fonksiyonları

| ui'da çıktı yeri | server'da üretici | Ne gösterir? |
|-----------------------------------|-----------------------------|---|
| <code>plotOutput()</code> | <code>renderPlot()</code> | base R ya da ggplot2 grafiği |
| <code>plotlyOutput()</code> | <code>renderPlotly()</code> | Etkileşimli plotly grafiği |
| <code>tableOutput()</code> | <code>renderTable()</code> | Tablo |
| <code>verbatimTextOutput()</code> | <code>renderPrint()</code> | Konsol çıktısı (ör. <code>summary()</code> , test sonucu) |
| <code>textOutput()</code> | <code>renderText()</code> | Düz metin |

⚠ En sık yapılan üç hata

1. ui'daki ad ile server'daki ad uyuşmuyor: `plotOutput("grafik")` ama `output$Grafik <- ...`
2. ui'daki öğeler arasına **virgül** konmamış (ya da son öğeden sonra fazladan virgül konmuş).
3. Bir `render*()` fonksiyonunun dışında `input$...` kullanılmış. Girdiler yalnızca reaktif bir bağlamda (`render*()`, `reactive()`, `observe()`) okunabilir.

14.5 Reaktiflik ve reactive()

Birden fazla çıktı aynı süzölmüş veriyi kullanıyorsa, süzme işlemini her çıktıda tekrar yazmak yerine bir **reaktif ifade** tanımlarız:

```
veri <- reactive({
  penguenler |> filter(species %in% input$turler)
})

output$grafik <- renderPlot({ ggplot(veri(), ...) }) # veri() -> parantezle
  ↳ çağrılır!
output$tablo <- renderTable({ veri() |> summarise(...) })
```

`reactive()` yalnızca içindeki girdiler değiştiğinde yeniden hesaplanır ve sonucu bütün çıktılarla paylaşır. Reaktif ifadeler bir fonksiyon gibi **parantezle** (`veri()`) çağrılır.

14.6 Bütünleştirici Örnek: Penguen Keşif Uygulaması

`shiny_ornekleri/02-penguen-kesif/app.R` dönem boyunca öğrendiklerimizi bir araya getirir. Kullanıcı:

- incelenen değişkeni, türleri ve cinsiyeti seçer,
- **Dağılım** sekmesinde `ggplot2` kutu grafiğini,
- **İlişki** sekmesinde iki değişkenin `ggplotly()` ile etkileşimli serpilme grafiğini,
- **Özet tablo** sekmesinde türlere göre betimsel istatistikleri,
- **Test** sekmesinde seçtiği testin (ANOVA, Welch ANOVA ya da Kruskal-Wallis) sonucunu görür.

```
# İST347 - 14. hafta: Penguen keşif uygulaması
# Görselleştirme (ggplot2 + plotly), betimsel istatistik ve hipotez testlerini
# tek bir etkileşimli uygulamada birleştirir.
# Çalıştırmak için: shiny::runApp("shiny_ornekleri/02-penguen-kesif")

library(shiny)
library(dplyr)
library(ggplot2)
library(plotly)
library(palmerpenguins)

penguenler <- penguins |> tidyr::drop_na()
tur_renkleri <- c(Adelie = "#D55E00", Chinstrap = "#0072B2", Gentoo = "#009E73")
sayisal <- c("Gaga uzunluğu (mm)" = "bill_length_mm",
             "Gaga derinliği (mm)" = "bill_depth_mm",
             "Yüzgeç uzunluğu (mm)" = "flipper_length_mm",
             "Ağırlık (g)" = "body_mass_g")

ui <- fluidPage(
  titlePanel("Palmer Penguenleri: keşif ve karşılaştırma"),
  sidebarLayout(
    sidebarPanel(
```

```

selectInput("y", "İncelenecek değişken:", choices = sayisal, selected =
  ↪ "body_mass_g"),
selectInput("x", "Serpilme grafiği için x eksenini:", choices = sayisal,
  selected = "flipper_length_mm"),
checkboxGroupInput("turler", "Türler:", choices = names(tur_renkleri),
  selected = names(tur_renkleri)),
radioButtons("cinsiyet", "Cinsiyet:",
  choices = c("Hepsi" = "hepsi", "Dişi" = "female", "Erkek" =
    ↪ "male")),
radioButtons("test", "Türleri karşılaştıran test:",
  choices = c("ANOVA" = "anova", "Welch ANOVA" = "welch",
    "Kruskal-Wallis" = "kruskal")),

width = 3
),
mainPanel(
  tabsetPanel(
    tabPanel("Dağılım", plotOutput("kutu", height = 380)),
    tabPanel("İlişki", plotlyOutput("serpilme", height = 420)),
    tabPanel("Özet tablo", tableOutput("ozet")),
    tabPanel("Test", verbatimTextOutput("test_sonuc"))
  ),
  width = 9
)
)
)

server <- function(input, output, session) {

  # Seçimlere göre süzölmüş veri: reactive() bir kez hesaplar, her çıkttı kullanır
  veri <- reactive({
    validate(need(length(input$turler) > 0, "En az bir tür seçin."))
    d <- penguenler |> filter(species %in% input$turler)
    if (input$cinsiyet != "hepsi") d <- d |> filter(sex == input$cinsiyet)
    d
  })

  etiket <- function(deger) names(sayisal)[sayisal == deger]

  output$kutu <- renderPlot({
    ggplot(veri(), aes(species, .data[[input$y]], fill = species)) +
      geom_boxplot(alpha = 0.6, show.legend = FALSE, outlier.shape = NA) +
      geom_jitter(width = 0.15, alpha = 0.4) +
      stat_summary(fun = mean, geom = "point", shape = 23, size = 3, fill = "white") +
      scale_fill_manual(values = tur_renkleri) +
      labs(x = NULL, y = etiket(input$y)) +
      theme_minimal(base_size = 14)
  })

  output$serpilme <- renderPlotly({
    g <- ggplot(veri(), aes(.data[[input$x]], .data[[input$y]], colour = species,
      text = paste("Ada:", island, "<br>Cinsiyet:", sex))) +
      geom_point(alpha = 0.7) +

```

```

    scale_colour_manual(values = tur_renkleri) +
    labs(x = etiket(input$x), y = etiket(input$y), colour = "Tür") +
    theme_minimal()
  ggplotly(g)
})

output$ozet <- renderTable({
  veri() |>
  group_by(Tür = species) |>
  summarise(n = n(),
            Ortalama = mean(.data[[input$y]]),
            SS = sd(.data[[input$y]]),
            Ortanca = median(.data[[input$y]]),
            ÇAG = IQR(.data[[input$y]]))
}, digits = 1)

output$test_sonuc <- renderPrint({
  d <- veri()
  validate(need(n_distinct(d$species) >= 2, "Test için en az iki tür seçin. "))
  formül <- reformulate("species", response = input$y)
  switch(input$test,
        anova = summary(aov(formül, data = d)),
        welch = oneway.test(formül, data = d),
        kruskal = kruskal.test(formül, data = d))
})
}

shinyApp(ui, server)

```

Bu uygulamada dikkat edilecek noktalar:

- Değişken adı kullanıcıdan **karakter** olarak geldiği için ggplot2’de `.data[[input$y]]` kullanıldı (Bölüm 6’deki grafik fonksiyonlarında olduğu gibi).
- `validate(need(...))` kullanıcı hiç tür seçmezse hata yerine anlaşılır bir mesaj gösterir.
- `switch()` (Bölüm 5) kullanıcının seçtiği teste göre farklı bir fonksiyon çalıştırır.
- `reformulate("species", response = input$y)` karakter olarak gelen değişken adından `body_mass_g ~ species` gibi bir formül oluşturur.

14.7 Uygulamayı Çalıştırmak ve Paylaşmak

```

shiny::runApp("shiny_ornekleri/01-histogram")
shiny::runApp("shiny_ornekleri/02-penguen-kesif")

```

| Yol | Açıklama |
|-------------------------------|---|
| Kendi bilgisayarınızda | <code>runApp()</code> ya da Run App ; uygulama tarayıcıda ya da Viewer’da açılır |
| shinyapps.io | Posit’in ücretsiz katmanı olan bulut hizmeti; RStudio’daki Publish düğmesi ile birkaç tıklamada yayınlanır |

| Yol | Açıklama |
|-------------------------------------|--|
| Shiny Server / Posit Connect | Kurum içi sunucular |
| Shinylive | Uygulama tarayıcıda (WebR ile) çalışır, sunucu gerekmez; statik bir web sitesine konabilir |
| Quarto içinde | <code>server: shiny</code> seçeneğiyle bir Quarto belgesine etkileşimli öğeler eklenebilir |

14.8 Dönem Sonu Projesi İçin Öneriler

1. **Bir soru ile başlayın.** “Bu uygulamayı kim, hangi soruyu cevaplamak için kullanacak?”
2. Önce analizi bir **Quarto belgesinde** yapın; çalışan kodu sonra uygulamaya taşıyın.
3. Tekrarlanan kodu **fonksiyonlara** ayırın; uygulama dosyası kısa ve okunur kalsın.
4. Her grafiğin eksen adları, birimleri ve başlığı olsun; renkler her sekmede aynı gruba aynı rengi versin.
5. Kullanıcının hata yapabileceği her yere `validate(need(...))` ekleyin.
6. Testin sonucunu yalnızca p değeriyle değil, **etki büyüklüğü ve yorumuyla** birlikte gösterin.

Kendini Sına

- 💡 1. İlk uygulamaya histogramın rengini seçmek için bir `selectInput()` ekleyin.

ui'da `selectInput("renk", "Renk:", choices = c("steelblue", "darkorange", "grey50"))`, server'da `hist(..., col = input$renk)`.

- 💡 2. Penguen uygulamasına seçilen değişkenin histogramını gösteren yeni bir sekme ekleyin.

ui'daki `tabsetPanel()` içine `tabPanel("Histogram", plotOutput("hist"))`, server'a:

```
output$hist <- renderPlot({
  ggplot(veri(), aes(.data[[input$y]], fill = species)) +
    geom_histogram(bins = 25, alpha = 0.6, position = "identity") +
    scale_fill_manual(values = tur_renkleri)
})
```

- 💡 3. `veri()` yerine `veri` yazarsanız ne olur?

`veri` reaktif ifadenin **kendisidir** (bir fonksiyon), değeri değil. `ggplot2` bir data frame yerine fonksiyon aldığı için hata verir. Reaktif ifadeler her zaman parantezle çağrılmalıdır.

- 💡 4. Test sekmesine, ANOVA seçildiğinde Tukey HSD sonuçlarını da yazdırın.

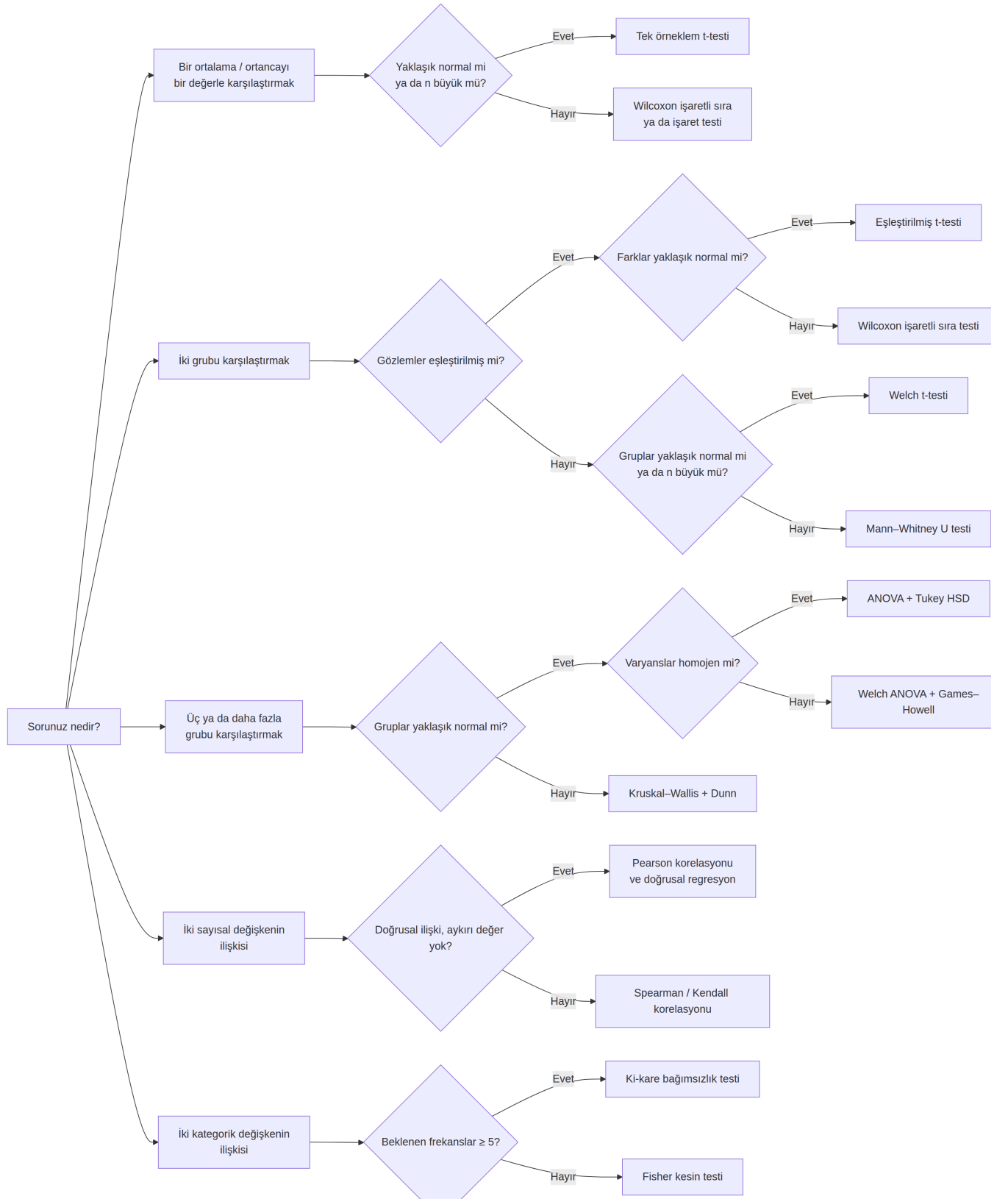
`switch()` içindeki `anova` satırını şöyle değiştirin:

```
anova = { m <- aov(formul, data = d); print(summary(m)); TukeyHSD(m) }
```

A Hangi Test? Hızlı Başvuru Rehberi

Bu ek, Bölüm 11, Bölüm 12 ve Bölüm 13'te ele alınan yöntemlerden hangisinin hangi durumda kullanılacağını özetler.

A.1 Karar Şeması



A.2 R Fonksiyonları Özeti

Tablo A.1: Testler ve R fonksiyonları

| Amaç | Parametrik | R | Parametrik olmayan | R |
|-----------------------------|-----------------------|--|---------------------------------------|--|
| Tek örneklem | t-testi | <code>t.test(x, mu = m0)</code> | Wilcoxon işaretli sıra / işaret testi | <code>wilcox.test(x, mu = m0) / binom.test()</code> |
| İki bağımsız grup | Welch t-testi | <code>t.test(y ~ g)</code> | Mann–Whitney U | <code>wilcox.test(y ~ g)</code> |
| İki eşleştirilmiş ölçüm | Eşleştirilmiş t-testi | <code>t.test(x1, x2, paired = TRUE)</code> | Wilcoxon işaretli sıra | <code>wilcox.test(x1, x2, paired = TRUE)</code> |
| İki varyans | F testi | <code>var.test(y ~ g)</code> | Fligner–Killeen | <code>fligner.test(y ~ g)</code> |
| k bağımsız grup | ANOVA / Welch ANOVA | <code>aov(y ~ g) / oneway.test(y ~ g)</code> | Kruskal–Wallis | <code>kruskal.test(y ~ g)</code> |
| k grup, çoklu karşılaştırma | Tukey / Games–Howell | <code>TukeyHSD()</code> | Dunn / Holm düzeltilmeli Mann–Whitney | <code>rstatix::dunn_test() / pairwise.wilcox.test()</code> |
| Varyans homojenliği | Bartlett | <code>bartlett.test(y ~ g)</code> | Levene | <code>car::leveneTest(y ~ g)</code> |
| İki faktör | İki yönlü ANOVA | <code>aov(y ~ a * b)</code> | — | — |
| İki sayısal değişken | Pearson | <code>cor.test(x, y)</code> | Spearman / Kendall | <code>cor.test(x, y, method = "spearman")</code> |
| Tahmin / açıklama | Doğrusal regresyon | <code>lm(y ~ x)</code> | — | — |
| Tek oran | Binom testi | <code>binom.test(x, n, p)</code> | — | — |
| İki oran | Oran testi | <code>prop.test(c(x1, x2), c(n1, n2))</code> | — | — |
| Kategorik dağılım | Ki-kare uyum iyiliği | <code>chisq.test(tablo, p = ...)</code> | — | — |
| İki kategorik değişken | Ki-kare bağımsızlık | <code>chisq.test(tablo)</code> | Fisher–Perman testi | <code>fisher.test(tablo)</code> |
| Normallik | Shapiro–Wilk | <code>shapiro.test(x)</code> | Q-Q uyumu | <code>qqnorm(x); qqline(x)</code> |

A.3 Etki Büyüklükleri

| Test | Etki büyüklüğü | Küçük / orta / büyük (kabaca) |
|------------|----------------|--|
| t-testleri | Cohen'in d | 0.2 / 0.5 / 0.8 |
| ANOVA | η^2 | 0.01 / 0.06 / 0.14 |
| Korelasyon | r | 0.1 / 0.3 / 0.5 |
| Regresyon | R^2 | bağlama göre |
| Ki-kare | Cramér'in V | sd'ye bağlı (2×2 için 0.1 / 0.3 / 0.5) |

Bu eşikler yalnızca kaba yol göstericidir; bir etkinin büyük ya da önemli olup olmadığı **alanın bağlamına** göre değerlendirilmelidir.